

EXPERT INSIGHT

Microsoft Power Apps Cookbook

Build user-friendly apps, troubleshoot challenges,
and navigate the evolving Power Apps landscape

Third Edition

Eickhel Mendoza



<packt>

Microsoft Power Apps Cookbook

Third Edition

Build user-friendly apps, troubleshoot challenges, and navigate the evolving Power Apps landscape

Eickhel Mendoza



Microsoft Power Apps Cookbook

Third Edition

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Senior Publishing Product Manager: Larissa Pinto

Acquisition Editor – Peer Reviews: Jane Dsouza

Project Editor: Parvathy Nair

Content Development Editor: Matthew Davies

Copy Editor: Safis Editing

Technical Editor: Gaurav Gavas

Proofreader: Safis Editing

Indexer: Tejal Soni

Presentation Designer: Ganesh Bhadwalkar

Developer Relations Marketing Executive: Sohini Ghosh

First published: January 2021

Second edition: August 2022

Third edition: October 2024

Production reference: 1281024

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83546-515-8

www.packt.com

To my wife, Pili, the strongest woman who ever lived, to my daughter, Amanda, the spark of my life, and our little pulse of energy, Chispa.

Without their love and support, nothing would make sense.

– Eickhel Mendoza

Contributors

About the author

Eickhel Mendoza is a Microsoft Business Applications MVP with many years of experience in project management, Microsoft Azure development, and Microsoft Power Platform technologies.

He is a team lead of the Business Apps department and oversees all Microsoft 365 and Power Platform projects. He has contributed to significant community events such as the Power Platform World Tour, Global Azure Bootcamp, Microsoft 365 Developer Bootcamp, and Dynamics 365 Saturdays.

He coordinates the TenerifeDev and Power Platform Canarias user groups with a group of like-minded developers eager to share their knowledge in different technologies. Eickhel is also a member of the organizing committee of the Business Applications Summit Spain.

I want to express my gratitude to all the individuals who have influenced my passion for technology: my parents Numa and Yenny, my teachers, my colleagues, and Osvaldo, who have consistently motivated me to create great things.

About the reviewers

Yves Habersaat is a business applications consultant and AI tech lead at Sword Group in Switzerland, specializing in building applications and solutions with Microsoft 365, Azure, Dynamics 365, and the Power Platform.

With a passion for modern development technologies and application lifecycles, Yves focuses on Microsoft 365 services such as Microsoft Teams, SharePoint Online, OneDrive for Business, Microsoft Viva, and Microsoft Copilot. He possesses expertise in modern development tools including **SharePoint Framework (SPFx)**, Teams SDK, Microsoft Graph, Git, TypeScript, React, Visual Studio, and Azure DevOps.

He actively shares his knowledge and experience through speaking engagements and his blog, always ready to support and assist the community. Yves served as a reviewer for *Microsoft 365 and SharePoint Online Cookbook*, by Gaurav Mahajan, Sudeep Ghatak, Nate Chamberlain, and Scott Brewster.

Rebekka Aalbers has over 15 years of experience in the IT industry. Midway through her career, she transitioned to working with Microsoft Partners. Currently, she is a Power Platform Technical Consultant at Fellowmind, a Microsoft partner active in multiple Northern European countries. Rebekka specializes in building low-code solutions with the Power Platform, striving to maximize the platform's out-of-the-box capabilities and resorting to custom development only when it adds significant value.

Passionate about helping others embark on their Power Platform journey, Rebekka is an active member of the technical community. Her contributions were recognized in 2019 when she received the Microsoft Business Applications MVP award. Known for her fondness of unicorns, she often incorporates them into her presentations, adding a unique and engaging touch.

Rebekka served as a reviewer for both the first and second editions of *Microsoft Power Apps Cookbook* by Eickhel Mendoza. She resides in a rural part of the Netherlands with her husband and two cats.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>

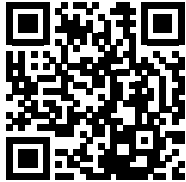


Table of Contents

Preface	xxiii
Chapter 1: App Maker Basics	1
Technical requirements	1
How to build a Power Platform solution	2
Getting ready • 2	
How to do it... • 2	
How it works... • 3	
Preparing tables in Dataverse	3
Getting ready • 4	
<i>Team players</i> • 4	
How to do it... • 4	
How it works... • 6	
Building metadata-enabled storage – SharePoint lists and libraries	6
Getting ready • 7	
<i>Player documents</i> • 7	
How to do it... • 7	
How it works... • 9	

Gathering information with Microsoft Forms	9
Getting ready • 9	
<i>Summer basketball camp registration • 10</i>	
How to do it... • 10	
How it works... • 12	
Setting up a cloud data source – Azure SQL Database	12
Getting ready • 12	
How to do it... • 12	
How it works... • 14	
 Chapter 2: Building Pixel-Perfect Solutions with Canvas Apps	 17
Discovering best practices when building canvas apps	18
Variable naming • 18	
Control usage • 18	
Coding methodology • 19	
Copilot assistance • 20	
There's more... • 20	
Creating an incident tracking solution – setting up the data source	20
Getting ready • 20	
How to do it... • 21	
How it works... • 22	
Creating an incident tracking solution – building the user interface	23
Getting ready • 23	
How to do it... • 23	
How it works... • 25	
There's more... • 27	
Improving SharePoint list forms with the help of Power Apps	27
Getting ready • 27	
How to do it... • 28	
How it works... • 29	
There's more... • 31	

Creating a canvas app from existing data 31

- Getting ready • 31
- How to do it... • 32
- How it works... • 36
- There’s more... • 37

Chapter 3: Building from Data with Model-Driven Apps 39

Technical requirements 40

- Licensing requirements • 40

Exploring Dataverse 40

Configuring your environment 41

- Getting ready • 41
- How to do it... • 42
- How it works... • 45

Building a Power Platform solution 45

- Getting ready • 45
- How to do it... • 46
- How it works... • 46

Defining data structures 47

- Getting ready • 47
 - Ticket* • 47
 - Ticket operation* • 47
 - Project* • 48
 - Account* • 48
- How to do it... • 48
- How it works... • 53
- There’s more... • 54

Building the model-driven app 54

- Getting ready • 54
- How to do it... • 55
- How it works... • 56

Setting up business process logic	56
Getting ready • 56	
How to do it... • 57	
How it works... • 59	
Designing dashboards to visualize data	61
How to do it... • 61	
How it works... • 63	
There's more... • 64	
Integrating canvas apps inside model-driven apps	64
How to do it... • 64	
How it works... • 66	
Improving model-driven apps with custom pages	67
Getting ready • 67	
How to do it... • 68	
How it works... • 73	
There's more... • 73	
Deploying your solution to your production environment	74
Getting ready • 74	
How to do it... • 74	
How it works... • 76	
There's more... • 76	
Discovering the Copilot experience when building model-driven apps	77
Getting ready • 77	
How to do it... • 78	
How it works... • 79	
 Chapter 4: Choosing the Right Data Source for Your Applications	 81
Technical requirements	82
Data sources and the licensing model	82
Power Apps/Power Automate for Microsoft 365 • 84	

Choosing the right data source – real-world tests	85
Getting ready • 85	
How to do it... • 85	
How it works... • 89	
There's more... • 90	
<i>SharePoint Online performance considerations • 90</i>	
Reducing your application complexity by using Azure SQL database views	91
Getting ready • 91	
How to do it... • 91	
<i>Provisioning the data source • 92</i>	
How it works... • 95	
There's more... • 95	
Integrating on-premises data with canvas apps	96
Getting ready • 96	
How to do it... • 97	
How it works... • 100	
Improving application speed and responsiveness by using static data	101
Getting ready • 101	
How to do it... • 101	
How it works... • 101	
There's more... • 102	
Consuming external data by using Dataflows	102
Getting ready • 103	
How to do it... • 103	
How it works... • 108	
There's more... • 109	
 Chapter 5: Automating Processes with Power Automate	 111
Technical requirements	112
The different types of flows	112
Automated cloud flow • 112	

Instant cloud flow • 112	
Scheduled cloud flow • 113	
Business process flow • 113	
Desktop flow • 113	
Creating a sales survey solution with Microsoft Forms	114
Getting ready • 115	
How to do it... • 115	
How it works... • 119	
There's more... • 120	
Building a file processing automation for SharePoint libraries	120
Getting ready • 120	
How to do it... • 121	
How it works... • 126	
There's more... • 127	
Overcoming Power Apps delegation with Power Automate	127
Getting ready • 128	
How to do it... • 129	
How it works... • 133	
There's more... • 134	
Creating a modular solution with Power Automate	135
Getting ready • 135	
How to do it... • 136	
How it works... • 140	
There's more... • 141	
Automating applications using desktop flows	142
Getting ready • 143	
How to do it... • 144	
How it works... • 147	
There's more... • 147	
Integrating Google Forms with Dataverse using cloud flows	148
Getting ready • 148	

How to do it... • 149	
How it works... • 152	
There's more... • 152	
Chapter 6: Extending the Platform	153
Technical requirements	154
User interface guidelines	154
Setting up your first canvas app component	155
Getting ready • 156	
How to do it... • 156	
How it works... • 158	
There's more... • 160	
Building a floating action button component	160
How to do it... • 160	
How it works... • 162	
There's more... • 162	
Creating a vertical navigation component	163
How to do it... • 164	
How it works... • 168	
There's more... • 169	
Using Power Apps component libraries	169
Getting ready • 170	
How to do it... • 170	
How it works... • 172	
There's more... • 173	
Expanding communications with custom connectors	173
Getting ready • 174	
How to do it... • 174	
How it works... • 179	
There's more... • 180	

Integrating map capabilities into canvas apps	180
Getting ready • 181	
How to do it... • 182	
How it works... • 184	
There's more... • 184	
Extending canvas solutions using mixed reality	185
Getting ready • 185	
How to do it... • 186	
How it works... • 189	
There's more... • 190	
 Chapter 7: Improving User Experience	 191
Technical requirements	192
Enhancing the design of our application interfaces	192
Getting ready • 193	
How to do it... • 193	
<i>Improving the Browse screen • 193</i>	
<i>Refining the Detail screen • 194</i>	
<i>Finishing touches for the Edit screen • 196</i>	
How it works... • 197	
There's more... • 198	
Building a responsive password manager in Power Apps – setting up the Azure Key Vault service	199
Getting ready • 199	
How to do it... • 199	
How it works... • 200	
Building a responsive password manager in Power Apps – designing an adaptive application	201
Getting ready • 201	
How to do it... • 201	
How it works... • 207	
There's more... • 209	

Chapter 8: Power Apps Everywhere	211
Technical requirements	212
Discovering applications through Power Apps mobile	212
How to do it... •	212
How it works... •	214
There's more... •	215
Leveraging modern browsers for Power Apps	216
Getting ready •	216
How to do it... •	216
<i>Microsoft Edge</i> •	217
<i>Google Chrome</i> •	218
<i>On both browsers</i> •	218
How it works... •	218
There's more... •	219
Improving SharePoint document libraries with Power Apps	220
Getting ready •	220
How to do it... •	221
How it works... •	223
There's more... •	225
Embedding Power Apps in SharePoint pages	225
Getting ready •	225
How to do it... •	225
How it works... •	228
There's more... •	228
Making Power BI reports interactive with embedded Power Apps	229
Getting ready •	229
How to do it... •	229
<i>Installing and configuring Power BI Desktop</i> •	229
<i>Building the Power App</i> •	232
How it works... •	234

Working with Power Apps in Microsoft Teams channels	234
Getting ready • 234	
How to do it... • 235	
<i>Building the canvas app • 235</i>	
<i>Microsoft Teams integration • 241</i>	
How it works... • 242	
There's more... • 243	
Integrating Power Apps inside Teams	243
Getting ready • 244	
How to do it... • 245	
<i>Getting application identifier • 245</i>	
<i>Configuring Microsoft Teams policies • 245</i>	
<i>Building a secure app registration • 246</i>	
<i>Automatic deployment of Teams • 248</i>	
How it works... • 250	
There's more... • 251	
Building apps with Dataverse for Teams	252
Getting ready • 252	
How to do it... • 253	
How it works... • 258	
There's more... • 260	
 Chapter 9: Empowering your Applications with AI Builder	 263
Technical requirements	263
Creating a customer success solution using sentiment analysis	264
Getting ready • 264	
How to do it... • 264	
<i>Creating the SharePoint list • 265</i>	
<i>Building the complaints form • 266</i>	
<i>Building the cloud flow • 267</i>	
<i>Building the canvas app • 267</i>	

How it works... • 270	
Building a text recognition system using canvas apps	271
Getting ready • 272	
How to do it... • 272	
<i>Building the app • 272</i>	
How it works... • 275	
Gathering insights from data with the Copilot Answer component	277
Getting ready • 278	
How to do it... • 278	
How it works... • 279	
There's more... • 280	
Using Power Automate to create an invoice classification system	280
Getting ready • 280	
How to do it... • 281	
<i>Configuring the AI model • 281</i>	
<i>Building the cloud flow • 284</i>	
How it works... • 288	
There's more... • 289	
 Chapter 10: Discovering the Power Platform Admin Center	 291
Technical requirements	291
Managing Power Platform environments	292
How to do it... • 293	
<i>Creating an environment without a database • 293</i>	
<i>Creating an environment with a database • 295</i>	
How it works... • 295	
There's more... • 296	
Gathering Power Platform analytics	297
Getting ready • 297	
How to do it... • 297	
<i>Power Platform admin center • 297</i>	

- Power Apps maker portal* • 298
- SharePoint list Power Apps* • 299
- Power Automate* • 299
- How it works... • 300
- There’s more... • 301
- Handling connectors through data policies 301**
 - Getting ready • 302
 - How to do it... • 302
 - How it works... • 303
 - There’s more... • 304
- Chapter 11: Tips, Tricks, and Workarounds 307**
- Technical requirements 307**
- Sharing OneDrive data between tenants 308**
 - Getting ready • 308
 - How to do it... • 309
 - Uploading the data* • 309
 - Building the cloud flow to get Excel data* • 309
 - Building the cloud flow to communicate with Power Apps* • 311
 - Building the app* • 311
 - How it works... • 313
- Playing with vectors – SVG in canvas apps 314**
 - How to do it... • 315
 - How it works... • 317
 - There’s more... • 318
- Transferring SharePoint list apps from one site to another 318**
 - Getting ready • 319
 - How to do it... • 319
 - Building and customizing the list on the source site* • 319
 - Exporting the app* • 320
 - Configuring the destination site* • 321

- Transforming the exported package* • 323
 - Importing the new package* • 324
 - How it works... • 325
 - There's more... • 326
- Troubleshooting using Power Apps Monitor 326**
 - Getting ready • 326
 - How to do it... • 326
 - Provisioning the SharePoint list* • 326
 - Building the app* • 327
 - How it works... • 329
 - There's more... • 331
- Extending screen real estate using the canvas control 331**
 - Getting ready • 331
 - How to do it... • 331
 - How it works... • 335
- Updating Azure SQL Server connections in Power Apps 336**
 - Getting ready • 337
 - Azure SQL databases* • 337
 - How to do it... • 337
 - Creating the environments* • 338
 - Building the app* • 338
 - Transforming the exported package* • 340
 - How it works... • 342
 - There's more... • 343
- Renaming files in SharePoint document libraries 343**
 - Getting ready • 344
 - How to do it... • 344
 - Creating the SharePoint document library* • 344
 - Building the cloud flow* • 345
 - Configuring the document library* • 347
 - How it works... • 347

Chapter 12: Advanced Techniques with the Power Apps Component Framework	351
Technical requirements	352
Configuring your development environment	352
Getting ready • 353	
How to do it... • 353	
<i>Node.js/npm • 353</i>	
<i>Editor – VS Code • 354</i>	
<i>Editor – Visual Studio 2017 or later • 354</i>	
<i>Microsoft Power Platform CLI • 355</i>	
<i>Configuring the Power Platform environment • 355</i>	
How it works... • 355	
There's more... • 356	
Setting up Font Awesome	356
How to do it... • 357	
How it works... • 358	
Creating the project	358
Getting ready • 358	
How to do it... • 358	
How it works... • 360	
Building the application logic	361
How to do it... • 361	
How it works... • 363	
There's more... • 363	
Deploying the solution	363
How to do it... • 364	
<i>Building the solution • 364</i>	
<i>Deploying to the environment • 364</i>	
How it works... • 365	

Testing the component on a canvas app 365

- How to do it... • 365
- How it works... • 367
- There’s more... • 368

Chapter 13: Reaching Beyond the Organization with Power Pages 369

Technical requirements 370

Setting up Power Pages 370

- Getting ready • 370
- How to do it... • 371
- How it works... • 371
- There’s more... • 372

Extending the Dataverse solution 372

- Getting ready • 372
- How to do it... • 372
- How it works... • 374

Building the help desk interface 374

- Getting ready • 375
- How to do it... • 375
- How it works... • 378
- There’s more... • 379

Configuring user access and security 379

- Getting ready • 380
- How to do it... • 380
 - Setting up permissions • 380*
 - Securing page access • 382*
- How it works... • 383
- There’s more... • 383

Automating record attributes 384

- Getting ready • 384
- How to do it... • 384

How it works... • 386	
There's more... • 386	
Adding custom fonts to the portal	387
Getting ready • 387	
How to do it... • 388	
<i>Preparing the local version of the portal • 388</i>	
<i>Using preinstalled fonts • 389</i>	
<i>Using fonts from the Google Fonts repository • 390</i>	
How it works... • 391	
There's more... • 392	
Other Books You May Enjoy	395
Index	399

Preface

Power Apps is a low-code platform for building applications from Microsoft. With this platform, you can create solutions to your business needs while integrating with other components of the Power Platform, such as Power Automate and Power BI.

Quite different from the complete online documentation, you will find that this is not your regular reference book. Instead, this book exposes real-world scenarios and experiences to help you get a headstart in your Power Apps projects.

By exploring a carefully selected collection of chapters, you will delve into various aspects of Power Apps. This includes learning the fundamentals of creating canvas apps, designing model-driven solutions, enhancing apps with custom connectors, and integrating with other platforms. You will also gain insight into the pro-developer aspects, such as the Power Apps Component Framework and developing website experiences for external users with Power Pages.

Who this book is for

Since we cover practical use cases, basic knowledge of building applications using Power Apps is required to take advantage of the solutions explored in this book. You will get a step-by-step tutorial on building the recipes crafted for each chapter.

What this book covers

Chapter 1, App Maker Basics, serves as the base instructions for building the subsequent recipes. Instead of repeating these steps in each recipe, we concentrate them in to one chapter for tasks such as creating a Dataverse solution or building a SharePoint list.

Chapter 2, Building Pixel-Perfect Solutions with Canvas Apps, starts with the best practices for building canvas apps and continues with creating sample solutions, including coverage of the data source handling.

Chapter 3, Building from Data with Model-Driven Apps, continues our journey by following a collection of recipes joined together to create an all-around help desk solution. We will also explore the Entity Relationship Diagram capabilities from the maker Copilot in Power Apps.

Chapter 4, Choosing the Right Data Source for your Applications, will help you to make a sound decision when determining the data source of your applications. This chapter also explains the importance of the licensing model on this platform.

Chapter 5, Automating Processes with Power Automate, focuses on several use cases, from inside and outside the Microsoft ecosystem, to improve business processes using this component of the Power Platform.

Chapter 6, Extending the Platform, builds upon the concept of enhancing the application-building process by using components. We will also learn how to extend the platform by creating custom connectors, using geospatial capabilities, and integrating mixed reality.

Chapter 7, Improving User Experience, looks at how to enrich your user interfaces to make your applications more appealing to end users. This chapter will also cover how to create a responsive application using the latest techniques available in Power Apps.

Chapter 8, Power Apps Everywhere, explains all the possible ways to use and integrate Power Apps on many platforms, from mobile device consumption to embedding scenarios on SharePoint and Power BI, without forgetting the latest on Microsoft Teams development, including Dataverse for Teams.

Chapter 9, Empowering Your Applications with AI Builder, focuses on using AI Builder solutions to improve our applications and processes by bringing artificial intelligence into the mix, including Copilot.

Chapter 10, Discovering the Power Platform Admin Center, is about learning how to manage the Power Platform using the tools and settings available in this admin center.

Chapter 11, Tips, Tricks, and Workarounds, offers a collection of hints from the application-building experience in Power Apps with a set of topics that will help solve or improve a wide variety of scenarios.

Chapter 12, Advanced Techniques with Power Apps Component Framework, tackles the pro-developer side by building a Power Apps Component Framework component from scratch. We will look at setting up our development environment, deploying the component to an environment, and using it on a canvas app.

Chapter 13, Reaching Beyond the Organization with Power Pages, introduces the Power Pages component. It offers a solution to allow external users to interact with our Dataverse data. This chapter will extend the help desk solution built in previous chapters.

To get the most out of this book

Before diving into building solutions with Power Apps, you are going to need a Microsoft 365 subscription.

You might have one already from your organization, but if you want to have a playground to build apps, I'm going to give you two suggestions:

- **Microsoft 365 Developer Program**

This program allows you to have a Microsoft 365 subscription with many features available: 25 E5 user licenses, apps such as SharePoint, learning resources, and more. It's the perfect sandbox environment to create your apps, not only to build Power Apps but also to learn other technologies from the whole Microsoft 365 ecosystem. This subscription renews automatically every three months as long as you actively use it.

To get more information, please refer to <https://developer.microsoft.com/en-us/microsoft-365/dev-program>.

- **Power Apps Developer Plan**

This option is ideal if you want a more focused approach to the Power Platform. It offers a free environment for individual use with the same advantages as a paid plan, including premium connectors. However, some restrictions exist, such as app sharing and the need for a Microsoft organizational account.

This subscription has no renewal process; it's perpetual. For more information, please visit <https://www.microsoft.com/en-us/power-platform/products/power-apps/free>.

Power Platform licensing requirements

Power Apps paid plans

The licensing model on this platform depends on the type of connector needed for your data sources. Standard connectors, such as the one used for SharePoint, don't require an additional license besides Microsoft 365, but premium or custom connectors do require a Power Apps license:

- **Per app plan**

Allows running one app (canvas or model-driven) or portal per user.

- **Per user plan**

Allows building and using unlimited apps and portals (within service limits).

- **Pay-as-you-go plan**

Allows a per user plan that charges for the number of apps or portals runs by a user each month (note that this plan requires an Azure subscription).

To get more insight into the licensing model, please visit <https://www.microsoft.com/en-us/power-platform/products/power-apps/pricing/>.

Power Automate paid plans

In Power Automate, three licensing plans allow access to data using premium connectors, custom connectors, and on-premises data. The difference lies in the capacity of these plans and who will be the end-user of them:

- There are two user plans: the **Power Automate Premium** and the **Power Automate Hosted Process**. Both let you create unlimited flows, including AI builder service credits, but the latter adds unattended robotic process automation in a hosted virtual machine.
- On the other hand, the **Power Automate Process** licenses a bot that runs unattended robotic process automation.

Using either of these options depends entirely on the analysis of the business process that requires automation. For detailed information about the pricing of these plans, please refer to <https://www.microsoft.com/en-us/power-platform/products/power-automate/pricing>.

Download the example code files

The code bundle for the book is hosted on GitHub at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition>. We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Download the color images

We also provide a PDF file with color images of the screenshots/diagrams used in this book. You can download it here: <https://packt.link/gbp/9781835465158>.

Conventions used

There are a number of text conventions used throughout this book.

Code in text: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, and user input. Here is an example: “Repeat *step 9* for the Ticket Priority column, using the following choices: **Low**, **Medium**, and **High**.”

A block of code is set as follows:

```
Filter(  
    vw_TicketData,  
    customer_id = CustomersCbx.Selected.id  
)
```

Any command-line input or output is written as follows:

```
Set-ExecutionPolicy -ExecutionPolicy Bypass
```

Bold: Indicates a new term, an important word, or words you see onscreen. For example, words in menus or dialog boxes appear in the text like this. An example is: “Choose a name, a region close to you, and describe the purpose. For the **Type**, you can select **Production**, **Sandbox**, or **Trial**”



Warnings or important notes appear like this.



Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, mention the book title in the subject of your message and email us at customer care@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit <http://www.packtpub.com/support/errata>, selecting your book, clicking on the **Errata Submission Form** link, and entering the details.

Piracy: If you come across any illegal copies of our works in any form on the Internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com>.

Leave a review!

Thank you for purchasing this book from Packt Publishing—we hope you enjoy it! Your feedback is invaluable and helps us improve and grow. Once you’ve completed reading it, please take a moment to leave an Amazon review; it will only take a minute, but it makes a big difference for readers like you.

Scan the QR code below to receive a free ebook of your choice.



<https://packt.link/NzOWQ>

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily.

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below:



<https://packt.link/free-ebook/9781835465158>

2. Submit your proof of purchase.
3. That's it! We'll send your free PDF and other benefits to your email directly.

1

App Maker Basics

When comparing technical and culinary cookbooks, you will find certain similarities. Both are based on recipes that introduce the reader to creating something by following detailed instructions.

For example, in Italian culinary books, there is often a chapter describing how to prepare the sauce that many recipes will use throughout the book. The same concept applies to this cookbook. This chapter will explain the basic requirements for the most common scenarios when working on Power Platform recipes.

The following recipes will serve as a template for specific situations throughout the book:

- How to build a Power Platform solution
- Preparing tables in Dataverse
- Building metadata-enabled storage – SharePoint lists and libraries
- Gathering information with Microsoft Forms
- Setting up a cloud data source – Azure SQL Database

Technical requirements

Depending on the data source you'll be working with, you might need proper licensing. Start from the Microsoft 365 Business Basic license for SharePoint through to the Power Apps Premium needed for Dataverse. Please refer to the *Preface* to get more information about the licensing requirements.

How to build a Power Platform solution

A Power Platform solution consists of a package that stores all the pieces that compose the solution you've created to solve your business need. It aids in deployment, distribution, and management. Examples of these pieces can be Power Automate cloud flows, model-driven apps, Power BI reports, or Dataverse tables.

Creating a solution is a straightforward process, but that doesn't mean you don't need to follow specific steps to include best practices for your solutions.

For example, if you need to interact with a particular service, the platform uses a connection reference. When working with solution components such as cloud flows, the platform will try to help you by creating connection references for you but using random naming. It becomes even more complicated if multiple solutions exist within a specific environment. One component from one solution might use a connection reference from another without you noticing.

Setting up the connection references before creating components can solve these issues easily. This way, the platform will automatically use the solution's connection references and the naming pattern you decide.

Getting ready

After creating the solution, we will need to choose a data source to define the steps required to configure a connection reference. For this example, we will use SharePoint.

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com>, select **Solutions** from the left pane, and click + **New solution** from the toolbar.
2. Before describing the solution, click + **New publisher** and enter the display name and name in the **Display name** and **Name** fields respectively. Enter a meaningful two-to-eight-letter **Prefix** value that will be used for all the elements you will create. Check the **Preview of new object name** for an example of how a component will be named. Keep the default **Choice value prefix** and click on **Save**.
3. Back in the **New solution** pane, enter the desired **Display name** and **Name** for the solution, select your newly created **Publisher**, and click on **Create**:

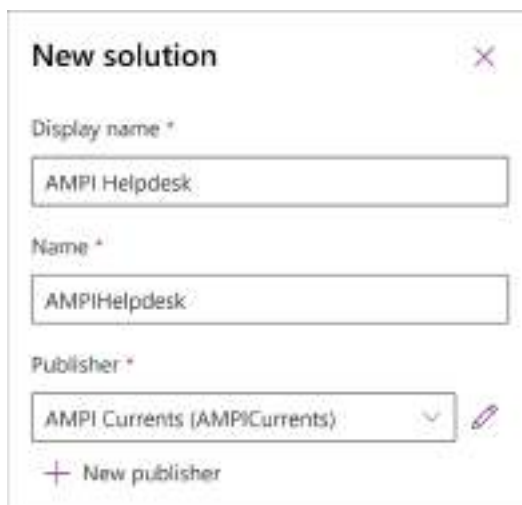


Figure 1.1: Solution settings

4. Once the solution is created, from the toolbar, click **New | More | Connection Reference**.
5. From the panel that opens, set the **Display name** to **AMPI-SharePoint**, leave the **Name** as it is, and choose **SharePoint** from the **Connector** dropdown. If you already have a connection, select it. If not, create one with your credentials by selecting **New connection**.
6. Repeat *Steps 4* and *5* for any other service your solution might require.

How it works...

This creation process will prepare the solution and open it automatically. The SharePoint connection reference will be available to every component that can use it, such as a cloud flow or a canvas app. This way, we will get a more organized solution for our projects.

Regarding the publishers, from the left pane, click **More, Discover all**, and then on **Publishers** inside the **App Management** section to see the list of all publishers included in the current environment.

Preparing tables in Dataverse

Tables are essential to every data source since they represent the backbone of the data structure. Their designs include column definitions, indexing mechanisms, relationship configurations, and more.

Almost every Power Platform solution requires related tables to support the business process, and that's why Microsoft Dataverse is a compelling choice. It was created as a business-oriented relational database.

To discover more about the distinctive features that set Microsoft Dataverse apart, please visit <https://learn.microsoft.com/en-us/power-apps/maker/data-platform/why-dataverse-overview>.

In this recipe, we will learn how to build tables for this database. This will become a standard task for many projects, such as the one showcased in *Chapter 3, Building from Data with Model-Driven Apps*, where we will create a help desk solution based on Dataverse.

Getting ready

Each table on a database serves a specific purpose. For this example, we will create a table that will hold the data of all the members of a girls' youth basketball team.

Team players

Display name	Data type	Format
Full name	Text Plain text (Single line of text)	Text
Jersey number	Number Whole number	None
Position	Choice Choice	-
Gear size	Single line of text	Text
Emergency contact information	Multiple lines of text	Text

Table 1.1: Team players table columns

How to do it...

1. Go to the Power Apps maker portal and open a solution from the list of solutions for the current environment.
2. From the toolbar, click **New | Table | Table**. Set the **Display name** to **Team player**. The **Plural name** will change to **Team players** automatically. Depending on your table's needs, you might want to **Enable attachments**, for example, including extra information about the team player, such as parental consent or team enrollment.

3. Click on the **Primary column** tab. Enter **Full name** as the **Display name**. Remember to choose something your end users can use to select the records when listing them.
4. Once you have entered all the required information, click **Save**. The system will start creating the table with the specified primary column and the rest of the business-oriented columns. The table will open automatically once finished.
5. Click on **New | Column** from the toolbar. For the **Display name** field, enter **Jersey number**, set the **Data type** to **Number | Whole number**, and click **Save**.
6. Click **New | Column** again to include the **Position** column. For **Data type**, select **Choice | Choice**. Under the **Sync this choice with** dropdown, click on **New choice**:
 - a. Set **Position** as the **Display name**, and under the **Choices** section, add the term **Point Guard**.
 - b. Click on **New choice** to add each of these items: **Shooting Guard**, **Small Forward**, **Power Forward**, **Center**.
 - c. Click **Save**.
7. Back on the column configuration, select the previously created choice from the **Sync this choice with** dropdown, and click **Save** to continue.



Using global choices is a great way to reuse a Choice column for different tables if they are going to hold the same elements.

8. Repeat *Step 5* to add the **Gear size** column using **Single line of text** as the **Data type** and **Text** as the **Format**.
9. Repeat *Step 5* to include the **Emergency contact information** column using **Text | Plain text** from the **Multiple lines of text** section of the **Data type** dropdown. Leave **Text** as the **Format**.

How it works...

Now that we have added all the required columns, the table can be used in our solutions. The following is an example of the Team player table:

Dino Shakers > Tables > Team player

Table properties

Properties

Tools

Schema

Name

Team player

Type

Standard

Primary column

Full name

Last modified

20 minutes ago

Description

Columns

Relationships

Keys

Team player columns and data

Team player	Full name	Jersey number	Status	Version N...
2702a13e-4f52-ef11...	Melini	2	Active	3,339,711
674ee234-4f52-ef11...	Noe	7	Active	3,339,720
e890e72e-4f52-ef11...	Izumiya	9	Active	3,339,717
34cbe205-4f52-ef11...	Amanda	19	Active	3,339,701
30104318-4f52-ef11...	Alba	20	Active	3,339,712
d0b69d9f-4f52-ef11...	Ana Karina	21	Active	3,339,704
7b42a11e-4f52-ef11...	Marta	33	Active	3,339,708
Enter text	Enter number			

Figure 1.2: Team player table base structure

Building metadata-enabled storage – SharePoint lists and libraries

SharePoint Online, included in Microsoft 365, provides the perfect platform for secure content management, collaboration, and communication through dedicated sites.

This platform is commonly used as a data source for Power Apps solutions since it provides Microsoft Lists. This feature allows you to build a table-like storage with rich metadata columns. It's not comparable to a full-fledged relational database, but it's simple to use and comes included on most Microsoft 365 licenses. Please refer to *Chapter 4, Choosing the Right Data Source for Your Applications*, where we will perform a data source performance comparison, including SharePoint.

It also provides document libraries, which are file containers that inherit the same capabilities as Microsoft Lists, enhancing categorization and organization of the file storage, which helps improve business processes.

Getting ready

Since we are using SharePoint as our data source, there are no additional requirements besides a regular Microsoft 365 license. We will create a document library with some metadata columns to improve the library's categorization.

Player documents

Here are the metadata columns to be added to the document library:

Type	Name
Choice: Photo ID, Medical certificate, Enrollment form	Document type
Date and time	Expiration date
Yes/No	Requires signature

Table 1.2: Library columns

How to do it...

1. Go to the SharePoint site, which will hold the document library. On the main site page, click on **New | Document library**, as seen in the following screenshot:

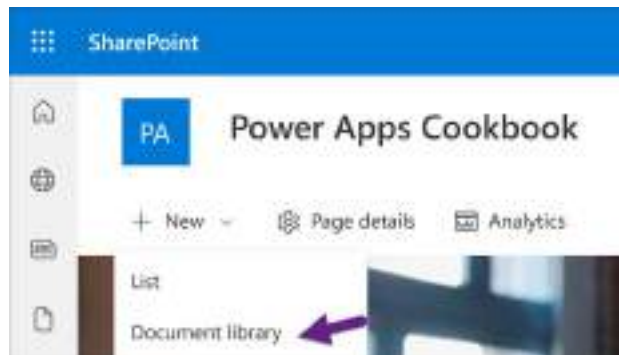


Figure 1.3: Document library creation

This action will open a dialog with options to create the library. Click **Blank library** and then enter the name and description of the library. Enter **Player documents** for the name, leave the rest as default, and click **Create**.



Whether creating lists or document libraries, the platform will try to help you when designing them. Providing options such as **From existing list/library**, **From Excel/CSV** will allow you to speed up the creation process.

Templates is another capability that provides metadata examples gathered from many business scenarios.

2. Once the library opens, click on the **+ Add column** link at the end of the column header, as seen in the following screenshot:

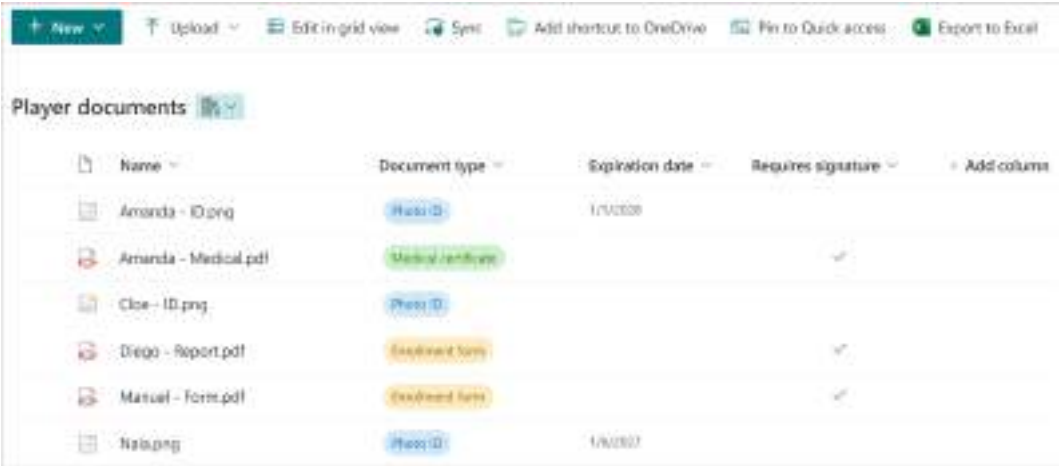


Figure 1.4: The + Add column option

3. Select the **Choice** data type from the open dialog and click **Next**. Enter **Document** type for **Name** and click each of the choice examples to replace them for **Photo ID**, **Medical certificate**, and **Enrollment form**. Once completed, click **Save**.
4. Repeat *Step 2* to add another column. Select **Date and time** and click **Next**. Enter **Expiration date** for the **Name** and leave the rest as default. Click **Save** to finish.
5. Again, repeat *Step 2* to add the last column. Select **Yes/No** and click **Next**. Enter **Requires signature** for the **Name** and click **Save** to complete the process.
6. Back in the document library, reorder the columns as you see fit by dragging them. You can also hide columns by clicking on a particular column and selecting **Columns settings** | **Show/hide columns**.
7. Add files to the library by selecting **Upload** | **Files** from the toolbar or by dragging them. Once the upload is complete, click **Edit in grid view** to update the file's metadata in bulk, like when working with an Excel spreadsheet. Click on **Exit grid view** when finished.

How it works...

The document library creation process is now complete. If you upload some files, your library should look like the one in the following figure:



Name	Document type	Expiration date	Requires signature
Amanda - ID.png	Photo ID	1/1/2028	
Amanda - Medical.pdf	Medical certificate		✓
Cloe - ID.png	Photo ID		
Diego - Report.pdf	Examination form		✓
Manuel - Form.pdf	Examination form		✓
Nate.png	Photo ID	1/6/2027	

Figure 1.5: Player documents library

Gathering information with Microsoft Forms

Web forms are an excellent tool for gathering information. Whether you require feedback from a specific process, an event registration, or even a simple contact form, you must clearly define the fields to collect the data in these forms.

Microsoft Forms gives you an easy way to create them with an easy-to-use interface that allows you to quickly create forms, quizzes, and polls. Most importantly, it offers a way to easily gather information from external users to your organization, as we will discover in *Chapter 9, Empowering Your Applications with AI Builder*, where we will use this tool to gather possible complaints from our customers.

Getting ready

Microsoft Forms is a service that goes beyond Microsoft 365. You can even use it with your personal Microsoft accounts. However, you must create the forms inside your tenant to use it with Power Platform. For this recipe, we will learn the necessary steps to build a form.

Summer basketball camp registration

Questions to be added to the form:

Field type	Required	Text	Extra
Text	Yes	Child’s Full Name	
Text	Yes	Parent / Guardian	
Choice	Yes	Campus options	Choices: 2 weeks regular student price 2 weeks external student price 2 weeks sibling’s price

Table 1.3: Form fields configuration

How to do it...

1. Head to Microsoft Forms by typing this address: <https://forms.microsoft.com>, and once there, click on the **New Form** button. This action will open the designer for you to start setting up the form.
2. Click on the **Untitled form** text to define the name, description, and image. You can also click **Style** on the toolbar to change to one of the built-in themes.

The following is an example using one of the immersive styles:



Figure 1.6: Microsoft Forms style

- Now, let's start including fields. A list of options appears to select the type. You can choose from the following field types:



Figure 1.7: Field types available in Microsoft Forms

- Click **Quick start with** and select **Text**. Change the Question to Child's Full Name and enable the **Required** setting.
- Repeat *Step 4* to add the Parent / Guardian question.
- Again, click on + **Add new question**, but this time, select **Choice**. Replace the Question with Campus options. Replace Option 1 and Option 2 with 2 weeks regular student price and 2 weeks external student price, respectively. Click + **Add option** to include 2 weeks sibling's price.
- Click **Preview** on the toolbar and, depending on the layout and style of the form, click **Start now** afterward. You should have a form just like the following:

A screenshot of a Microsoft Forms preview titled 'Summer basketball camp registration'. The form includes a header with a close button, a disclaimer, and three numbered questions. Question 1 is 'Child's Full Name' with a text input field. Question 2 is 'Parent / Guardian' with a text input field. Question 3 is 'Campus options' with three radio button options: '2 weeks regular student price', '2 weeks external student price', and '2 weeks sibling's price'. A blue 'Submit' button is at the bottom left.

Figure 1.8: Registration preview

8. From the toolbar, click on **Back** to leave the preview mode and click the gear icon to configure who can fill out the form. Regularly, you should select **Anyone can respond**. You can change it as desired.

How it works...

Now that our form is ready, we could start building Power Automate actions that will get triggered when we receive responses.

Setting up a cloud data source – Azure SQL Database

Azure SQL is a suite of SQL database services on the Azure cloud. It offers automated backups, high availability, and advanced security, making it ideal for traditional SQL database needs with cloud scalability and integration.

While Azure SQL provides extensive control for database management, Dataverse offers a higher level of abstraction for faster and easier development. Depending on the needs of your project, sometimes having granular control of your database is a must.

This recipe will help you build an Azure SQL database that might be needed for an app.

Getting ready

To start working with Azure SQL databases, you will need an Azure subscription. You can request a free version with credit for 30 days to test all the services available on this cloud platform. Apply for it at <https://azure.microsoft.com/en-us/free>.

How to do it...

1. Go to the Azure portal at <https://portal.azure.com> and click **Create a resource**. This action will take you to Azure Marketplace. Using the search field, enter **SQL Database** and hit *Enter*.
2. From the list of services, select **SQL Database** and then click **Create**. This action will open the **Create SQL Database** page, as seen in the following screenshot:

Basics Networking Security Additional settings Tags Review + create

Create a SQL database with your preferred configurations. Complete the Basics tab then go to Review + Create to provision with credit details, or visit each tab to customize. [Learn more](#) >

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Database details

Enter required settings for the database, including picking a logical server and configuring the compute and storage resources.

Database name *

Server * [Create new](#)

Want to use SQL elastic pool? ☐ Yes ☒ No

Workload environment: ☒ Development ☐ Production

Compute + storage * [Configure database](#)

Standard server (Serd), 1 vCore, 32 GB storage

Cost summary

General Purpose (GP, 1 Core, 1 vCore)
 Compute (Serd) cost: 0.01\$
 Max storage (server) cost: 0.01\$
 ESTIMATED STORAGE COST / MONTH: 0.00\$
 COMPUTE COST / HOUR: 0.00111\$
 STORAGE COST / HOUR: 0.00111\$

NOTES
 * Some databases are billed in a different way than the ones listed here. See the pricing page for more details.

Figure 1.9: Creating an SQL database from the Azure portal

3. Select your subscription and a resource group (create a new resource group if necessary). Enter the database name in the **Database name** field and select a server.

If you don't have an existing server, click the **Create new** link below the servers list and enter the required information: a server name and a location close to you for better response times. Leave the default **Use Microsoft Entra-only authentication** as the **Authentication method** and click **Set admin** to look for one user, group, or enterprise application to act as the database administrator. After choosing one, click **Select** and then **OK** to create the server.

4. Select **No** for the elastic pool setting and choose **Development** for the **Workload environment** setting. Click **Configure database** on the **Compute + storage** section, change the **Service tier** to **Basic (For less demanding workloads)**, and click **Apply**. The **Estimated storage cost/month** in the **Cost summary** section on the right should be around USD 5.
5. Select **Locally-redundant backup storage** in the **Backup storage redundancy** section.
6. Go to the **Networking** tab and select **Yes to Add current client IP address**.
7. Click the **Additional settings** tab and select **Sample** from the **Use existing data** section. Click **OK** on the dialog that appears.
8. Select the **Review + create** tab and then click on **Create**. This action will start the provisioning of the database.

How it works...

Once the provisioning process is completed, you will be able to access your database.

You can use Azure Data Studio at <https://learn.microsoft.com/en-us/azure-data-studio/download-azure-data-studio> or the Azure portal itself.

Accessing it through the portal is straightforward as you don't need to install any software. Just navigate to your database, and from the left pane, click **Query editor (preview)** and select **Continue as <your username>** from the **Microsoft Entra authentication** section.

After authentication, you will be able to interact with the database, as seen in the following figure:

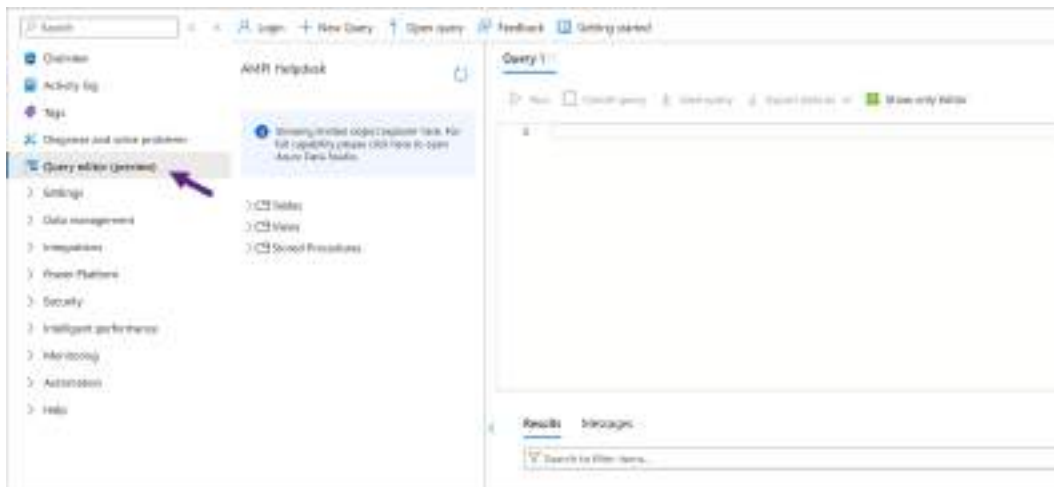


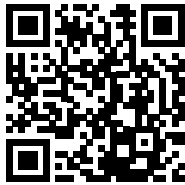
Figure 1.10: Query editor from the Azure portal

This editor will allow you to run scripts from the **Query** tab or upload scripts using the **Open query** option from the toolbar.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



2

Building Pixel-Perfect Solutions with Canvas Apps

Power Apps is the platform of choice for building business solutions using a low-code method. The platform is suitable for individuals within an organization eager to learn about the technology while bringing their business process expertise to the table.

Power Apps' canvas apps allow you to create pixel-perfect implementations of user interfaces. As the name suggests, they offer various tools to build any imaginable design in your applications, allowing you to design and develop apps of any type, whether it is a critical business system or a simple tracking application.

During this chapter, we will discover how to create a set of applications that will give you an insight into the different approaches to building canvas apps. We will also learn how to set up various data sources for our applications.

This chapter consists of the following recipes:

- Discovering best practices when building canvas apps
- Creating an incident tracking solution – setting up the data source
- Creating an incident tracking solution – building the user interface
- Improving SharePoint list forms with the help of Power Apps
- Creating a canvas app from existing data

Discovering best practices when building canvas apps

Setting up data sources, defining business process flows, and creating user interfaces – all these tasks are pieces of an application-building process. These pieces come together to accomplish one main goal: to build a solution that solves a specific need.

One of the things you need to consider is the maintainability of your app. Whether fixing bugs or adding new features, using best practices is always a good idea. No matter your technology, well-documented code is easier to maintain.

Even though Power Apps is a low-code platform, you must consider certain things before building applications. Like any other developer team, you need to establish code standards. These standards allow your team to be more productive by setting predefined patterns for variable naming, control usage, and coding methodology.

Variable naming

Proper naming gives your developers instant insight into the scope of your variables. For example, if a variable name prefix starts with `lc1` (short for **local**), it means its value will only be available on the current screen. On the other hand, using `gb1` (short for **global**) means that this variable is accessible across the whole application.

These examples might seem trivial, but if another developer needs to maintain your app or your app serves as a template for other apps in your organization, setting these patterns from the start can help the application-building process and maintainability.

Control usage

One key point of an application's success is performance. If an application starts slowly or takes several seconds to perform a task, it hurts user adoption.

Here are a couple of examples of this:

- A great-looking app that uses many controls to build its user interface but hurts performance each time the screens are rendered.
- Displaying data to a user using a gallery inside a gallery. This approach might be tempting to present master-detail data, but this would be a significant slowdown in your application.

To avoid this, you need to learn the performance points of your platform. In Power Apps, one of the main recommendations for improving responsiveness is to reduce the number of controls.

Make sure you keep an eye on the built-in **App checker** to prevent any issues that your app might encounter.

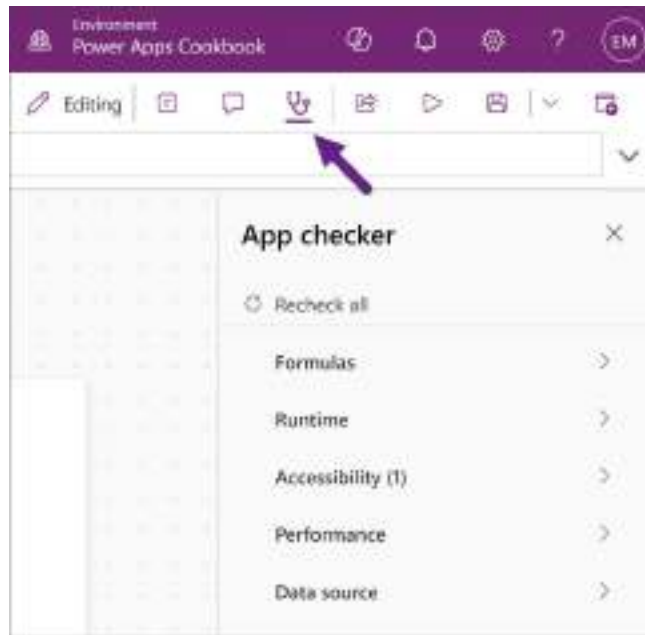


Figure 2.1: App checker inside Power Apps Studio

Coding methodology

This concept describes a set of rules to regulate the development process in a low-code team. Your solution infrastructure can also help you make an informed decision on how to build your application:

- If you have data that rarely changes, you can create collections to avoid round-trips each time data is needed. Even more, if you have data that never changes, you can import this as static data inside your app for speedier access. Refer to *Chapter 4, Choosing the Right Data Source for Your Applications*, for a recipe example on this.
- Taking advantage of the features available in your data source can significantly improve your application performance. For example, when using Dataverse or any other relational database, there is a significant difference when querying data if you use data source views instead of building the actual query in your application logic, especially if it needs complex relationships. Using these views gives you cleaner code while also enhancing performance. This improvement relies on the fact that the data source engine is the one executing the data processing instead of the application itself.

Copilot assistance

When starting with canvas apps, you will find that application templates can help you learn how to build your application's logic by taking advantage of all the capabilities of Power Fx used in those templates.

In other cases, you might be in charge of maintaining a complex application in your organization. For these cases, you leverage Copilot's formula assistance to learn what is actually happening on a particular formula. Just click the Copilot icon and select **Explain this formula** to get insight into a specific piece of Power Fx code.

There's more...

You can coordinate these practices with your team when building apps. For a detailed list of best practices, please refer to <https://www.microsoft.com/en-us/power-platform/blog/power-apps/powerapps-canvas-app-coding-standards-and-guidelines>.

Creating an incident tracking solution – setting up the data source

Tracking processes is a necessity for customers everywhere. As in any development process, the first step is to gather all the requirements needed to fulfill the business need; this will help us design the data structure to support our application.

As for the data source, this recipe will set up the required fields in a SharePoint list while also applying specific settings to make it as performant as it should be.

Getting ready

Before diving into developing solutions with Power Apps, you will need a Microsoft 365 subscription and, depending on the required features, an independent Power Platform license. Please refer to the *Preface* section for suggestions on getting a playground to build your apps.

We will use SharePoint as the data source for this recipe, so please refer to *Chapter 1, App Maker Basics* for detailed instructions on how to handle SharePoint lists and libraries. We will need two lists: Client and Incident. The structure is as follows:

Column data type	Column name	Required	Extra
Multiple lines of text	Address	No	-
Text	Phone	No	-
Text	Email	No	-

Table 2.1: Client list columns

Column data type	Column name	Required	Extra
Choice	Priority	Yes	High, Medium, and Low
Lookup	Customer	Yes	Source list: Customers
Date and time	Date	Yes	-
Multiple lines of text	Comments	No	-

Table 2.2: Incident list columns

The list creation process is pretty straightforward, leaving plenty of time for our application design.

How to do it...

1. Go to your desired SharePoint site and create the Client list.
2. Once created, click the down arrow next to the Title column and select **Column settings | Rename**. Set it as Name and click **Save**. We are doing this for user interface reasons. This column lets you open the selected record quickly from the list view. We will set it as the client's name.
3. Now, create the Incident list. Rename the **Title** column Incident and set the rest of the columns as mentioned in the *Getting ready* section.



The **Choice** type means that it will use a list of items to choose from, and the **Lookup** type indicates that this column will link to another list that holds another set of data. In this case, we will use this column to relate the incidents to the clients.

Now that we have configured our lists, let's start using them.

How it works...

After setting up our data source, we can enter some test data for our app:

1. Click on the gear icon in the top-right corner, select **Site contents**, and then select the Customers list:

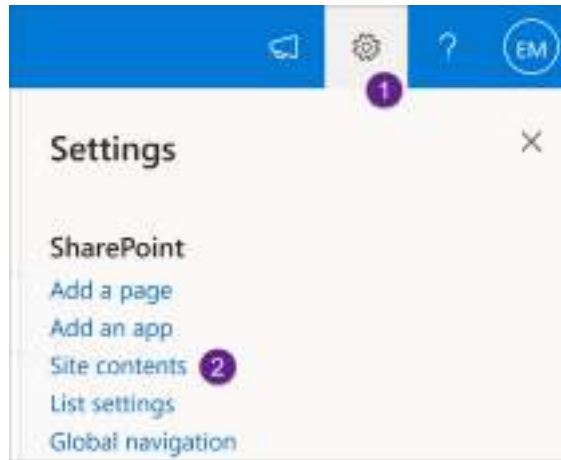


Figure 2.2: Site contents option

2. Now, let's click on + **New** to add some random test data:
 - **Name:** Yennu Enterprises
 - **Address:** One Yennu Road
 - **Phone:** +1 555 800 5555
 - **Email:** sales@yennu.com
3. Add as much data as you like. Repeat the same steps with the Incident list by clicking the gear icon and selecting the list from **Site contents**.

4. For the **Customer** column, you will see data loaded from the Customers list, and the **Priority** column will let you choose from the previously defined elements:
 - **Incident:** Failure in the programming of the executive elevator
 - **Customer Name:** Yennu Enterprises (link)
 - **Date:** 05/02/2024
 - **Priority:** High
 - **Comments:** The elevator returns itself to the last floor when idle
5. Again, add as much data as you want. Now, we will move on to the next section, where we will build an app from this data structure.

Creating an incident tracking solution – building the user interface

Before designing interfaces, we must ask ourselves which requirements our application will solve. This analysis will help us decide which technology to use, what infrastructure holds every piece of our solution, and even how our end users will consume it.

Getting ready

With the help of the previous recipe, we have solved the data source section of our infrastructure. It's time to leverage Power Apps to automate creating the building blocks of our application.

Power Apps deeply integrates with SharePoint as a tool to build solutions rapidly with its low-code principles. This integration allows the creation of a completely functional application in a few steps.

In this recipe, we will discover the classic approach to developing apps with SharePoint. The evolution of the platform allows newer ways to tackle these tasks. Please refer to the *Creating a canvas app from existing data* recipe to learn the modern approach of building apps from SharePoint. Keep in mind that this approach doesn't allow you to choose the environment you will be working on; it creates the application in the default environment.

How to do it...

1. Go to the SharePoint site that has your lists. Click the gear icon in the top-right corner, select **Site contents**, and then select the Incident list.

2. On the list's toolbar, select **Integrate | Power Apps | Create an app**. This action will transfer you to Power Apps Studio:

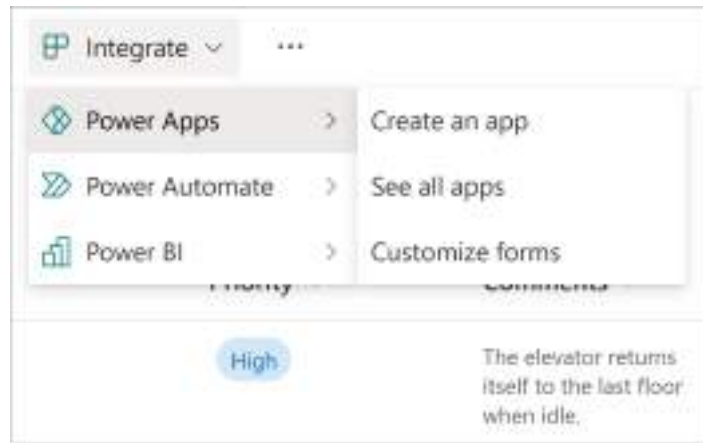


Figure 2.3: Power Apps integration inside SharePoint

3. After a little while, a brand-new application is created with the base functionalities: data manipulation, search, listing, and sorting. It's now time to polish the application to have the desired results.
4. First, let's edit the application title to be different than the list's name. Go to **Tree view**, expand **BrowseScreen1**, and select the **LblAppName1** label. Change the label's value on the right pane by changing the **Text** value. Do the same for the rest of the screens.
5. Next, let's improve **EditScreen1** to allow more space for the comments data card. At this moment, we won't be using the attachments data card, so by expanding **EditForm1** and selecting **Attachments_DataCard1**, we can uncheck its **Visible** property on the properties pane.
6. Increase the **Comments_DataCard2**'s **Height** value by setting the **Y** property to `Parent . Height - Self.Y` using the formula bar; this will change the size to the available screen space.
7. To match the new size, do the same for the **DataCardValue10** control using this formula on the **Height** property: `Parent.Height - Self.Y - 30`

8. Set the **Mode** property to **Multiline** to fix the text's vertical alignment.

The screenshot shows a form titled 'Incidents' with a blue header bar. The form contains several fields: a 'Title' text box with the value 'Failure in the programming of the executive', a 'Customer' dropdown menu with 'Yennu Enterprises', a 'Date' field with a calendar icon and the value '5/2/2024', a 'Priority' dropdown menu with 'High', and a 'Comments' multiline text area with the text 'The elevator returns itself to the last floor when idle,'.

Figure 2.4: EditScreen1 changes

9. Finish up by changing the colors, fonts, and control alignment to match your style. *Chapter 7, Improving User Experience*, will cover some techniques to apply a style makeover to your application.
10. Finally, save your application and provide a name for it.

How it works...

Now that we have created an application from SharePoint data, we can take it for a test drive. Power Apps Studio gives you a preview functionality for testing. You can do it in two ways:

- Return to **BrowseScreen1** and press the play button at the top right of the Studio interface. This action will execute the application so that you can interact with it as if running it from your device of choice. Speaking of which, there is a sidebar next to the running application that allows you to preview it on different form factors: tablet, phone, or browser.

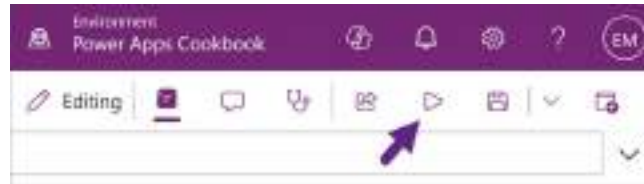


Figure 2.5: Power Apps Studio preview feature

- Another option to test functionality without entering into preview mode would be to use the *Alt* key (*option* on a Mac) to test a particular control. For example, if you press *Alt* and click a button, it will perform its `OnSelect` action. This way, you can test a specific control without leaving the Studio design mode.

Never rely on these testing methods to deploy a production application. Power Apps Studio gives you the preview button to test your app; while it serves you well for functionality testing, it's no match for accurate device testing. You need to check your app on the user's medium of consumption, such as on Teams or mobile.

This application is now ready to perform incident tracking for your organization. You can now test it by creating new incidents or editing existing ones. The Power Apps template should have already taken care of the core functionality, so verify your requirements, such as maximum text length, user interface design changes, etc. Making subtle user interface changes, such as font names and sizes, can improve the overall look of the application:

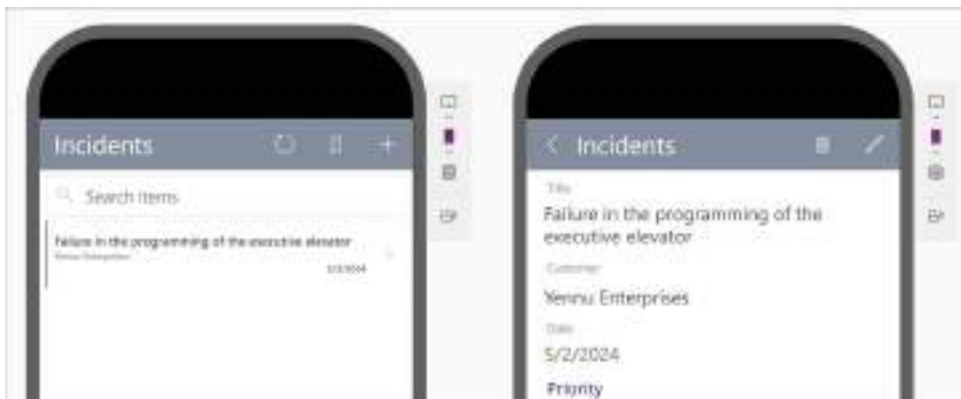


Figure 2.6: User interface changes for the incident tracking solution

There's more...

Whenever we build solutions, we constantly build them for connected devices. The capabilities of cloud services make our lives easier wherever we are, but what if our field technicians need to work someplace where there is no connectivity?

Power Apps helps you by including offline capabilities. For canvas apps, we can leverage Power Fx functions such as `LoadData` and `SaveData` to handle our data, depending on the connection state. In the upcoming chapters, you will explore the model-driven app, where Dataverse plays a crucial role in providing offline capabilities to tables and mobile offline profiles for you to configure.

To get more information, please visit this link for canvas apps: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/offline-apps>. For model-driven apps, visit this link: <https://learn.microsoft.com/en-us/power-apps/mobile/setup-mobile-offline>.

Improving SharePoint list forms with the help of Power Apps

Earlier, we configured our SharePoint data source in the *Creating an incident tracking solution – setting up the data source* recipe. We will use the same steps to create our data source for this recipe.

Getting ready

Planning for this data source requires the creation of related columns to track expenses. As a template, we could use the following options:

- **Projects:**

This list will hold project information, which will help us link the fees and the customer who owns the project. We will use the Customers list created in the previous recipe for this connection.

Column data type	Column name	Required	Extra
Lookup	Customer	Yes	Source list: Customers

Table 2.3: Projects list columns

- **Expenses:**

This list will save the information related to the actual expense. The structure required is as follows:

Column data type	Column name	Required	Extra
Date and time	Date	Yes	-
Choice	Category	Yes	Travel, Transport, Supplies, and Meals
Lookup	Project Name	No	Source list: Projects
Currency	Amount	Yes	-
Multiple lines of text	Comments	No	-

Table 2.4: Expenses list columns

How to do it...

1. Go to the desired SharePoint site and create the Projects list.
2. Once the list gets created, click the down arrow next to the **Title** column and select **Column settings | Rename**. Set it as Name and click **Save**. Add the Customer column.
3. Create the Expenses list and rename the **Title** column Expense.
4. With the list structure in place, add as much sample data to the list as you like.
5. On the list's toolbar, select **Integrate | Power Apps | Customize forms**. This action will automatically create an app based on the list. This time, it will build an app with only one form and one particular control called **SharePointIntegration**:

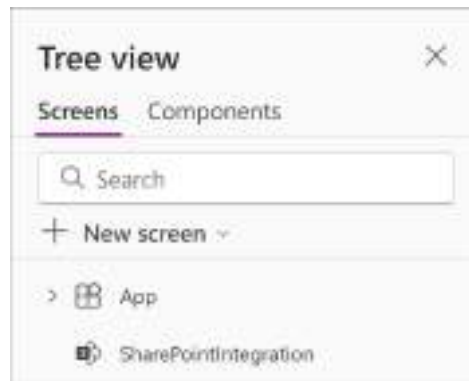


Figure 2.7: SharePointIntegration control inside Power Apps

6. Make relevant changes to your app, such as increasing the Comments column's height, resizing the attachment's control, and making style changes.
7. Save and publish your app to SharePoint.

How it works...

As you can see, the new application differs from the one created earlier because this one gets embedded inside the SharePoint list. It replaces the regular list's data entry mechanism by displaying the Power App whenever you want to view, edit, or add new records:

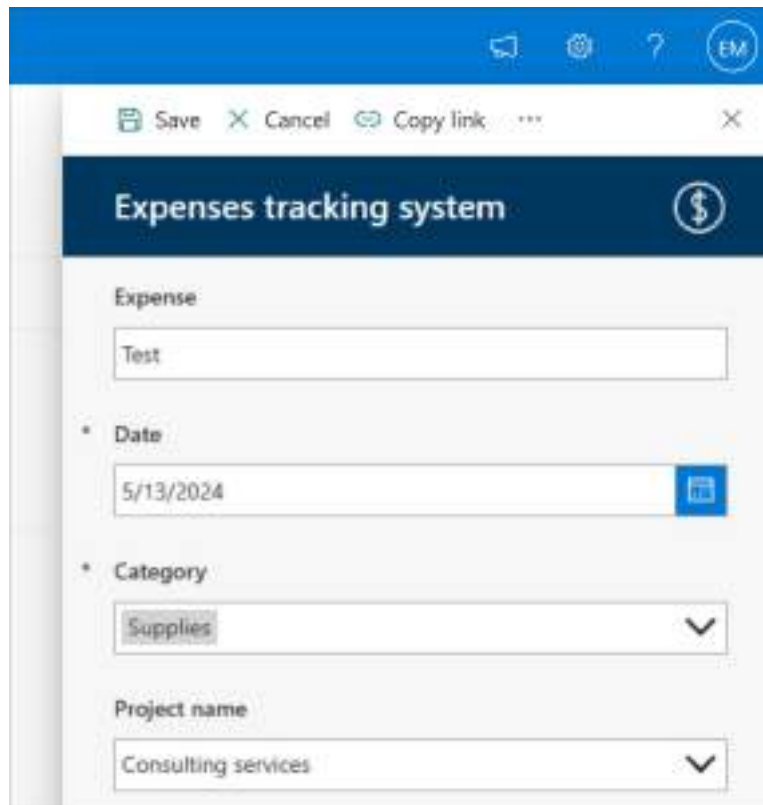
The image shows a screenshot of a SharePoint list interface. At the top, there is a blue header bar with icons for chat, settings, help, and a user profile labeled 'EM'. Below this is a white toolbar with buttons for 'Save', 'Cancel', 'Copy link', and a menu icon. The main content area features a dark blue header for the 'Expenses tracking system' app, which includes a dollar sign icon. The app form contains several fields: 'Expense' with a text input containing 'Test'; 'Date' with a date picker showing '5/13/2024'; 'Category' with a dropdown menu showing 'Supplies'; and 'Project name' with a dropdown menu showing 'Consulting services'.

Figure 2.8: SharePoint list Power App

This integration works by using the **SharePointIntegration** control. It acts as a bridge between SharePoint and Power Apps to catch the data events (view, create, and edit), while also allowing us to handle the save and cancel events:

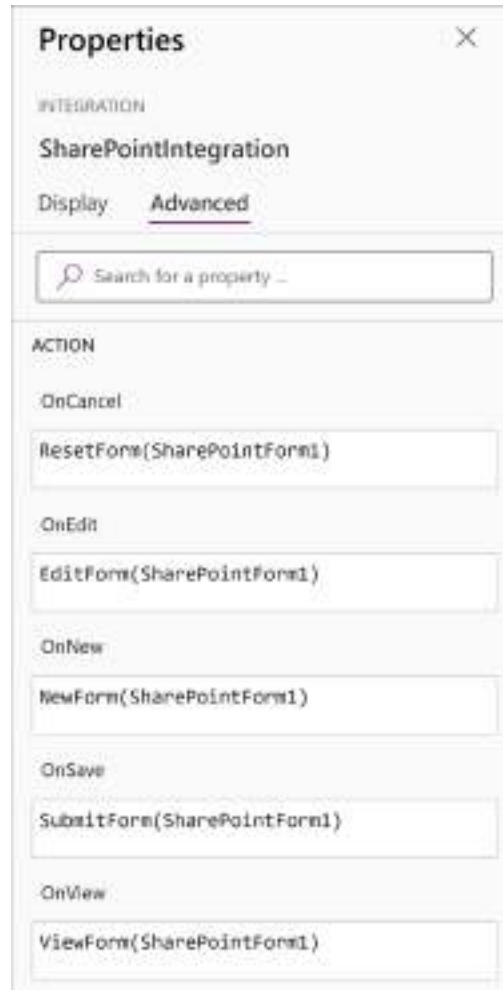


Figure 2.9: SharePointIntegration actions

These actions trigger Power Fx formulas that get triggered depending on each event.

Replacing a SharePoint form with a Power App gives us the flexibility to control how the data gets registered, improve the data validation, and even allow connections to other services besides SharePoint, to name a few examples. Always remember that these types of applications will live in the default environment.

To manage Power Apps replacing SharePoint forms, go to the list, click the gear icon at the top of the screen, click **List settings**, and finally, **Form settings**. From there, you will be able to reset the list to use the default form, modify the Power App, and review the version changes. If you ever need to delete the Power App, select to use the default form, and then click on **Delete custom form**.

There's more...

Think outside the box when building embedded apps. They are not only there to improve data entry forms. You can create full-fledged solutions inside your lists. Please refer to the following example showcased on the Power Apps blog: <https://powerapps.microsoft.com/en-us/blog/island-council-of-tenerife-organizes-community-events-using-the-power-platform>.

There are many more scenarios where you can enrich your current platforms by integrating Power Apps – Microsoft Teams and SharePoint pages, to name a couple. Want to know more? Please refer to *Chapter 8, Power Apps Everywhere*, to view more recipes that extend the use of Power Apps.

Creating a canvas app from existing data

Power Apps is the ideal choice when building enterprise solutions. These solutions are often needed to improve a particular process, especially regarding data handling. Sometimes, you will start from scratch, but most of the time, the data structure will be there already, just waiting for a robust and flexible application to handle it.

For these situations, there are some ready-made processes in the Microsoft ecosystem that allow you to create apps from a particular set of data. We will discover these from Azure and the Power Apps maker portal.

Getting ready

Microsoft Azure is a platform that provides a wide range of cloud services for all kinds of scenarios, such as application development, data analysis, and AI services. When working with Azure, you will need to have an active subscription. To test the capabilities of this platform, you can apply for a free account by signing up here: <https://azure.microsoft.com/en-us/free/>.

This recipe will require an Azure SQL database. Please refer to *Chapter 1, App Maker Basics* for instructions on how to build this database. For a quickstart on how to create one, please visit <https://docs.microsoft.com/en-us/azure/azure-sql/database/single-database-create-quickstart>.

How to do it...

To gather data from your existing Azure SQL database, take the following steps:

1. Go to the Azure portal, <https://portal.azure.com>, and from the main search, enter SQL databases and select the service from the list, as seen in the following screenshot:

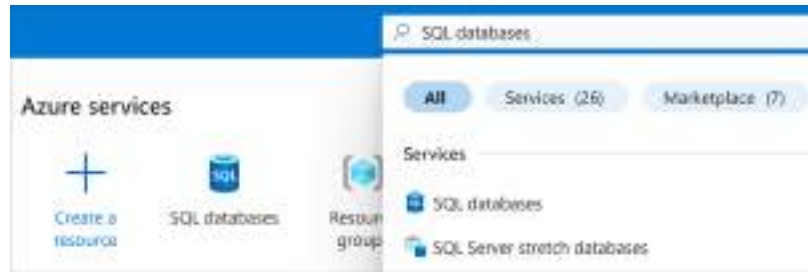


Figure 2.10: Azure SQL databases

2. A list of databases will appear for you to pick. Select the one that has the data needed for your application. This action will open the configuration page, sometimes called a *blade*, with all the settings to configure the database.
3. On the left pane of the configuration page, you will find a section called **Power Platform**, from which you can start working with any of the services available:
 - **Power BI** is used to connect and visualize the data from this database.
 - **Power Apps** will use this database as one of the data sources for your application.
 - **Power Automate** is used to automate processes using SQL Server templates.
4. Select **Power Apps**, and then click **Get started**, as seen in the following screenshot:

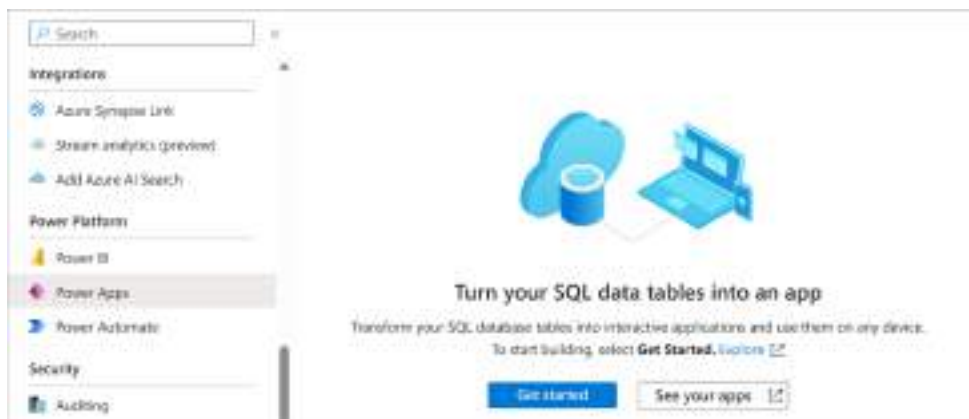


Figure 2.11: Power Apps option in Azure

5. The Azure portal will open a **Create an app** form to gather the required settings for your app. If this form is disabled, it means that you don't have an active Power Apps plan. You can apply for a trial by clicking the **Start trial** link on the form's header.
6. Enter the app name and the credentials to authenticate to the database in the **SQL Authentication** section. Keep in mind that from this wizard, you can't use Microsoft Entra authentication.
7. Finally, select a table from the list of tables in the database. The following is a sample form to create an app:

Power Apps

Verify the app name, enter your credentials, and select a table to get started. You will be taken to Power Apps Studio to view, customize, and share your fully functioning app.

Basic Information

App name * ✓

Server Name

SQL database name

SQL Authentication

Provide access to this database by supplying your credentials below.

Username *

Password *

Table Selection

The table you select and its schema will be used to create your app

Tables * ▼

Figure 2.12: Create an app form

8. Click **Create** to start the app-building process. If the Power Apps portal doesn't open, check whether your browser of choice has blocked the pop up. In the following screenshot, you can see what happens in Microsoft Edge. You can click the link or select **Always** to allow pop ups from the Azure portal:

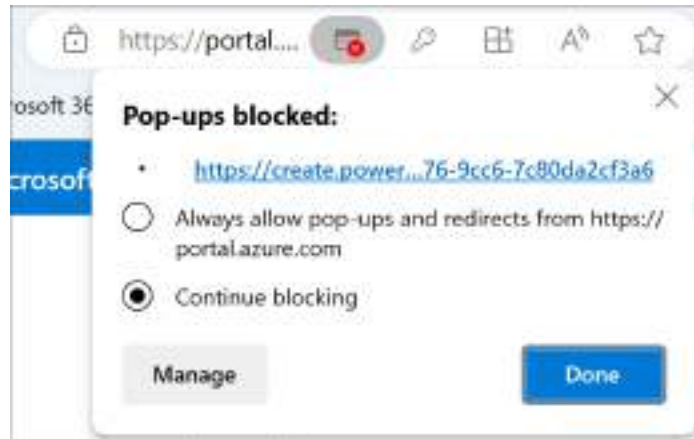


Figure 2.13: Pop-up blocking in Microsoft Edge

9. When the process finishes, you will see an application created from the table data with a browse, detail, and edit screen:

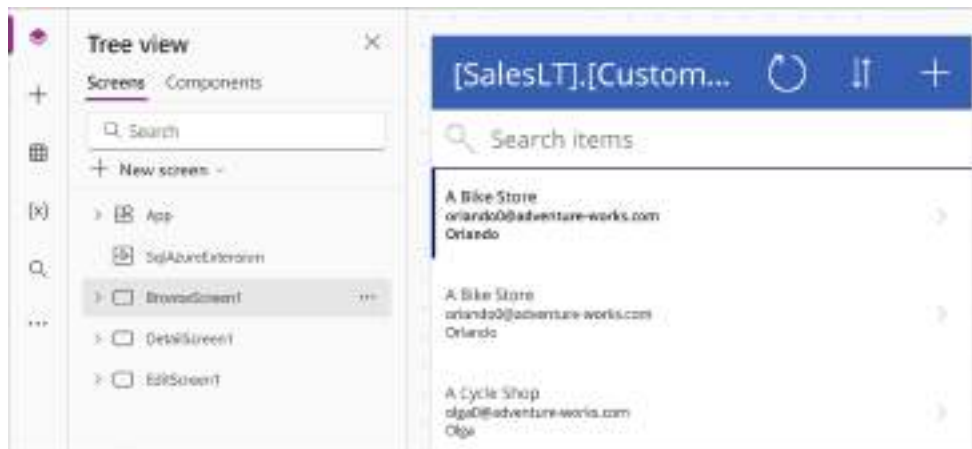


Figure 2.14: App from Azure SQL database

Keep in mind that this application will be created in the default environment.

Now, let's review the process of creating apps from data using the Power Apps maker portal:

1. Go to the Power Apps maker portal, <https://make.powerapps.com>, and from the left pane, select **Apps**. Click the **Start with data** button.
2. Under the **Other ways to get started** section, click **Connect external data**. In the next step, you will see some of the most popular data sources in the Microsoft ecosystem. Selecting any of the choices will automatically create a connection to the selected data source:
 - **From SharePoint:** You will be able to choose from the available sites and lists.
 - **From Excel:** This will display a list of locations where you can look for Excel files to select as your data source.
 - **From SQL:** You will need to provide the server and database information to see the tables you can choose from to build the app.
 - Depending on the service, you will be able to configure more options regarding the application data source connectivity. If you want more insight into some of these data sources regarding speed, please refer to the *Choosing the right data source – real-world tests* recipe in *Chapter 4, Choosing the Right Data Source for Your Applications*.
3. In the following screenshot, we will use **SharePoint** as our data source:

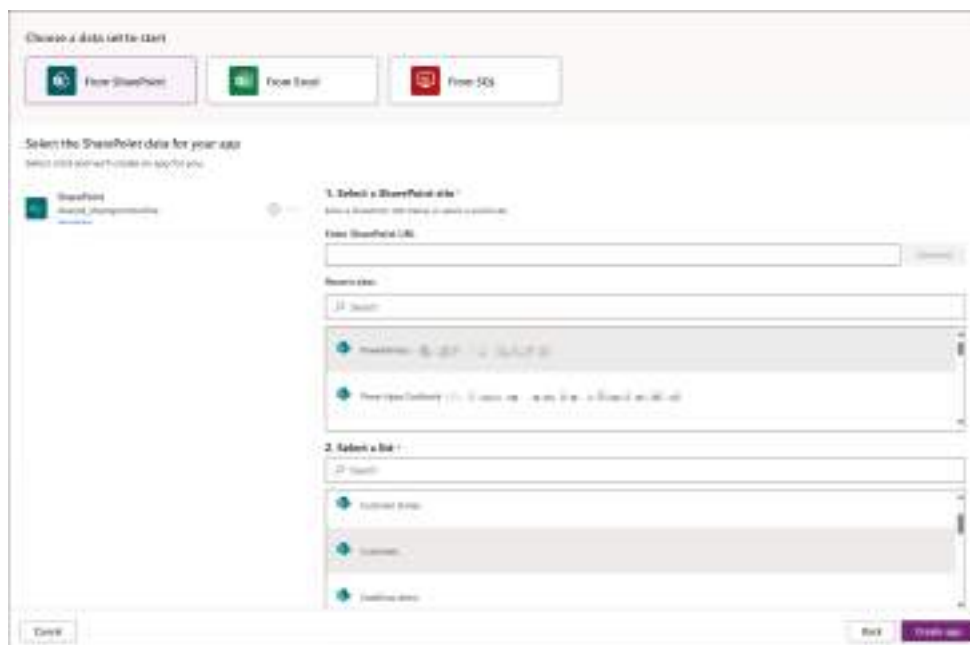


Figure 2.15: Connections page

4. After setting up the data source, click on **Create app**. Power Apps Studio will create an app based on the existing columns in the selected SharePoint list.

How it works...

Unlike the application created in our third recipe, the application improves in terms of design and responsiveness. When you use the **Start with data** wizard, Power Apps generates an application with a desktop format that can adapt to any screen, whether it is a mobile phone, a tablet, or a full-fledged desktop computer.

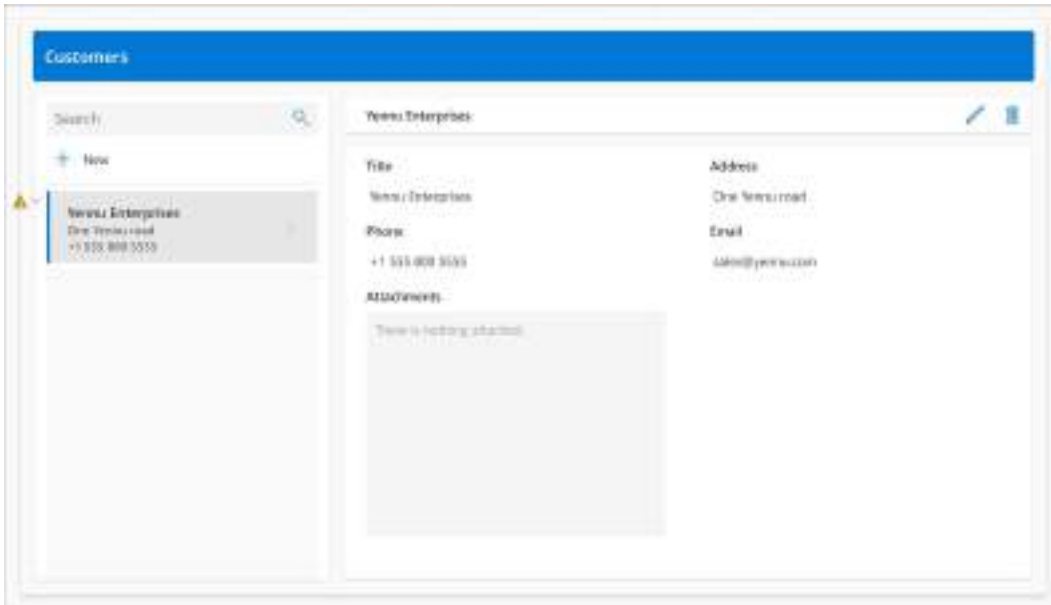


Figure 2.16: Canvas app to handle a SharePoint list

Regarding the Azure SQL version, there is an additional requirement if you want to edit database records. From Power Apps, you can only update or delete records from tables with primary keys. If the table you select doesn't have a primary key defined, you will only get **BrowseScreen** and **DetailScreen** in the created app.

Keep in mind that there are other known issues and limitations when working with Azure SQL databases. For more information, please visit <https://learn.microsoft.com/en-us/connectors/sql/#general-known-issues-and-limitations>.

Always refer to the connector's documentation of your data source to learn more about possible issues or limitations that could compromise your solution: <https://learn.microsoft.com/en-us/connectors/connector-reference/>.

There's more...

Even though we have discovered how to create modern-looking applications using the new wizards available in the Power Apps maker portal, we always need to have the concept of performance in our minds. A warning flag in our SharePoint example warns us about delegation: the idea of offloading data processing at the data source level and not at the application level. The flag appears because the Search function is not delegable to the data source when working with SharePoint. It means you can't ask SharePoint to do the actual search, so Power Apps needs to gather all the data and then do the search at the application level.

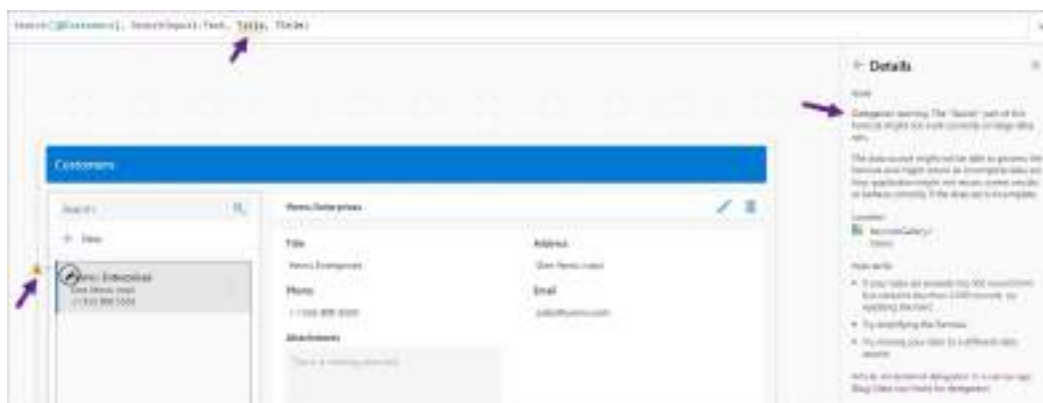


Figure 2.17: Delegation warning

A workaround to handle situations like this could be to use a cloud flow to do the legwork for the app, by doing the search at the API level. Documentation specific to this subject is available here: <https://learn.microsoft.com/en-us/sharepoint/dev/sp-add-ins/working-with-lists-and-list-items-with-rest#working-with-list-items-by-using-rest>.

You can also find an example in *Chapter 5, Overcoming Power Apps Delegation with Power Automate*.

Please refer to the official documentation for updates on functionality available on your data source of choice, for example, to have the Search as a delegable for SharePoint: <https://learn.microsoft.com/en-us/connectors/sharepointonline/#power-apps-delegable-functions-and-operations-for-sharepoint>.

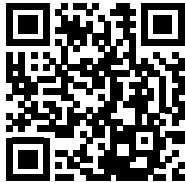
We also discovered how Copilot makes an appearance when dealing with the data sources of our apps. It's especially helpful when building data structures from scratch. Please refer to *Chapter 3, Building from Data with Model-Driven Apps*, where we will use these capabilities to create an application.

Some of the applications created in this chapter are fully functional for handling data but lack appeal in the user interface department. To enhance their design, please refer to *Chapter 7, Improving User Experience*, to see how to apply an application makeover.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



3

Building from Data with Model-Driven Apps

Model-driven apps are the data-centric siblings of canvas apps. They get their foundation from the business data model, taking the approach of placing building blocks on top of data. These blocks spread in a standard responsive layout, which allows the creation of a solution that adapts to any device.

Even though analyzing and setting up the data source is one of the main objectives of any application, this is the primary objective in model-driven apps. A complete understanding of the business process allows you to design your data model with a solid structure to support the proposed solution. The last step would be to build the app on top of this model.

This chapter will explore the model-driven world of Power Apps while building a Helpdesk solution. Through these recipes, you will get a clear view of all the components available on this platform:

- Exploring Dataverse
- Configuring your environment
- Defining data structures
- Building the model-driven app
- Setting up business process logic
- Designing dashboards to visualize data
- Integrating canvas apps inside model-driven apps

- Improving model-driven apps with custom pages
- Deploying your solution to your production environment
- Discovering the Copilot experience when building model-driven apps

Technical requirements

The proposed Helpdesk example requires addressing the following recipes sequentially to build a complete model-driven solution.

The complete version of this application is available from our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-03>.

Licensing requirements

When building model-driven solutions, there are some specific licensing requirements. Please refer to the *Preface* section of this book to get more information about the licensing model of this platform.

Exploring Dataverse

When we talk about the data model for model-driven apps, we mean Dataverse. The data gets stored in tables just like traditional databases. The difference is the rich set of business-oriented features:

- Use an out-of-the-box set of standard business tables with the ability to add your custom tables
- Create a table, and the platform will automatically add all columns to address the underlying process requirements, such as owner info, tracking, status, versioning, and the like
- Structure your data with various column data types, which helps in complex data modeling scenarios
- Design relationships and define keys for standard and custom tables to ensure data integrity across the service
- Enforce security on roles, records, and column levels, giving you complete flexibility when setting up data access
- Add business features right from the table configuration: data consumption through views, data manipulation through forms, data visualization through dashboards and charts, and last but not least, business rules to apply a set of validations and logic without writing code

Here's an example of what tables look like:

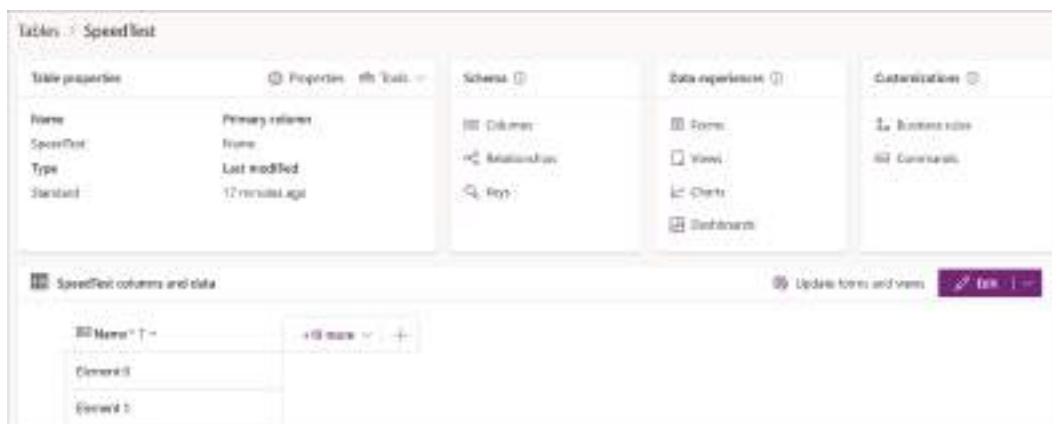


Figure 3.1: Table example in Dataverse

To store these tables, Dataverse uses **environments**. Environments serve as a container for data and all the components that interact with it, such as applications, cloud flows, and business processes.

Inside these environments, Dataverse manages **solutions** to package apps and components that serve a specific purpose. This feature allows the transfer of these packages between environments and, thus, the implementation of **application lifecycle management (ALM)** for the whole platform. For more information, please refer to <https://docs.microsoft.com/en-us/power-platform/alm/basics-alm>.

Dataverse is also the service that most Dynamics 365 applications use to store their data – this means that you can interact with the data that your business is already using.

Configuring your environment

Configuring a Dataverse database for the first time will create a set of business-centric tables, such as **Accounts** and **Contacts**. Starting from these, you can add the custom tables your processes might need to achieve their goal. In this recipe, we will start creating the building blocks of an application to handle a Helpdesk service.

Getting ready

To build our solution, we will start by setting up a new environment that will act as our development area. Then, we can pack our solution and deploy it wherever it is needed.

How to do it...

1. Go to the Power Apps admin center by visiting <https://admin.powerplatform.microsoft.com> or <https://aka.ms/ppac> for short. You can also navigate from the Power Apps maker portal by clicking the upper-right gear icon and selecting **Admin center**.
2. Once in the admin center, select **Environments** in the left pane and click on **New** from the toolbar. This action will open a pane on the right where you will need to define the properties.
3. Choose a **Name** and a **Region** close to you.



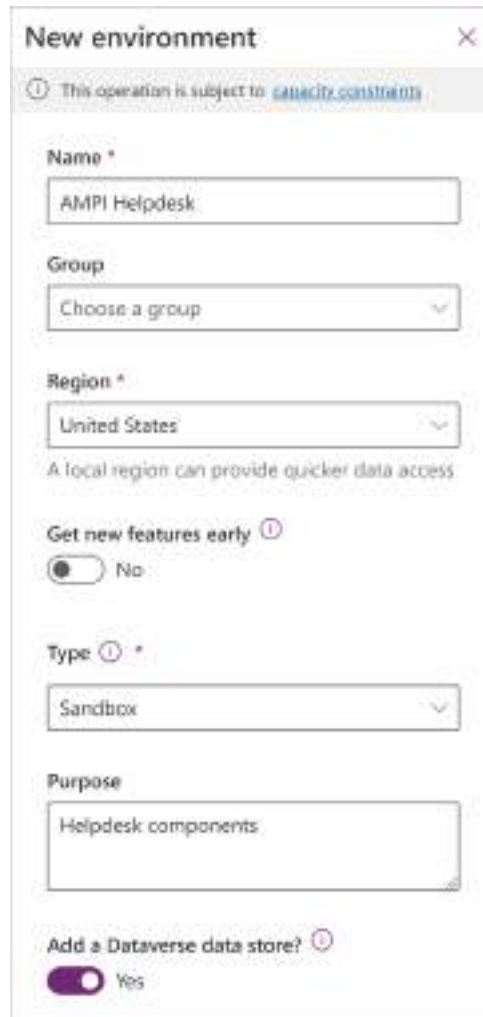
The **Group** property configures the **Environment groups** feature, which helps you organize your environments as if they were folders on your device. Besides helping you structure your platform, it includes more capabilities to handle environments as a group. Visit <https://learn.microsoft.com/en-us/power-platform/admin/environment-groups#production>.

4. For **Type**, you can select **Production**, **Sandbox**, **Developer**, or **Trial**.



Production is where your solutions operate for end users, **Sandbox** and **Developer** are for developing or testing solutions, and **Trial** is a 30-day environment meant for the short-term testing of new features. For more information, please visit <https://docs.microsoft.com/en-us/power-platform/admin/environments-overview#types-of-environments>.

5. Describe the **Purpose** and select **Yes** for **Add a Dataverse data store?** as we will need one database for our solution. Click **Next** to continue:



The screenshot shows the 'New environment' settings pane. At the top, there is a title bar with 'New environment' and a close button. Below the title bar, a message states: 'This operation is subject to: capacity constraints'. The form contains several fields: 'Name' with the value 'AMPI Helpdesk'; 'Group' with a dropdown menu showing 'Choose a group'; 'Region' with a dropdown menu showing 'United States' and a note 'A local region can provide quicker data access.'; 'Get new features early' with a toggle switch set to 'No'; 'Type' with a dropdown menu showing 'Sandbox'; 'Purpose' with a text box containing 'Helpdesk components'; and 'Add a Dataverse data store?' with a toggle switch set to 'Yes'.

Figure 3.2: Environment settings

6. In the next pane, you can set the default **Language** for the interface and the required **Currency** for reporting purposes. You should pre-define the access to your environment by selecting a **Security group**. You can also select **None** to allow access to all tenant users. Please refer to this documentation for more insight into security: <https://learn.microsoft.com/en-us/power-platform/admin/control-user-access>.

7. You can also change the URL prefix of your new environment, decide whether you want to enable Dynamics 365 apps (given that you have the required licenses), and choose whether you're going to deploy sample apps and data. Click on **Save** when finished:

The screenshot shows the 'Add Dataverse' configuration window. At the top, there are two informational messages: 'This operation is subject to [security constraints](#)' and 'Assign a security group to this environment or opt for open access before saving'. The form contains several sections: 'Language' with a dropdown set to 'English (United States)' and a description 'Default language for user interfaces in this environment'; 'Currency' with a dropdown set to 'EUR (€)' and a description 'Reports will use this currency'; 'Security group' with a description 'Restrict environment access to members of a security group or select None to opt for open access across your tenant. [Learn more](#)' and a '+ Select' button; 'URL' with a description 'A unique domain name will be generated. Click [here](#) to enter a custom domain'; 'Enable Dynamics 365 apps?' with a description 'In addition to Power Apps. [Learn more](#)' and a radio button set to 'No' with a note 'If not enabled, you won't be able to install Dynamics 365 apps on this environment. This decision is irreversible'; and 'Deploy sample apps and data?' with a radio button set to 'No'.

Add Dataverse

① This operation is subject to [security constraints](#)

① Assign a security group to this environment or opt for open access before saving

Language *

English (United States)

Default language for user interfaces in this environment

Currency *

EUR (€)

Reports will use this currency

Security group *

Restrict environment access to members of a security group or select None to opt for open access across your tenant. [Learn more](#)

+ Select

URL

A unique domain name will be generated. Click [here](#) to enter a custom domain

Enable Dynamics 365 apps?

In addition to Power Apps. [Learn more](#)

☒ No

If not enabled, you won't be able to install Dynamics 365 apps on this environment. This decision is irreversible

Deploy sample apps and data?

☒ No

Figure 3.3: Dataverse settings

8. The admin center will start the provisioning of your environment. After a few minutes, it will appear as **Ready**, and then you can go to the Power Apps maker portal to begin working with it.

How it works...

From the Power Apps maker portal, select your newly created environment by clicking the selector at the top right of the interface:

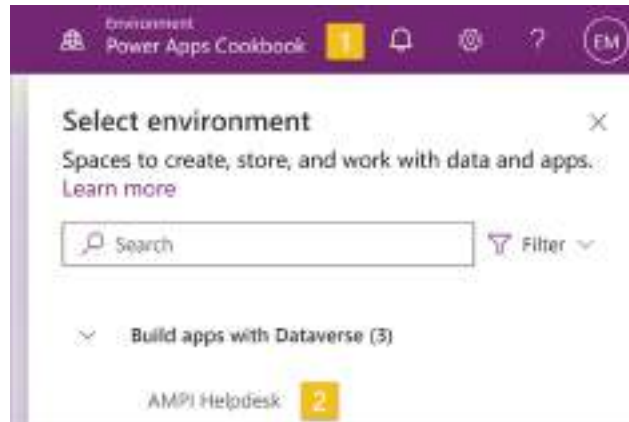


Figure 3.4: Environment list

After making this selection, we will only see information related to this environment, such as apps, cloud flows, and connections.

In the *Defining data structures* recipe, we will define the custom tables needed for our Helpdesk solution.

Building a Power Platform solution

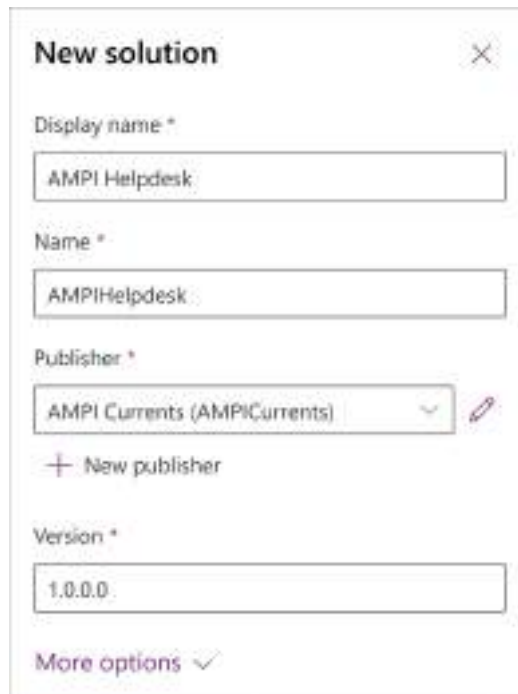
If environments serve as containers for all our Power Platform components, then a solution is the container for each development we build using those components. A solution creates a package to store all the pieces that compose your application, which aids in deployment, distribution, and management. For more information on this subject, follow this link: <https://learn.microsoft.com/en-us/power-apps/maker/model-driven-apps/distribute-model-driven-app>.

Getting ready

To build our solution, we will start by setting up the publisher, which defines the solution's developer, and also the prefix for all your custom tables and columns.

How to do it...

1. From the Power Apps maker portal, select **Solutions** from the left pane, and then click on **New solution** from the toolbar.
2. Before describing the solution, click **New publisher** and enter the **Display name** and **Name**. Enter a meaningful two-to-eight-letter **Prefix** for all the elements you will create. Keep the default **Choice value prefix** and click on **Save**.
3. Back in the **New solution** pane, enter the desired **Display name** and **Name** for the solution, select your newly created **Publisher**, and click on **Create**:



New solution ✕

Display name *

AMPI Helpdesk

Name *

AMPIHelpdesk

Publisher *

AMPI Currents (AMPICurrents) ▼ ✎

+ New publisher

Version *

1.0.0.0

More options ▼

Figure 3.5: Solution settings

How it works...

This process will open the solution automatically. If you wish to return, click on the arrow at the top of the left pane. The new solution will now appear in the **Solutions** section of the Power Apps maker portal. To view all your publishers, from the left pane, click on **More | Discover all** and then on **Publishers** inside the **App Management** section.

Defining data structures

For our Helpdesk solution, we will use related tables as our data source. We will design a base column template to keep track of all the information needed. You can customize it to your own needs.

Getting ready

The following tables will make up our solution’s data structure, with the main focus on the Ticket table, to which all the tables will be related. Let’s start with this one and then move on to the rest.

Ticket

This table will hold the base information of the Helpdesk ticket and will be our data model’s primary object. The column structure is as follows:

Display name	Data type	Format
Title	A single line of text	Text
Description	Multiple lines of text	Text
Ticket Status	Choice	-
Priority	Choice	-
Ticket validation	Choice	-
Resolution	Multiple lines of text	Text

Table 3.1: Ticket table columns

Ticket operation

Related to the Ticket table, this will hold the specific operations executed to resolve the ticket. The structure is as follows:

Display name	Data type	Format
Title	A single line of text	Text
Description	Multiple lines of text	Text
Operation Status	Choice	-
Duration	Whole number	Duration
Latitude	Decimal	(6 decimal places)
Longitude	Decimal	(6 decimal places)

Table 3.2: Ticket operation table columns

Project

All tickets are going to be associated with a customer's project. The structure is as follows:

Display name	Data type	Format
Title	A single line of text	Text
Description	Multiple lines of text	Text
Start	Date and time	Date only
End	Date and time	Date only

Table 3.3: Project table columns

Account

We will take advantage of one of the default business tables included with Dataverse. This table will hold information for our customers.

How to do it...

1. From the list of solutions, click to open the one you created in the previous recipe.
2. From the toolbar, click on **Add existing | Table**, and look for the **Account** table. Select it and click on **Next**. On the **Selected tables** pane, ensure that the **Include all objects** and **Include table metadata** checkboxes are not selected because we want to include a reference to the table, not all of its details. Click on **Add** to finish. For more information regarding table metadata, please visit <https://learn.microsoft.com/en-us/power-platform/alm/segmented-solutions-alm>.
3. To add our custom tables, click on **New | Table | Table**. Set the **Display name** to Ticket. The **Plural name** will change to **Tickets** automatically. Depending on your table's needs, you might want to enable attachments. For example, including extra information about the incident that started the ticket might be helpful, such as screenshots or PDF files.
4. Click on the **Primary column** tab. Enter Title as the **Display name**. Remember to choose something your end users can use to select the records when listing them.
5. Once you have entered all the required information, click **Save**. The system will start creating the table with the specified primary column and the rest of the business-oriented columns. The table will open automatically once finished.

The following is an example of the Ticket table:

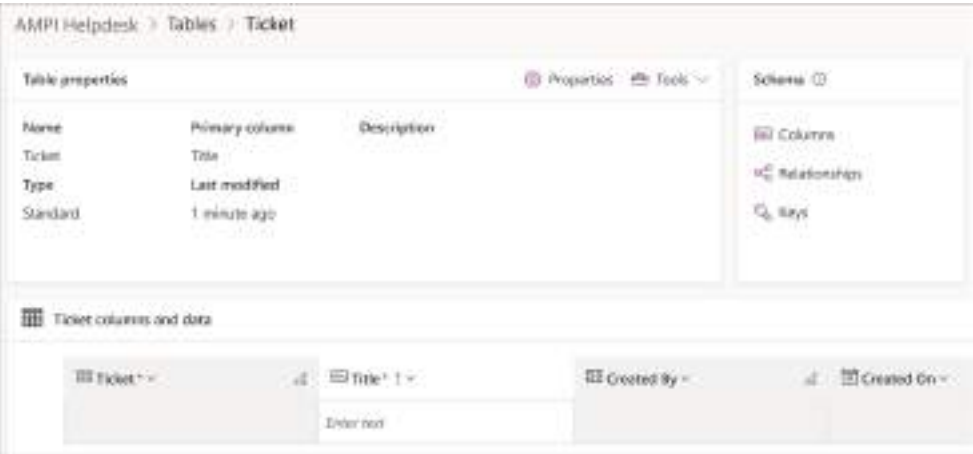


Figure 3.6: Ticket table base structure

- 6. Click on **New | Column** from the toolbar. For the **Display name** field, enter **Description**, set the **Data type** to **Text | Multiple lines of Text | Plain text**, and click **Save**.
- 7. Repeat *Step 6* to add the **Resolution** column.
- 8. Click **New | Column** again to include the **Ticket status** column. For **Data type**, select **Choice | Choice**. Under the **Sync this choice with** dropdown, click on **New choice**. Set **Ticket status** as the **Display name**, and under the **Choices** section, add the term **New**. Click on **New choice** to add each of these items: **Pending**, **Resolved**, and **Closed**. Finally, click **Save** and then **Save** again to continue.

Using global choices is a great way to reuse a Choice column for different tables if they are going to hold the same elements.

- 9. Repeat *Step 8* for the **Ticket priority** column, but this time, select **No** for the **Sync with global choice?** option. Under the **Choices** section, add the term **Low**. Click on **New choice** to add each of these items: **Medium** and **High**. Click **Save** to continue.
- 10. Repeat *Step 9* for the **Ticket validation** column, adding **Phone** and **Email** for the choices.

Here’s an example of the **Ticket validation** column settings:

Display name *

Ticket validation

Description ⓘ

Data type * ⓘ

Choice

Behavior ⓘ

Simple

Required ⓘ

Optional

☒ Searchable ⓘ

☐ Selecting multiple choices is allowed

Sync with global choice? *

☐ Yes (recommended)
Can be used in multiple tables, and will stay updated everywhere.

☒ No
Creates a local choice that can only be used in one table. People using it can add new choices.

Choices

Sort ▾

	Label *	Value *	
⋮	Phone	750.120.000	⋮
⋮	Email	750.120.001	⋮

Figure 3.7: Ticket validation properties

11. Click on **Tables** on the left pane and then **New | Table | Table** from the toolbar to create the `Ticket` operation table. For the **Primary column**, use **Title** and then click on **Save**.
12. When the table completes the provisioning, click on **Columns** inside the **Schema** section and then click on **New column** on the toolbar to add the `Description` column using **Text | Multiple lines of Text | Plain text** as its **Data type**, and click **Save**.
13. Click on **New column** again to add the `Operation status` column. For **Data type**, select **Choice | Choice**. From the **Sync this choice with** dropdown, choose the one we created in *Step 8*, `Ticket status`, and then click **Save**.
14. Again, click on **New column** to create the `Duration` column. For **Data type**, select **Number | Duration** and click **Save**.
15. Finally, click on **New column** to add the `Latitude` and `Longitude` columns. For **Data type**, select **Number | Decimal**. Click **Save** when finished.
16. Return to the tables list by clicking on **Tables** on the left pane and then clicking on **New | Table | Table** from the toolbar to add the `Project` table. Use **Title** for **Primary column** and click on **Save**.
17. Once the provisioning is complete, click on **Columns** inside the **Schema** section and click on **New column** to add the `Description` column using **Text | Multiple lines of Text | Plain text** as its **Data type**, and click **Save**.
18. Click on **New column** again to create the `Start` column. Set **Data type** to **Date and time | Date Only** and then click **Save**. Repeat this step to make the `End` column.
19. Let's configure the relationship between tables to maintain data integrity in our solution. Expand the `Ticket` table from the list of **Tables** and click on **Relationships**. On the toolbar, click on **New relationship | One-to-many**. This kind of relationship roughly means that one element from one side (`Ticket`) can have many child items on the other (`Ticket operations`).
20. When the panel opens, select `Ticket operation` from the **Related (Many)** list. This action will create a lookup column to connect both tables. You can configure more settings in the **Advanced options** section regarding which action gets taken when a record gets deleted. To maintain data integrity, you might want to avoid deleting a ticket without removing the ticket operations first. In this case, we will select **Restrict** from the **Delete** dropdown. When you've finished, click on **Done**.

21. Repeat *Steps 18, 19, and 20* to configure the **One-to-many** relationship between **Project (one)** and **Ticket (many)** and then between **Account (one)** and **Project (many)**:

One-to-many

Choose the **Related table** from which to create your relationship lookup. [Learn more](#)

Current (One)	Related (Many)
Table *	Table *
Ticket	Ticket operation

1 — *

Lookup column display name *

Ticket

Lookup column name *

ticket

Lookup column requirement *

Optional

☒ Searchable

> General

Advanced options

Relationship behavior

Type of behavior * ⓘ

Referential

Delete *

Restrict

Done Cancel

Figure 3.8: Relationship example

22. On the **Ticket** table on the left pane, click on **Forms**. Select the three dots next to the form with the **Form type** of **Main** and click on **Edit | Edit in new tab**.
23. Once the form designer opens, you will see that only the **Title** column appears. On the **Information** pane on the right, change the **Display Name** to **Main**

24. On the left tree view, select the second option, **Table columns**. Click on all the columns we created for this table to add them to the form. You can also drag and drop the columns to place them as you see fit. For each field, you can change the properties on the right pane to increase the height, hide the labels, and the like. You can also click on the first option of the tree view, **Components**, to add section dividers, tabs, and more. These components allow you to improve the user interface of your forms:

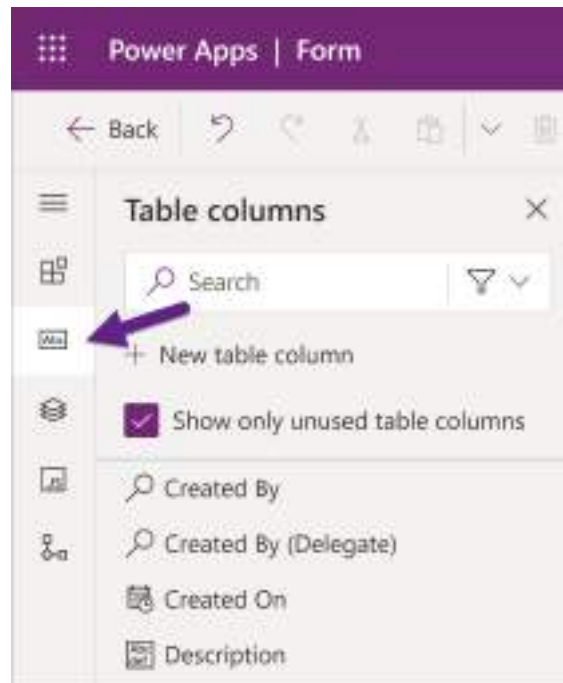


Figure 3.9: Form designer for the Ticket table

25. Complete the form changes, click **Save** and **Publish**, and then close the designer.

How it works...

The data structure is now in place, and the next step is to enter some sample account data. There are two ways for us to do this:

- From the **Tables** list, select to open the **Account** table and click on the **Edit** button just above the **Account columns and data** section. Go to the end of the columns list and click on the dropdown to open the **Show existing column** pane, then select **Account Name (Primary)**, **Email**, and **Description**. From the toolbar, click on **New row** and fill in all the desired columns.

- Go to the Power Platform admin center, <https://aka.ms/ppac>, and open your environment to see its details. From there, click on **Settings** on the toolbar and then expand **Data management**. Click the **Sample data** link to add mockup data to this environment automatically.

In this recipe, we have defined a data structure by relating tables to each other. Regarding relationships, please refer to <https://docs.microsoft.com/en-us/powerapps/maker/common-data-service/data-platform-entity-lookup#add-advanced-relationship-behavior> for more information.

There's more...

When building tables, the primary column data type is always **Text**. This configuration suits most situations, but sometimes, you might need another data type.

As a workaround, open the table columns, click on the column marked with the label **Primary Name Column**, and from the **Properties** panel, change the **Data type** from **Multiple lines of Text** to **Autonumber** and click **Save**. This change will allow the platform to manage the contents of this column automatically so you can build your table upon another column as your primary column.

Building the model-driven app

Creating a low-code application entails a different methodology for the application-building process. When using other technologies, you might need to choose the language, infrastructure, database system, and more. For model-driven applications, the essential principle is to create everything from the data structure and then build the components from there.

Getting ready

Per our example, the Helpdesk solution will package all elements required for our application, such as tables, dashboards, and so on. You can export all components in just one package and then import them into the destination environment.

We will use the **Modern app designer** to build the application. This designer allows more rapid application development, although it doesn't have the complete list of features that the classic designer has. However, there is always a **Switch to classic** button to fall back to the classic designer when needed.

How to do it...

1. Open your solution and click on **New | App | Model-driven app**. Enter your application's **Name** and **Description**, and click **Create**. This action will generate a blank application.
2. From the toolbar, click on **Add page**, select **Dataverse table**, and click **Next**. Look for the **Ticket** table in the list and click **Add**. Make sure that the **Show in navigation** option is selected.
3. In the **Pages** tree view on the left, select **Tickets view**, and in the right pane, click on the three dots next to the **Active Tickets** view and click **Edit in new tab**.
4. When the view designer opens, change the name of the view in the right pane to Pending Tickets.
5. From the left pane, drag the **Ticket priority** and **Ticket status** columns after the **Title** column. Expand the header of each column to set the width as desired.
6. In the right pane, click the **X** next to the **Title** column in the **Sort by ...** section. This action will show a **Sort by ...** dropdown. Click on it to select the **Created On** column.
7. In the **Filter by ...** section, click on **Edit filters ...** to open the **Filter** pane. Click on **Add** and select **Add row**. From the first dropdown, choose **Ticket status**. From the second, pick **Does not equal**, and choose **Closed** and **Resolved** from the third. Click **OK** to close the panel.
8. The **Pending Tickets** view should look like the following screenshot:

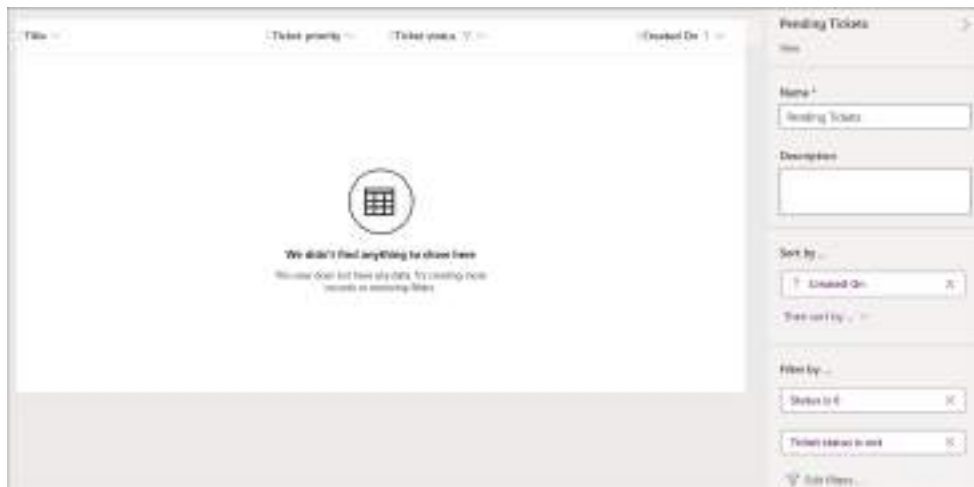


Figure 3.10: Pending Tickets view example

9. Finally, click on **Save and publish**. Close the view designer.

10. Back in the app designer, select **New group** and, in the right pane, change the **Title** to Helpdesk.
11. Now that we have configured our application, we can click **Publish** to save and finish.

How it works...

After completing the previous steps, we can test our application to verify whether it works as intended by clicking **Play**. This action will open the application with a menu on the left with a **Tickets** option inside a **Helpdesk** group and the new **Pending Tickets** view in the main display.

Add some records by clicking on the **New** button from the toolbar. If you create one with a **Ticket status** of **Closed** or **Resolved**, it should not appear in the main view. In your solution, add as many views as needed to make it easier for your users to track information:



Figure 3.11: Model-driven application running

In the following recipes, we will add application logic for data validation. We will also include a dashboard to provide a more informative view of the business process data.

Setting up business process logic

Every business process has a step-by-step procedure that defines every requirement, objective, and resolution from start to end. Companies build on and improve these procedures depending on the business needs and even create departments to work exclusively on this.

Getting ready

Model-driven apps help set up these rules by providing several tools to define business logic at various platform levels. You can incorporate data validation rules, structured data flows to guide the end user, and even trigger workflows for specific events in your processes. These are just some examples of what you can achieve with this platform.

Let's improve our Helpdesk application by adding business logic in two ways:

- Using a business rule, hiding or making a column visible depends on the ticket status. In our case, there is no reason for the **Ticket validation** column to be visible unless the case is in a **Resolved** state.
- Guiding ticket processing by ensuring the user inputs the required information in each defined step with the help of a **business process flow (BPF)**. This component also allows you to execute workflows or Power Automate cloud flows when a defined trigger gets fired.

We will explore column visibility in *Steps 1–7* of this recipe. In *Steps 8–13*, we will use the BPF.

How to do it...

1. First, open the Helpdesk solution to start adding data validation rules.
2. Expand the **Tables** section from the tree view and then expand the **Ticket** table. Select **Business rules**, and on the toolbar, click on **New business rule**; this will open a designer where you can define actions based on a condition.
3. Select the default condition, **New condition**, and on the right panel, change the **Display Name** to **Ticket validation visibility**.
4. In the **Rules** section, set it to check when the **Ticket status** field equals **Resolved** or **Closed** and click on **Apply**.
5. Click **Add** on the main toolbar to include an **Add Set Visibility** action. Once you choose this option, two plus signs will appear next to the primary condition: when it is valid (checkmark symbol) and another when it isn't (X). Place one for the checkmark and then add another for the X.
6. Select each new action, set the following properties on the right pane, and then click on **Apply**:

Condition	Display Name	Field	Visible
Valid (checkmark symbol)	Show Ticket Validation	Ticket validation	Yes
Invalid (X symbol)	Hide Ticket Validation	Ticket validation	No

Table 3.4: Business rule conditions

7. Finally, click on **Ticket: New business rule**, located at the upper left of the screen, to set the business rule's name. Enter **Ticket validation visibility**, hit **Save** on the toolbar, and then on **Activate** to make it available for this table. Close the business rules designer afterward.

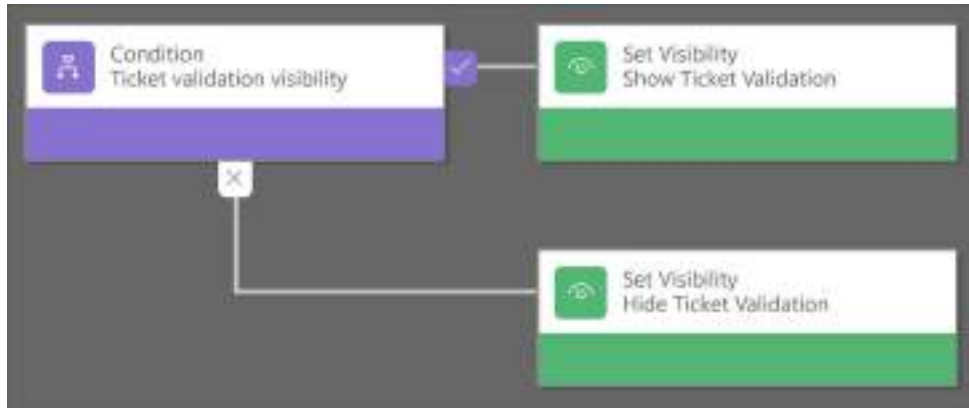


Figure 3.12: Ticket validation business rule

8. Now, let's create a BPF to guide our users during ticket processing. From the solution explorer, click on **All** in the tree view, and from the toolbar, click on **New | Automation | Process | Business process flow**.
9. Once the **New business process flow** pane opens, enter **Ticket** process as the **Display name**, leave **ticketprocess** as the **Name**, select **Ticket** from the **Table** dropdown, and click **Create**.
10. Once the designer opens, you will find a default stage. These are the steps of your guided flow. Let's add three more stages by dragging them next to each other from the components panel on the right. For each one, set the **Display Name** as follows: **New**, **In Progress**, **Resolved**, and **Closed**. Click **Apply** after each update.
11. Each stage comes with a data step by default, which lets us specify the column displayed in each step of our BPF and, most importantly, which are required when moving from one stage to another.

12. Let's configure the data steps by expanding the **Details** on each stage and setting these values:

Stage	Data Field	Required
New	Description	No
In Progress	Description	Yes
Resolved	Resolution	Yes
Closed	Ticket validation	Yes

Table 3.5: BPF data steps configuration

13. The BPF should look like the following figure:

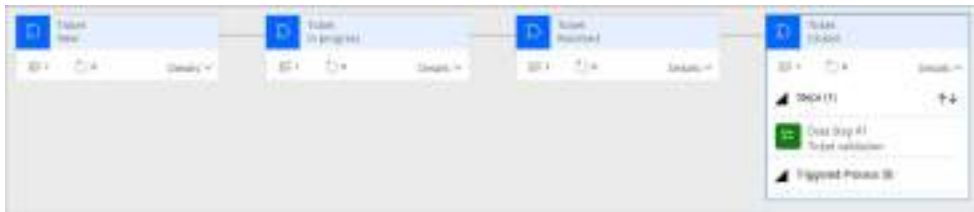


Figure 3.13: Ticket process BPF

14. After completing these changes, on the top toolbar, click on **Validate** to check everything is correct, and then click **Save**. Finally, click **Activate** to enable this BPF and close the business process flow designer.
15. Back in the solution, to have all the required components, click on **Add existing | Table** and look for **Ticket** process. The reasoning is that the business process requires the table with the same name as the BPF to hold the states of each process. Click **Next**, check **Include all objects**, and finally, click **Add**.

How it works...

Once we have configured the business process logic for **Ticket**, every time this table gets invoked, it will carry on these defined components. To see them working, go to your solution, select the model-driven app, and click **Play**; this will open the app on the **Pending Tickets** view. Click on **New** to add a ticket.

You will see that the form now includes your **BPF**, letting you visualize which step of the process your ticket is. After saving it, if you try to move, for example, beyond the **Resolved** stage, it will ask you to set the value for the required column:

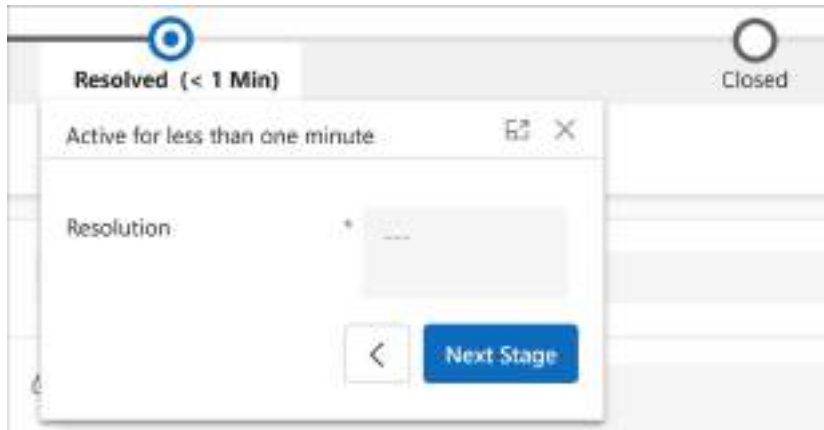


Figure 3.14: BPF in action

As for the business rule, the **Ticket validation** column will remain hidden until the ticket status changes to **Resolved**. Once we enter this column's data, it is then that we can finish the **BPF**:



Figure 3.15: Ticket processing completed

If you don't see the **BPF** on previously created records, click **Process | Switch Process** on the toolbar, select **Ticket process** from the list, and click **OK**.

Designing dashboards to visualize data

When working with business processes that deal with data, one of the main goals for organizations is to gain insights. Knowing where your operations are failing or where there is a market opportunity is vital, so having an easy way to interact with data is very important.

Dashboards are a feature in Dataverse that allows you to display a mix of graphs, grids of data, and web resources in one place. For example, web resources can include **iframe** objects, which can display content from another website inside a model-driven application.

Building on the example used throughout this chapter, let's add more value to the Helpdesk solution by including a ticket dashboard.

How to do it...

1. Let's create the charts that we will use in our dashboard. Starting from our solution, expand the **Ticket** table and click on **Charts**. From the toolbar, click on **New chart**. This action will open a new tab to define the graph's data, type, and visualization rules.
2. First, choose the view that will filter the data, then select the legend entries, which are the graph series, and finally, the horizontal axis, which will categorize your data. Let's create three charts using the following configurations; after completing each one, click on **Save & Close**:

Chart Type	View	Chart Name	Legend Entries	Horizontal Axis
Pie	Pending tickets	Ticket by Priority	Ticket	Ticket priority
Pie	Pending tickets	Ticket by Project	Ticket	Project
Column	Pending tickets	Ticket by Status	Ticket	Ticket status

Table 3.6: Charts configuration

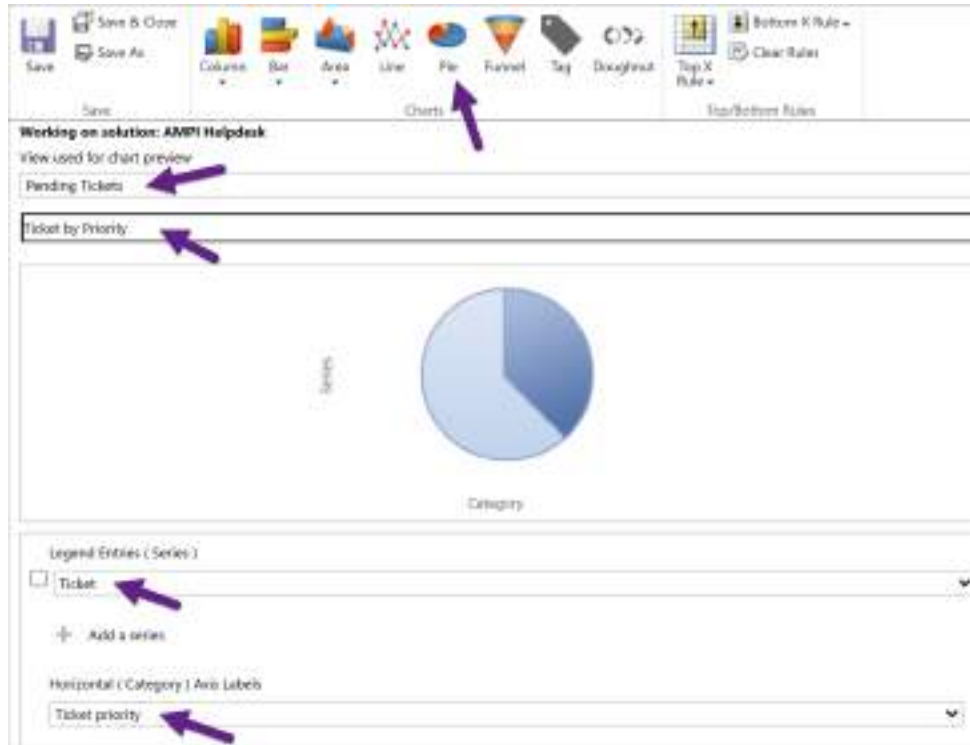


Figure 3.16: Chart configuration

3. Now that we have defined the charts, we can place them on a dashboard. From the solution, click **All** on the tree view and then click on **New | Dashboard | 3-Column overview**. In the dashboard designer, set the dashboard's name to **Tickets dashboard**. Select the 5th and 6th sections and press **Delete** to remove them. Select the 4th section and, from the toolbar, click on the ellipsis and select **Increase Width** twice to fill the lower section of the dashboard.
4. For the top area, select the first section and click on **CHART**. In the dialog, choose **Ticket** as **Record Type**, **Pending Tickets** from the **View** list, and pick one of the charts we created in *Step 2*. Repeat this for the other two sections at the top.
5. As for the lower section, select **LIST** from the toolbar. Choose **Tickets** as **Record Type** and **Pending Tickets** from the **View** list and click **Add**. Setting this will display a grid of records for easy access. Hit **Save** on the toolbar and close the window.
6. From the list of dashboards, back in the solution, select the three dots next to **Tickets dashboard**, and select **Publish**.

7. Select **Tables** from the tree view, click on the three dots next to the **Ticket** table, and select **Publish**.
8. Select **Apps** on the tree view and edit the model-driven app by clicking on the three dots next to it and selecting **Edit | Edit in new tab**.
9. Once the designer opens, click + **New** and select **Dashboard** from the dialog. From the list of dashboards, search for **Tickets dashboard** and click **Add**. It might take a while for the dashboard to appear. Refresh your browser tab as necessary.
10. Click on **Save** and then on **Publish** from the designer toolbar.

How it works...

After completing the dashboard setup, we can see it working in our model-driven application by selecting it from our solution and then clicking **Play**.

When the app opens, you will see the new **Dashboards** item on the navigation bar. At a glance, this display gives insights into the current state of the Helpdesk system.

You might need to hard-refresh your browser window if you don't see the changes immediately. This behavior is due to the cache mechanism.



Figure 3.17: Tickets dashboard

Open the model-driven app. Refresh a few times, and then you should see the dashboard.

There's more...

Classic dashboards are a great way to visualize data from Dataverse tables, not only by using charts or lists but also by web resources and **iframes**.

Model-driven apps also support embedding Power BI reports or dashboards to improve data analysis by implementing features such as enhanced visualizations, complex formulas, and multiple data sources. For more information on this topic, please visit <https://learn.microsoft.com/en-us/power-apps/maker/model-driven-apps/create-edit-powerbi-embedded-page>.

Integrating canvas apps inside model-driven apps

One of the best features of canvas apps is the ability to access multiple connectors from the same app. We can use this strength by embedding a canvas app as a component.

These are some scenarios in which this can be very helpful:

- A canvas application that is already in use by the organization, and we want to integrate it into our model-driven solution.
- A canvas app that connects to a legacy system using the on-premises data gateway.
- An employee tracking system that gives us information about which of our customer locations our support staff is working at any given moment. This data could be helpful to see whether we already have someone from support at the customer's location when a new ticket arrives.

Now, let's see how to integrate a canvas app into our model-driven solution.

How to do it...

1. From the solution, expand the **Project** table and click on **Forms**. Edit the **Information** form of the **Main** type.
2. Once the designer opens, select the **Switch to classic** option from the toolbar.



Power Apps is still evolving regarding the form designer experience, so we still need to use the classic version to add some components.

3. Select where you want your canvas app to appear, and then, from **Field Explorer** on the right, drag a column to that position. Remember that this column needs to have data to make the app visible. In this example, I've chosen **Created On**.
4. Double-click on the column and, in the **Label** section, uncheck **Display label on the form** so that it doesn't appear next to our app.
5. Go to the **Controls** tab and click on **Add Control...**, select **Canvas app** from the list, and click on **Add**. This action will add the canvas app property section. On the list of controls, select **Canvas app** and choose these radio buttons: **Web**, **Phone**, and **Tablet**.
6. In the **Canvas app** property section, you will see the settings required to link a canvas app. To add a new one, click on **Customize**; this will create a new canvas app with a form using the **Project** table as its data source.
7. You will notice a new control called **ModelDrivenFormIntegration**, shown in the following figure, which acts as a gateway between the model-driven form and the canvas app:

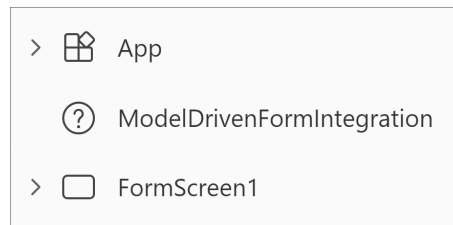


Figure 3.18: The *ModelDrivenFormIntegration* control

8. It works like the **SharePointIntegration** control we saw in the previous chapter's *Embedding an expense tracking list* recipe but with only one action available. Its primary purpose is to act on data refresh.
9. Design your app to match your needs. Keep in mind that one of the primary purposes of this integration is to have the ability to link the data from the Dataverse table with our canvas app. This connection gets done by the integration control using the **Item** property, which carries the column data from the selected item in the model-driven app:

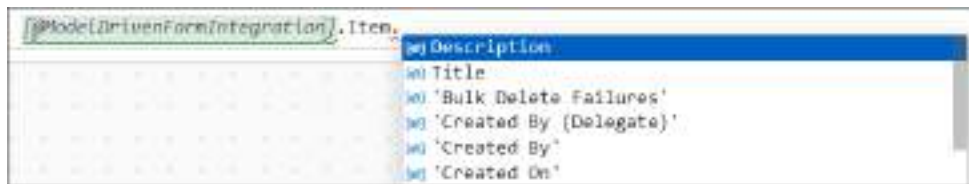


Figure 3.19: Project columns in the canvas app

10. Once you finish building your app, click the Save icon on the toolbar and publish it to make it available in our model-driven app. Close the canvas app studio.
11. Back in the classic form designer, you will see that the updated canvas properties reflect the new app's link. Click **OK** to save the field properties, save the form, and publish it. Close the classic form designer.
12. Go back to your solution. You will see that the canvas app appears automatically in the list of components.

How it works...

Let's pretend our company is in the middle of its cloud migration, and we still use a legacy **enterprise resource planning (ERP)** system to control customers' support rights. We can build a canvas app with a custom connector, a data gateway, or a Power Automate cloud flow to get data from the on-premises system. This solution can give support representatives more insight into the customer's rights related to the ticket.

Open a ticket with an associated project. If you click on the **Project** column value, it will navigate to the project form to display the related information about the project:



Figure 3.20: Project table navigation

As soon as the form appears, the model-driven integration control will notify the canvas app of the currently selected record and execute the control's **OnDataRefresh** action:

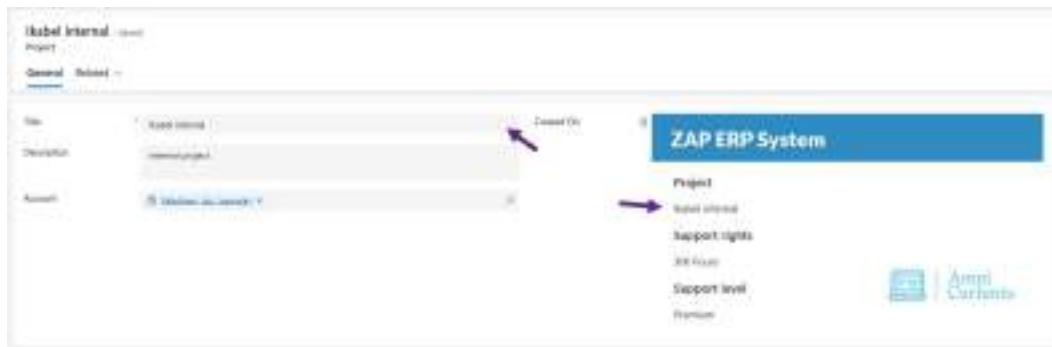


Figure 3.21: Canvas app integrated into the model-driven form

This integration shows just one of the situations where a canvas app can aid a model-driven solution to accomplish something that would require adding a code solution or a third-party component.

Improving model-driven apps with custom pages

Model-driven apps offer many capabilities focused on data-driven solutions. These solutions use the features available from Dataverse, but what if you need to go beyond what model-driven apps provide?

The Power Apps roadmap hints about a convergence model in which model-driven and canvas apps use the same designer to build solutions. This feature offers the ability to unite the excellent capabilities available in canvas apps within model-driven applications. More information on this subject is available here: <https://powerapps.microsoft.com/en-us/blog/custom-pages-for-converging-model-driven-apps-and-canvas-apps/>.

With this recipe, we will use canvas apps to use one of the mobile device's native capabilities – in this case, the **Global Positioning System (GPS)**.

Getting ready

Building on this chapter's solution, we will improve the data handling of the **Ticket operations** table. Before we begin, update the table's **Main** form to include all the custom columns created for this table. Also, in the **Ticket** table, include a sub-grid in the **Main** form to view all the related ticket operations.

The form should look like the following figure after these changes:

Figure 3.22: Ticket operation main form

This recipe will make use of static maps inside a custom page. We will leverage the HERE Maps service since it offers a free alternative to map services. A HERE Maps API key will be required to visualize the location information gathered from our custom page. Review the **Developer Guide** for instructions on how to create a free account and to obtain documentation of the Image API used in this recipe: <https://www.here.com/docs/bundle/map-image-developer-guide-v3/page/topics/quickstart.html>

Download the `ApplicationCode.js` file from the code folder in the GitHub repository: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-03>.

How to do it...

1. Open the Helpdesk solution and select **Apps** from the left pane. Click the three dots next to your model-driven app and select **Edit | Edit in new tab**.
2. From the toolbar, click **New** and select **Custom page**. On the next page, click **Create a new custom page**, give it a name, such as `Location handler`, and click **Create**.
3. Once the studio opens, click on the **Data** panel (database symbol) and click on **Add data**. Search for `Ticket operations` and select it.
4. Go back to the tree view and click on **App**. Using the formula bar, set the **OnStart** code to the following:

```
Set(
    gblRecord,
    If(
        IsBlank(Param("recordId")),
        First('Ticket operations'),
        LookUp(
            'Ticket operations',
            'Ticket operation' = GUID(Param("recordId"))
        )
    )
)
```

Using the ellipsis next to **App**, select **Run OnStart**.

5. From the toolbar, click on **Settings | Display**. Change the **Orientation** to **Portrait** and click **Apply** in the **Change app orientation** dialog. Close the **Settings** dialog.

6. Select **Screen1** on the tree view and rename it **MainScreen**. Using the formula bar, change the **Fill** color to `RGBA(102, 138, 166, 1)` and the **OnVisible** property to `Select(RefreshIcn)`



Ignore the error. It will resolve itself later on.

7. Using the **Insert** panel on the left, search for **reload** and select this icon. Set the following properties:
 - Using the **Properties** pane, change the **Color** to **White** and **Y** to 95
 - Using the formula bar, set **X** to `(Parent.Width - Self.Width) / 2`, **HoverColor** to `ColorFade(Self.Color, -10%)`, and the **OnSelect** property to the following:

```
UpdateContext(
{
    varLatitude: Location.Latitude,
    varLongitude: Location.Longitude
})
```

- Rename this icon **RefreshIcn**



If you receive a message from your browser asking for location permissions, allow it.

8. From the **Insert** panel, click on **Rectangle**. Set the following properties:
 - Using the **Properties** pane, change **X** to 0
 - Using the formula bar, set the **Fill** property to `RGBA(237, 237, 237, 1)`, **HoverFill** to `Self.Fill`, **Width** to `Parent.Width`, **Height** to `Parent.Height / 2`, and **Y** to `Parent.Height / 2`
 - Rename this rectangle **BackgroundRct**
9. Expand **Media** from the **Insert** panel, and click on **Image**. Set its properties as follows:
 - Using the **Properties** pane, change **Image position** to **Fill**, and **Border style** to **Solid**

- Using the formula bar, change **BorderColor** to `RGBA(202, 202, 202, 1)`, **HoverBorderColor** to `Self.BorderColor`, **Width** to `Parent.Width - 80`, **Height** to `Parent.Height / 1.5`, **Y** to `(Parent.Height - Self.Height) / 2`, and **Image** to the following code:

```
"https://image.maps.hereapi.com/mia/v3/base/mc/
center:{varLatitude},{varLongitude};zoom=17/1280x720/
png?apiKey={HEREMapsAPIKey}&style=explore.
day&overlay=point:{varLatitude},{varLongitude};icon=poi;
size=large"
```



Remember to swap {HEREMapsAPIKey} with your own HERE Maps API key.

- Rename this image `StaticMapImg`
- From the **Insert** panel, click to add a **Label**. Using the **Properties** pane, change **Text** to **Latitude**, **Font size** to 16, **Color** to **White**, **X** to 40, **Y** to 40, and **Height** to 40. Rename this label `LatitudeLbl`
 - Add a **Text box** from the **Insert** panel:
 - Using the **Properties** pane, set the **Font size** to 16, **X** to 40, **Y** to 90, and **Height** to 42
 - Using the formula bar, change **Value** to `gblLatitude` and **Width** to `(Parent.Width - 80) / 2.3`
 - Rename this text box `LatitudeTxt`
 - Repeat *Steps 10* and *11* to create the longitude items. Keep the same properties but change **X** to `StaticMapImg.X + StaticMapImg.Width - Self.Width` and set `gblLongitude` as the **Value** for the text box. You can also use copy and paste, but remember to rename the controls.
 - From the **Insert** panel, add a **Button**. Set the following properties:
 - Using the **Properties** pane, set **Text** to **Save location**, **Font size** to 18, **Font weight** to **Semibold**, **X** to 40, and **Height** to 80
 - Using the formula bar, change **Width** to `Parent.Width - 80`, **Y** to `Parent.Height - Self.Height - 40`, **FillColor** to `Parent.Fill`, and **HoverFillColor** to `ColorFade(Parent.Fill, -10%)`

- Rename this button to `SaveLocationBtn`

The application should look like the following figure:

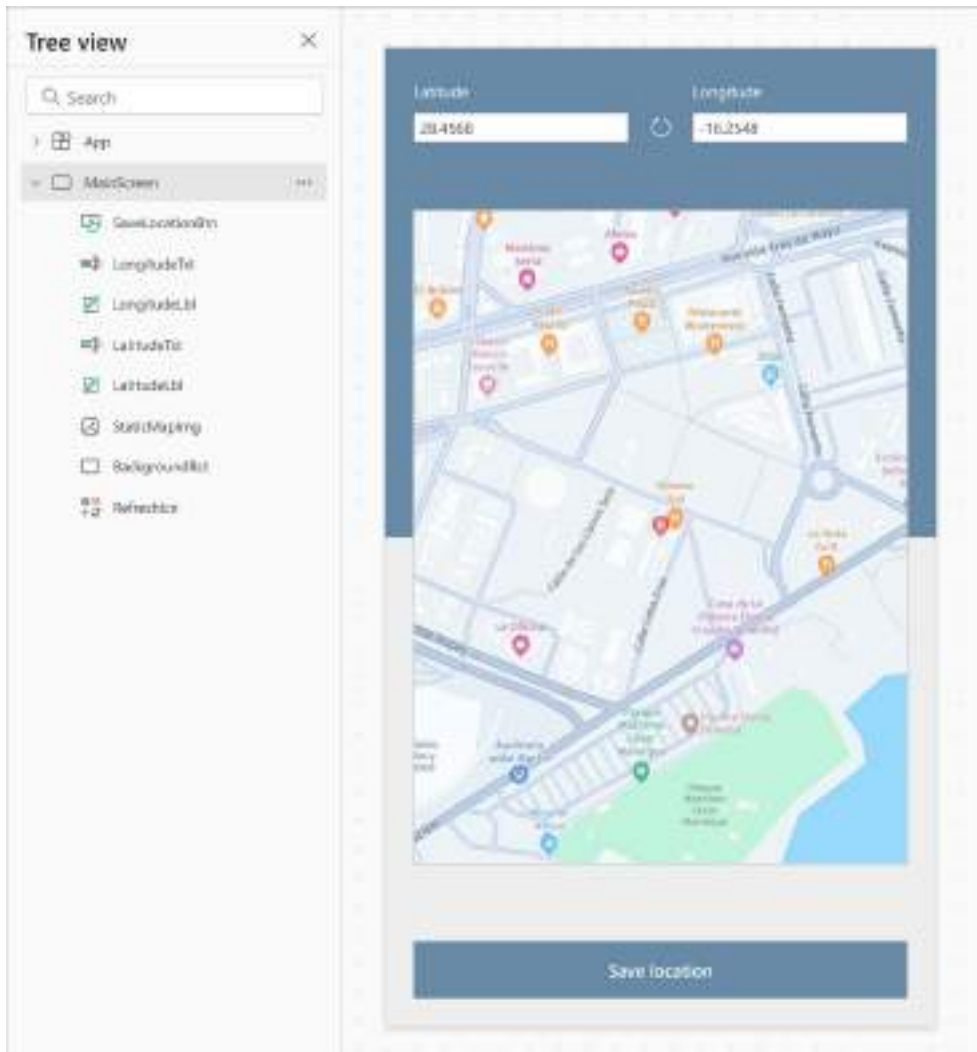


Figure 3.23: Location handler app

14. The last step is to add the following code to the `OnSelect` property of `SaveLocationBtn`:

```
Patch(
    'Ticket operations',
    gblRecord,
    {
        Latitude: gblLatitude,
        Longitude: gblLongitude
    }
);
Back()
```

15. From the toolbar, click on **Save**, and in the dialog that opens, click on **Save** again. Click on **Publish** to finish and **Back** to return to the solution designer after confirming to leave the app.
16. From the list of **Pages**, note the **Name** of your custom page. It should be something like `apc_locationhandler_89a8c`. Open the `ApplicationCode.js` file downloaded from the GitHub repository. Replace `<logical page name>` with the name of your custom page. Save your file.
17. From the toolbar, click on **New | More | Web resource** and, in the side dialog, click **Choose file** and select your `ApplicationCode.js` file. Set `Application Code` as the **Display name**, `ApplicationCode` as the **Name**, and keep **JavaScript (JS)** as the **Type**. Click **Save** when finished.
18. Go to the **Apps** section, click the ellipsis next to your model-driven app, and select **Edit | Edit in new tab**.
19. Once the designer opens, go to the second option on the left pane, **Navigation**, click the ellipsis next to the **Location handler** element, and click **Remove from navigation**.
20. From the toolbar, click on **Add page** and then **Dataverse table** in the opening dialog. Search for `Ticket operation`, select it, untick **Show in navigation**, and click **Add**.
21. From the **All other pages** pane on the left, click the ellipsis next to **Ticket operation** and select **Edit command bar | Edit in new tab**. If asked, click on **Save and continue**.
22. In the command bar dialog, select **Main form** and click on **Edit**.
23. From the left pane, click on **New | Command**. A dialog will open asking you to create a component library. Select **JavaScript** and click **Continue**. Set the following properties on the right panel: Get `Location` as **Label**, select **Use Icon** for **Icon**, and look for the **Pin** icon. Click on **Add library** in the **Action** section. Using the search field, look for `Application Code`, and click on **Add**. On the **Function name** field, enter `OpenSidePanel`. Click on **Add parameter**, and using the **Parameter 1** dropdown, select **PrimaryControl**.

24. From the toolbar, click on **Save and Publish**; when it finishes, click **Play**.

How it works...

From the application, open a ticket and then add a **New Ticket operation** from the related sub-grid:



Figure 3.24: New Ticket operation

Enter some sample data and save the ticket operation. Once saved, click the **Get Location** button to get your current location. When clicking **Save location**, this information will get transferred to the corresponding fields of the form.



Sometimes, the application takes some time to gather the location information. By pressing the reload icon, we can request this data again.

If the custom page doesn't appear, publish the model-driven application and try again.

There's more...

With the continuous convergence of the platform, canvas apps offer tighter integration with model-driven solutions by improving application needs such as building panels, dialogs, or even a main page inside your model-driven app.

Whenever there's a user interface requirement outside of what a model-driven form can do, you can provide it by creating small low-code interfaces using canvas apps. You can then integrate them using JavaScript code or the modern app designer for model-driven apps.

For more information regarding this, please visit these links:

- Overview of custom pages for model-driven apps: <https://learn.microsoft.com/en-us/power-apps/maker/model-driven-apps/model-app-page-overview>
- Navigating to and from a custom page using the client API: <https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/navigate-to-custom-page-examples#sometimes>

These features are evolving as we speak. For example, not all canvas controls are available for custom pages. As an unsupported workaround, open the regular canvas studio on one tab and the custom page designer on another. Copy and paste the required controls from the regular one to the custom page.

There are even more capabilities coming with the following platform releases. Keep an eye on the roadmap for more information: <https://learn.microsoft.com/en-us/dynamics365/release-plans/>.

Deploying your solution to your production environment

Whether building a full-fledged model-driven or canvas application, you should consider building solutions with an ALM insight. This concept allows you to differentiate between a development, testing, and production environment, their role in your application-building process, and how your solution will get deployed through them. These environment types are just an example of an ALM implementation. You can discover more information about ALM in Power Platform here: <https://learn.microsoft.com/en-us/power-platform/alm/basics-alm>.

Power Platform leverages the concept of pipelines as the tool to connect environments and then deploy solutions between them. They can be as straightforward or complex as you or your organization want them to be. To get more information, follow this link: <https://learn.microsoft.com/en-us/power-platform/alm/pipelines>.

Getting ready

To fully integrate ALM into your solutions, you should keep in mind that it's only available for environments with a Dataverse database. That's one of the selling points of this critical element of Power Platform: it maintains your data and helps you manage the solutions you create to handle this data.

To build this recipe, we will need to have at least another environment to deploy the solution. Please refer to the first recipe of this chapter, *Configuring the environment*, to create a new environment.

How to do it...

1. Open your solution from the Power Apps maker portal.
2. On the left pane, you will find a rocket icon. This icon refers to the pipeline we will use to deploy our solution to another environment. Click on it to open this section. The first time you open the pipelines, it will take a while to set up everything for you.

3. From the toolbar, click on + **Create pipeline**. On the panel that opens on the right, give your pipeline a **Name** and **Description**. The current environment will appear as the **Source Environment**. Select the **Target Environment** from the list, as seen in the following figure. Hit **Save** when finished.

Create pipeline [X]

Name *

Deploy to Playground

Description

Source Environment

Power Apps Cookbook

Target Environment *

Your solution will be copied to the environment you choose. [Learn more](#)

Search [Filter]

Demos

Playground

Save Cancel

Figure 3.25: Create pipeline

4. This process will take a couple of seconds. When it finishes, the pipeline will be available from the list of pipelines to use. To test it, click the **Deploy here** button at the bottom of the screen.
5. The first step will be to set the schedule. You can do it on demand or at a later date and time. Leave **Now** selected and click **Next** to start the validation process.
6. When the process finishes, you will end up in the **Summary** section. Click **Deploy** to start the deployment.

How it works...

From there, all the components will start traveling through the defined pipeline from one environment to the other. The validation process should have reviewed any possible errors regarding the required components or any issues with the destination environment. If everything checks out, you should be seeing the deployment progress as seen in the following figure:



Figure 3.26: Deployment progress

When the deployment finishes, you can click on the environment selector at the top of the maker portal to go to the destination environment. You will see the same solution deployed as a managed solution.

There's more...

The concept of having a tool to deliver your solutions from one environment to another doesn't start with Power Platform pipelines. Azure DevOps has been around for many years, allowing makers to have more granular control over the changes a solution requires when changing environments, such as environment variables and connection references. We also have another flavor if we opt to use GitHub as our deployment mechanism.

To get more information regarding Azure DevOps, please visit <https://learn.microsoft.com/en-us/power-platform/alm/devops-build-tools>, and if you plan to use GitHub, review the documentation here: <https://learn.microsoft.com/en-us/power-platform/alm/devops-github-actions>.

Discovering the Copilot experience when building model-driven apps

So far, we have followed the complete process of building a solution using Power Platform. From creating the environment to the deployment process of our Helpdesk application. Every step of this process follows a specific purpose, allowing a team of makers to focus on particular tasks. One can be in charge of the data model, another can build the app, and another can create a custom page to extend capabilities. In this era of artificial intelligence assistance in every platform, why not assign one of these tasks to the Power Platform Copilot?

You will discover that Copilot is present in certain parts of the application-building process. We can find it when building canvas apps, dealing with Power Fx code, and creating the data model. Soon, it will play a more significant role in various aspects of Power Platform. To keep track of all the improvements, follow this link: <https://learn.microsoft.com/en-us/power-platform/copilot>.

Getting ready

We will take advantage of the data model capabilities of Copilot in Power Apps to prepare some of the tables mentioned in this chapter, but only for creating the new tables since Copilot is still learning how to deal with existing tables from an actual prompt. From there, we could continue building the app as we did through this chapter.

We will recreate the *Defining data structures* recipe with the help of Copilot. Once the environment and solution are ready by following the first two recipes, we will land in the data structure creation process. Using an AI prompt, we will create the following tables: Project, Ticket, and Ticket operation.

Copilot doesn't have a parameter to define the solution it will be working on. As a workaround, we will adapt the maker portal to have the Helpdesk solution as default for new objects in the environment. This way, everything done by Copilot will be included in our solution.

How to do it...

1. From the Power Apps maker portal, click on the gear at the top right of the screen and click **Set preferred solution**. From the dialog that opens, select your solution. If you already have the Helpdesk solution complete, you can create a new solution to test this feature.

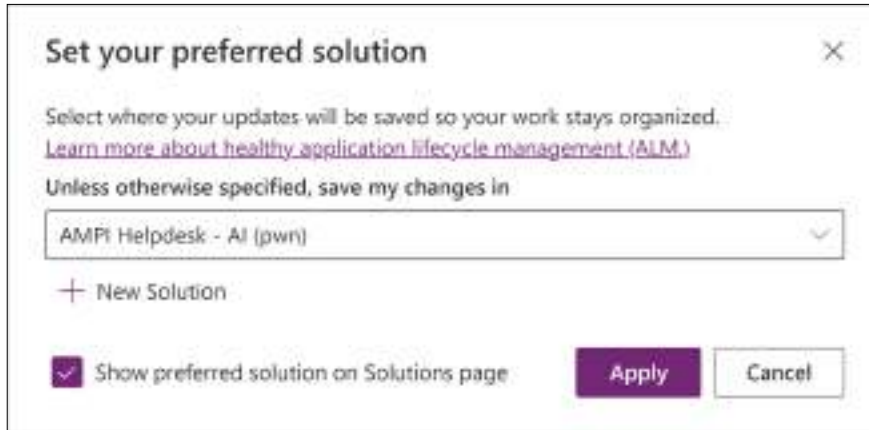


Figure 3.27: Preferred solution dialog

2. Go to the maker's portal homepage. At the time of writing, using the AI assistance when building tables is in preview, so make sure that you have enabled the **Try the new data experience** toggle.
3. Go to the **Tables** section on the left pane. Click on **Create new tables** and then select **Start with Copilot** from the **Choose an option to create tables** section. This option will open a dialog to enter the required prompt to build your tables. For our example, enter this and click **Generate** when finished:

I'm building a help desk solution.

I will need a Tickets table. Each ticket will have a description, a resolution, and a choice of priority, status, and validation.

I will need a Projects table. Each project will have a name, description, start date, and end date

I will need a Ticket operations table. Each Ticket operation will have a description, duration, latitude, longitude, and the operation status.

Each ticket will have one related Project and many Ticket operations.

When invoking Copilot from the maker's homepage, the main focus is to make a UI, so we have limited characters for our prompt. That's not the case when creating the data structures. Keep that in mind when explaining your request.

However, if you invoke Copilot from the **Apps** section of the maker portal by clicking on **Start with Copilot**, it will launch the AI table creation process and, from there, it will allow you to create a canvas app or a model-driven app from the toolbar, as seen in the following image:

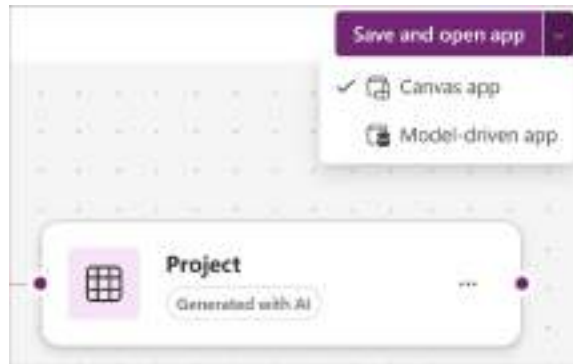


Figure 3.29: App creation experience with Copilot

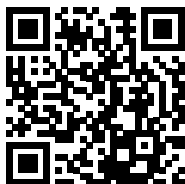
Another great feature of Copilot is that the tables are prefilled with data, which helps identify the most suitable column type and its properties. On top of that, the forms include the requested fields by default, giving you a headstart on the form-building process. The same goes for the views; they contain more meaningful columns from the beginning.

In essence, Copilot is our trusty ally, empowering us to concentrate on building solutions to address specific requests, much like the entire Power Platform does.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



4

Choosing the Right Data Source for Your Applications

Selecting the right data source is one of the most important decisions when designing your application infrastructure. Many factors come into play, such as features, security, speed, pricing, and integrations.

Thankfully, your solution's needs will help you select the right one. For example, it's better to use a database with a certified connector rather than building it yourself; if your application needs to store a specific data type, it's best to choose a database that can handle it natively. Another example is when working with canvas apps: setting up different data sources on the same application opens many possibilities when interacting with data.

Before diving into this chapter, we will go into more detail about Power Platform's licensing model, a crucial topic for Power Platform makers. Having covered this concept, we will be able to discuss data in the Power Platform with the help of these recipes:

- Reducing your application complexity by using Azure SQL database views
- Choosing the right data source – real-world tests
- Integrating on-premises data with canvas apps
- Improving application speed and responsiveness by using static data
- Consuming external data by using dataflows

Technical requirements

To thoroughly test all the recipes tackling various forms of data sources, we will need a Power Apps or Power Automate licensing plan. Please refer to the *Preface* to get more information about the licensing requirements.

Visit our GitHub repository for the code samples of this chapter: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-04>.

Data sources and the licensing model

We need to understand the unique differentiator that makes us need a license: whether a connector is labeled premium. This tag activates the end-user license requirement when using the app you created, even if you own a license to build it.

The current licensing model for end users is reminiscent of the SQL Server **Client Access Licenses (CALs)**, in which every user accessing the server requires a pass. It is the same for the Power Platform. You add capacity to your environments to allow unlicensed users to access your app.

The platform recognizes which apps are built with premium connectors to access data or services. When you access the details of your apps or flows, you will see a **License designation** or **Plan** that will inform you about this:



Figure 4.1: License designation for a canvas app

These changes came on October 1, 2019, when many apps already used premium connectors, mainly Azure services, which were initially standard. Microsoft then offered a **grandfathering period** to allow the use of these apps without the license requirement for apps published before October 1, 2019. For more information regarding this exception, please visit <https://learn.microsoft.com/en-gb/power-platform/admin/powerapps-flow-licensing-faq#how-does-the-change-to-power-apps-and-power-automate-use-rights-for-microsoft-365-applications-affect-me-if-i-purchased-the-subscriptions-prior-to-oct-1-2019-will-my-existing-apps-and-power-automate-workflows-continue-to-work>.

When designing your infrastructure, always consider the licensing model beforehand to avoid surprises at the end of your project when the user base starts growing.

However, you need to be conscious of the value of this consumption model. Previously, you would need to buy licenses for every user using your app at any moment instead of purchasing passes only for the actual users. Let me explain this with an example.

Let's pretend we have an organization with 10 employees that would use your app at some specific points in time:

- Five administrative employees will use the app daily
- Four managers will use the app quarterly
- One employee from marketing will use the app occasionally

In the previous model, if the license price were 10 USD, you would have had to pay 100 USD per month. With the consumption model, you can make calculations based on usage to allow a certain amount of simultaneous users, say six, which would reduce the price to 60 USD per month.

Keep in mind that the Power Apps per-app licenses used for the consumption model can only be acquired through Microsoft Partners. Please refer to this link to get the current Power Apps pricing plans: <https://www.microsoft.com/en-us/power-platform/products/power-apps/pricing>.

Power Apps/Power Automate for Microsoft 365

You don't always need a license to work with Power Apps or Power Automate. Starting from the regular Microsoft 365 Business Basic license, you can build apps in the context of Microsoft 365 using standard connectors.

Real-world applications use SharePoint as their data source without incurring extra costs, even for mobile apps with offline capabilities. To get more information regarding this topic, visit <https://learn.microsoft.com/en-us/power-platform/admin/pricing-billing-skus#power-apps-and-power-automate-for-microsoft-365>.

Furthermore, **Dataverse for Teams** allows us to build apps with a light version of Dataverse in the Microsoft Teams platform, using the same license required to use Teams. Refer to *Chapter 8, Power Apps Everywhere*, for recipe examples.

Let's deep dive into the data source recipes with a clear concept of the licensing model in mind.

Choosing the right data source – real-world tests

Every data source has unique features that make it ideal for specific scenarios, whether you need a robust relational database, a document-centric infrastructure, or a lightweight approach for mobile applications.

This recipe will test some of Microsoft’s most well-known players in the data source world: **Dataverse**, **SharePoint**, and **Azure SQL Database**. With the help of Power Automate, we will test both the reading and writing speeds of these three contestants.

Getting ready

The first requirement is to have the same data structure for each service:

- A list with the default `Title` column in SharePoint
- A table with a `Title` field in Azure SQL using the Basic performance tier
- A table with a `Title` column in Dataverse

Refer to *Chapter 1, App Maker Basics*, for instructions on creating these data sources. Once complete, proceed to the Power Automate portal to build our tests at <https://make.powerautomate.com/>.

At the time of writing, Power Automate is transitioning to a new designer based on the one used in Azure Logic Apps, the so-called father of Power Automate. This new designer includes excellent features to help makers build their cloud flows faster and more reliably. An example is the ability to save cloud flows as drafts, even with errors.

For the sake of having the most up-to-date version, we will use the new designer throughout the book. However, when creating cloud flows from solutions, we land on the now-called Classic designer. To use the new designer, save the flow and open it again.



Another workaround is to enable **Create new canvas apps and cloud flows in Dataverse solutions in your environment** from the Power Platform admin center and then set your working solution as the preferred one. For more information on these topics, please visit <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/add-app-solution-default> and <https://learn.microsoft.com/en-us/power-apps/maker/data-platform/preferred-solution>.

How to do it...

1. We will need some data to run our test, so let’s build a Power Automate cloud flow to write data into SharePoint. First, go to **Solutions** and create a new solution named `Speed tests`.

2. First, let's prepare our solutions' connection references so we can name them appropriately. From the toolbar, click **New | More | Connection Reference**. From the open panel, set the **Display name** to SpeedTest - SharePoint, leave the **Name** as it is, and choose **SharePoint** from the **Connector** dropdown. If you already have a connection, select it. If not, create one with your credentials by selecting **New connection**.
3. Repeat *step 2* to create a **Dataverse** connection reference and then again for an **SQL Server connection**. Name them accordingly.
4. The next step is to create environment variables for the SharePoint site and list that we will use in our cloud flows. From the toolbar, click **New | More | Environment variable**. From the open panel, set the **Display name** to SpeedTest-SharePoint-Site, choose **Data source** for the **Data Type**, and **SharePoint** for the **Connector**. Select a previously created connection or create one from the **Connection** dropdown. Set **Site** for the **Parameter Type**, click + **New site value**, and select the site that holds the list required from the *Getting ready* section.
5. For the List environment variable, repeat *step 4*, but set **List** for the **Parameter Type**. Select the previously created Site environment variable from the **Site** dropdown, and click on + **New list value** to select the list from the site.
6. From the toolbar, click **New | Automation | Cloud flow | Instant**.
7. Set the **Flow name** as Speed Test - SharePoint insert, choose **Manually trigger a flow**, and click on **Create**.
8. We will load 500 records into each data source. To do this, we need to create a loop that will load the data. Click on the plus sign to reveal the **Add an action button** and search for the **Initialize variable** action to add it, **Name** it autoNumber, set the **Type** as Integer, and set the **Value** to 0

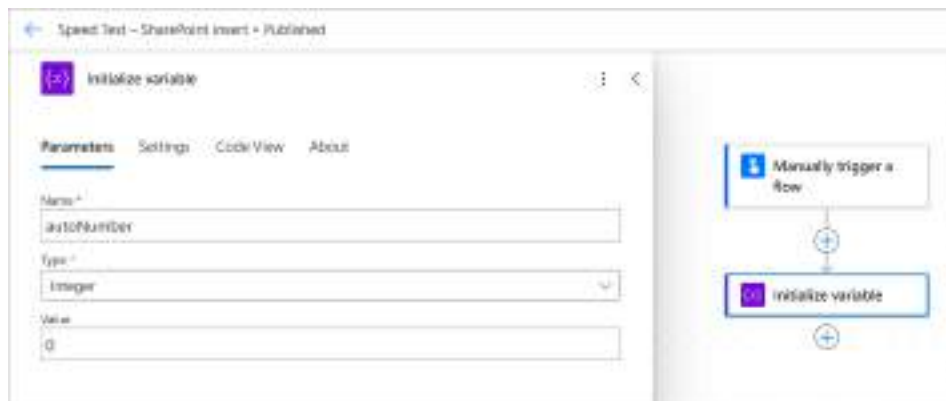


Figure 4.2: Loop variable initialization

9. Add a new **Do until** action, which will run our data input loop. Set this action to execute until autoNumber is equal to 500. Click the **Change limits** link and set the **Count** to 600
10. Insert an action inside this loop. Search for **SharePoint** and then look for **Create item**. Configure it to use your SharePoint site and list environment variables by selecting **Enter custom value** from the respective dropdowns and then clicking on dynamic data (the lightning arrow). Set the **Title** to Element, and select the autoNumber variable.
11. Add one last **Increment variable** action to increase the autoNumber value by 1. Setting this will make sure our loop keeps running until the value reaches 500:

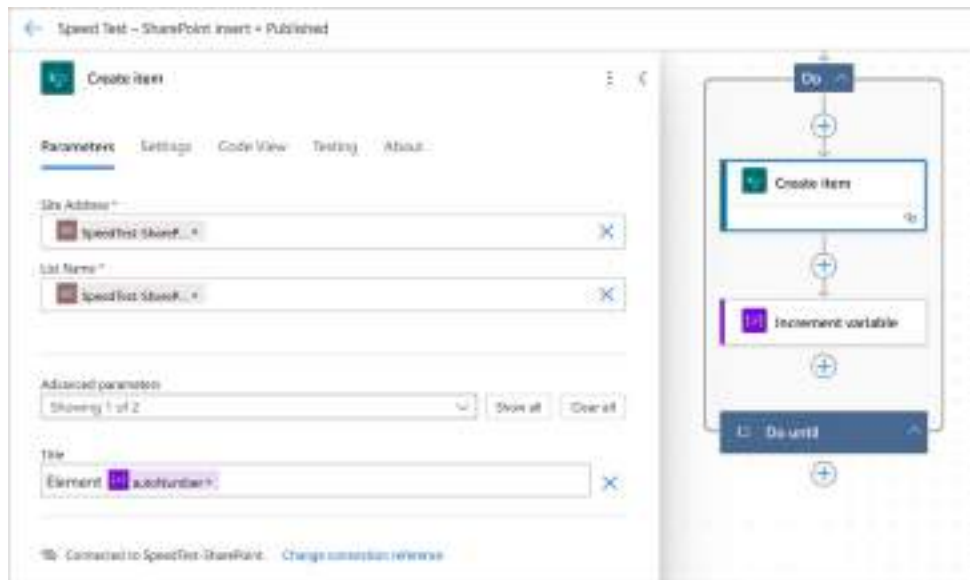


Figure 4.3: Load data actions

12. From the toolbar, click **Save draft** and then run it by clicking on **Test**. It will create the items in SharePoint.
13. Create a new cloud flow repeating *steps 6-12* for Azure SQL, using **Insert row (V2)** in *step 10* with your server, database, and table names. Name this flow **Speed Test - Azure SQL insert**
14. To build our last test, create another cloud flow and repeat *steps 6-12* for Dataverse using the **Add a new row** action in *step 10*, setting the table name to the one you created in the *Getting ready* section. Name this flow **Speed Test - Dataverse insert**

15. Now is the turn of the reading tests. From the solution, create a cloud flow for Dataverse, repeating *steps 6-7*, and then add an action called **List rows**. On the **Settings** tab of this action, enable **Pagination** and set a **Threshold** value of 500. Configure it to the same table as the writing tests. Name this flow Speed Test – Dataverse read. If you receive a warning message, you can safely ignore it for the sake of this test.



Connectors retrieve data in small portions (by default, 100 items), which serves as a protective measure to prevent accidentally querying large amounts of data. Consider a scenario when querying a massive database without applying the correct filters; the platform would have to handle a significant amount of work.

For specific scenarios where you need to read more than the default, we can use pagination. This concept translates into getting a predefined amount of data in chunks. To get more information about this, please visit <https://learn.microsoft.com/en-us/power-automate/dataverse/list-rows?tabs=edit-with-copilot#turn-on-pagination-to-request-more-than-5000-rows>

16. For Azure SQL, repeat *step 15* with the **Get rows (V2)** action. Map it to the previously used server, database, and table name. Name this flow Speed Test – Azure SQL read
17. Lastly, create one last cloud flow for SharePoint and repeat *step 15* using the **Get items** action. Set your SharePoint site address and list name. Name this flow Speed Test – SharePoint read. Ignore the warning again if you receive one.
18. For each cloud flow, click on **Save** and **Test**:

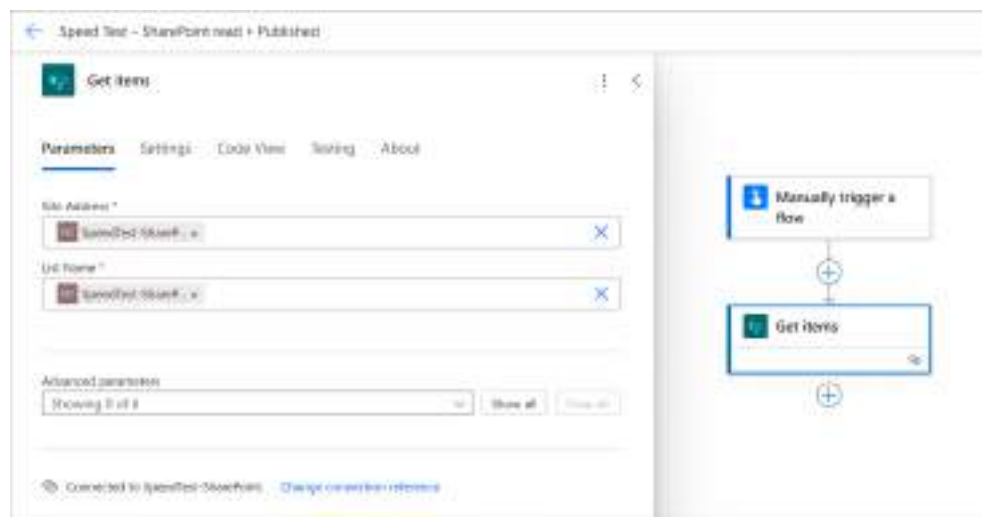


Figure 4.4: Read data actions

How it works...

The six cloud flows we have just created will help you get an overview of these data sources’ data-handling speeds. You will need to execute the insert cloud flows to have data for our reading tests. In my test scenarios, these were the execution times I obtained on an average of three runs:

Test	SharePoint	Azure SQL	Dataverse
Writing test	04:08 minutes	04:14 minutes	04:24 minutes

Table 4.1: Write results

Keep in mind that these results may vary depending on your current location and the location of your Power Platform environment.

If we look at the reading test, we get a surprising result on an average of three runs:

Test	SharePoint	Azure SQL	Dataverse
Reading test	5 seconds	347 ms	2 seconds

Table 4.2: Read results

We got 500 records really fast. This kind of outcome is mainly due to the caching mechanisms that all three have in their infrastructure. When returning data, it also depends on each specific platform: SharePoint and Dataverse return metadata with each query, but Azure SQL does not. If we improve the requests by specifying only the fields we need, we will get faster response times. For example, querying only the Title column in Dataverse lowered the results to approximately 320 milliseconds.

However, this opens a whole new debate. If the data in our application is mainly for reading purposes, then why not use SharePoint? It is cheaper and, while not the quickest, still performed well in the reading test; this, of course, will depend on the complexity of your entities. Using only one field is not the same as including complex data types in the mix. Still, this specific SharePoint result is something to bear in mind.

There's more...

The platform used for SharePoint and Dataverse has predefined performance limits to comply with SLAs and response times. For the Azure SQL test, the database was running on the Basic tier level. Increasing the tier level improves performance on memory or throughput levels. You can even upgrade the database to a more robust and performant level. Consider these skills when deciding on your data sources.

I hope this workshop sparks your curiosity regarding the complexity behind the data sources you will use in your developments.

SharePoint Online performance considerations

When working with SharePoint Online, you need to remember that you are working with a web application with specific response and performance levels to provide the best user experience. Microsoft has designed a set of rules to meet this, for example, setting a view threshold of 5,000 elements per list. Even though a list can hold up to 30 million items, querying data exceeding this threshold limit will result in platform errors, making the list unresponsive.

These are a few recommendations to keep your lists performant:

- Set an index for the columns you wish to filter. Remember to do this beforehand since you cannot change it if you have exceeded the threshold.
- Prepare segmented views for lists that might carry a large number of items. Examples can be categories, years, and departments.
- To avoid performance issues, build views with fewer than 12 People, Lookup, or Managed metadata columns.

For reference on this subject, check out the following links:

- <https://support.microsoft.com/en-us/office/manage-large-lists-and-libraries-in-sharepoint-b8588dae-9387-48c2-9248-c24122f07c59>
- <https://support.microsoft.com/en-us/office/use-filtering-to-modify-a-sharepoint-view-3d8efc52-0808-4731-8f9b-3dfaeacea3d4>

Reducing your application complexity by using Azure SQL database views

Performance is one of the main objectives of almost all data source engines. Database developers tweak the innards of their core systems to make them as efficient as possible while also adding new features for app makers. Knowing this, why not leverage the efficiency of these platforms?

That's the primary purpose of this recipe: to learn how to improve our apps' responsiveness and complexity by making the data sources work for us.

Getting ready

To build our use case, we will improve an application connected to an Azure SQL database. The database needs five tables that will make up our data catalog. The scripts to create them and the base apps are available in the GitHub repository mentioned in the *Technical requirements* section.

If you don't have an Azure subscription, you can request a free version with credit for 30 days to test all the services available on this cloud platform. Apply for it at <https://azure.microsoft.com/en-us/free/>.

How to do it...

Please refer to *Chapter 1, App Maker Basics*, for steps that will help you build the Azure SQL database needed for the app. First, download all the files from the repository and have them ready.

Provisioning the data source

1. Navigate to the Azure portal, <https://portal.azure.com>.
2. Go to your Azure SQL database to execute the scripts provided from the Database scripts folder. These scripts will prepare the tables needed for our app. To complete this task, you can use either the Azure Data Studio, <https://learn.microsoft.com/en-us/azure-data-studio/download-azure-data-studio>, or the Azure portal itself. Accessing it through the portal is straightforward; you don't need to install any software. Load the scripts using the **Open query** option in the toolbar and execute them one by one by clicking **Run** on the toolbar. In the following figure, you can see how to access the **Query editor**:

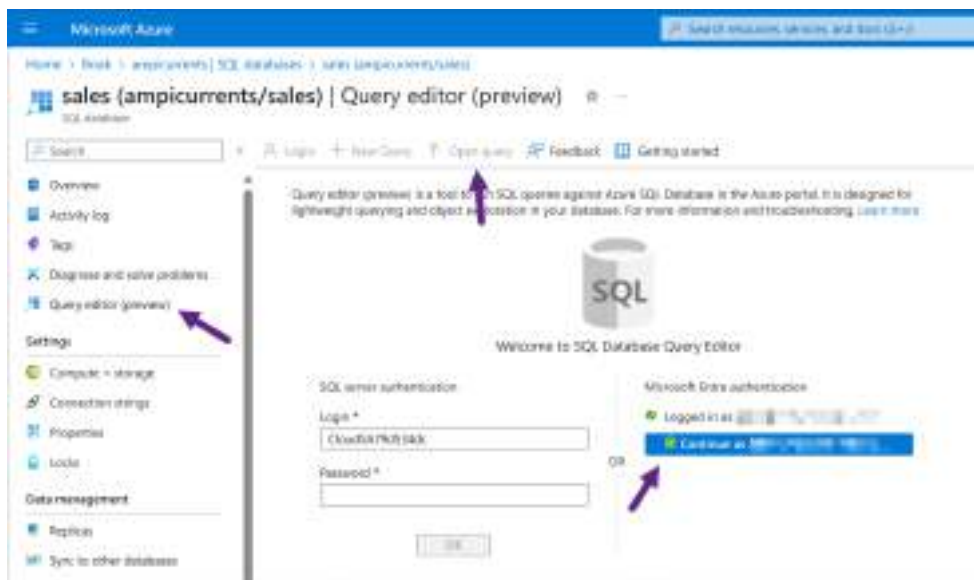


Figure 4.5: Query editor in the Azure portal

3. Go to the Power Apps maker portal, <https://make.powerapps.com>, click on **Solutions** on the left pane, and from the toolbar, click **Import solution**. On the pane that opens on the right, click **Browse**, select the AzureSQLViews.zip package file you downloaded from the recipe's GitHub folder, and click **Next** and then **Next** again to begin the import process.
4. In the **Connections** section, click **Create** to configure the connection to Azure SQL. Select the **Microsoft Entra ID Integrated** as the **Authentication type**, and click **Sign in**. Enter the credentials for your database and then click **Import**.

5. The import pane will close, and you will return to the **Solutions** section of the maker portal. When the import process completes, you will receive a green banner letting you know the process has finished successfully:

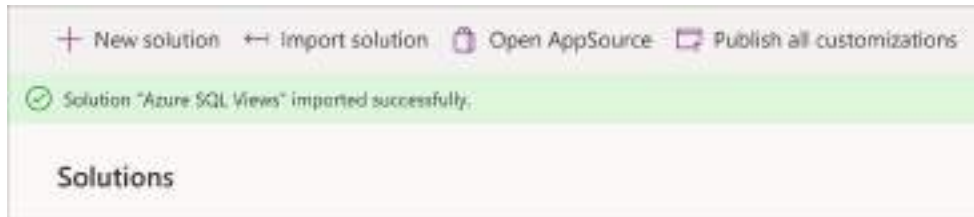


Figure 4.6: Import process complete

6. From the solution's list, click to open **Azure SQL Views**. Click the 3-dots menu beside the **PowerTrack SQL** app and select **Edit**. On the open dialog, click **Allow** to approve using the **SQL Server** connector.
7. Review the app and test it to see that it's getting the data from your Azure SQL Server database. You should be able to press play on Power Apps Studio to test your app. When selecting customers from the combo box, the related tickets should appear in the gallery, as you can see in the following screenshot:

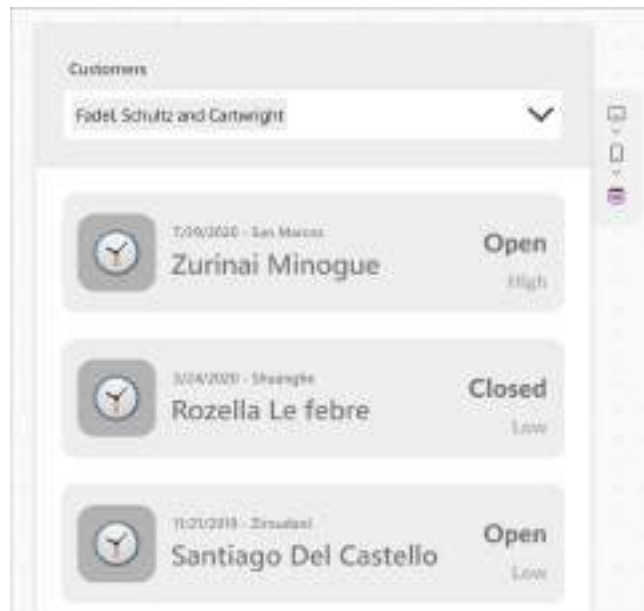


Figure 4.7: List of tickets

If you can't get the data from your database, remove all the tables already connected to the application and add them again.

8. Let's inspect the gallery's Items property:

```
Filter(
    AddColumns(
        tickets,
        Technician,
        LookUp(
            technicians,
            id = technician_id, full_name
        ),
        Status,
        LookUp(
            ticket_status,
            id = status_id,
            description
        ),
        Priority,
        LookUp(
            ticket_priority,
            id = priority_id,
            description
        )
    ),
    customer_id = CustomersCbz.Selected.id
)
```

As you can see, several complex formulas accomplish the relationships between the five tables. First, we filter the tickets for the selected customer, and then we add columns to get the technician's name and the descriptions for the statuses and priorities.

Having this many formulas to query data reduces performance because we make multiple calls to the backend to get the data. It also adds a maintenance overhead for your app. Let's improve this by using an Azure SQL view.

9. Connect to your Azure SQL database using your preferred method and execute the provided script, VIEW - TicketData.sql

10. To include the `vw_TicketData` view in the app, click on the **Data** panel on the left (represented by a database icon), click **Add data**, search for SQL, and pick **SQL Server**. Select your previously configured connection, choose the `vw_TicketData` view from your database, and click **Connect** when you're finished.

Using this database view, we can accomplish several tasks:

- Reduce the complexity of the gallery's `Items` property by removing all the formulas, leaving just the filter for the selected customer:

```
Filter(  
    vw_TicketData,  
    customer_id = CustomersCbx.Selected.id  
)
```

- Eliminate the need for the lookup tables, as they don't need to be connected to our app to get the descriptions of the statuses, priorities, or technician data. You can even remove the `tickets` table because the view provides all the information needed, leaving you just with `customers` and `vw_TicketData` in the **Data** panel of your app.

How it works...

The application performance improves because the relationship process is executed at the database level and not from the Power Apps app. We also decreased the data transfers between the data source and our app when we reduced the tables connected to the app. The code also looks cleaner, and it's easier to maintain.

The script to create the view links the database tables using the built-in SQL language. It gets only the fields we need and joins the four tables by their identity fields.

This approach also adds another enhancement regarding development workload: the team in charge of database maintenance and performance can provide us with these views because they have better insights on the best way to obtain this data.

There's more...

This recipe is just an example of how the capabilities of our chosen data source can improve our app. It will be helpful if you know the features available in your data service so you can take advantage of them when building your apps.

Another similar example is the use of Dataverse table views. It works on the same principles as the SQL database views, giving us just the columns we need with the required data. We can then use these views in our apps by invoking them in our Filter clause:

```
Filter(Tickets, 'Tickets (Views)'. 'Active Tickets')
```

The list doesn't stop here; you can use store procedures to build code into your SQL database, apply business rules to your Dataverse tables, improve user authentication, and more. These are just examples of the capabilities available for these two data sources. Make your application-building process more straightforward by leveraging them.

Integrating on-premises data with canvas apps

Not all data lives in the cloud; sometimes, organizational restrictions, legacy platforms that are too expensive to migrate, or complex network infrastructures make our job a little more complicated.

There are different ways to overcome these challenges, such as synchronization processes and exposing application interfaces outside the local infrastructure. Thankfully, one of the most significant features of the Power Platform is the ability to connect to on-premises services using a data gateway.

The on-premises data gateway acts as a bridge between the local services and Azure. There is no need to set authentication methods for the services you need to access; this only allows the platforms to be visible to each other—authentication takes place afterward.

Gateways not only allow you to connect to databases; they also include access to the file system, local SharePoint servers, and other on-premises services.

Getting ready

Once you have identified the on-premises system you want to access, you need to choose the type of gateway you want to use: regular or personal mode. As the names imply, the difference lies in the number of users. The personal mode only allows one user, and the other shares the gateway between multiple users.

There are specific use cases for each. For example, why configure a gateway for the entire organization if only the data analyst needs to gather information from the local SQL server? She can install the personal gateway on her computer to collect data.

Regarding technical requirements, you will need a Power Apps license to access on-premises data. As discussed in the *Preface* of this book, you can use a **Premium, per app**, or **Pay as you go** plan. For personal use, you can also take advantage of the Power Apps Developer plan, which lets you access premium connectors and on-premises gateways.

The sample application and the images are available from the GitHub repository located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-04>.

How to do it...

1. For this recipe, we are going to use a personal gateway. To add one, go to the Power Apps maker portal and, from the left pane, Click **More, Discover all**, and then in the **Data Management** section, click on **Gateways**. From the toolbar, click **New gateway**; this will open a new web page prompting you to get the **On-Premises Data Gateway**. Download and run the installation wizard:

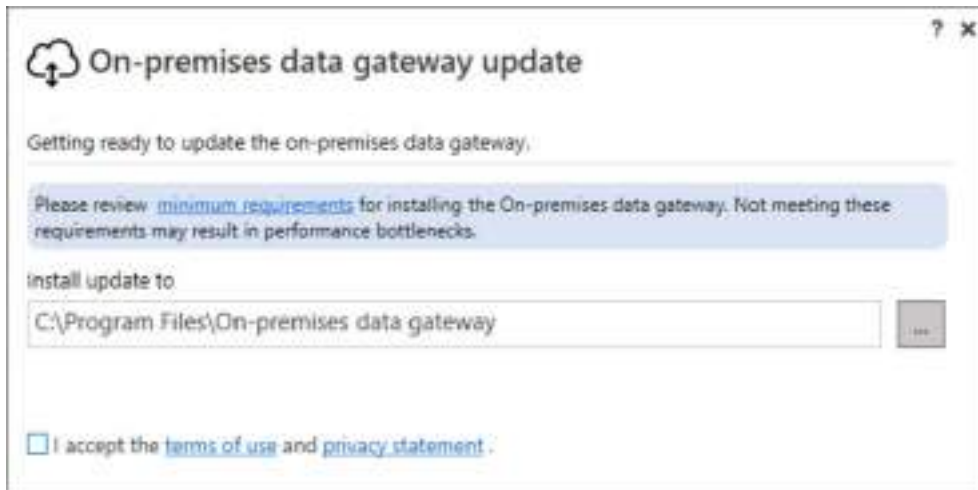


Figure 4.8: Data gateway installation

If the download doesn't start, use this link instead: <https://go.microsoft.com/fwlink/?LinkId=2116849&clcid=0x409>

2. Configure it by using an organizational account to sign in. Register the new gateway and give it a name, such as `Personal connection`. Set a recovery key required to recover the gateway and hit **Configure**. After the configuration is complete, you should see a green checkmark beside Power Apps, Power Automate, and Power BI, as shown:

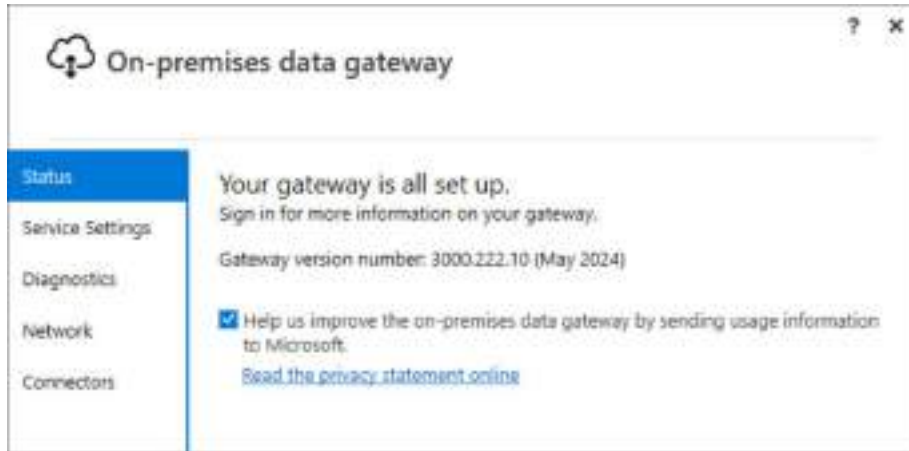


Figure 4.9: Successful gateway installation

3. To verify the visibility from the cloud, go back to the Power Apps maker portal, and you should see your newly created gateway:

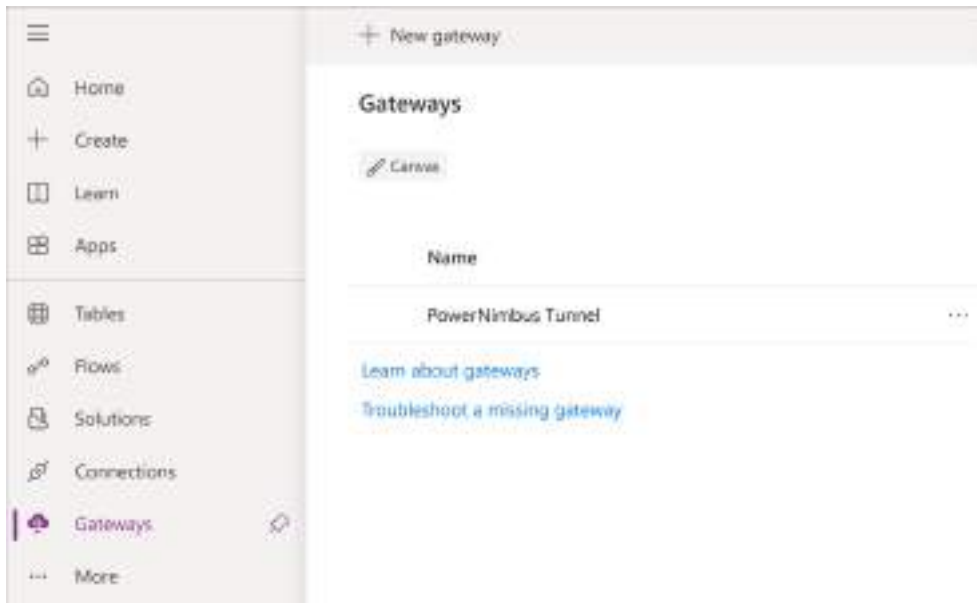


Figure 4.10: Local gateway in the Power Apps maker portal

4. Create a new solution with a name such as **Power Premises**. From there, create a new canvas app with the phone layout. From the left pane, select **Data**, click on **Add data**, search for **File System**, and then click on **Add connection**.
5. A right pane will open where you need to set the root folder from where the connection will display the list of files and folders, for example, **C:.** As for authentication, you will need to enter a local username and password to ensure the platform can connect. The gateway should appear on the gateways list. Click **Connect** when you're done.



If you use Windows 10/11 with a configuration to log in using an organizational account, you can find your username by going to **System Settings | Accounts | Other users**. Your username will be on the **Workplace or school users** list starting with **AzureAD\<YourFullName>**. For local accounts, the default is **WORKGROUP\<username>**.

6. We will connect through the gateway to get images from a local folder. We have provided sample images in the GitHub repository. Place them in a folder on the device with the data gateway configured, for example, **C:\Images**. Remember to use the same root folder as in *step 5*.
7. From the left pane, go to the **Insert** panel (plus symbol) and click on **Vertical gallery**. From the properties pane located on the right, change the **Layout** to **Title and subtitle**. Using the formula bar, set the **Data source** property to `FileSystem.ListFolder("Images")`.
8. Inside the gallery, remove **NextArrow2** and set the **Title2 Text** property to `ThisItem.DisplayName`, and set the **Subtitle2 Text** property to `ThisItem.MediaType`. Name the gallery **ImagesGly**.
9. Click anywhere on the canvas to deselect the gallery, go to the **Insert** panel, expand **Media**, and click on **Image**. Using the formula bar, set the **Image** property to `FileSystem.GetFileContentByPath(ImagesGly.Selected.Path)`
10. Rearrange the controls as you see fit and **Save** the app.

How it works...

In the following screenshot, the example application lists all images found in the **Images** folder of the local computer connected to a data gateway.

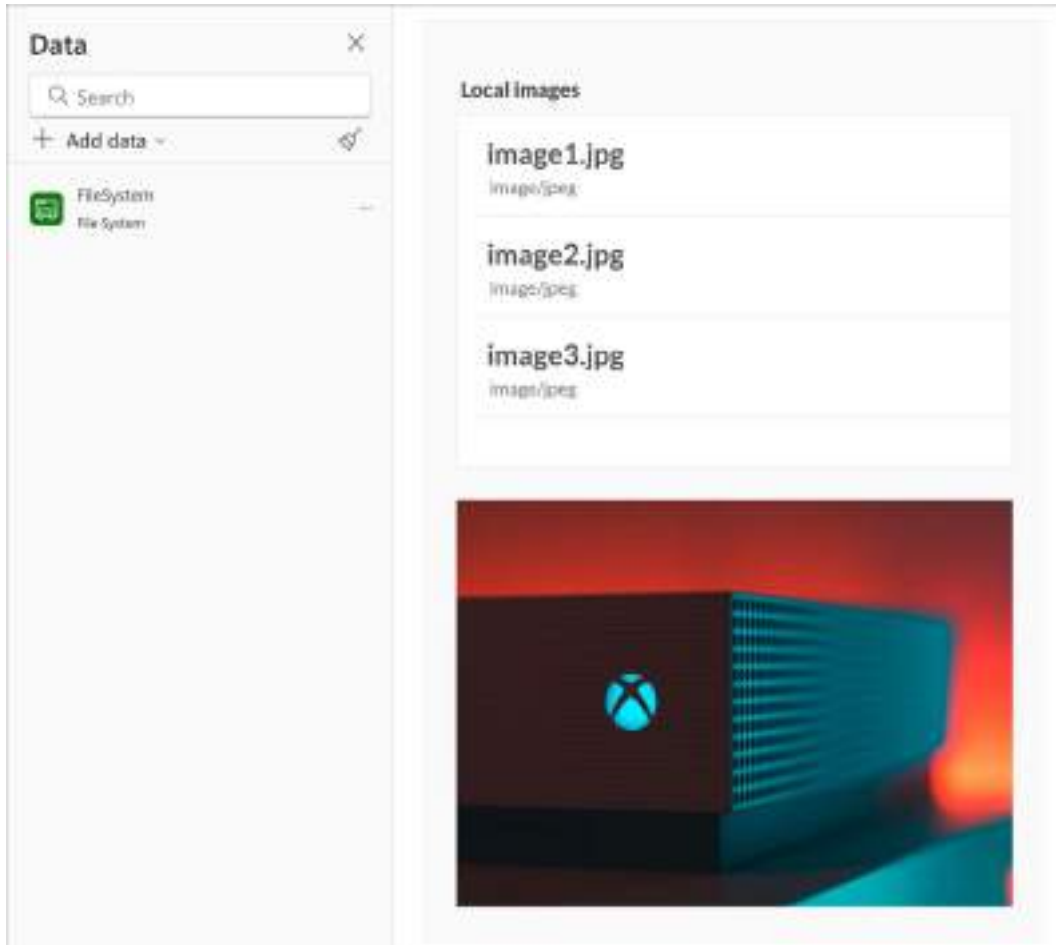


Figure 4.11: Canvas Power Apps app using local data

Clicking on each file will load them on the media control at the bottom of the gallery. As you can see, we can use the gateways for other services besides relational data. Match this with the capability of canvas apps to use several connectors at once, and the possibilities are endless.

Improving application speed and responsiveness by using static data

When working with cloud data, there are situations where the data collection process can be slow. Many factors can add delays when obtaining data: network speed, signal strength, and data source infrastructure, to name a few.

That's why you need to consider the data you are working with. Where is it located? What is its availability? How changeable is it?

In this recipe, we will try to add a solution to the last question. If data doesn't change, why not make it static? We are not talking about adding the data in the actual code; we will add the data as a resource for the application, just like when adding an image.

Getting ready

As an example of data that rarely changes, we have provided a list of countries in an Excel file, `countries.xlsx`, in this chapter's repository: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-04>.

How to do it...

1. Go to the Power Apps maker portal and create a new canvas app; select either phone or tablet layout. From the left pane, select **Data**, click on **Add data**, search for **Import from Excel**, and choose this connection.
2. This action will prompt you to select an Excel file from your file system; pick the provided one or any other from your computer, given that the data on this file has a table defined. For more information on this requirement, visit <https://support.microsoft.com/en-us/office/overview-of-excel-tables-7ab0bb7d-3a9e-4b56-a3c9-6c94334e492c>.
3. Once the file finishes uploading, the right pane will display a list of tables inside the file for you to select. Choose one and click **Connect**. The selected Excel table will appear on the list of data sources as if it were a connected one.

How it works...

If your data rarely changes, you can add it as a resource in your application. Keep in mind, though, that the amount of data will increase the size of your app. A 3 MB Excel file is no candidate for this. Think of these resources as you would think about images. The smaller the file size, the better.

With the provided file, you can build an app that can help you search for a list of countries:

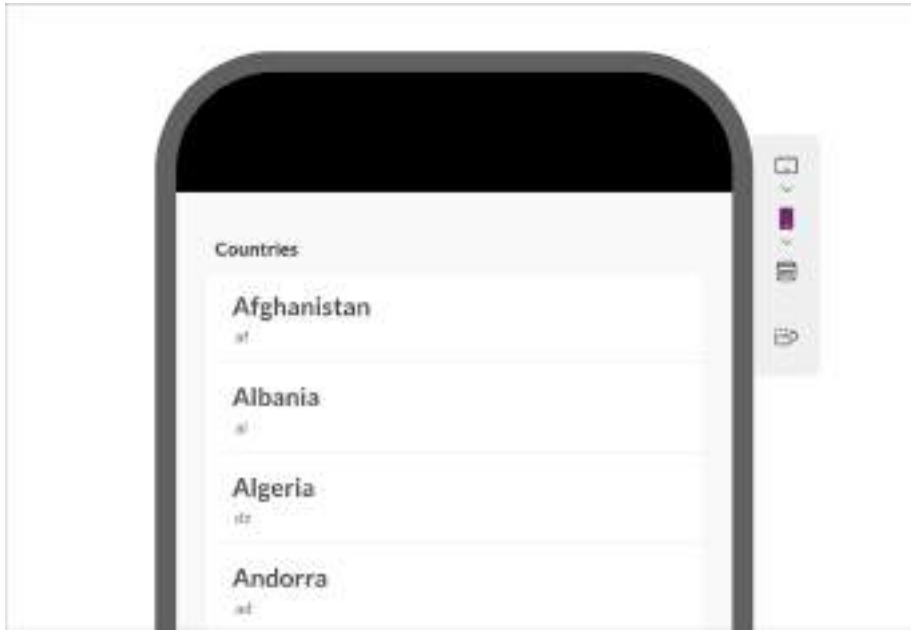


Figure 4.12: List of static countries' data

Now, you can check your apps for static data candidates and improve their performance by reducing data sources' connections.

There's more...

There's another approach to static data, this time with custom pages. This type of Power App includes the capacity to import web resources (.resx) commonly used for model-driven apps. In this case, we can use these resource files to add localization for our applications. To get more information, visit <https://learn.microsoft.com/en-us/power-apps/maker/model-driven-apps/custom-page-localize>

Consuming external data by using Dataflows

The data for your solutions can come from various sources, such as relational databases, on-premises data, analytics data, APIs, and even regular files. These sources can be located anywhere and may not have the infrastructure to provide data.

Dataflows aim to be an instrument for gathering all kinds of data sources. With this tool, you can consolidate all the required data, related or not, into Dataverse, where you can improve the collected information by adding views, relationships, and the like. Dataflows not only gather data but also allow you to transform it while doing it.

Power Query, the same technology used in Power BI, helps with these transformations, giving you a complete set of features to shape your data. For more information about Power Query, please refer to <https://learn.microsoft.com/en-us/power-query/power-query-what-is-power-query>.

Dataflows can run manually or on a schedule. They live in your environments, so you can have them running in your test environment, gathering data from test sources, and have different ones in production.

For this recipe, we will use a different approach other than connecting to a database. Thanks to the AccuWeather developer API, we will get weather data and save the forecast into a Dataverse table.

Getting ready

According to the current documentation, you should need a license to use Dataflows because it is not part of the features included in a Developer plan. However, you can create and use Dataflows anyway. This is something that needs clarification from Microsoft's documentation and has been reported to them already.

The next thing we will need to do is register for a free developer account with AccuWeather at [https://developer .accuweather .com](https://developer.accuweather.com). After registering, go to **MY APPS** and create an app with an API key to access their system. The limited trial is a free service that allows 50 calls daily and just one key per developer, but it suits our objective well. Make sure you select **Core Weather Limited Trial** under **Products** when creating the app.

How to do it...

1. Go to the Power Apps maker portal to create a solution for this recipe. From this solution, click on **New | Automation | Dataflow**. In the open dialog, name it, for example, Weather data, and click **Create**.

2. The Power Query dialog will display the built-in data sources that are available for you to use. Every option has different kinds of settings. The authentication type and credentials are configured based on connection parameters and privacy levels, which determine the data source's isolation level. For this example, select **Web API**:

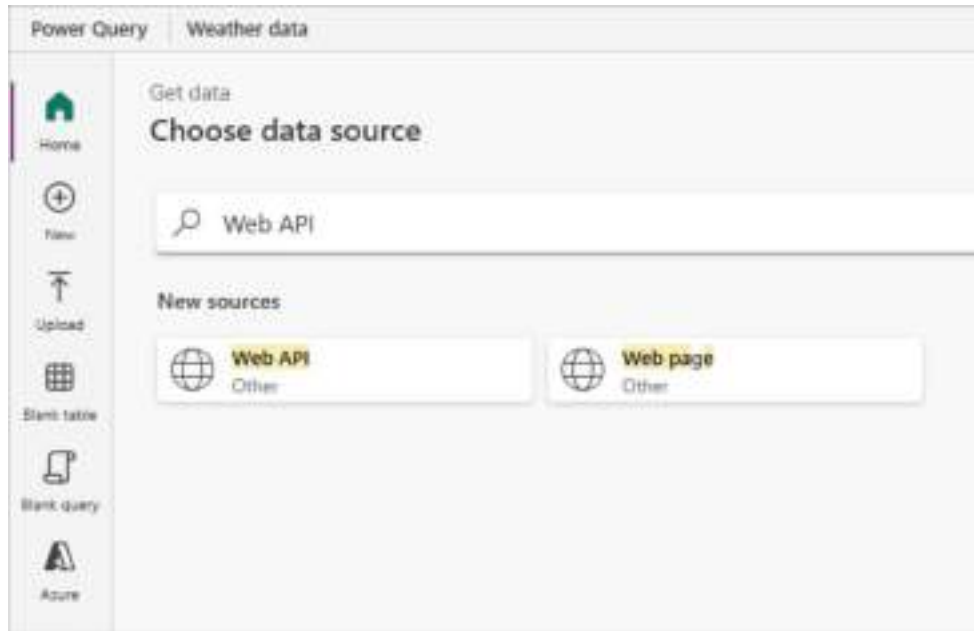


Figure 4.13: List of data sources available for Power Query

3. In the Web API configuration dialog, enter the following URL, `http://dataservice.accuweather.com/forecasts/v1/daily/5day/349727?apikey=`, plus your API key. This URL gets the 5-day daily forecast of New York City (location code 349727). Set the desired name for **Connection name** and leave all the other values as default. The information we are going to receive gets structured as one record, as shown here:

```
{
  "Headline": {
    "EffectiveDate": "2024-04-28T20:00:00-04:00",
    "EffectiveEpochDate": 1714348800,
    "Severity": 5,
    "Text": "A thundershower this evening",
    "Category": "thunderstorm",
    "EndDate": "2024-04-29T02:00:00-04:00",
    "EndEpochDate": 1714370400,
```

```
    "MobileLink": "http://www.accuweather.com/en/us/new-york-ny/10021/daily-weather-forecast/349727?lang=en-us",
    "Link": "http://www.accuweather.com/en/us/new-york-ny/10021/daily-weather-forecast/349727?lang=en-us"
  },
  "DailyForecasts": [
    {
      "Date": "2024-04-28T07:00:00-04:00",
      "EpochDate": 1714302000,
      "Temperature": {
        "Minimum": {
          "Value": 65,
          "Unit": "F",
          "UnitType": 18
        },
        "Maximum": {
          "Value": 78,
          "Unit": "F",
          "UnitType": 18
        }
      },
      "Day": {
        "Icon": 16,
        "IconPhrase": "Mostly cloudy w/ t-storms",
        "HasPrecipitation": true,
        "PrecipitationType": "Rain",
        "PrecipitationIntensity": "Light"
      },
      "Night": {
        "Icon": 15,
        "IconPhrase": "Thunderstorms",
        "HasPrecipitation": true,
        "PrecipitationType": "Rain",
        "PrecipitationIntensity": "Moderate"
      },
      "Sources": [
        "AccuWeather"
      ]
    }
  ]
}
```



```

"MobileLink": "http://www.accuweather.com/en/us/new-york-ny/10021/daily-weather-forecast/349727?day=1&lang=en-us",
"Link": "http://www.accuweather.com/en/us/new-york-ny/10021/daily-weather-forecast/349727?day=1&lang=en-us"
}...

```

4. The following dialog is the actual query editor, where you can transform the incoming data. Depending on the source data, Power Query can apply certain transformations automatically to help you shape your data. With the AccuWeather information, the result is shown in the following screenshot:

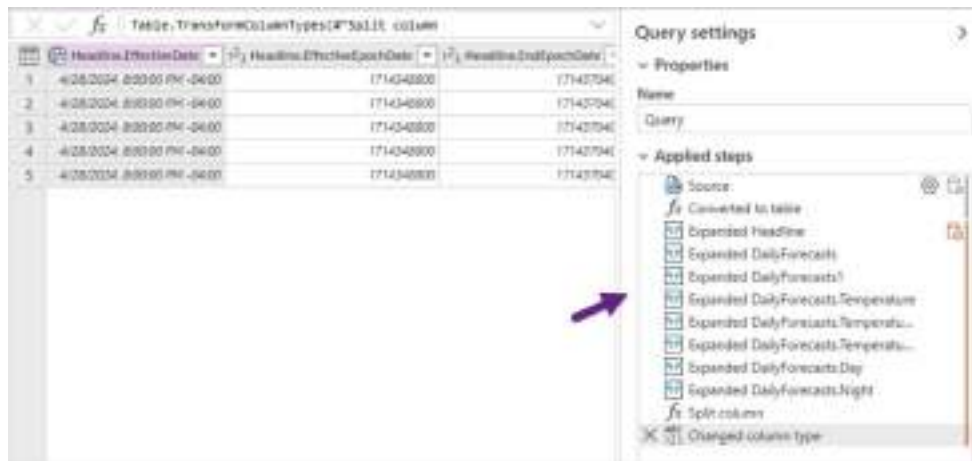


Figure 4.14: Query settings for Power Query

5. With the help of the *Ctrl* key, select the following columns:

- **DailyForecasts.Date**
- **DailyForecasts.Temperature.Minimum.Value**
- **DailyForecasts.Temperature.Maximum.Value**

Right-click on them and choose **Remove other columns**.

6. Right-click on the **DailyForecasts.Date** column and select **Transform column | Day | Name of day**, as shown in the following screenshot:

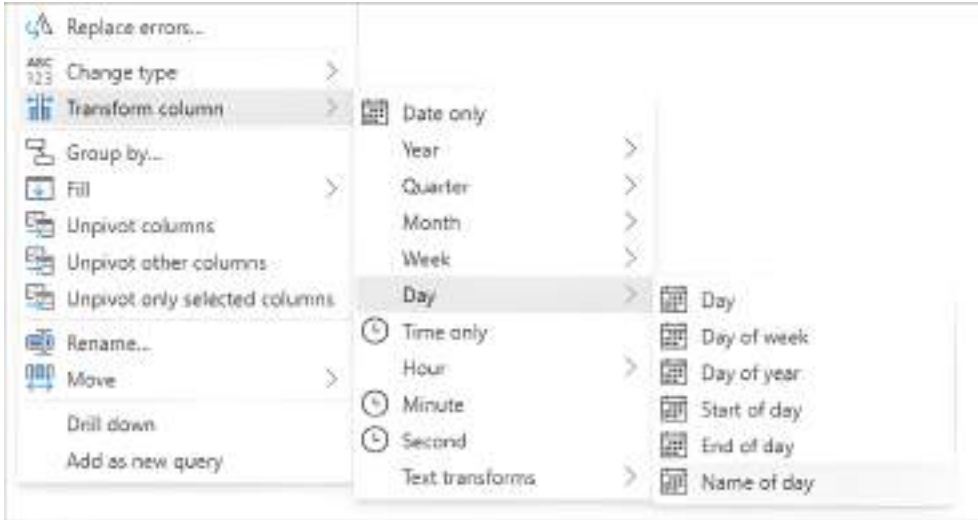


Figure 4.15: “Name of day” transformation

- 7. The last step is to rename the columns to get a more detailed description of the data columns. Right-click on each column and select **Rename** (or just double-click on the column header). Set the names to Weekday, Minimum, and Maximum
- 8. Once all these changes are in place, we should see the data transformed, as seen in the following screenshot. Try other transformations if you like until you get the desired result. Click on **Next** when you’re done:

	Weekday	Minimum	Maximum
1	Sunday	18	24
2	Monday	24	32
3	Tuesday	34	36
4	Wednesday	39	43
5	Thursday	41	50

Figure 4.16: Data transformation complete

9. Now is the time to place the transformed data into Dataverse. You will get the option to load the data into a new table or use an existing one. Let's use a new one by giving it a **Table name** and **Table display name** of New York Forecast. Set an **Auto-generated** unique primary column and name it ID. Once you're ready, click on **Next**. You should get the following result:



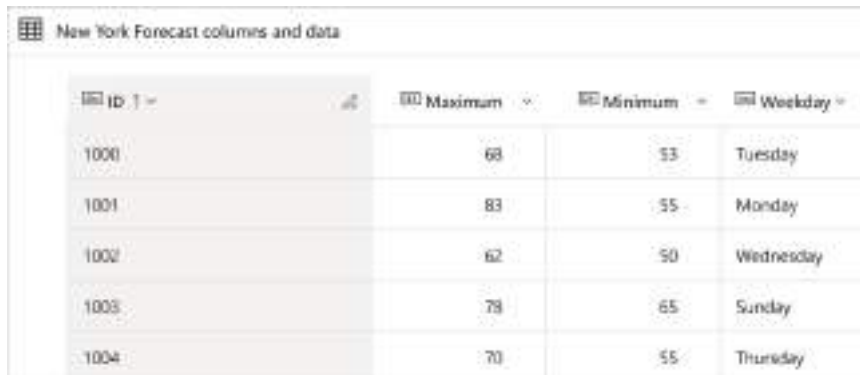
Source columns	Destination columns	Destination column type
Maximum	c11_Maximum	Integer
Minimum	c11_Minimum	Integer
Weekday	c11_Weekday	Text

Figure 4.17: Data load settings

10. Once you're ready, click **Next** and **Publish** to complete the process.
11. You should see your dataflow in the list, and in a minute or two, you should see the **In progress** status, meaning that a first refresh is executing. When it finishes, you will see the updated timestamp in the **Last Refresh** column and the scheduled **Next Refresh**.

How it works...

Now that we have completed the steps to build the dataflow, we can verify that the data is, in fact, in Dataverse. Go back to your solution and, from the toolbar, click on **Add existing | Table** and look for the table you defined in the dataflow's load data step, in our case, New York Forecast. Click **Next**, select **Include all objects**, and then click on **Add**. Open the table from the list and verify that the 5-day forecast data is there:



ID	Maximum	Minimum	Weekday
1000	68	53	Tuesday
1001	83	55	Monday
1002	62	50	Wednesday
1003	78	65	Sunday
1004	70	55	Thursday

Figure 4.18: Current forecast view

There's more...

Dataflows are an excellent tool for ingesting data from different sources without the need to build an infrastructure to gather or transform the data. However, there are some limitations that you might need to consider, which you can learn about here: <https://learn.microsoft.com/en-us/power-bi/transform-model/dataflows/dataflows-features-limitations>.

A helpful feature of dataflows is that they also serve as synchronization tools. When defining the load data settings, there is an option called **Delete rows that no longer exist in the query output**. This change requires the alternate key field to be mandatory as it uses it for comparison purposes when loading new data:



Figure 4.19: Dataflow load settings for sync

This setting, tied to a scheduled refresh, turns dataflows effectively into a sync mechanism. In our tests, we loaded data from an Azure SQL database into a technicians table with these columns: ID, Full name, Email, and Location. After the first load, we deleted a record from the SQL database, and on the next refresh, the deleted record was removed on Dataverse.

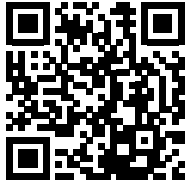
One more concept that we need to consider is that you can configure a dataflow to load several tables simultaneously. As the previous figure shows, we are loading both SalesLT Products and technicians on the same dataflow.

As you can see, this is an excellent tool for integrating external data into Dataverse. Once there, you can have complete control over security, automatization, and data consumption.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



5

Automating Processes with Power Automate

Power Platform provides different solutions to solve business organizations' needs: Power BI to analyze data, Power Apps to build applications, Power Pages to build customer-facing websites and portals, Copilot Studio to interact with users, and Power Automate to automate processes effectively.

In this chapter, we will discuss Power Automate, which aims to improve business processes with a series of actions to help streamline existing operations. Examples of these actions include executing procedures based on specific events, building data integration systems, automating repetitive tasks, and more.

Before diving into this chapter, we will also cover a key topic regarding the different types of flows. Having understood this concept, we can then build recipes using examples for diverse scenarios:

- Creating a sales survey solution with Microsoft Forms
- Building a file-processing automation for SharePoint libraries
- Overcoming Power Apps delegation with Power Automate
- Creating a modular solution with Power Automate
- Automating legacy applications using desktop flows
- Integrating Google Forms with Dataverse using cloud flows

Technical requirements

There might be a license requirement depending on the services you need to connect to Power Automate. Please refer to the *Preface* section of this book for more information.

At the time of writing, Power Automate does not use the new designer when creating a flow from a solution. The workaround is to create it, save it, and then open it again.

If you see the classic designer, add a **New step**, search for **Compose**, and enter any text as the **Inputs**. We are doing this workaround to be able to save the cloud flow, close it, and then open it again to get the new flow designer. You can delete the **Compose** action afterward.

This chapter will follow the new designer's steps to provide the most up-to-date instructions.

The code for this chapter's recipes is available from our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05>.

The different types of flows

Power Automate offers five types of flows to provide organizations with as many options as possible. Each of these types aims to solve specific scenarios.

Automated cloud flow

These cloud flows are executed when a specific action triggers them – for example:

- When a new email arrives in Gmail
- When a new file gets uploaded to SharePoint

You can build solutions around them, such as the following:

- When a new email has the word “invoice” in the title, extract the attachments and copy them to the Finance SharePoint document library.
- When a new tweet gets posted, send the content to AI Builder for sentiment analysis, and then send a Microsoft Teams notification to the marketing department if the result is negative.

Instant cloud flow

If you need to trigger a cloud flow manually, an instant cloud flow is the option you want. The following are examples of this kind of situation:

- Clicking a button on your mobile phone

- Executing flows from a canvas app
- Selecting a file in a SharePoint document library

These cloud flows can trigger solutions such as:

- Creating an on-demand report from the current sales records
- Notifying a user through Microsoft Teams from a canvas app
- Requesting a signature for a selected file in SharePoint

Scheduled cloud flow

As the name implies, these cloud flows are executed at a specific date and time and with a defined recurrence. These are some of the possible use cases:

- Performing a monthly backup of a particular folder to your personal OneDrive
- Creating and sending timesheet reports every two weeks
- Performing a daily cleanup of an Azure SQL table

Business process flow

As mentioned in *Chapter 3, Building from Data with Model-Driven Apps*, these flows act as a guiding system for Dataverse tables to help users through the data-gathering process. Examples include the following:

- Turning leads into opportunities for the sales department
- Managing a support case in a help desk system

Desktop flow

These desktop flows bring **Robotic Process Automation (RPA)** to Power Automate by allowing you to record your interaction with web and desktop applications, which avoids repetitive tasks by automating the playback of these interactions. Using these can help in the following cases:

- Data entry on a legacy ERP system that has no API to interact with
- Recording the customization process of a particular application to replicate it on different devices

As you can see, there is a diversity of flows aimed at solving almost every business requirement, and it doesn't stop there; the platform keeps evolving to offer even more features to help you achieve your goals. The following recipes will help you get a taste of all the possibilities that Power Automate offers.

Creating a sales survey solution with Microsoft Forms

Web forms are an excellent tool for gathering information. Whether you require feedback from a specific process, an event registration, or even a simple contact form, you must clearly define the fields to collect the data in these forms.

Microsoft Forms gives you an easy way to create them with an easy-to-use interface that allows you to quickly create forms, quizzes, and polls. We will use this service to complete our solution by integrating an automated cloud flow using Power Automate.

The first step is to design a sales survey, and then, depending on the answers received, we will notify a team to further engage with the customer.

The way we notify people in our organizations has evolved since emails. The norm is using email for external users and collaboration tools for internal ones. Users prefer receiving rich notifications with which they can interact directly using their platform of choice.

Adaptive Cards help improve notifications by applying user interface elements, which act and look native in the platform where they are delivered.

The following is an example of an Adaptive Card:

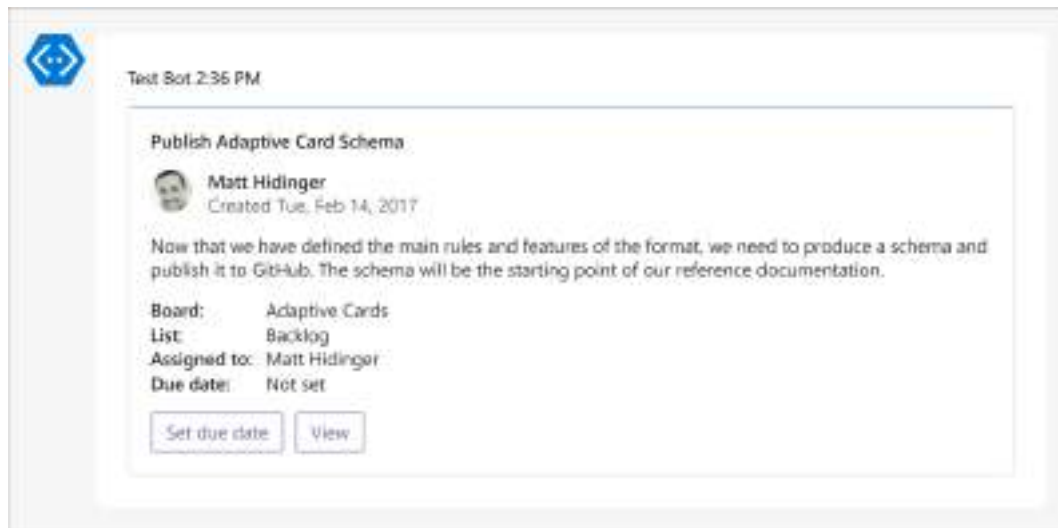


Figure 5.1: Adaptive Card example in Microsoft Teams

Targets of Adaptive Cards include mobile devices, desktop applications, bot platforms, and collaboration tools, such as Microsoft Teams. To get more information, please refer to <https://adaptivecards.io>.

Getting ready

To recreate this recipe, you will need a regular Microsoft 365 Business plan (Microsoft 365 Business Basic and up) or a Microsoft personal account (Outlook, Hotmail, or Microsoft). Both account types allow you to use Microsoft Forms for free.

For this recipe, we will prepare a solution containing an automated cloud flow that will automate the responses from a sales survey, created using Microsoft Forms. Follow the instructions found in *Chapter 1, App Maker Basics* to create the solution and build the survey. Use the following fields for the form:

Field type	Required	Text	Extra
Text	Yes	Customer name	
Choice	Yes	How did you first learn about our services?	Choices: Advertisement, Trade show, Search engine, and Other
Text (long answer)	No	Please describe the main reasons you selected our services	
Rating (5 stars)	Yes	Overall rating of our services	
Choice	Yes	Would you like a follow-up call?	Choices: Yes and No

Table 5.1: Form fields' configuration

We will use a standard connector on the Power Automate side; either account type works for our scenario. We will also use a marketing team in Microsoft Teams as an example.

How to do it...

1. Head to Microsoft Forms to build the required form. Once ready, we can start building actions that will get triggered when we start receiving the responses.
2. Go to the Power Automate portal, <https://make.powerautomate.com>, and create a solution to hold the cloud flow.

3. Create a new cloud flow inside the solution by clicking **New | Automation | Cloud flow | Automated** from the toolbar. This action will open a dialog to choose the trigger.
4. Give it a name, such as Sales survey handler, and search for the trigger **When a new response is submitted on Microsoft Forms**, and then click on **Create**:

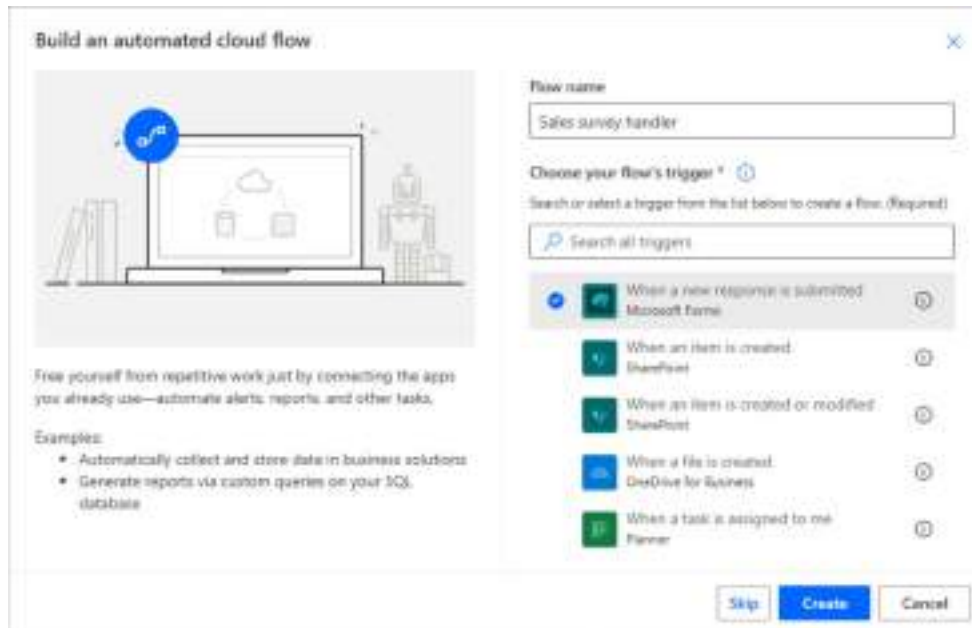


Figure 5.2: Automated cloud flow creation dialog

5. When the designer opens, you will need to select the form from which we expect answers. Click on the first action of the designer, and then choose the one you just created from the left pane.
6. The next step will be to get the actual response data by choosing the **Get response details** action, using the previous step's result as the **Response Id** data.

7. Now, we need to redirect the flow depending on the *Would you like a follow-up call?* question. To do this, we need to add a **Condition** action to check whether the answer was Yes. Select this question in the left value field, select **is equal to** in the condition dropdown, and then, in the right value field, enter Yes:



Figure 5.3: Response details action and condition

8. On the **True** route, let's add a Microsoft Teams notification to our marketing team using an Adaptive Card. Add a new action, and then search for Post adaptive card and wait for a response. By using this, we can notify a specific channel, chat, or group chat and wait for feedback. Leave **Flow bot** in the **Post as** field and select **Channel** for **Post in**. Select the **Team** and **Channel** where you would like to send the notification. Leave the default **Update message**.
9. For the **Message** field, paste the code of the sample Adaptive Card from this recipe's folder in our GitHub repo, located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05>.

10. Replace the values in braces with the corresponding response details. For example, **{rating}** links to the **Overall rating of our services** response. Please refer to the following screenshot for instructions on how to replace these values:

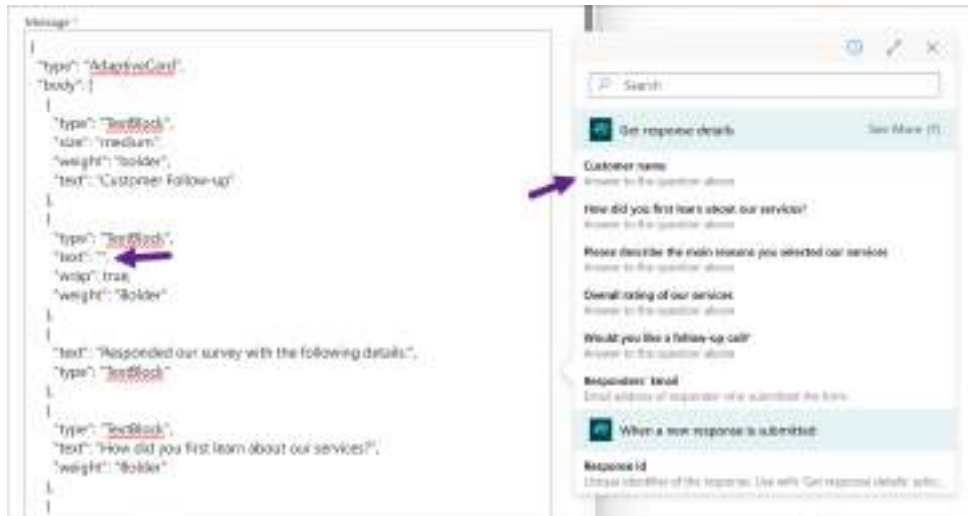


Figure 5.4: Updating the Adaptive Card with data from previous steps

11. On the **False** route, we could, for example, add a record to a **Ratings** list in SharePoint to keep track of the ratings we receive from our customers. Then, we could build **Key Performance Indicators (KPIs)** using Power BI. The following screenshot shows the condition routes configured:

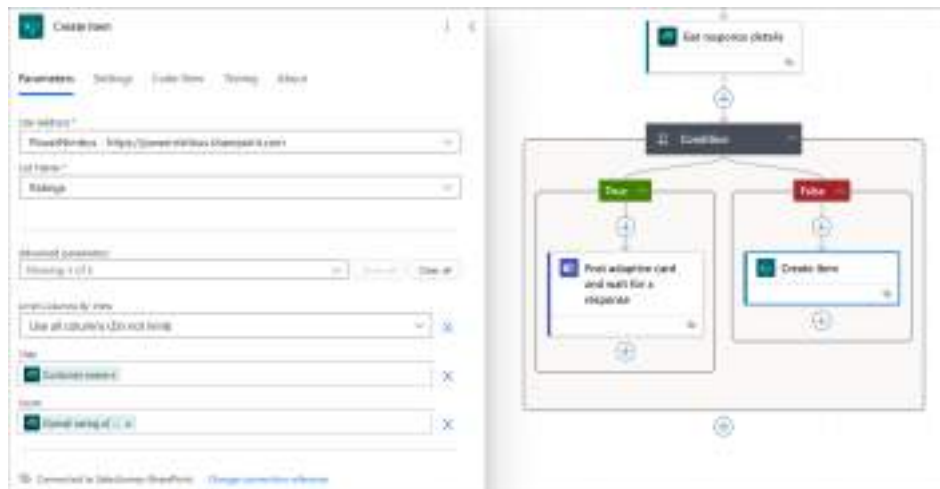


Figure 5.5: Adaptive Card and Create item record actions

When automating responses from Microsoft Forms, you can change these actions to your specific needs.

How it works...

You need to send some responses using the form to test this recipe. Whenever you select **Yes** in the follow-up question, the team you defined in the cloud flow will receive a message telling them that a customer wants to continue engaging with the company. In the following screenshot, you can see the notification:

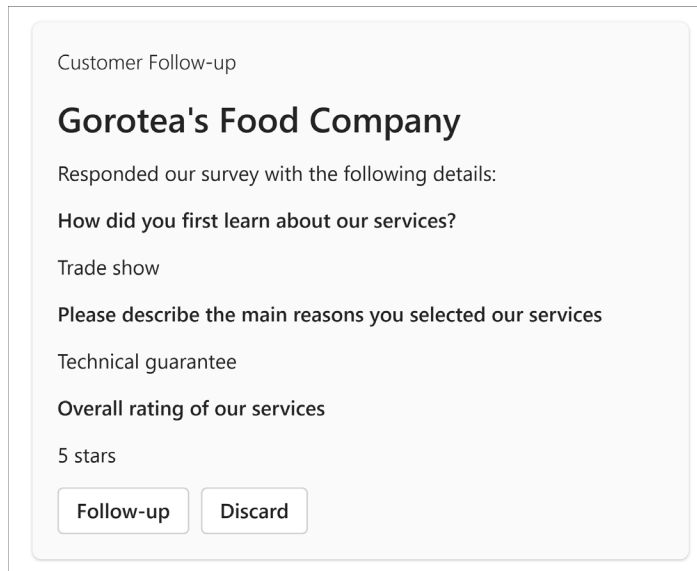


Figure 5.6: Adaptive Card in action

We can continue to iterate with the responses from our notification by executing more tasks, including the following:

- Creating a follow-up Microsoft Teams meeting
- Sending an automated email to the customer
- Adding an activity to the customer record in the CRM system

These are just examples of what you can achieve by automating customer satisfaction feedback using Power Automate, but it doesn't end there. Imagine an educator who builds quizzes using Microsoft Forms and then auto-grades his students using Power Automate, while feeding a database with the scores for further analysis. The possibilities are endless.

There's more...

Another option for sending rich notifications, available exclusively for Microsoft Teams, is **Cards for Power Apps**. This lightweight version of canvas apps features a simplified set of controls while allowing the introduction of logic through Power Fx. Additionally, it supports connectors for seamless data and service integration.

To get more information, please visit <https://learn.microsoft.com/en-us/power-apps/cards/overview>.

In this recipe, we have followed the steps to build the cloud flow manually. We could leverage Power Automate's Copilot to help us kickstart the flow creation by going to the maker's portal and entering this prompt:

```
When a new response is submitted to a Microsoft form, check the response for certain conditions
```

From there, we can adapt the flow configuration and conditions to fulfill our needs. This is just an example of how Copilot can aid in the solution-building process.

Building a file processing automation for SharePoint libraries

Improving business activities, especially recurring ones, is a top priority in every organization. Repeating a set of actions for a business process can be prone to errors because the personnel tends to replicate actions without validation. Power Automate builds upon this need to help organizations automate their tasks so that people can focus on the business.

In this recipe, we will automate a business process using a SharePoint document library that handles the quotes of a given company. Microsoft Word will generate these quotes with defined metadata to help classify their contents.

Users can manually execute a cloud flow when a quote is complete, converting the document into a PDF file with the same metadata as the original one. It also notifies the Finance team if the quote is over \$100,000.

Getting ready

When using SharePoint, as far as licensing goes, a regular Microsoft 365 Business Basic license is required to include this service in your organization. We won't need an additional license, as we will use standard connectors on the Power Automate side. Regarding the document library, please refer to *Chapter 1, App Maker Basics*, for instructions on how to create it.

How to do it...

- 1. First, let’s start by building the document library. For this recipe, name it **Quotes** and add the following columns:

Column type	Name	Extra
Text	Quote	
Choice	Department	Choices: Finance, HR, Marketing, and IT
Currency	Amount	
Number	Fiscal year	Default value (Use calculated value): =YEAR(TODAY)

Table 5.2: Quote columns

Reorder the columns to get the desired result. The following screenshot shows the **Quotes** library with the previous columns and a few loaded files:



Figure 5.7: Quotes document library

- 2. Navigate to the Power Automate maker portal and select the default environment. You need to create the cloud flows in this environment to make the SharePoint integration work.
- 3. Once in the Power Automate portal, click **My flows** on the left pane, and then click **New flow | Instant cloud flow** to build an on-demand cloud flow. In the open dialog, set a name, choose **For a selected file in SharePoint**, and click **Create**.
- 4. The first action that appears acts as a link from our cloud flow to the selected document library. Click on it to open the configuration pane. Choose your SharePoint site address and the **Quotes** library.

5. We need to get the actual properties of the file requesting the automation. Click the plus sign just below the first action to add a new one. On the pane that opens, search for **Get file properties** in SharePoint. Set the same site address and library as the previous action, and select the **Id** from the result of the last step:

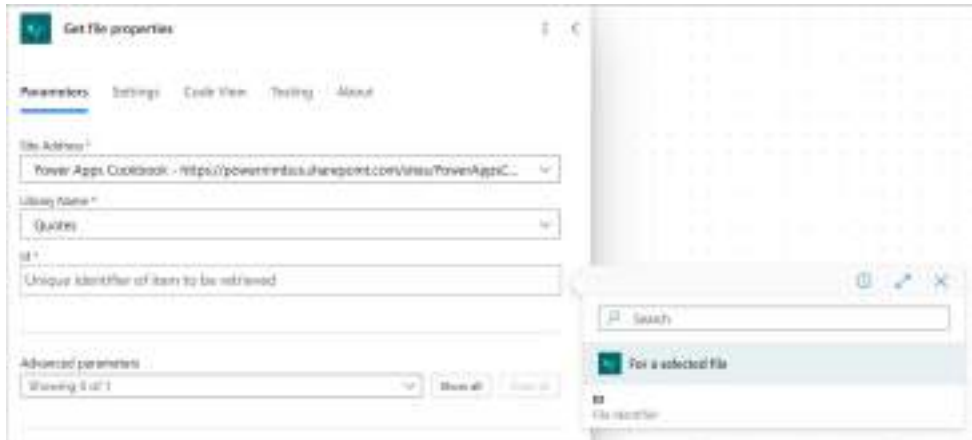


Figure 5.8: Get file properties configuration

6. To create our resulting PDF file, we need to define its name. To do this, we will use the **Compose** action to build a formula to generate it. Add this new action, and for **Inputs**, use the following expression, using the expression editor:

```
replace(triggerBody()?['entity']?['fileName'], '.docx', '.pdf')
```

7. With this formula, we use the same filename but change the extension from Word (.docx) to PDF (.pdf). To tidy up our cloud flow, let's rename this action from **Compose** to **PDF filename** by clicking on its title.
8. Now that we have all the metadata, we can start the actual automation. Let's get the file content to use in the conversion. Add a new action and search for **Get file content** in SharePoint. Set the same site address as before, and for the **File Identifier**, use the Identifier collected from the **Get file properties** action.
9. To convert the file, we will use a feature of the OneDrive/OneDrive for Business connector, so the first thing we need to do is create the file in OneDrive. Add a new action and search for **Create file in OneDrive/OneDrive for Business** (we will use the Business version from now on). Sign in if this is your first time using this connector. For **Folder Path**, choose the root folder or any other folder to hold the temporary file needed for the conversion.

As for the **File Name**, we will need to set a random name to prevent concurrency issues if this cloud flow gets executed more than once. We can use a formula for the name, such as `concat(ticks(utcNow()), '.docx')`. For **File Content**, select the result of the previous step.

10. To do the conversion, we need to add a new action. Search for **Convert file in OneDrive for Business**. For **File**, select the **Id** of the previous step and set **PDF** as the **Target type**.
11. Let's add a new action to delete our temp file and avoid clutter in our OneDrive. Search for **Delete file in OneDrive for Business**. For **File**, select the **Id** of the **Create file** action:

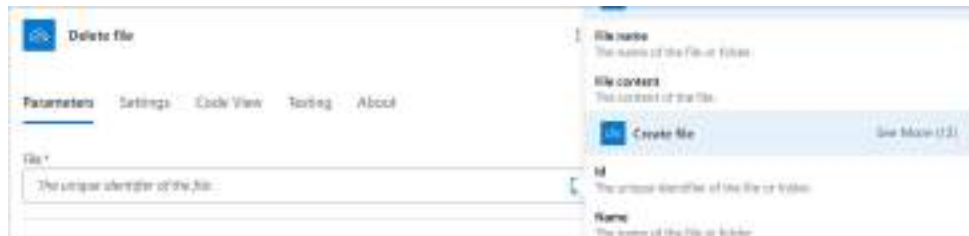


Figure 5.9: OneDrive actions

12. The next step is to get the PDF file back to SharePoint, but before we do, we need to consider the scenario that this might be a second execution of the cloud flow, so we will need to do some cleanup first: if the file exists already, let's delete it. Add a new action, and search for **Get file metadata using path in SharePoint** to check whether the PDF file already exists. Set the same **Site Address** as before and, in **File Path**, input `/Quotes/`. Then, from the **Dynamic Content** selector, choose **Outputs** from the **PDF filename** action.
13. Add a new action called **Delete file in SharePoint**. Set the same **Site Address** as before, use the **Id** from the previous step as the input of **File Identifier**, and rename this action **Delete PDF file**:

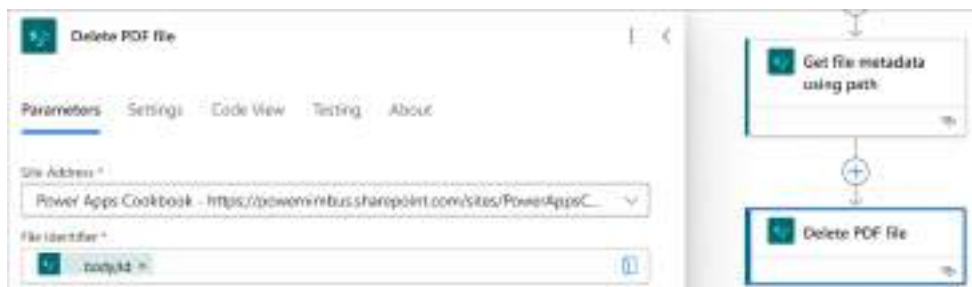


Figure 5.10: SharePoint actions

14. Keep in mind that the first time this cloud flow gets executed, the PDF file won't exist, so we need to inform Power Automate that if any of the previous two steps fail, it should continue to run the rest of the actions. To do this, we will add a new action called **Scope**. This action allows us to encapsulate several steps together. If one fails, the entire scope fails. Rename this action **Check for existing file**
15. Drag the **Get file metadata using path** and **Delete PDF file** actions inside the scope, preserving the same order.
16. Add a new action and search for **Create file** in SharePoint. Set the same site address and path as in previous actions. For **File Name**, use the **Outputs** from the **PDF filename** action, and for **File Content**, get the conversion result of the **Convert file** step. Go to the **Settings** tab of this action, and under the **Run After** section, check **Has failed**. Rename this action **Create PDF file**:

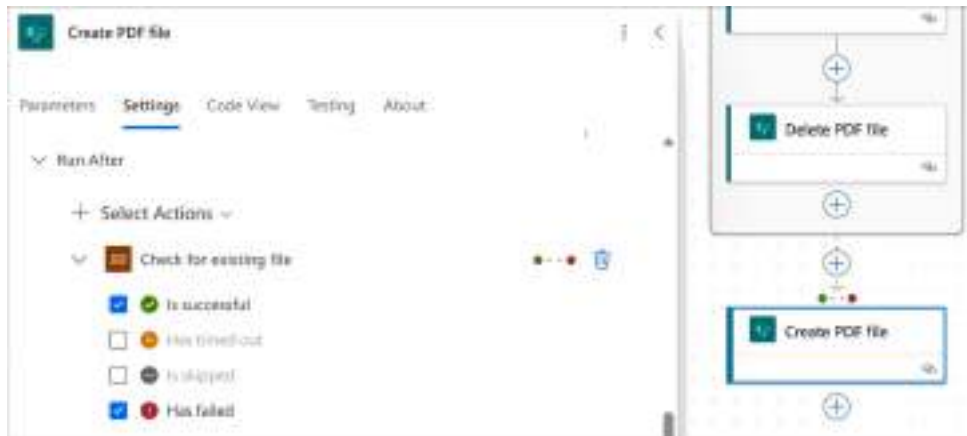


Figure 5.11: Run After configuration

17. We now need to replicate the same metadata as the original file. Add a new action and search for **Update file properties** in SharePoint. Set the same site address and library as before, and set the **ItemId** from the last file creation step as the **Id**. The rest of the fields come from the **Get file properties** result. Remember to click the **Show all** button to display all the library columns.

Keep in mind that the **Department** field is of the **Choice** type, so we need to use the actual value by searching for **Department Value** and entering it as a custom value:



Figure 5.12: Column metadata replication

18. The last step is to notify the Finance team if a quote is over \$100,000. Add a new action called Condition. Using the action properties, build an expression to check whether the Amount value of the **Get file properties** result is **greater than** \$100,000. Rename this condition Amount value check
19. On the **True** route, add a new action called **Post message in a chat or channel in Microsoft Teams**. Sign in if this is the first time using this connector in this environment, select **Channel** in the **Post in** field, set the desired **Team** and **Channel**, and build a message using the **Get file properties** values. Rename this action Post a message to Finance

The following screenshot shows the amount check condition and a notification message example:

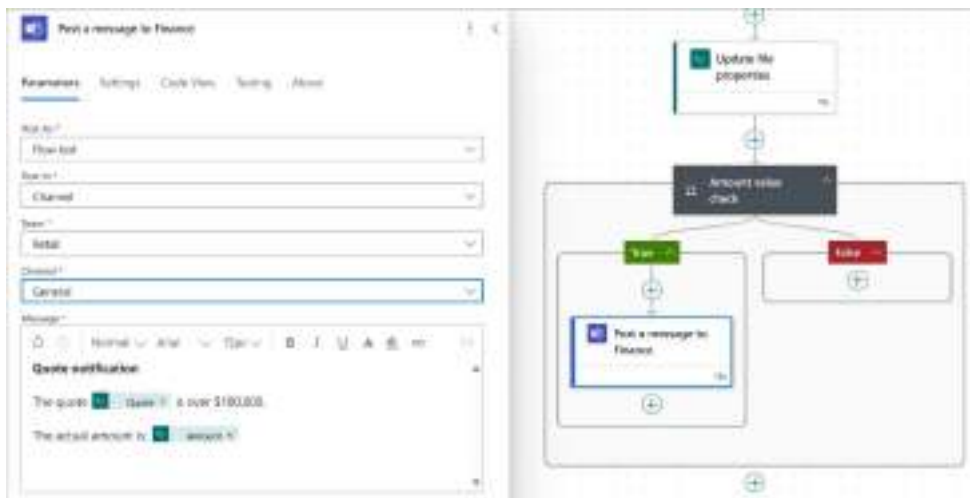


Figure 5.13: Amount check condition and the notification message

20. Add any action for the **False** route, but for now, click **Save** on the toolbar and close the designer.

How it works...

Returning to the document library, you will notice that if you select a file and click **Automate** on the toolbar, you will get your cloud flow listed and ready to process the chosen file. The first execution will ask permission to use the connectors defined in your cloud flow—in our case, SharePoint, OneDrive, and Microsoft Teams. Click **Continue** and then **Run flow** to start processing the file:



Figure 5.14: Flow activation on a selected file

When the automation is complete, the converted file will appear in the same document library with the .pdf extension. If the amount is over \$100,000, a notification will appear in the **Retail** team inside the **General** channel, alerting them about this quote, which looks like this:

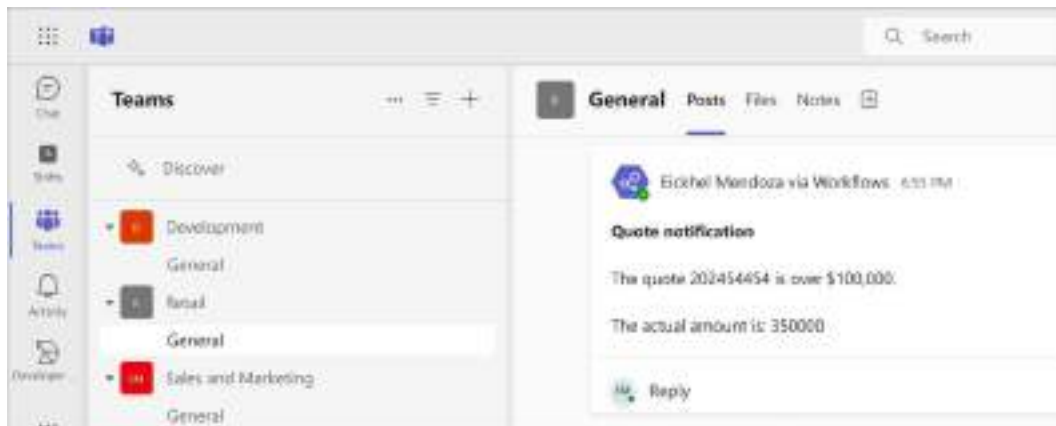


Figure 5.15: Teams channel notification

This solution is an all-around example of a quote processing system. You can add more steps, validations, and notifications depending on the needs of your retail or finance department.

You can improve the solution by doing the following:

- Add actions to the **False** route of the **Amount value check** condition
- Include a document signing solution to sign the final quote
- Send the final quote via email to the customer

These are just examples of the automation possibilities that SharePoint and Power Automate can offer.

There's more...

Remember to leverage Copilot's capabilities when building expressions in Power Automate. Sometimes, getting the correct syntax to get specific parameters isn't easy; this is when AI can help us.

For instance, when creating our flow, we can enter this prompt in Copilot's chat window to generate the expression required for *step 6* in our previous cloud flow:

```
Add a compose action that gets the filename from the trigger and replace  
' .docx' with ' .pdf'
```

Copilot is constantly evolving, so give it a try when creating expressions to help you achieve your goal faster.

Overcoming Power Apps delegation with Power Automate

Canvas apps offer a wide range of features. Among them, we can highlight the ability to connect to multiple data sources simultaneously for a given application. To support this, the platform applies specific techniques to remain performant. One of these techniques is Delegation.

Delegation is a term used in canvas apps that means the platform delegates the data processing to the data source, rather than doing it in the application itself. As querying data uses memory and processing power, the less our application uses, the better. That's why using data sources that support delegation is a great idea. To get more insight into this matter, please refer to <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/delegation-overview>.

By default, Power Apps limits the number of records retrieved to 500. Even if your data source supports delegation, requesting more items will throw a warning because the platform won't be able to retrieve all the requested data.

The following screenshot shows the sign on top of the label that calculates the number of records of a SharePoint list. The list has 3,000 items, and the app can only count the first 500 received by the SharePoint connector:



Figure 5.16: Delegation warning for a SharePoint list with 3,000 items



USER EXPERIENCE ADVICE

When designing your application, the best approach is to work with filtered data. It makes no sense to throw 3,000 items into a gallery for the user to browse. Refining by categories, indexed fields, or any other column that lets you work with limited data sets is the best option.

There are times when delegation does not work in our favor. As indicated in *Figure 5.16*, if we need to count our data source's total number of records, we will hit delegation issues sooner or later.

We might have missed this when developing our app, or perhaps our customer demands to display all data, irrespective of the number of records. For these situations, we can leverage Power Automate and the strength of the formulas available in Power Apps to solve these issues.

Getting ready

To use this recipe, we need to have a SharePoint list with at least 3,000 items. Please refer to *Chapter 4, Choosing the Right Data Source for Your Applications*, to use the cloud flow example that loads 500 items as a template.

The files needed for this recipe are available through the GitHub repository located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05#USER>.

How to do it...

1. Go to the Power Automate portal, <https://make.powerautomate.com>, and create a solution for this recipe.
2. Let's prepare environment variables inside the solution to set the SharePoint parameters. Click on **+ New | More | Environment variable**. Set the **Display** name to something relevant, such as `Delegation-SharePointSite`

For the **Data Type**, select **Data source** and then **SharePoint** for the **Connector** dropdown. Select an existing connection or create one from the **Connection** dropdown. For the **Parameter Type**, select **Site**. Finally, click **+ New site value** and set your SharePoint site from the **Current site** dropdown.

3. Repeat *step 2* to create an environment variable to hold the name of the SharePoint list, so for this case, select **List** for the **Parameter Type**.
4. Now, we are ready to create the cloud flow. Click **+ New | Automation | Cloud flow | Instant**. Name it, select **When Power Apps calls a flow (v2)** as the trigger, and click **Create**.
5. The first step is to get data from our SharePoint list, filled with 3,000 items. There are ready-made actions to retrieve this data from this data source, but we will introduce the SharePoint REST API to acquire these items. Why? A picture is worth a thousand words:

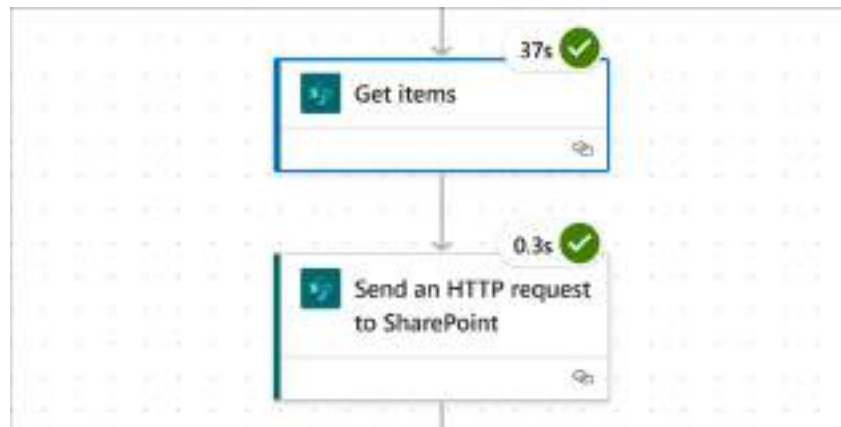


Figure 5.17: Speed comparison between two actions

Imagine your users waiting more than half a minute to get the data; this won't help your solution's adoption.

6. Add a new action and search for Send an HTTP request to SharePoint. Set the **Site Address** to the environment variable we created in *step 2*, leave **Method** as **GET**, as we are querying data, and for **Uri**, enter the following command:

```
_api/web/lists(guid'')/items?$select=Title&$top=3000
```

Let's break this code down:

- **_api**: The SharePoint endpoint where we will establish the communication.
 - **web**: Methods to access information from a given site.
 - **lists**: The endpoint for list handling methods.
 - **guid('')**: This is where we indicate that it should look for the list we selected in *step 3*. Insert the environment variable between the quotes using the dynamic data selector.
 - **items**: Gets all the list's items.
 - **select**: A modifier to specify that we only want the Title column.
 - **top**: A modifier to indicate that we want to limit the result to the first 3,000 elements. Keep in mind that going over 5,000 items requires further tweaking and pagination.
7. In the **Headers** section, add these items:
 - **Key**: accept – **Value**: application/json
 - **Key**: odata – **Value**: nometadata

nometadata indicates that we don't want extra metadata information. We do this to clean as much data from the response as possible, thereby reducing bandwidth and increasing speed.

8. Querying SharePoint data using this action returns data in JSON format. To understand this data and use it, we need to parse it. Add a new step and search for Parse JSON. For **Content**, select the body of our previous step, and for **Schema**, click **Use sample payload to generate schema** and copy the contents of the `schema.json` file provided by our GitHub repository. Another way to get this schema is to run the cloud flow with just the query action and get the body from the output, as seen in the following screenshot:

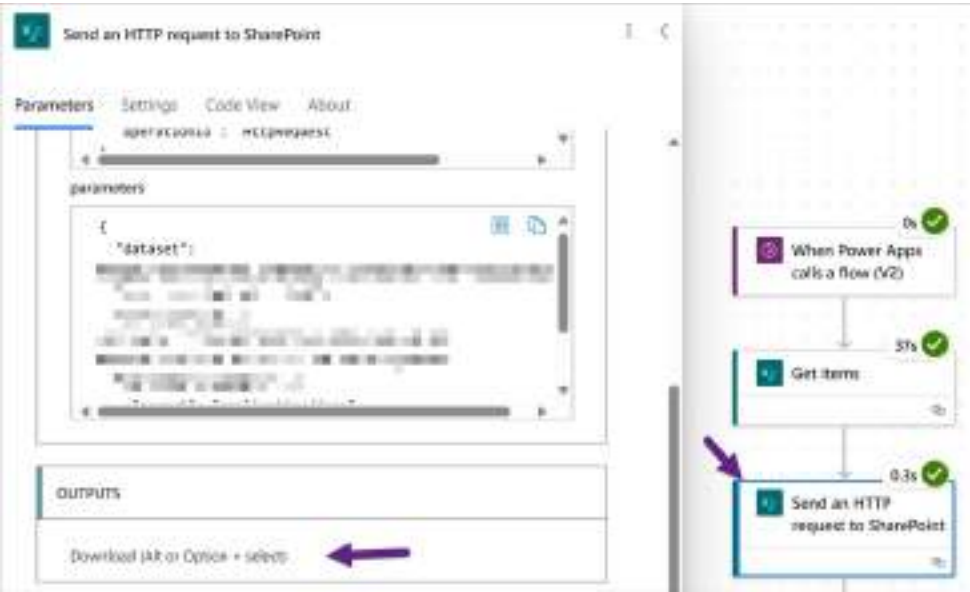


Figure 5.18: Response output from SharePoint

9. Now, let's clean up the response even further by using the **Select** action. This option allows us to select only the necessary columns from the previously received data.

Add a new action and search for **Select**. For the **From** field, use **Body** value from the **Parse JSON** action. In the **Map** section, set these values:

- **Key:** Name – **Value:** Title

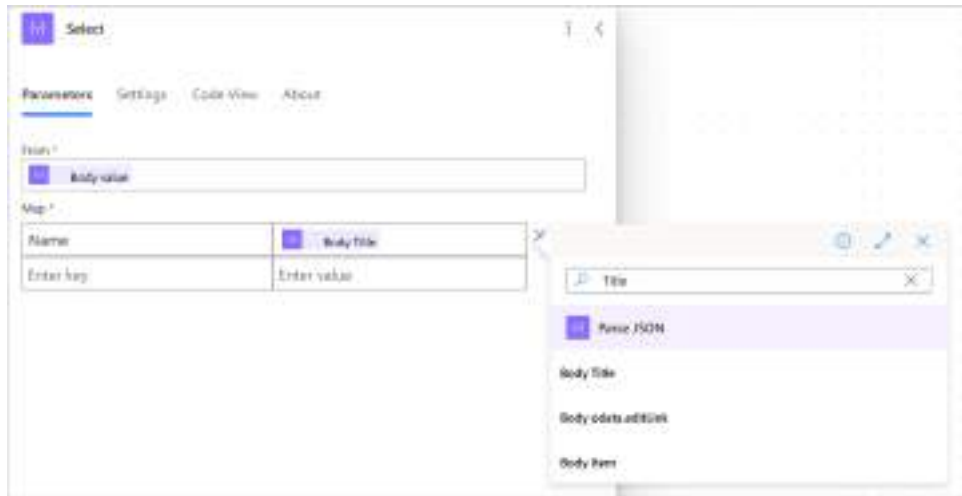


Figure 5.19: Select action mapping

10. To send the response to Power Apps, we only have a handful of data types: Text, Yes/No, File, Email, Number, and Date. None of these is a complex data type such as an object or an array. To solve this, we are going to use the Join action. As the name implies, this action joins an array to a string using a delimiter. Add a new step and search for Join. Use **Output** from the **Select** action in the **From** field. For the **Join with** field, use the semicolon.
11. Add a new action and search for Respond to a PowerApp or flow. Click on **Add an output**, select **Text**, and use data in the **Enter a name field**. For the **Enter a value to respond with**, select **Output** from the **Join** action.
12. From the toolbar, click on **Save as draft** and then **Publish**. Close the designer and go back to the solution.
13. Click on **New | App | Canvas app**. Name it and choose either the **Phone** or **Tablet** layout.
14. Insert a **Button**, a **Label**, and a **Gallery**. Change their visual properties to match your desired style.
15. From the Power Automate panel on the left, click **Add flow** and select the cloud flow you've just created from the list.
16. Using the formula bar, change the **OnSelect** property of the button to the following code:

```
ClearCollect(
    Elements,
    ForAll(
        Split(
            Delegationhelper.Run().data,
            ";"
        ),
        With(
            Match(
                Value,
                "{\\"Name\\"\\:\\\"(?:(<name>.+)\\\"}\\\"}"
            ),
            {Name: name}
        )
    )
)
```

Let's break down the previous code:

- ClearCollect creates a new collection to hold the data coming from Power Automate.
 - ForAll iterates through the array in parallel threads.
 - Split creates an array from the string received from Power Automate by breaking it, using the semicolon used to join it. Please note that Delegationhelper is the name of the connected cloud flow. Change it to the name of your cloud flow.
 - With executes a regular expression formula using the Match function, exposing the result to the name variable.
17. Hold *Alt* and click the button to execute the code and create the collection.
 18. Select the gallery control and set its data source to the Elements collection. Adapt the elements inside this control to display the collected data, using `ThisItem.Name` as its value.
 19. Select the label and set its text property to `CountRows(Elements)`. Counting the elements of the collection won't throw the delegation warning, since we have already received the complete data in the application.

How it works...

After completing the steps, you will have an application just like the following:



Figure 5.20: Delegation example Power App

As you can see, the SharePoint connector can only count the first 500 elements, while Power Automate handles the whole 3,000.

We take advantage of various actions to get data from our data sources to overcome delegation issues. Using regular expression is one way to deal with them; you could also use the ParseJSON function to handle the response from Power Automate or manipulate the received array with another approach, using the string formulas available in Power Fx.

The code we used to transform the data in Power Apps iterates through the data received from Power Automate, extracting the fields from each record and collecting the result in the Elements collection.

The extraction expression looks for patterns that match this example data, {"Name": "Element 2088"}, and then it uses capture groups to extract the fields into one record:



```
With(
  Match(
    Value,
    "{ \"Name\": \"(?<name>.+)\" }"
  ),
  {Name: name}
)
```

Figure 5.21: Capture groups in regular expressions

For more information regarding this technique, please refer to <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expression-language-quick-reference#grouping-constructs>.

Power Automate is an integration tool at its core, so it's prepared to handle a large amount of data without dealing with application responsiveness, user interaction, or adoption. This capability plays a role in our favor to solve delegation issues.

There's more...

A setting in Power Apps can bend the limit of 500 records when delegation issues appear, but it only raises the limit to 2,000.

From Power Apps Studio, click **Settings** from the main toolbar, and then change the **Data row limit** value. This might be another escape route if you can't solve your delegation issue at the data source level and your data is under the 2,000-item limit. Again, these are only workarounds for specific situations; remember that data always grows.

Regarding the use of SharePoint REST APIs, there is much more that you can do than just query data: you can create, update, and delete elements, lists, and even SharePoint sites. Please check the following link for more information about the features available in this API: <https://learn.microsoft.com/en-us/sharepoint/dev/sp-add-ins/get-to-know-the-sharepoint-rest-service>.

Creating a modular solution with Power Automate

Power Automate is a powerful tool for automating repetitive tasks and addressing business needs with its wide range of features. For example, you may have created an efficient solution using a cloud flow that manages multiple tasks with a single click.

As a use case, a marketing department uses an instant cloud flow to take care of multiple activities using an email address as input. It can:

- Send a pre-built email to that email address
- Save a logging record to a table in Dataverse
- Send a notification to the marketing team in Microsoft Teams, informing them that an email activity has been completed

What would happen if another department in your company needs the same activities that have already been created in your solution? One option would be to recreate the same actions in a new flow, but what about maintenance? If, later on, you need to change something, you would need to make the change twice.

In this recipe, we will learn how to create modular solutions in Power Automate that let us break one big solution into pieces, or reuse existing ones to improve your flows' maintenance and readability.

Getting ready

To create a modular solution, we need the help of **Power Platform Solutions**, which lets you pack several components into a single distribution package. These components include canvas apps, model-driven apps, cloud flows, Dataverse tables, and more. Please refer to *Chapter 1, App Maker Basics* for more information about solutions.

We'll start with our previous example, *Building a file processing automation for SharePoint document libraries*. The idea is to build cloud flows as if they were reusable components in your solution.

A SharePoint list, called *Marketing Emails*, is also required with the following structure:

Column name	Column data type	Required
Customer	Text	Yes
Address	Multiple lines of text	No
Email	Text	Yes

Table 5.3: *Marketing Emails' SharePoint list structure*

Add sample data to this list. You could use a service to help you create mock data, for example, Mockaroo: <https://mockaroo.com/>.

The complete solution package is available in our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05>.

How to do it...

1. Go to the Power Automate portal, <https://make.powerautomate.com>, and create a new solution.
2. Once the solution opens, add components by clicking **+ New | More | Connection Reference**. Set the **Display name** to *Modular-Office365* and choose **Office 365 Outlook** as the **Connector**. Select an existing **Connection** or create one from the dropdown menu by clicking **+ New connection**. Hit **Create** when finished.
3. Repeat *step 2* to create three more connection references: one for Microsoft Dataverse, one for SharePoint, and one for Microsoft Teams.
4. Let's add a table to log all marketing emails. Click on **+ New | Table | Table**. Set the **Display name** to *Marketing log* and the **Plural name** to *Marketing logs*. Go to the **Primary column** tab and set the **Display name** to *Email*. Leave the rest as default and hit **Save**.
5. Select **All** from the **Objects** pane on the left, and from the toolbar, click on **+ New | Automation | Cloud flow | Instant**. Name the cloud flow as you wish, for example, *Marketing Process*. Select **Manually trigger a flow** as the trigger and click **Create**. This action will open the flow designer. If you see the classic designer, follow the workaround discussed in this chapter's *Technical requirements* to get the new designer.

6. On the trigger action, click on **Add an input** and choose **Email** as the type. Name it Marketing Email.
7. Add a new action and look for **Send an email (V2)** in **Office 365 Outlook**. Insert the Marketing Email parameter into the **To** field by enabling **Advanced mode** and then clicking the Dynamic Content selector (the lightning icon). Enter some sample data in the **Subject** and **Body** fields:

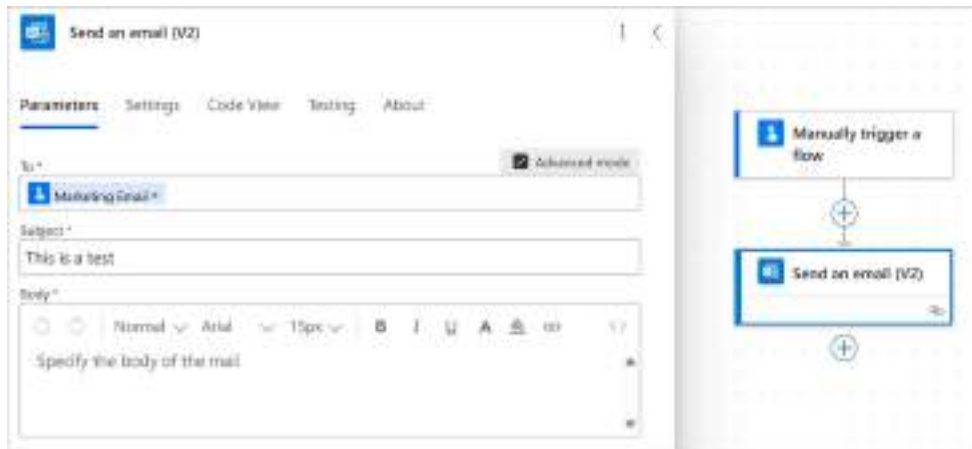


Figure 5.22: Cloud flow parameters and email action

8. Add another action and look for **Add a new row** under **Microsoft Dataverse**. Select **Marketing logs** from the list and enter the **Marketing Email** parameter in the **Email** field. This action is an example of a logging table.
9. Add one more action and look for **Post message in a chat or channel** in **Microsoft Teams**. Select **Channel** in the **Post in** field, choose any **Team** and **Channel**, and write a **Message** such as **The marketing email was delivered and logged**. Between the words **email** and **was**, insert the **Marketing Email** parameter.

10. The last step must be a response, so add a new action and look for **Respond to a Power App or flow**. Click on **Add an output** and select a data type to respond, for example, a text output with the name `Logged_email` and the **Marketing Email** parameter as the value.

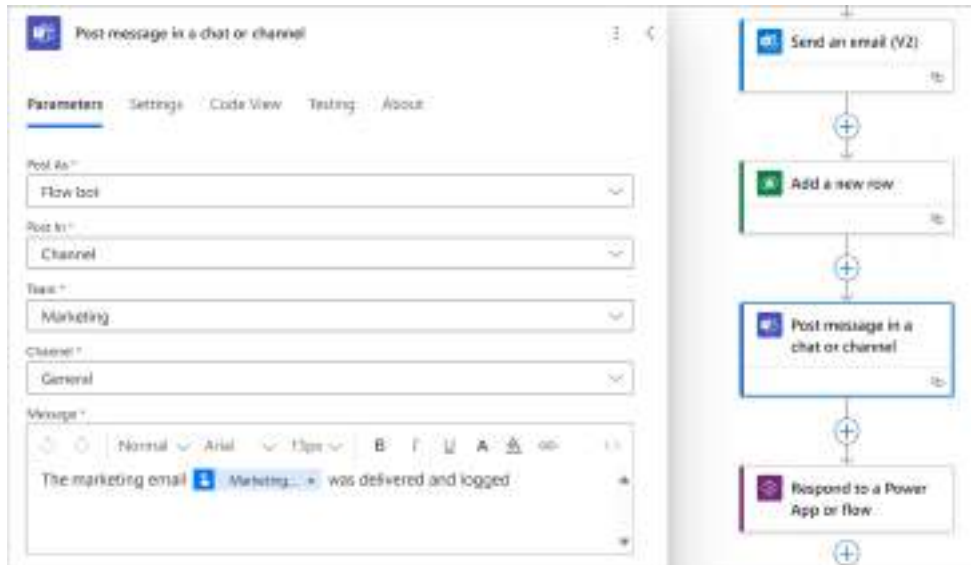


Figure 5.23: Teams action in the Marketing Process cloud flow

11. Now that this cloud flow is complete, hit **Save as draft** and **Publish**. Click the back arrow beside the cloud flow name to return to its settings.
12. Click on **Edit** on the **Run only users** section and, under **Connections Used**, set all connectors to use a specific connection and not **Provided by run-only user**, as shown in the following screenshot:

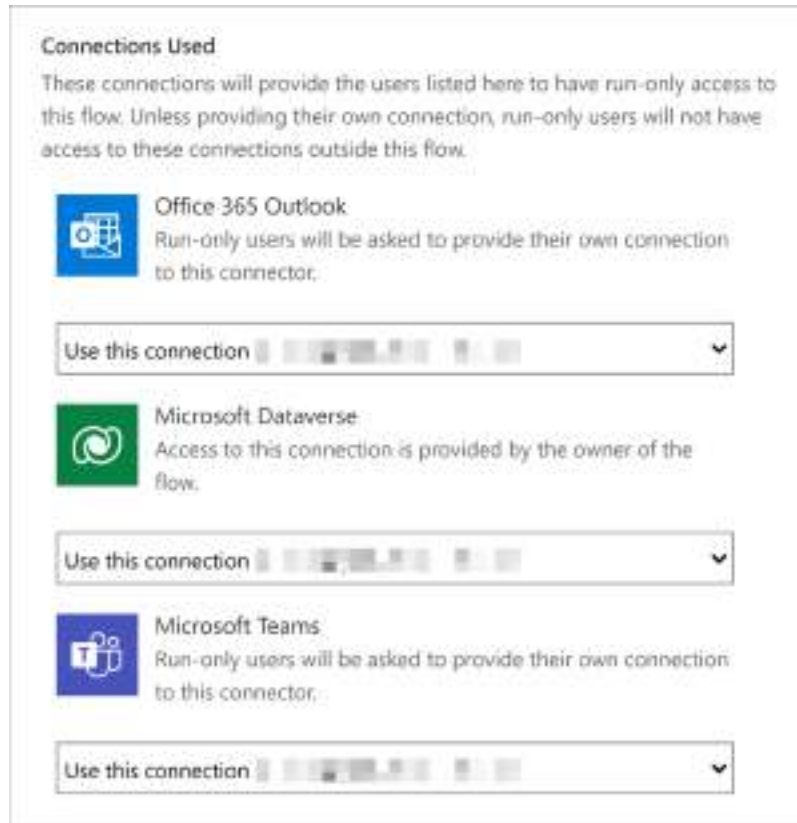


Figure 5.24: Run-only users configuration

13. Once all the connections have the right setting, click **Save** and close this browser tab.
14. Back in the solution, let's create another flow by clicking **+ New | Automation | Cloud flow | Instant** from the toolbar. For demo purposes, use the same trigger as before, **Manually trigger a flow**, and name it **Marketing Lists**. Click **Create** to continue.

Apply the same workaround as *step 5* if you land in the classic designer.

15. As an example, we will create just one action that retrieves emails from a SharePoint list and use them as input parameters for our **Marketing Process** flow. Add a new action and search for **Get items** under **SharePoint**. Configure it to get the data from the **Marketing Lists** mentioned in the *Getting ready* section of this recipe.
16. Add one more action and search for **Run a Child Flow** under **Flows**. This action only appears when designing flows inside solutions and only shows cloud flows within solutions. Select the previously created flow from the list. Once selected, it will ask for the email parameter. Use the **Email** column featured in the SharePoint list.
17. As we are dealing with a list of items received from SharePoint, a **For each** loop will be autogenerated to handle all elements in the list:



Figure 5.25: Main flow calling the child flow

18. Click **Save as draft** and then **Publish** to finish. Ignore the warning regarding the filtering of items from the SharePoint request.

How it works...

While still on the **Marketing Lists** cloud flow, click on **Test** from the toolbar to test the connected flows. Choose **Manually**, and then click **Test**, **Continue**, **Run Flow**, and **Done**.

The **main** flow should start moving the emails from the SharePoint list to the **child** flow, **Marketing Process**. This is done, in part, thanks to the **Run only users** configuration.



This configuration is necessary because when a cloud flow gets executed, it uses the user's credentials. When a parent flow calls a child flow, the latter loses the security context of the parent. For this reason, we need to configure the child flow to use the connection credentials configured in each action.

Each action has its associated connection reference, as you can see in the following image:

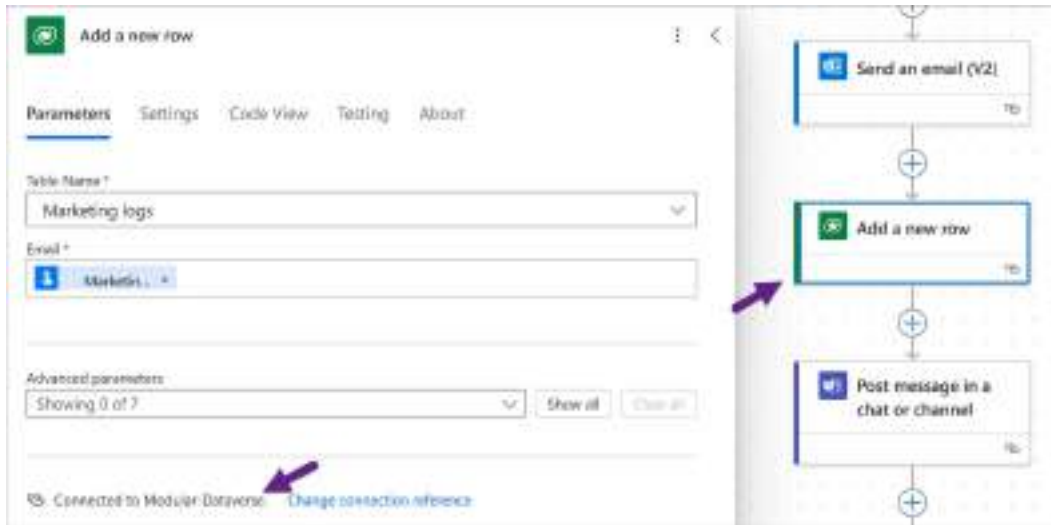


Figure 5.26: Connections used by actions

Clicking on **Change connection reference** will show you a list of all connection references configured in your environment for an action's specific service.

There's more...

Using this technique requires that your flows are part of a solution. However, there's a workaround to make this feature available for cloud flows outside solutions.

If you change the **child** flow trigger to an HTTP request, you will be able to call it using the **HTTP POST URL**. When you make this change, the platform creates a random URL to trigger this cloud flow. You can also set the **JSON Schema** of the request so that you can pass information to your cloud flow, as seen in the following screenshot:

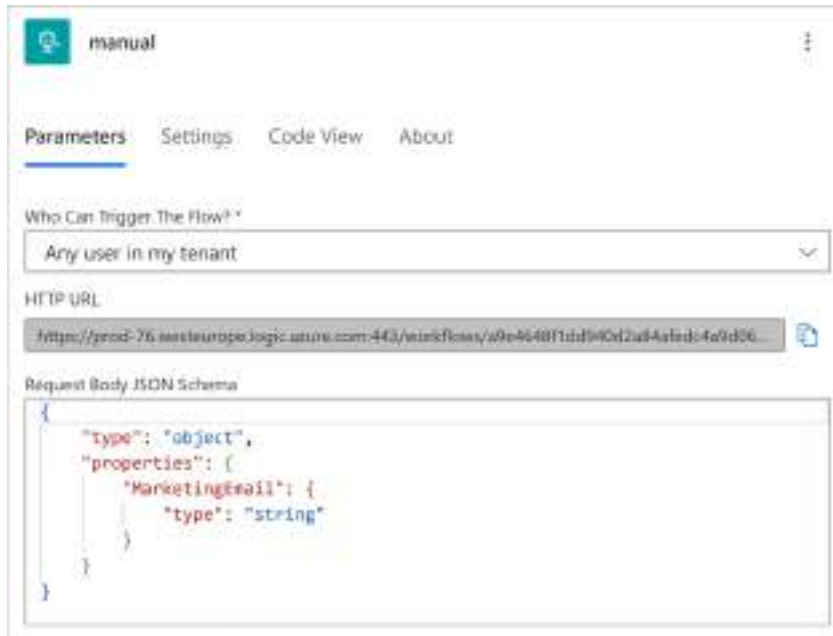


Figure 5.27: HTTP request trigger

With this alternative, you can organize all your cloud flows inside a solution and call them using the address provided by other cloud flows or external applications. This trigger also serves as a solution whenever you need an external entity to interact with your organization's resources.

However, don't forget to secure this HTTP request by setting the **Who Can Trigger The Flow?** option. Please visit <https://learn.microsoft.com/en-us/power-automate/oauth-authentication> to get more information regarding this.

Automating applications using desktop flows

Building automation solutions between connected systems is an easy task using Power Automate. Develop the logic, tie everything together using connectors, and solve your business requirements using actions in a cloud flow. But what if the systems/components we need to automate don't have a native way to connect? What if the integration involves a legacy system without API access or database connectivity?

RPA aims to solve these questions through UI-based automation. It's a specific add-on for Power Automate, which allows you to build steps just like when creating cloud flows, but this time using UI activities such as opening applications, sending keystrokes, or simulating mouse actions.

This feature also helps solve those repetitive tasks people tend to do every day. Imagine a situation where you might be working with an ERP system tied to a mainframe, giving you only a console to communicate with. The business requirement is to transfer all the invoices from the ERP system to Dynamics 365 Sales. Since the mainframe doesn't have an API, the only way to get this data is by sending keystrokes to the console to get the data.

Desktop flows can automate this task by sending predefined commands to repeat the same process for you, successfully integrating both systems without manual input from a user. It helps with legacy systems and modern applications, like the ones you use on your web browser.

This recipe will use RPA features to automate the creation of PowerPoint templates for all sessions at a major tech community event: Bizz Summit ES.

Getting ready

Working with RPA requires using a special Power Automate license called **Power Automate Premium**, which allows the licensed user to build solutions using this technology. For more information regarding this, please visit <https://www.microsoft.com/en-us/power-platform/products/power-automate/pricing>.

In your Microsoft 365 subscription, you can opt in for a free 1-month trial to thoroughly test these features. Go to the admin center of your tenant, and under **Billing**, click on **Purchase services**, search for premium, and view the **Details** of this license. From there, follow the steps after clicking on **Start free trial**.

Follow this link for a shortcut to the offer: <https://admin.microsoft.com/#/catalog/offer-details/power-automate-per-user-with-attended-rpa-plan/738E8152-767C-4E17-AE67-7545C2D2EA67>.

The following requirement is a Dataverse table to hold a list of **Tech Sessions**. Refer to *Chapter 1, App Maker Basics* to learn more about data table handling. The table needs to have the following structure:

Column name	Column data type	Required
Session name	Primary Name Column/Text	Required
Speaker	Text	Required

Table 5.4: Tech sessions' table structure

Enter some sample data into this table before building this recipe. Keep in mind that the **Session name** field should only contain characters that are allowed in filenames.

The final requirement is to install the **Power Automate machine runtime app** on the destination computer executing the RPA process. Please refer to this site for installation instructions: <https://learn.microsoft.com/en-us/power-automate/desktop-flows/install>.

Make sure you sign in using the same credentials used to build the desktop flow.

For this recipe, you will need to have PowerPoint installed on your device. The PowerPoint template and the complete solution are available on the GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05>.

How to do it...

1. Download the PowerPoint template from the GitHub repo and place it under a local specific folder, for example, C:\Slides
2. Go to the Power Automate portal, <https://make.powerautomate.com>, and create a new solution.
3. Once the solution is created, add components by clicking **+ New | Automation | Desktop flow** from the toolbar. Give the desktop flow a name, such as `Slides automation`, and click **Launch app**. You will see a dialog asking you to install the latest version of the desktop flow designer, or if you already have it on your device, it will prompt you to launch it.
4. On the right side, you will see the **Variables** pane (denoted with {x}). In the **Input/output variables** section, click the plus (+) icon and select **Input**. Set both the **Variable name** and **External name** to `Title`. For testing purposes, enter sample data for the **Default value**, and then click **Save**. Repeat this step to add the **Subtitle** variable.
5. In the **Actions** pane at the left, use the search field to search for `Run application`. Double-click on it or drag it to the main canvas. For the **Application path**, click the file icon and look for the PowerPoint template in the folder specified in *step 1*. Set the **Window style** to `Maximized` and the **After application launch** to `Wait for application to complete`, enter a **Timeout** of 3 seconds, and click on **Save**. Depending on the device and the time required to open the template, you might need to increase the timeout.
6. Look for the **Send keys** action on the left pane and add it to the canvas. Set the **Send keys to** field to `Title` and/or `class` and the **Window class** to `PPTFrameClass`. Enter the following in the **Text to send** field:

```
{Control}({H})SESSION NAME{Tab}%Title%{Alt}({A})
```

This code will replace the session name in PowerPoint. Hit **Save** to continue. The action should look like the one in the following screenshot:

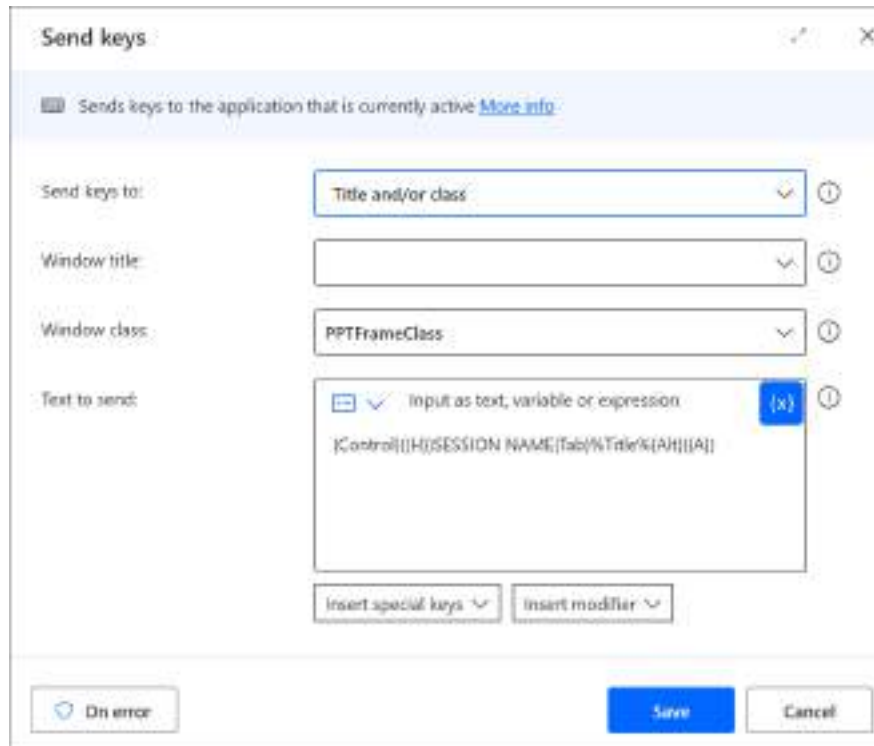


Figure 5.28: Send keys action

7. Now, search for **Wait**, enter 1 second, and click **Save**.
8. Insert another **Send keys** action, leave **Foreground** window in the **Send keys to** field, enter `{Return}{Escape}` for the **Text to send**, and hit **Save**.
9. Repeat *steps 6–8*, but in *step 6*, change the **Text to send** content to:

```
{Control}({H})SPEAKER NAME{Tab}%Subtitle%(Alt)({A})
```

You can use copy and paste to do this.

10. Add another **Send keys** action, and set the **Send keys to** field to **Title and/or class** and the **Window class** to **PPTFrameClass**. Enter the following code to open the *Save as* dialog in PowerPoint:

```
{Shift}({Control}({S}))
```

Hit **Save**.

11. Insert a **Wait** action, enter 1 second, and click **Save**.
12. Include a **Send keys** action, leave the **Send keys to** field to **Foreground window**, and for the **Text to send**, enter:

```
Session - %Title%.pptx{Return}
```

Hit **Save**.

13. Insert another **Wait** action of 1 second and click on **Save**.
14. The last step is to include another **Send keys** to close PowerPoint. Set the **Send keys to** field to **Title** and/or **class** and the **Window class** to **PPTFrameClass**. Enter the following code to open the **Save as** dialog in PowerPoint:

```
{Alt}({F4})
```

Hit **Save**. If you want, do a test run to verify that all steps work as expected.

15. On the main toolbar, click on **Save**, and then close the designer.
16. Back in the solution editor, click on **+ New | Automation | Cloud flow | Instant** from the toolbar. Name it as you wish – for example, *Slides Automator*. Select **Manually trigger a flow** as the trigger and click on **Create**. Apply the workaround mentioned in the *Technical requirements* to get the new designer.
17. Add a **List rows** action from **Microsoft Dataverse** and look for the **Table Name** mentioned in the *Getting ready* section of this recipe, *Tech Sessions*.
18. Add a new action and search for **Run a flow built with Power Automate for desktop** in desktop flows. This action will open the **Create Connection** pane. Set a **Connection Name**, such as **Slides-DesktopAutomation**, choose **Connect with username and password** for the **Connect** field, select your device from the **Machine or Machine Group** dropdown, set the credentials to log in, and click **Create New**.



If your device uses a Microsoft account to log in, creating a connection using those credentials may be challenging, since they tend to use multi-factor authentication. To solve this, create a local account on your machine and use these credentials to connect from Power Automate.

19. Once the connection configuration is complete, select the **Desktop flow** you created from the dropdown, choose **Attended (runs when you're signed in)** as the **Run Mode**, and use the **Dynamic Value** popup to set the **Title** and **Subtitle** fields from the **List rows** action. Setting these fields will auto-create a **For each** action to handle all the records created in the Dataverse table, as seen in the following screenshot:

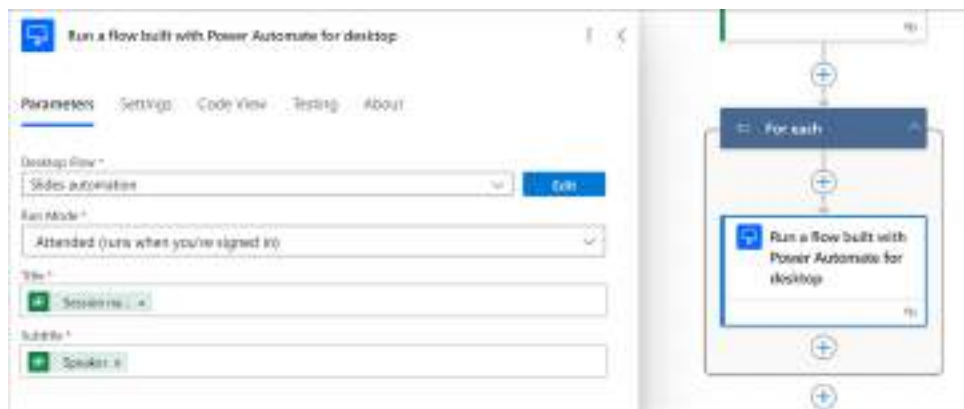


Figure 5.29: Run desktop flow action

20. Hit **Save draft**, click **Publish** on the main toolbar, and then click **Test** to try the cloud flow.

How it works...

Once the cloud flow begins, after a couple of seconds, PowerPoint should start executing the tasks step by step on the destination computer, replacing **SESSION NAME** and **SPEAKER NAME** and then creating a copy of the presentation by performing a **Save as** operation, setting the name to **Session -** and the session title. Finally, PowerPoint will close, and everything will start again for each record on the Dataverse table.

This recipe is an example of the automation possibilities to run tasks on a destination device already logged in. You could include even more actions, such as clicking UI elements, running PowerShell scripts, downloading files from the web, and more. You can even apply conditional logic with the **If**, **Switch**, and **Case** actions, to name a few.

There's more...

This recipe has been created to be executed in attended mode, meaning that the destination computer must be on and logged into. A different license, **Power Automate Process**, allows the execution of unattended desktop flows on a target machine with all users signed out. Please visit this site to see the requirements to make it work: <https://learn.microsoft.com/en-us/power-automate/desktop-flows/run-unattended-desktop-flows>.

For users running Windows 11, Power Automate Desktop comes preinstalled to automate tasks immediately. More information on this topic is available here: <https://www.microsoft.com/en-us/power-platform/blog/2021/10/04/automation-in-windows-11-enables-users-to-reduce-repetitive-tasks-and-boost-productivity/>.

Integrating Google Forms with Dataverse using cloud flows

Connectors in the Power Platform play a significant role when building solutions that incorporate services from Microsoft, other companies, or even on-premises systems. As far as other companies are concerned, many connectors (more than 1,200 at the time of writing) allow connectivity for such services.

In Google’s case, you will find connectors for Calendar, Contacts, Drive, Sheets, and Tasks, but no connector exists for Google Forms. You can build a custom connector to make this possible, but handling all the connectivity and security requirements might be challenging.

For this recipe, we will leverage the capabilities of Google’s Apps Script, which is included in Google Forms. This feature allows you to build JavaScript code with triggers to perform actions. In this case, we will transfer the form’s data on every submission to a cloud flow endpoint that sends a message to Microsoft Teams.

Getting ready

To build this recipe, you must have a Google account to create a Google Form with sample questions. Create a form using this template:

Question	Type	Required
Email	Short answer	Yes
How did you discover us?	Multiple choice: Search engine, Social media, or Blog	No
How was the event?	Linear scale: 1 – Dull to 5 – Great	Yes

Table 5.5: Tech sessions’ table structure

On the Microsoft side, this cloud flow will need a Power Automate license because it will use the premium HTTP trigger. For more information on licensing, please refer to the *Preface* of this book.

The resources for this recipe are available on the GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-05>.

How to do it...

1. Go to the Power Automate portal, <https://make.powerautomate.com>, and create a new solution.
2. Once the solution has been created, click **+ New | Automation | Cloud flow | Automated** from the toolbar. Name it as you wish – for example, **Google Forms Integrator**. Select **When an HTTP request is received** as the trigger, and click **Create**. Apply the workaround mentioned in the *Technical requirements* to get the new designer.
3. From the trigger, change the **Who Can Trigger This Flow** dropdown to **Anyone**.
4. Add a new action, search for **Compose**, and add it to the cloud flow. As the input, enter **test**, and then, from the toolbar, click **Save as draft** and then **Publish**. This action is needed to be able to save the cloud flow and get the **HTTP POST URL** from the trigger action. Copy this URL to use it later.

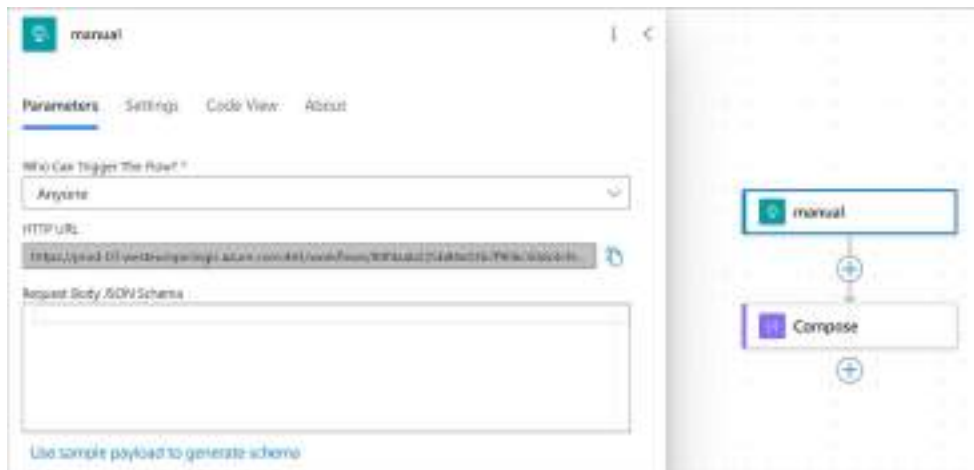


Figure 5.30: When an HTTP request is received trigger

5. Go to Google Forms, <https://docs.google.com/forms>, and click on the plus sign on the lower right of the screen or **Blank form** under **Start a new form**. This action will open the form designer.

6. Add the questions mentioned in the *Getting ready* section. Name your form **Sample form**. Once completed, your form designer screen should look like the following:

The image shows a Google Form titled "Sample form". It has a light blue header bar. Below the title, there is a text input field for "Form description". The second question is "Email:" followed by a text input field and a red asterisk indicating it is required. The third question is "How did you discover us?" with three radio button options: "Search Engine", "Social media", and "Blog". The fourth question is "How was the event?" with seven radio button options: "Dull", "1", "2", "3", "4", "5", and "Great".

Figure 5.31: Sample Google form

7. Click on the ellipsis at the top right of the screen and select **Script editor**. A new tab with the **Apps Script** code editor should appear. Copy the contents of the `Code.js` file located in the book's GitHub repository, and replace the existing code in the script editor by pasting this in. Replace the `{URL}` text near the end with the URL from *step 4*. Click the save icon on the toolbar to save the project.
8. On the left pane, click the clock icon to configure the triggers. Click on **Add Trigger** at the bottom-right corner of the screen. Leave all settings as default on the open dialog, but change the **Select event type** to `On form submit` and click **Save**. You might get a Google security screen warning you about the app you're building with Apps Script. Click **Advanced** at the bottom left, **Go to Untitled project (unsafe)**, and then **Allow**. By doing this, you will validate yourself as the app owner.
9. Close the **Apps Script** tab, and back in the form, click the eye icon to preview it. Enter some sample data and hit **Submit**.

10. Go back to Power Automate and open the cloud flow details. You will notice an execution on the **28-day run history** list. Click on the date to open it and then expand the trigger action. Scroll to the **OUTPUTS** section and then to the **body** result. Copy this information, as seen in the following screenshot:

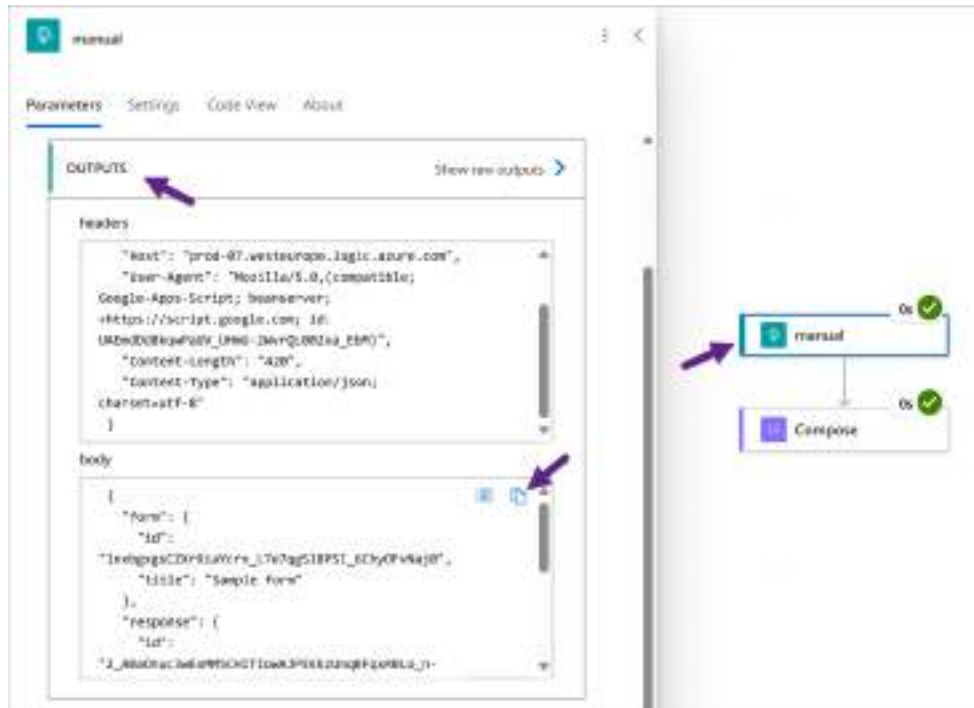


Figure 5.32: Request received from Google Forms

11. Click **Edit** on the top right of the screen.
12. Delete the **Compose** action and add a **Parse JSON** in **Data Operation**. For the **Content** field, select **Body** from the trigger action using the **Dynamic Value** panel. Click on **Use sample payload to generate schema**, paste the body portion from *step 11*, and click **Done**.
13. Add a new step called **Post message in a chat or channel in Microsoft Teams**. Select **Channel** in the **Post in** field, set the desired **Team** and **Channel**, and build a message using the **Parse JSON** result.
14. Click **Save draft** and **Publish**, and then close the designer.
15. Go back to the Google Forms portal and open the **Sample form**. Click on the eye icon again to preview it. Enter the sample data again and click on **Submit**.

How it works...

The Google Apps Script module acts as a Power Platform connector, by handling the trigger action that would have been found in the native Google Forms connector if it existed. Leveraging the JavaScript code that transforms the received data into a JSON response makes integrating with Power Automate seamless.

You could use this data in many other ways, such as saving these surveys into Dataverse, feeding the information into a Power BI report, or handling this information on Dynamics 365 Customer Insights.

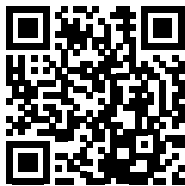
There's more...

You can find the original code used to transform the survey data into a JSON response on the GitHub Gist at <https://gist.github.com/lifehome/db101c66fe547a8ffbffe10d1bc0b426>, but there is another version called **Google Forms JSON exporter** that can handle more Google Forms field types. The repository for this solution can be found here: <https://github.com/stevenschmatz/export-google-form>.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



6

Extending the Platform

Power Platform includes several tools and services dedicated to building solutions that help improve organizational procedures. The tools we use to create applications and integrations offer a range of standard features in their field while leaving room for extensibility.

When building applications, you need to create interfaces that interact with your end users; in this process, you might need to repeat a set of controls such as headers, menus, loaders, etc. This task can be tedious as it slows development when a new change is needed.

This chapter will teach us how to speed up the application-building process using **components**. These elements allow the creation of small building blocks that focus on a specific task by encapsulating a set of controls that can be reused across your application. You will also learn how to create custom connectors to integrate with data sources using tools your developers already use, such as **Postman**. Before diving in, we will cover the user interface guidelines developed by today's tech giants, further speeding up the app-building process.

The following recipes, which we will cover in this chapter, are going to define the many ways to extend the platform:

- User interface guidelines
- Setting up your first canvas app component
- Building a floating action button component
- Creating a vertical navigation component
- Using Power Apps component libraries
- Expanding communications with custom connectors

- Integrating map capabilities into canvas apps
- Extending canvas solutions using Mixed Reality

Technical requirements

Building and using components for Power Apps application development requires no additional license. However, regarding the custom connector recipe, we will require any of the plans described in the *Preface* of this book.

For a more in-depth insight into licensing, please review *Chapter 4, Data Sources and the Licensing Model*, to get a better understanding, complete with some real-world examples.

The completed recipes of this chapter are available from our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-06>.

User interface guidelines

Designing intuitive applications has become an art form over the years. Initially, developers did not consider the applications' usability; they only focused on providing the required functionality.

Since the mobile revolution back in 2007, an app's design has become as important as its purpose. Applications that are easier to use are more successful than others with more features. That's why all major players in the mobile industry started to define their approaches to user interface design by creating guidelines for their devices. Toolbars were initially at the bottom and then at the top; now, they're back at the bottom again. There have been many iterations due to the changes in device sizes, screen aspect ratios, and more.

Today's main guidelines come from Google, Apple, and Microsoft. It doesn't matter what type of device it is; there are guidelines for it. The idea is to give app designers and developers cues on how to design their apps. The approach is to have one unique backend but different interfaces depending on the mobile ecosystem.

The applications we develop should use cues from these guidelines because users already know what these designed controls do. Let's take the floating action button, for example. Its function is quite well known; it is the main activity in the application. The following screenshot represents the use of this element in an app:



Figure 6.1: Floating action button in action

This control is just an example of all the definitions you can find in these guidelines. To gather more ideas from them, refer to these links:

- Google: Material Design:
<https://m3.material.io/>
- Apple: Human Interface Guidelines: <https://developer.apple.com/design/human-interface-guidelines/>
- Microsoft: Fluent Design System:
<https://fluent2.microsoft.design/>

Besides these links, refer to these books for further information:

- *Designing and Prototyping Interfaces with Figma*, Second Edition, by Fabio Staiano
- *101 UX Principles*, Second Edition, by Will Grant
- *Designing Interfaces*, Third Edition, by Jenifer Tidwell, Charles Brewer, and Aynne Valencia

Take into account that these guidelines not only cover how we need to design our controls but also offer specifications for accessibility and content design, to name a few.

Setting up your first canvas app component

After discussing the advantages of using cues from design guidelines, let's mix this concept using canvas app components. With the help of this recipe and the others that fall into the components category, we will recreate some of the most common controls in the design guidelines so you can reuse them in your apps.

Imagine building a canvas app with 30 screens using the same menu copied over and over. Eventually, the business process might require a new option to be included in the menu. You would have to apply the change to each of your application screens.

As mentioned before, a component encapsulates controls that serve one purpose. Working with them is like creating a new screen; the designer and the available controls are the same. The difference lies in the fact that you can set the dimensions to match a specific size.

Going back to the previous example, creating a menu component will allow replicating instances of this element on every screen while giving you one single place to update it. Once updated, all screens will reflect the change automatically.

Components also have custom properties that allow communication between them and the application. Let's create one from scratch to see these concepts in action.

Getting ready

The components feature comes enabled by default, even for old applications. You will see a new tab in the **Tree view** called **Components**, where you will find the list of components available. Let's create our first one from Power Apps Studio.

How to do it...

1. From the **Tree view**, go to the **Components** tab and select **New component**. This action will create a new one and adapt the canvas designer for editing.
2. From the right pane, change the component's **Height** to 100 and set **Width** to `App.Width`. Changing these values to formulas or variables requires the change to be done in the formula bar, as seen in the following screenshot:

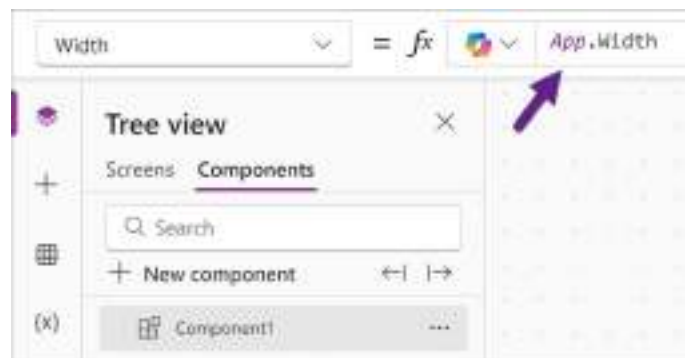


Figure 6.2: Changing properties in the formula bar



SPEED TRICK: if you click the label of the control's property, the formula bar will jump directly to that property.

3. Change the **Fill** property to your color of choice. I have set it to a dark blue.
4. Go to the **Insert** toolbar and select **Text label** to add it to the canvas. Set its **X** and **Y** properties to 0, the **Width** to `Parent.Width - 100`, and the **Height** to `Parent.Height`. Change the **Left** padding to 20. Set the font color to White and the size to 28. Hover over the control's name to click the edit icon and set the name of the label to `TitleLbl1`, as seen here:

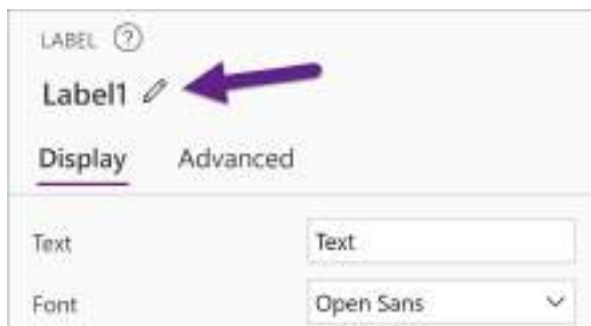


Figure 6.3: Control renaming in Power Apps

5. From the **Insert** panel, use the search field to look for Home and then select the **Home** icon. For the positioning, set the **Y** to 0 and the **X** to `Parent.Width - Self.Width`. Set the **Width** and **Height** to 100. It might look weird, but we'll fix it with the padding. Set all **Padding** properties to 20. Finally, set its **Color** to White. Name your icon `TitleIcon`.
6. The component should look like the following screenshot:



Figure 6.4: Title component in progress

7. From the **Tree view**, click on the component we created and set its name to `TitleCmp`, and from the right pane, click on **New custom property** for each of the following properties:

Display name	Name	Description	Property type	Data type
Title	Title	Title	Input	Text
IconName	IconName	Icon's name	Input	Text

Table 6.1: Component's custom properties

The custom properties will look like the following screenshot:

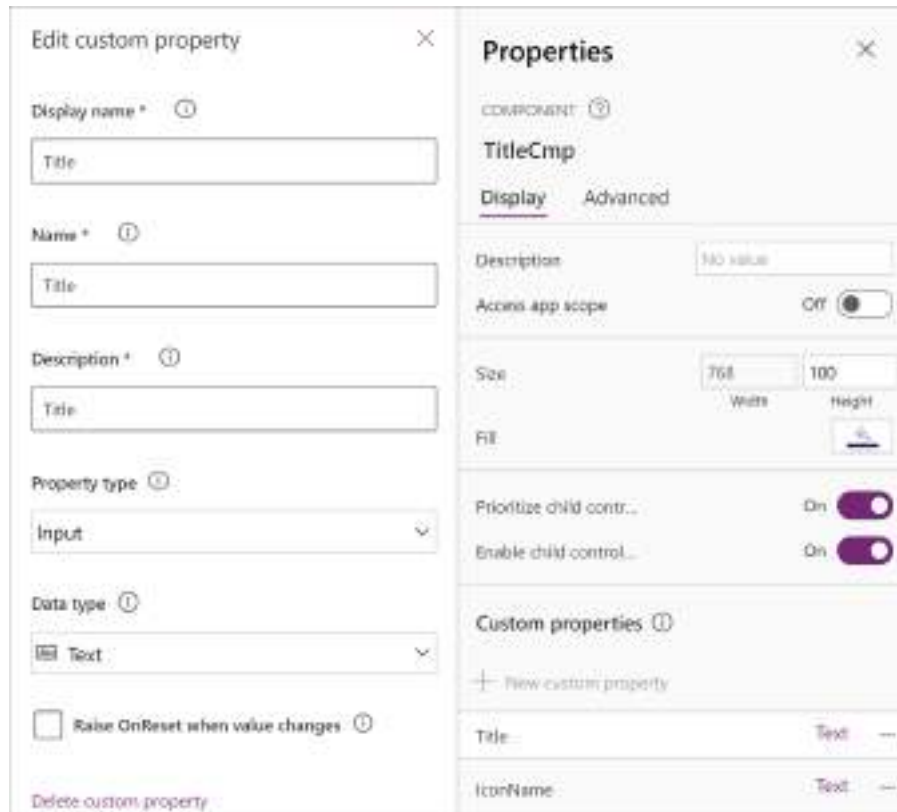


Figure 6.5: Component custom properties

8. From the **Tree view**, click on **TitleCmp** and, using the formula bar, change these properties: **Title** to `Screen.title` and the **IconName** to `Icon.Home`
9. Select **TitleLbl** from the **Tree view** and set its **Text** property to `TitleCmp.Title` using the formula bar. Now, select the icon and change its **Icon** property to `TitleCmp.IconName`, again, from the formula bar.

This component is now ready. Let's take it for a test drive.

How it works...

We have built a header-like component using custom properties, which allows changing internal properties from the app using it. To use it, select the **Screens** tab from the **Tree view** and, from the **Insert** panel, expand **Custom** and then choose the component we just created, **TitleCmp**, from the list.

This action will add it to our canvas and, because we've set the width to `App.Width`, it will expand automatically, no matter the layout of our app. The same goes for the controls inside the component. When using `Parent.Width`, the controls adapt to the width of their parent, in this case, the component.

While this is selected, let's head to the right pane to complete its configuration. Using the formula bar, change the **Title** property to `My component` and **IconName** to `Icon.Items`. Rename the component to anything you want.

The application will end up looking like this:

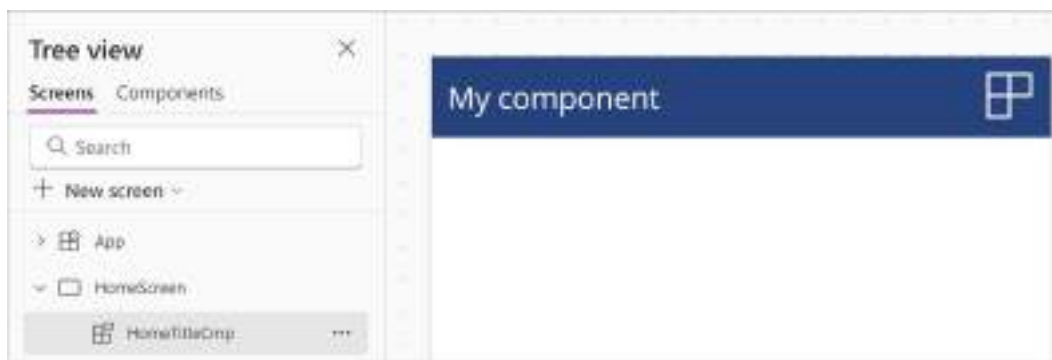


Figure 6.6: Custom properties in action

Custom properties offer more functionality to components than just encapsulating a series of controls. When adding screens, you can reuse this component and change the `Title` and `IconName` properties for each screen. Adding a background color property will allow you to create screens, as shown in the following screenshot:



Figure 6.7: Header component examples

Another added value to this technique is that it lets you maintain design consistency across your app by giving the user the same look for all the screens. Let's learn more about the components' functionality in the following recipes.

There's more...

Knowing the concept of components will make you think differently when designing your applications. From the very beginning, you will start with the idea of building elements as components to help you build applications faster.

Keep an eye on existing ones created by Microsoft and the community, such as the ones seen here: <https://powerusers.microsoft.com/t5/Canvas-Apps-Components-Samples/bd-p/ComponentsGallery?featured=yes>.

Building a floating action button component

One of the most recognizable features in a mobile design user interface is the **Floating Action Button (FAB)**. They represent the primary action of the active screen. Almost every user will know their purpose because they have seen them in all major applications, such as Microsoft Outlook:



Figure 6.8: Write new email FAB in Microsoft Outlook

We will learn how to replicate this element using components in this recipe.

How to do it...

1. Go to the Power Apps maker portal and create a solution.
2. Inside the solution, create an app by clicking **New | App | Canvas app**. Name it, select the **Phone** layout, and click **Create**.
3. From the **Tree view**, go to the **Components** tab and click on **New component**. From the right pane, name it FABComp. Set its **Width** and **Height** to 95 and its **Fill** color to transparent: `Color.Transparent`
4. Go to the **Insert** toolbar, search for the **Add** icon, and click on it to add it. Set its **X** and **Y** properties to 5 and its **Width** and **Height** to 80. All padding properties should be 20. Lastly, set its **Color** to White and name this control IconIcon

- Using the **Insert** toolbar, select **Circle** from the **Shapes** group. Set the same position and size properties as the icon we just inserted. Rename it to BackgroundCr1. Using the three-dot menu near the name of the control, **Reorder** it to place it at the bottom of the canvas so the icon is on top.
- We will add a shadow to the circle to make this component more fashionable. From the **Insert** toolbar, expand **Display** and then select **HTML text**. Set its **X** and **Y** properties to 0 and the **Width** and **Height** to 95. All padding properties should be 5. Using the formula bar, set the **HTMLText** property to this value:

```
"<div style='height: 80px; width: 80px; border-radius: 50px;
-webkit-box-shadow: 2px 2px 5px 0px rgba(0, 0, 0, 0.75); -moz-box-
shadow: 2px 2px 5px 0px rgba(0, 0, 0, 0.75); box-shadow: 2px 2px 5px
0px rgba(0, 0, 0, 0.75);'></div>"
```

- Reorder the HTML text control to the back as we did in *step 3* and name it ShadowHTML. Your FAB should have this look:

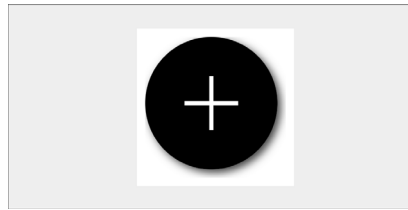


Figure 6.9: FAB component

- Go to the component properties and click on + **New custom property** for each of the following properties:

Name	Description	Property type	Data type
IconName	Name of the icon	Input	Text
CircleColor	Color of the circle	Input	Color
Screen	Screen name	Input	Screen

Table 6.2: FAB component's custom properties

- Move to the **Advanced** tab of the component properties and change the **IconName** default value from Text to Icon. Add

10. Select the **Icon** control and, using the formula bar, change the **Icon** property to `FABComp.IconName`. Then update the **OnSelect** property to this code:

```
Navigate(FABComp.Screen, ScreenTransition.Fade)
```

11. Select the **BackgroundCr1** control and, using the formula bar, change the **Fill** property to `FABComp.CircleColor`. You will see that your component changes its appearance to black. This variation is due to the change of its visual properties to formulas or variables.

This component is now ready to transform depending on the values set by the custom properties.

How it works...

Go back to the **Screens** list in the **Tree view**, and from the **Insert** panel, expand **Custom** and select **FABComp**. Set it at the bottom right of your screen.

You can refer to this guideline for information about placement: <https://m3.material.io/components/floating-action-button/specs#7712fa7f-cd29-4852-86a9-fa2f4a01f6bc>.

From the properties, change its **CircleColor** to match your application style. As for the rest, choose the icon name that best reflects the primary action and set the screen name that you want to navigate to when the user selects it. The following are examples of using this control:

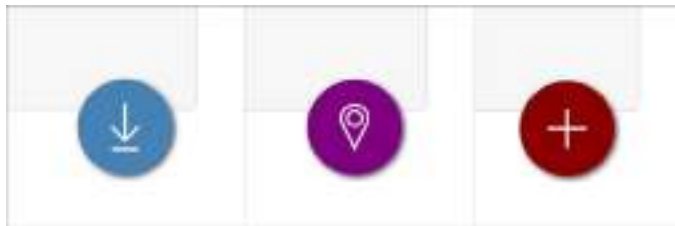


Figure 6.10: FAB component examples

If you run the application, selecting the FAB control will change the active screen to the one set in the custom properties, allowing you to redirect your user to a new screen where the main action will occur.

There's more...

Canvas components are still evolving. Enabling *Enhanced component properties* improves components' capabilities with **behavior properties**, allowing them to fire **custom events**. To get more information regarding these properties, please visit <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/component-properties>.

The recipe solution file, available from our GitHub repository, also includes the new FAB component that appears in Google's new Material Design 3 approach.

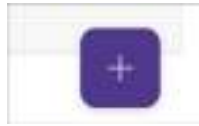


Figure 6.11: Material Design 3 FAB component

Creating a vertical navigation component

Guiding users through navigational controls is a known technique in almost all user interfaces. There are many ways to design them; examples are application wizards, tabs, and the like.

Power Apps Studio includes a ready-to-use template screen that uses a tutorial-like approach from the **New screen** selector:

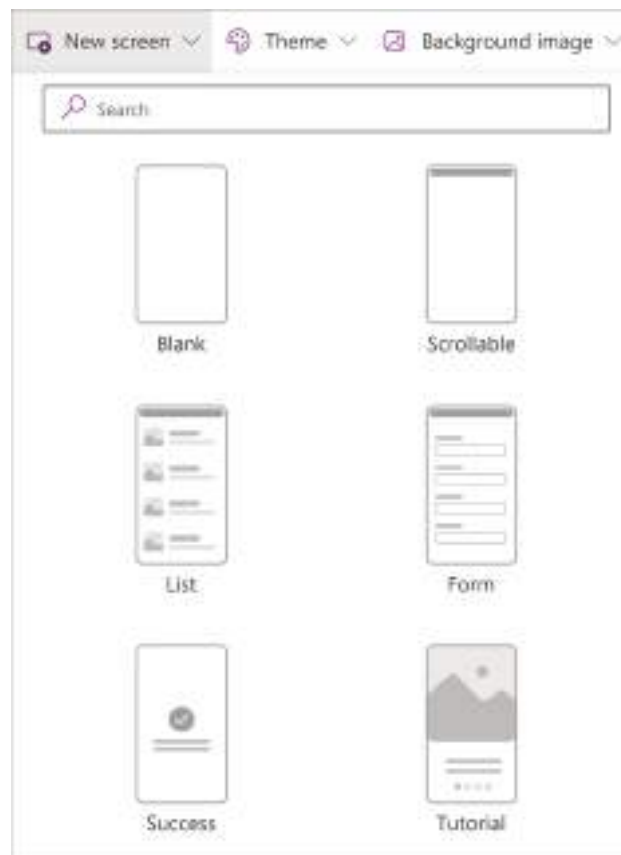


Figure 6.12: Tutorial screen template

You can take advantage of this feature to add a short tutorial for your new users.

This recipe will create a vertical navigational component suitable for desktop applications that will serve as a tool to guide your users.

How to do it...

1. Navigate to the Power Apps maker portal and create a solution.
2. Inside the solution, create an app by clicking **New | App | Canvas app**. Give it a name, and select the **Tablet** layout, as this component is more suitable for this form factor. Click **Create** to continue.
3. Go to the **Components** tab in the **Tree view** screen of Power Apps Studio and click **New component**. Set its **Width** to 180, the **Height** to `App.Height`, and, using the formula bar, the **Fill** to `RGBA(130, 141, 158, 1)`. Name this component `NavCmp`
4. From the **Insert** panel on the left, expand **Media** and select **Image**. Set the **X** and **Y** properties to 20. For the size, set the **Width** to 140 and **Height** to 60. Name this control `LogoImg`.
5. Expand the **Shapes** group of the **Insert** panel and select **Rectangle**. Set these properties: **X** to 15, **Y** to 100, **Width** to 150, and **Height** to 1. For the **Fill** property, enter the following value in the formula bar: `RGBA(237, 237, 237, 0.5)`. Set the name of this control to `TopDividerRct`
6. Add a gallery by expanding **Layout** and selecting **Blank vertical gallery** from the **Insert** panel. Position it to 10 on **X** and 120 on **Y**. Set the **Width** to 160 and the **Height** to `App.Height - 200`. Change its **Template size** property to 50. Name this control `OptionsGly`
7. Edit the gallery by clicking on the pencil icon, as shown in the following screenshot:

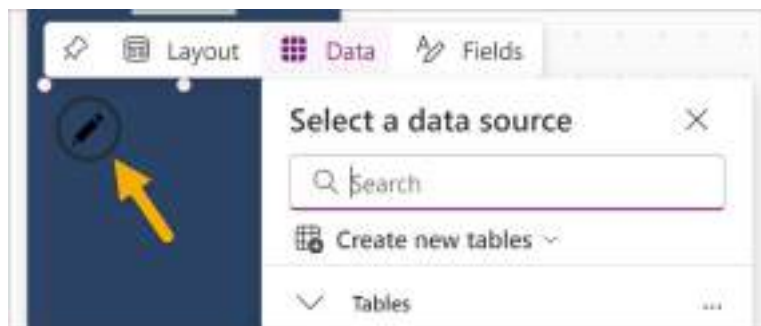


Figure 6.13: Gallery pencil icon

Add these controls from the **Insert** panel:

- From the **Media** group, select **Image**. Place it like this: 5 for **X**, 10 for **Y**. For the size, set the **Width** to 140 and the **Height** to 35. Set the **Border radius** to 12 and reset its **Image** property to **None**. Name this control `OptionBGImg`
 - Add a **Text label** and set its **X** value to 5 and **Y** to 10. Change the **Width** to 140 and the **Height** to 35. For the padding, set the **top** and **right** to 5, the **left** to 15, and the **bottom** to 7. Update its **Color** to **White** and name this control `OptionLbl`
 - Expand **Icons** and select the **Add** icon. Place it on 0 for **X**, 10 for **Y**, and set the **Width** to 150 and the **Height** to 40. Set the left padding to 150. Name this control `OptionIcon`
8. Copy the rectangle created in *step 4*. Change its properties to 15 for **X** and, using the formula bar, `App.Height - 60` for **Y**. Name this control `BottomDividerRct`
 9. Add a **Text label** from the **Insert** panel and, using the formula bar, place it on 0 for **X** and `App.Height - 53` for **Y**. Set the **Width** to `Parent.Width` and the **Height** to 40. Set the **Font** to something low, like 11, the **Text alignment** to **Center**, and the **Color** to **White**. Set the **Text** property to **Back** and name this control `BackLbl`
 10. Now, expand **Icons** and select the **Add** icon. Place it using `BackLbl` as the reference; **X** would be `BackLbl.X` and **Y** `BackLbl.Y`. Do the same for its size: **Width** would be `BackLbl.Width` and **Height** `BackLbl.Height`. Set the left padding to `Self.Width`
 11. Go to the **Advanced** tab of this control's properties and input `Back(ScreenTransition.Fade)` on the **OnSelect** property. Set the name of this control to `BackIcon`
 12. Go to the component's property pane and click + **New custom property** for each of these as an **Input** property type:

Display name	Name	Description	Property type	Data type
ApplicationLogo	ApplicationLogo	Application logo	Input	Image
NavColor	NavColor	Navigation primary color	Input	Color
Screens	Screen	List of screens	Input	Table
Active Screen	ActiveScreen	Active screen	Input	Screen

Table 6.3: Navigation component's custom properties

13. To be able to use these custom properties, we will set their values using sample data from the component itself. Go to the **Advanced** tab of the **NavCmp** properties and set the following data:

- **NavColor:** `ColorValue("#2A4365")`
- **Screens:** `Table({Name: "Home", Screen: App.ActiveScreen}, {Name: "Departments", Screen: App.ActiveScreen})`

14. Let's link these custom properties to the respective controls using the formula bar:

- Change the **Fill** property of the component to `Self.NavColor`
- Set the **Image** property of `LogoImg` to `NavCmp.ApplicationLogo`
- Change the **Data source** of `OptionsGly` to `NavCmp.Screens`
- Set the **OnSelect** property in the **Advanced** tab of the `OptionIcn` control to `Navigate(ThisItem.Screen, ScreenTransition.Fade)`
- Change the `OptionLbl` **Text** property to `ThisItem.Name`
- Set the `OptionBGImg` **Fill** property to `ColorFade(NavCmp.NavColor, -25%)` and the **Visible** property to `ThisItem.Screen = NavCmp.ActiveScreen`

It's easier to see how the component might look with the sample data and the custom properties in place. Check the following screenshot as an example:

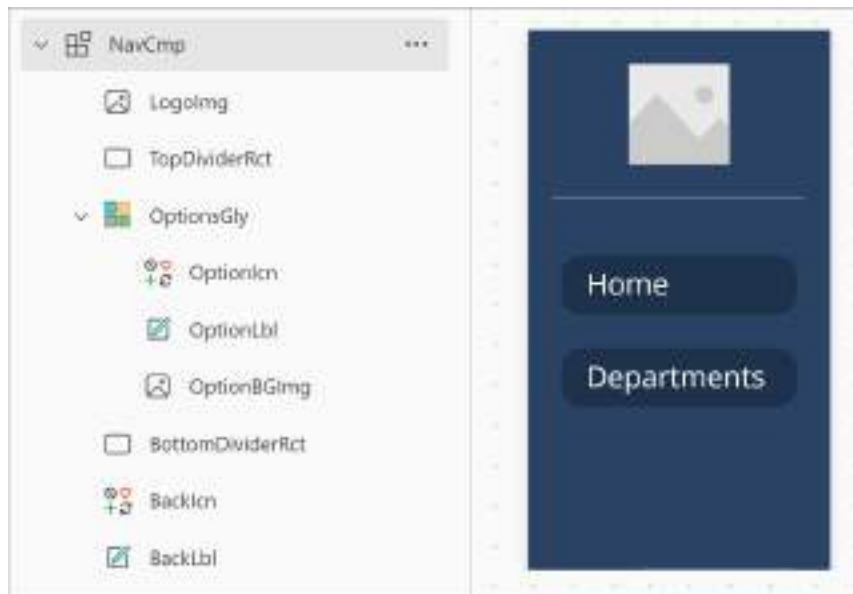


Figure 6.14: Navigation component with sample data

Ensure that all control placement follows the pattern seen in the previous screenshot. If you need to make changes, click on the ellipsis and reorder as required:

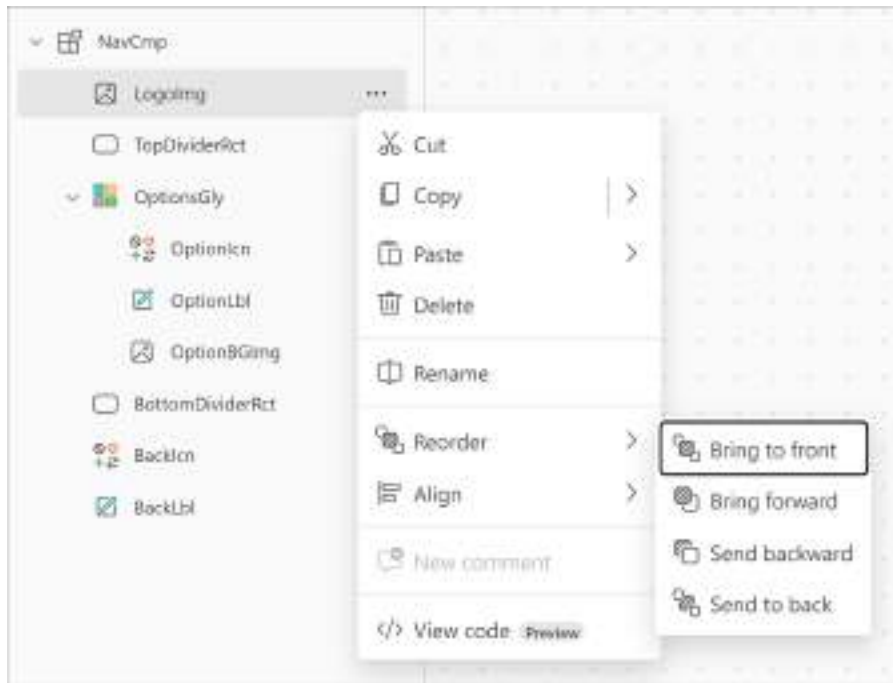


Figure 6.15: Control order inside the component

The component is now complete. Go back to the main canvas to complete the configuration.

15. Name your first screen **HomeScreen**, and create a new screen called **DocumentsScreen**
16. From the **Insert** toolbar, click on **Custom** and select **NavCmp**. Do this on both screens.
17. Upload an image through the media tab and set it as **ApplicationLogo** in the component's properties.
18. Select the **App** element in the **Tree view**. In the **Advanced** tab, select the **OnStart** property and set its value as the following:

```
Set(
  gblMenuOptions,
  Table (
    {
      Name: "Home",
      Screen: HomeScreen
    },
  ),
```

```

        {
            Name: "Documents",
            Screen: DocumentsScreen
        }
    )
)

```

19. Right-click the **App** element and select **Run OnStart**.
20. Set `gb1MenuOptions` as the component's **Screens** property.

Now that the component configuration is complete, let's test it by running the app.

How it works...

When the application starts, the component acts as a navigation system for all the defined screens, as shown in the following screenshot:

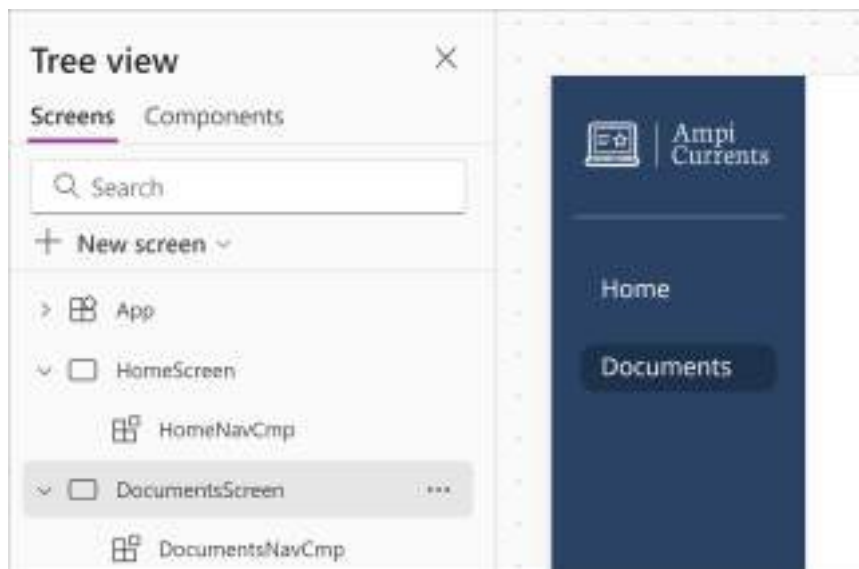


Figure 6.16: Navigation component in action

Because the component uses sample data, we can see examples of the data needed in each property, such as the required format for the **Screens** property table.

Let's explain some of the features of this component:

- If you need to add a new screen to your application, add a new record to the `gb1MenuOptions` variable, and all the components will reflect the change.

- The use of the `OptionIcn` and `BackIcn` icons is to imitate a *hand cursor* instead of a *text cursor*. Users expect this kind of behavior when selecting options using the mouse. You can see the result in the following screenshot:



Figure 6.17: Hand cursor over a Text label

- Adding a negative color fade to `OptionBGImg` creates a darker color in the component's custom property, making it suitable for a *selected* indicator.
- The lower elements, such as `BottomDividerRct` and `BackLbl`, use negative calculations based on the height of the application. This positioning technique makes them stay at the bottom, regardless of application size.
- Even though we created the `ActiveScreen` custom property without setting any value, it gets the default value from `App.ActiveScreen`, informing the component about the active screen, therefore helping us with the **Visible** property formula: `ThisItem.Screen = NavCmp.ActiveScreen`. If the active screen corresponds to one in the gallery, `OptionBGImg` becomes visible, acting as an indicator.

This component offers a simple solution for apps using the tablet layout by providing a set of customizations to make it flexible.

There's more...

Canvas components offer great extensibility when building user interfaces, but if you want to mix them with modern controls built by Microsoft, you could try the **Creator kit**: <https://learn.microsoft.com/en-us/power-platform/guidance/creator-kit/overview>.

This kit consists of a series of components based on the Fluent UI framework to improve the app-building process. You'll find components for both canvas and model-driven apps. Keep in mind that this kit has been built using the Power Apps Component framework, so you need to enable the use of code components in the environments you plan to use them in.

Using Power Apps component libraries

This chapter shows us the advantages of using components when building and maintaining applications. We encapsulate controls that work together for one purpose and then reuse them across our application.

However, if we wanted to use a specific component across several applications, our only resort would be to export this component and import it manually on each application. This process breaks maintainability because each application would have an independent copy of the component. If we update one component, all others remain unchanged. In fact, exporting and importing components is a feature that will be retired shortly.

Power Apps component libraries become a solution to this scenario. They act as an independent library that can hold all the components you want to share across your applications. The best part is that if you update one from the library, the rest of the applications connected to this library will get an update notification, so your apps can have well-informed component life cycle management. It's the preferred way to distribute components on the platform.

Getting ready

As a requirement for this recipe, we will use the component we created in our first recipe, *Setting up your first canvas app component*.

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com>, and create a solution.
2. Inside the solution, create an app by clicking **New | More | Component Library**.
3. Power Apps Studio will open just like when creating regular apps, but this time, it will open directly in the **Components** section of the **Tree view**. From there, click on the ellipsis to import a component, as you can see in the following screenshot:

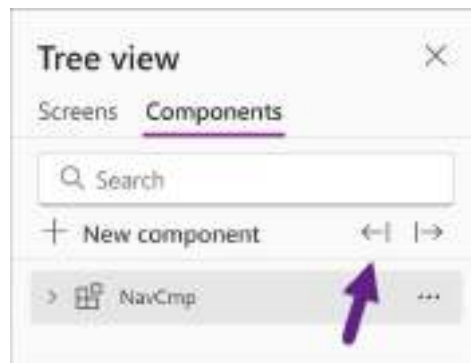


Figure 6.18: Component import feature

4. The import action will open a right pane with a list of applications containing components. If you already have the **BaseComponents** app in your environment, select it from the list and click **Import**.

5. The **TitleCmp** component should appear in the list for you to select. You can now delete the default component called **Component1** after importing **TitleCmp**. Keep in mind that at least one should exist in the library.
6. From the toolbar, click on **Save** to update your library. Once this process is complete, click **Back** to close Power Apps Studio.
7. Back in your solution, create a new app with the **Phone** layout. After Power Apps Studio is fully loaded, click on the **Insert** panel on the left and select the **Get more components** icon, as seen in the following screenshot:

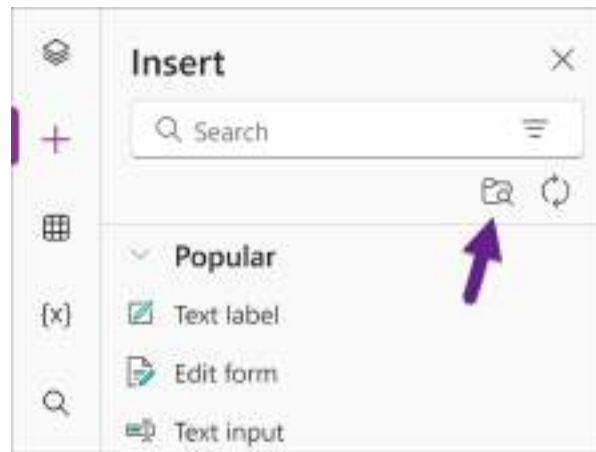


Figure 6.19: Component library access

8. This action will open the right pane with the libraries available in your environment. Select **TitleCmp** from the list and then click on **Import**. This action will add a new section to the list of elements called **Library components**. In this section, you will find the components imported from libraries.
9. Select the component to add it to the canvas, and set some custom properties for the title and icon. Click **Save**, name your app, and then close your app.
10. From the Power Apps maker portal, go to the list of component libraries and select the previously created library. Click on **Edit** from the toolbar.
11. Once Power Apps Studio opens, set the height of the **TitleCmp** component to 110 and its **Fill** color to `Color.Transparent`
12. Change the **Height** of `TitleLbl` to 85 and the **Width** to `Parent.Width - 85`. Set the **Fill** color to `TitleCmp.BackgroundColor`. Use the same for the **Fill** property of `TitleIcon`
13. For `TitleIcon`, set the **Width** and **Height** to 85

14. From the **Insert** panel, expand **Display** and select **HTML text**. Set its **X** position to 0 and its **Y** position to 85. Change the **Width** to `Parent.Width` and the **Height** to 25. Set all padding properties to 0 and, from the formula bar, set its **HtmlText** property to the following:

```
"<div style='margin-top: -15px; height: 15px; -webkit-box-shadow:
0 8px 16px 0 rgba(0, 0, 0, 0.16); -moz-box-shadow: 0 8px 16px
0 rgba(0, 0, 0, 0.16); box-shadow: 0 8px 16px 0 rgba(0, 0, 0,
0.16);'></div>"
```

15. Name the HTML text control `TitleShadow` and change its order to place it at the end. The component now has a shadow, as seen in the following screenshot:

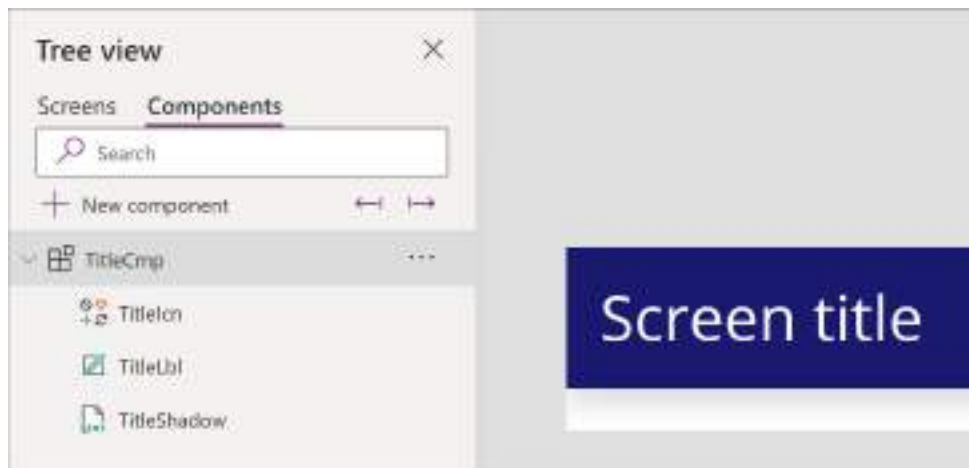


Figure 6.20: Title component with shadow

16. From the main toolbar, click the down arrow next to the **Save** icon and select **Save with version notes**. Get used to including version notes so you and other developers can take notice of the included changes. A good example would be `Added Title component shadow`. Click **Save** and then **Publish this version**. Go **Back** to close the library afterward. Always publish the changes to make them available to all apps.

After making this change, let's see how the library plays in our favor.

How it works...

Go back to the list of apps in the Power Apps portal and select the application you just created. Once it opens, you will see a banner letting you know that a component inside your application has an update, like the one shown in the following screenshot:



Figure 6.21: Component library update

Clicking the **Review** button will display a dialog listing all the libraries with updates. From there, you can check which one you want to get updates from, and clicking **Update** will apply them. Your header component now displays a shadow underneath.

This Power Apps feature offers great collaboration for app makers, allowing them to build components in just one place and share them across all solutions.

There's more...

When creating components for your organization, consider using the Power Platform Catalog. This tool offers your components excellent distribution, discovery, and usage analytics capabilities.

Some requirements exist for using the Catalog, such as having them inside Managed Environments. Please check the documentation to get more info:

<https://learn.microsoft.com/en-us/power-apps/maker/data-platform/catalog-overview>

Expanding communications with custom connectors

At the time of writing, more than 1,200 connectors are available for Power Platform. This number means that many organizations, besides Microsoft, support using this connectivity solution for Power Apps, Power Automate, and Logic Apps.

You can find connectors from Adobe, Amazon, Jira, and Google, to name a few. But what happens if you need to connect to an **Application Programming Interface (API)** created inside or outside your organization, or perhaps if you need to go beyond what the Jira connector offers? For these scenarios, there are **custom connectors**.

They allow the creation of a set of connection methods for a specific endpoint with the defined properties. These can then be shared across your organization and even certified by Microsoft to be available for all Power Apps, Power Automate, and Logic Apps users.

For more information regarding this, please visit the following links:

- Custom connectors: <https://learn.microsoft.com/en-us/connectors/custom-connectors/>
- API reference: <https://learn.microsoft.com/en-us/graph/use-the-api>

Getting ready

Using custom connectors will require that the user has a Power Apps or a Power Automate license. Please refer to the *Preface* section of this book for more details regarding licensing.

The service we are going to use in this recipe is AccuWeather. It's an online service that offers location and weather data. They offer an API for developers when you register at <https://developer.accuweather.com/>.

Upon completing the registration and login, go to **MY APPS** to create an app granting you access to their system through a key. Using the trial offer, you will have a limit of 50 calls per day and just one key per developer for free.

We will take advantage of the **Postman** application to build the connector. This tool is well-known among developers because it allows sending **Hypertext Transfer Protocol (HTTP)** requests to API endpoints. It means that developers can use it to send requests to online services using different methods and then verify the responses.

How to do it...

1. Download and install Postman from <https://www.postman.com/downloads/>. Upon running it, create an account or log in if you already have one. We need an account because we will use Postman's collection capabilities.
2. Make sure **Collections** is selected from the left pane, click the **plus** sign next to it, and choose **Blank collection**. Postman will create a collection named **New Collection**. Right-click the created collection and click on **Rename**. Name it AccuWeather:

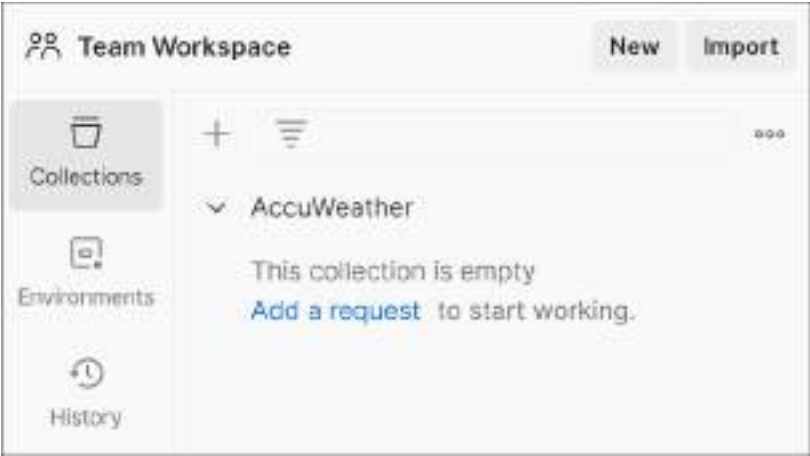


Figure 6.22: Postman collection creation

- 3. Click on **Add request**. On the tab that opens, set the name to City Search
- 4. Enter the following address in the **Enter URL or paste text** field: `http://dataservice.accuweather.com/locations/v1/cities/search`
- 5. This request needs parameters to work, so on the **Params** tab, enter the following:

Key	Value
apikey	<YOUR ACCUWEATHER API KEY>
q	Santa Cruz de Tenerife

Table 6.7: Request params

This request will search for a city named Santa Cruz de Tenerife.

6. Click **Send** to test the request. This action will execute it, and the online service will then return the data from the requested city in the **Body** section at the bottom of the screen, as seen in the following screenshot:



Figure 6.23: Postman request and response

7. With a successful request, click on **Save** near the **Send** button. This action is required to include the response in the request.
8. Now that the request configuration is complete, we can export the collection to build our custom connector. But first, we need to remove sensitive information, such as the API key, by cleaning this value from the **Params** tab.
9. Click on **Save** again, and from the **AccuWeather** collection, click on the ellipsis to expand the menu. Select **Export** and, on the open dialog, pick **Collection v2.1** and click **Export**. Save the file to a location on your computer:

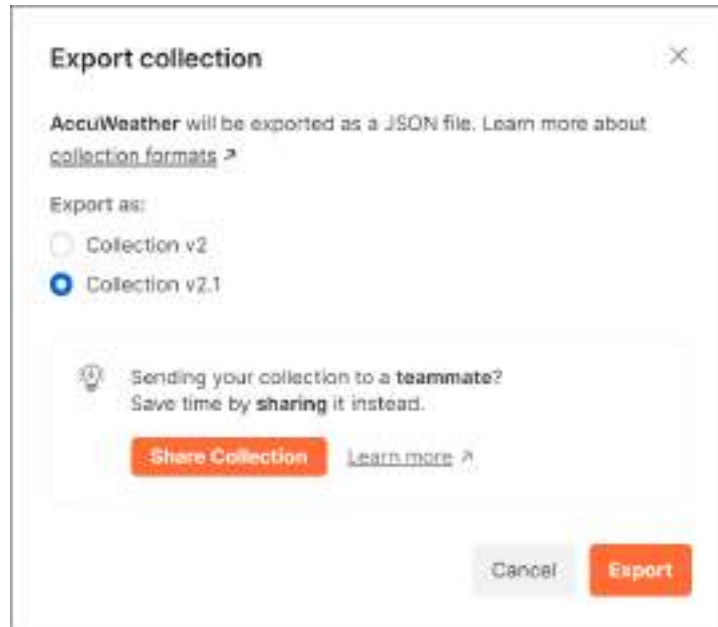


Figure 6.24: Collection export process

10. Go to the Power Apps maker portal, <https://make.powerapps.com>, and from the left pane, click on **More | Discover all** and then **Custom connectors** inside the **Data** section.
11. From the toolbar, click on the **+ New custom connector** dropdown and select **Import a Postman collection**.
12. Name your connector **AccuWeather Location** and click on **Import** to upload the collection file we exported from Postman. Click on **Continue** to start building the connector.
13. The build wizard opens to the **General** tab, where you can add more properties, such as the description or the connector icon. Set the **Description** to **Location service** provided from the **AccuWeather** service, the **Scheme** to **HTTPS**, and click on the **3. Definition** tab. We will skip the **Security** tab because our service relies on the API key configured in the **Definition** tab.
14. Review the information listed:
 - In the **General** section, you can describe your service. For example, **Operation ID** is the method that will appear in your apps.

- The **Request** section displays all the configurations gathered from the Postman file. Verify that the **URL** and the **Query** values are present, as seen in the following screenshot:

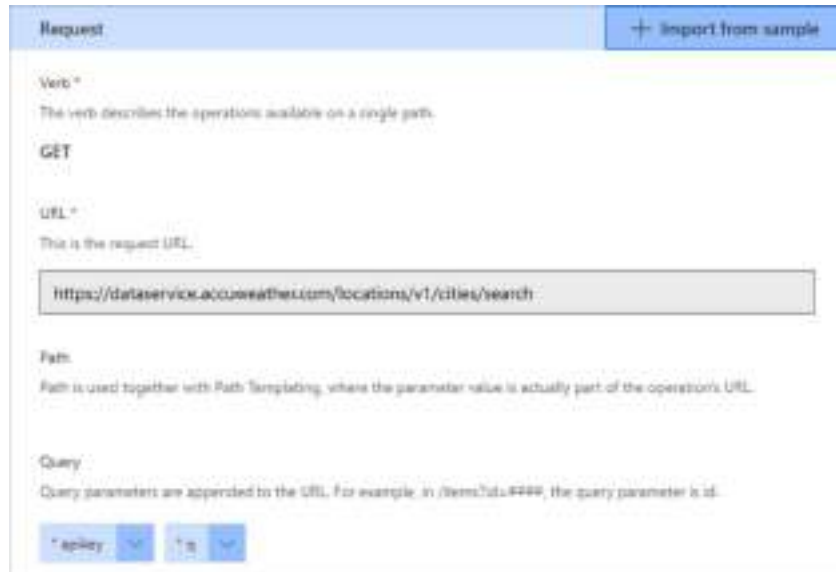


Figure 6.25: Request section of the custom connector

- At the end of this tab, there is a **Validation** section that will let you know if there are any errors in your connector. If everything checks out, click on **Create connector** at the top to finish the connector's creation. When the process is complete, click **Close**, as we will test it directly on an app.
15. Back in the Power Apps maker portal, create a solution for this recipe.
 16. From the toolbar, click on **Add existing | Automation | Custom connector**. On the pane that opens, select the **Outside Dataverse** tab and select the **AccuWeather Location** custom connector.
 17. Back in the solution, create a new canvas app by clicking **+ New | App | Canvas app**. Name it as you wish and use either the **Tablet** or the **Phone** layout.
 18. From the left pane, click on the **Data** panel and then **Add data** to search for the AccuWeather Location connector. Select it and click on **Connect**. A dialog might remind you of the license requirements when using custom connectors. Click **Got it** to dismiss it.
 19. From the **Insert** panel, add a **Text label**, a **Text input**, a **Button**, and a **Vertical gallery**. Set the following configuration:
 - Set the **Text** property of the label to **Location**. Name it **LocationLbl**

- Set the name of the text input to `SearchTxt`
- For the button, set the **Text** property to `Search` and, by using the formula bar, set the **OnSelect** property to the following:

```
UpdateContext({Locations: AccuWeather.CitySearch("<YOUR  
ACCUWEATHER API KEY>", SearchTxt.Text)})
```

20. Set the **Items** property of the gallery to `Locations`. Set its **Layout** to `Title` and `subtitle`. Edit the gallery by clicking on its pencil icon. Select the first text label which will be called **Title** followed by a number. Rename it to `CityNameLbl1` and set the **Text** property to `ThisItem.EnglishName`. For the second label, set the **Text** property to `ThisItem.Key` and name it `KeyLbl1`.

Adapt the design to your liking, and now let's test the connector.

How it works...

Building the connector from Postman allows validation that the required information gets requested correctly. It speeds up the custom connector creation by packaging the requests in a collection, making the process straightforward.

After setting up the app, you can test it by entering any city in the world in the **SearchTxt** text input and then clicking the **SearchBtn** button to load the results into **Locations**. This variable gets a table representation of the response from AccuWeather, which is then displayed in the gallery, as shown in the following screenshot:

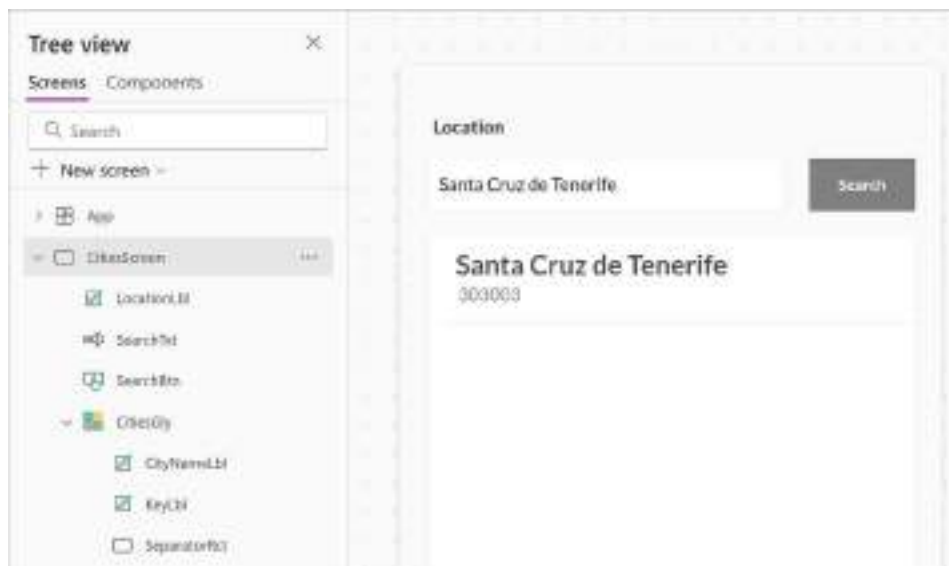


Figure 6.26: AccuWeather custom connector in a canvas app

We can extend this connector by adding more requests, such as weather data using the location key gathered using the `City` search method.

This sample application and the Postman collection file are available in this chapter's GitHub repository. Keep in mind that you need to import the custom connector solution first in order to import the sample application.

There's more...

The custom connector can also query location information in Power Automate. Add the actions of this connector from the **Custom** tab, as seen in the following screenshot:

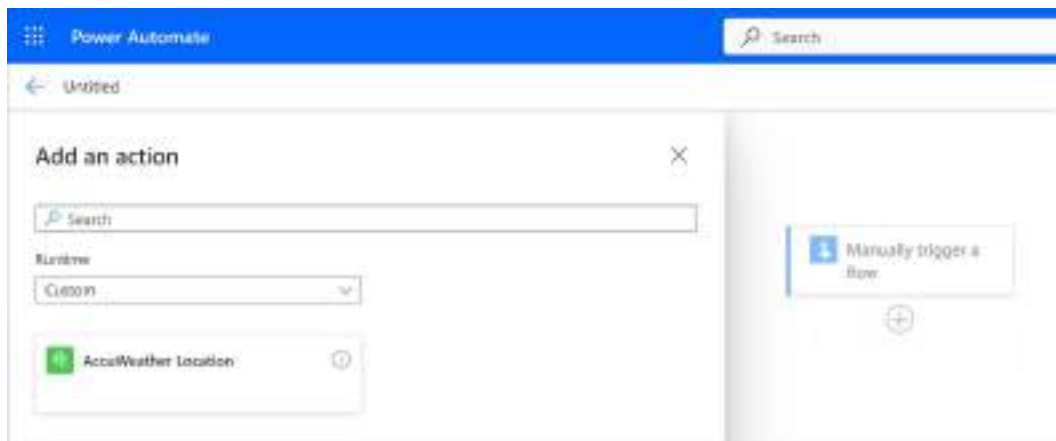


Figure 6.27: Custom connector in Power Automate

The methods configured in the connector will be exposed once the connector gets selected in the flow, and then you will only need to fill in the required query parameters: `apikey` and `q`.

Making a custom connector that works on both platforms is a great feature that, once again, improves the application-building process for app makers.

Integrating map capabilities into canvas apps

Extending device capabilities is one of the main aspects that differentiate canvas apps from other types of apps. Using features such as the camera and the **Global Positioning System (GPS)** allows the app maker to extend solutions beyond desktop devices.

The Power Platform provides two geospatial features that, together with the device's capabilities, can complement a powerful solution.

In this recipe, we will create an application to help home sellers of a real estate company pick up new opportunities when working outside the office.

Getting ready

When building apps, geospatial features are tagged as premium components. You can quickly distinguish which ones are premium by looking at the diamond icon next to them, as shown in the following screenshot:



Figure 6.28: Premium component in Power Apps Studio

This indicator means that the end users will require a Power Apps license. Please refer to this book’s *Preface* section for more details.

To use these features in Power Apps, you must enable them at the environment level using the Power Apps admin center. Please follow the instructions at this URL to activate them: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/geospatial-overview#enable-the-geospatial-features-for-the-environment>.

The following requirement is to create a Dataverse solution, which includes a table called Seller Opportunity to hold a list of locations. Refer to *Chapter 1, App Maker Basics*, to learn more about data table handling. The table needs to have the following structure:

Column name	Column data type	Required
Location name	Primary Name Column / Text	Required
Longitude	Decimal Number (7 decimal places)	Required
Latitude	Decimal Number (7 decimal places)	Required

Table 6.8: Seller Opportunity table structure

Since we are using geospatial features, you will be prompted by your browser to allow access to your location. Accept the request to test this recipe thoroughly.

How to do it...

1. First, download the `mock_data.csv` file from the GitHub repository.
2. Go to the Power Apps maker portal, <https://make.powerapps.com>, and open the solution holding the **Seller Opportunity** table mentioned in the *Getting ready* section. Select the table to open its properties.
3. From the toolbar, click on **Import | Import data from Excel**. Click on **Upload** and select the file downloaded in *step 1*. When the upload finishes, click on **Map columns** to map each column from the file to the table's columns, then click **Save changes**. Click **Import** from the toolbar to start the data loading. When the import completes, close the dialog box.
4. From the solution, click on **Apps** on the left pane, and from the toolbar, click **+ New | App | Canvas app**. Name it `Opportunity handler`, select the **Phone** format, and click **Create**.
5. Once Power Apps Studio opens, rename **Screen1** to `MainScreen`. Change the **Fill** property of the screen using the formula bar to `RGBA(242, 242, 242, 0.5)`
6. From the **Insert** panel on the left (denoted with a plus sign), click on **Rectangle** to add one. Set the following properties using the formula bar: `0` for **X** and **Y**, `Parent.Width` for the **Width**, and `160` for the **Height**. Change the **Fill** color to `RGBA(238, 238, 238, 1)`. Name this control `HeaderRct`.
7. Add a **Text label** from the **Insert** panel. Set these properties using the **Properties** panel on the right: `30` for **X** and `20` for **Y**. For the **Font**, choose **Segoe UI**, a size of `14`, and **Semibold** as the **Font weight**. Set the **Height** to `50` and change the **Text** to `Enter an address`. Name this control `HeaderLbl`
8. Go to the **Insert** panel, expand **Input** to click on **Address input**, and set the following properties: `30` for **X**, `76` for **Y**, `480` for **Width**, and `55` for **Height**. For the **Font**, choose **Segoe UI** and a size of `14`. Set the **Border thickness** to `1`, the **Border color** to a dark gray of your choice, and the **Border radius** to `0`. Change the **HoverFillColor** to white. Name this control `AddressTxt`
9. From the **Insert** panel, click on **Button** to add one. Set the following properties: `510` for **X**, `76` for **Y**, `100` for **Width**, and `54` for **Height**. Set the **Text** to `Add`, the **Font** to **Segoe UI**, the size to `16`, and the color to `Black`. Set the **Border thickness** to `1`, the **Border radius** to `0`, and the **Border color** to the same dark gray as before. Using the formula bar, change the **Fill** color to `RGBA(238, 238, 238, 1)`. Name this control `AddBtn`

The application should look like the following screenshot:

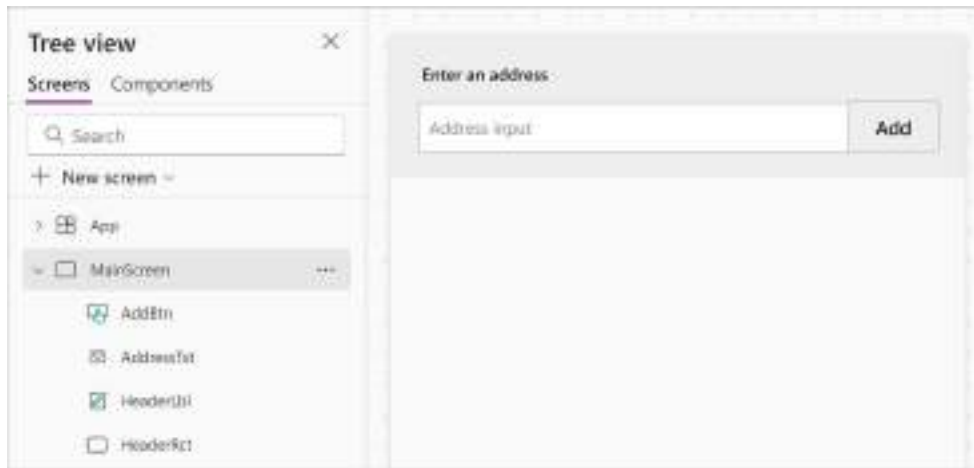


Figure 6.29: Address input on a canvas app

10. Go to the **Insert** panel and expand **Media** to select **Map**. From the **Select a data source** panel, search for **Seller Opportunities** and click on it. Set the following properties: 30 for **X**, 190 for **Y**, 580 for **Width**, and 900 for **Height**. Enable **Show current location**.
 - Using the formula bar, set **CurrentLocationLatitude** to `Location.Latitude` and **CurrentLocationLongitude** to `Location.Longitude`.
 - Using your internal Dataverse name columns, set **ItemsLabels** to `pn_locationname`, **ItemsLatitudes** to `pn_latitude`, and **ItemsLongitudes** to `pn_longitude`. Remember that `pn` is my publisher's prefix; change it to yours accordingly. Name this control `PanelMap`
11. Select the **AddBtn** control and, using the formula bar, set the following code to the **On-Select** method:

```
Patch(
    'Seller Opportunities',
    Defaults('Seller Opportunities'),
    {
        'Location name': AddressTxt.StreetName,
        Latitude: AddressTxt.SelectedLatitude,
        Longitude: AddressTxt.SelectedLongitude
    }
)
```

12. From the main toolbar, click on **Save** to complete the application.

How it works...

This application leverages both **Map** and **Address** inputs for its geospatial capabilities. It uses the **Address** input control to allow sellers to search and save addresses to the Dataverse table while updating the **Map** control to display every GPS point for visual access to this information.

The completed application should look like the following:

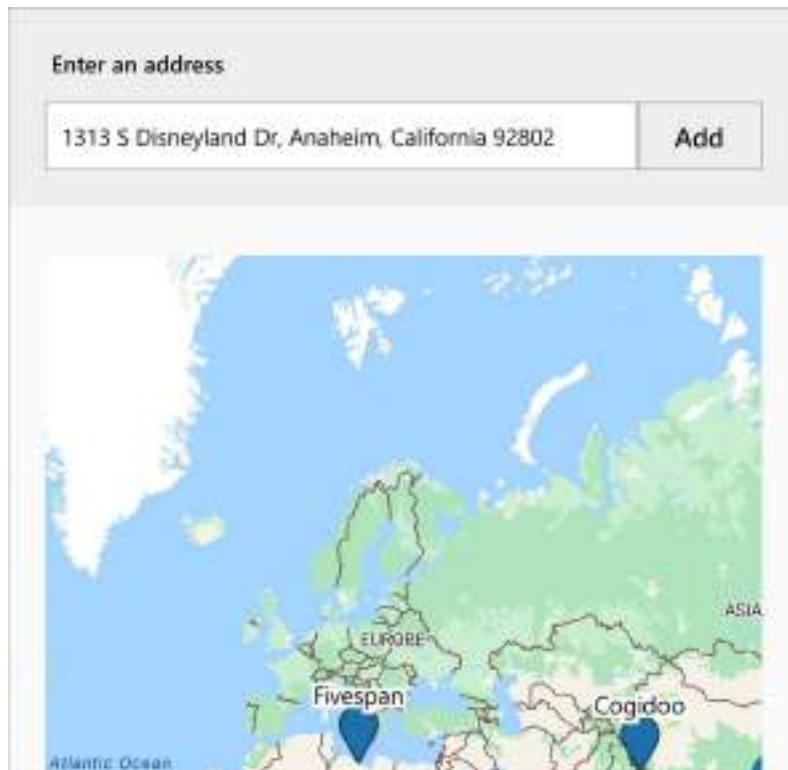


Figure 6.30: Complete seller opportunities application

The complete solution and the mock data file are available from the chapter's GitHub repository.

There's more...

The **Map** control has many properties that allow it to adapt and configure its visualization and features. Make sure to explore all the possibilities this control has to offer. For more information in this regard, please visit <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/geospatial-component-map#properties>.

Extending canvas solutions using mixed reality

Mixed reality is a technology that allows us to extend beyond device screens and into our living spaces.

This skill can bring solutions to different scenarios, such as the following:

- Measuring hard-to-reach areas or distances while adding this information to a data source
- As a medical equipment representative, being able to show the equipment to the hospital in mixed reality to check for fitting measurements
- Representing furniture in your living room to see if it will fit your style

Canvas apps allow the building of applications for these scenarios using three mixed reality controls: viewing 3D content, seeing shapes with pre-established dimensions, and performing measurements using mixed reality.

This recipe will take a more personal approach. We recently discovered that our water heater was broken. Its location was hard to reach because it was behind the washing machine and the dryer. It was then that I turned to **augmented reality (AR)** technology to measure the size of the water heater from afar so that I could replace it with a new one that would fit in the same place as the old one.

Getting ready

Using mixed reality technologies requires devices that support AR. Refer to the following links to verify if your device meets the requirements:

- Apple: <https://www.apple.com/augmented-reality>
- Google: <https://developers.google.com/ar/devices>

You will need to create three Dataverse tables to record the measurements made within the app. Refer to *Chapter 1, App Maker Basics*, to learn about data table handling inside a solution. The tables need to have the following structure:

- **Claims**

Column name	Column data type	Required
Number	Autonumber	Required
Description	Multiline Text	Optional

Table 6.9: Dataverse table structure for claims

- **Measurements**

Column name	Column data type	Required
Measurement	Autonumber	Required
Unit	Text	Optional
Height	Decimal Number (2 decimal places)	Optional
Length	Decimal Number (2 decimal places)	Optional

Table 6.10: Dataverse table structure for measurements

- **Images**

Column name	Column data type	Required
Name	Text	Required
Picture	Image	Optional

Table 6.11: Dataverse table structure for images

Before you begin, download the `Save.txt` file from the GitHub repository.

How to do it...

1. Create a solution for this recipe from the Power Apps maker portal and create the required tables.
2. From the **Schema** section of the **Claims** table, click on **Columns**. Select the **Number** column (tagged as the **Primary Name Column**) and, from the **Properties** pane on the right, change the **Data type** to **Autonumber** and click **Save**.
3. Go to the **Forms** section of the **Measurements** table and click to open the **Information** form with a **Form type** of **Main**. Design it as you see fit to include the previously mentioned table columns. Do the same for the **Images** table.
4. Open the **Claim** table and go to the **Relationships** section. Click on **+ New relationship** | **One-to-many**, choose **Measurement** from the **Related (Many)** dropdown, and click **Done**. Repeat this step for the **Images** table.
5. Go to the **Forms** section and click to open the **Information** with a **Form type** of **Main**. Design it as you see fit, and to display related data from the **Measurements** and the **Images** tables, add two **Subgrids**, as shown in the following screenshot:

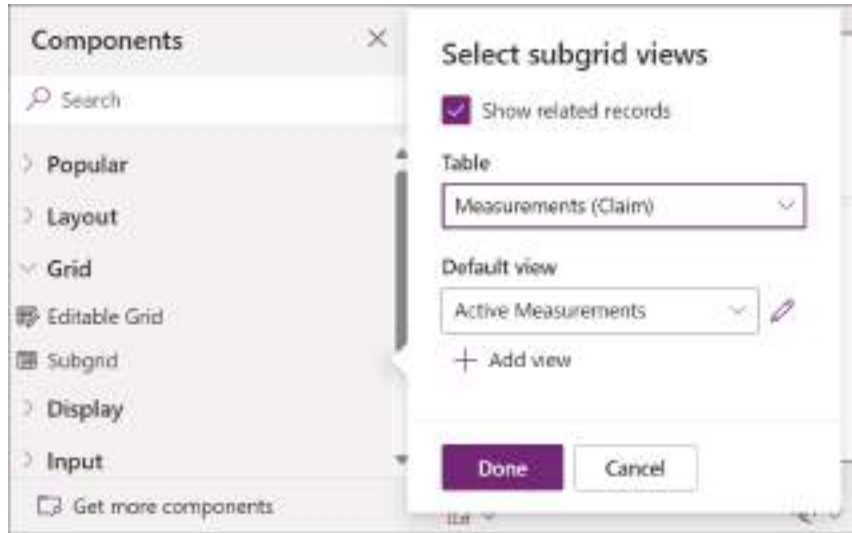


Figure 6.31: Subgrids for related data

6. Click **Save and publish** on the form and click **Back** to return to the solution designer.
7. Click **All** on the left pane and click **+ New | App | Canvas app**. Name it **Claims handler**, choose the **Phone** format, and click **Create**.
8. Once Power Apps Studio opens, rename **Screen1** to **Main Screen**. Change the **Fill** property of the screen to **RGBA(242, 242, 242, 0.5)** using the formula bar.
9. From the **Insert** panel on the left (denoted with a plus sign), click on **Rectangle**. Set the following properties using the formula bar: **0** for **X** and **Y**, **Parent.Width** for **Width**, and **100** for **Height**. Change the **Fill** color to **RGBA(50, 86, 160, 1)**. Name this control **HeaderRct**.
10. Add a **Text label** from the **Insert** panel. Set these properties using the **Properties** panel on the right: **30** for **X** and **15** for **Y**. For the **Font**, choose **Lato Black**, a size of **24**, and the **Color** to **White**. Set the **Height** to **70**. Change the **Text** to **Claim handler**. Name this control **HeaderLbl**.
11. Go to the **Insert** panel and click on **Vertical gallery**. Using the **Select a data source** panel, search for and select the **Claims** table. Set the following properties using the formula bar: **0** for **X**, **100** for **Y**, and **Parent.Height - Self.Y** for **Height**. Change the **Layout** to **Title and subtitle**. On the **Properties** pane, click on **Edit** in the **Fields** section. For the **Title**, select **Number**; for the **Subtitle**, choose **Description** and close the pane. Name this control **ClaimsGly**.
12. On the **Tree View**, expand the **ClaimsGly** gallery and delete the **NextArrow** icon.

13. From the **Insert** panel, expand **Icons** and select the **Add** icon. Set these properties using the **Properties** panel: 560 for **X**, 18 for **Y**, and **White** for the **Color**. Set all padding properties to 10. Using the formula bar, set the **OnSelect** property to `Navigate('ClaimScreen', Fade)`. Name this control **AddIcon**
14. Click on **+ New screen | Blank** from the Tree view. Name this new screen **Claim Screen**.
15. From the **Tree view**, select **HeaderLbl**, **HeaderRct**, and **AddIcon** and copy them from the **MainScreen** to the **ClaimScreen**. Rename them to **ClaimHeaderLbl**, **ClaimHeaderRct**, and **ClaimSaveIcon** accordingly.
16. Add a **Text** label from the **Insert** panel. Set these properties using the **Properties** panel on the right: 30 for **X**, 125 for **Y**, and 30 for **Height**. For the **Font**, choose **Lato** and a size of 16. Change the **Text** to **Description**. Name this control **ClaimDescriptionLbl**
17. Go to the **Insert** panel and click on **Text input**. Set these properties using the **Properties** panel on the right: 30 for **X**, 160 for **Y**, 580 for **Width**, and 830 for **Height**. Clear the **Default** text. For the **Font**, set it to **Lato** and a size of 18. Set the **Mode** to **Multiline**. Set all padding properties to 12. Change the border width to 1, use any dark gray as the border color, and change the **Border radius** to 0. Name this control **ClaimDescriptionTxt**
18. From the **Insert** panel, expand **Mixed Reality** and click on **Measure in MR**. Set these properties using the **Properties** panel: 30 for **X**, 1030 for **Y**, and 580 for **Width**. Change the **Font** to **Lato** and a size of 22. Set the **Border radius** to 0. Name this control **MeasureMR**
19. Using the formula bar, set the **Text** property for **MeasureMR** to the following code:

```
With(
    {
        measurements: Coalesce(
            CountRows(Self.Measurements),
            0
        )
    },
    If(
        measurements = 0,
        "Measure in MR",
        "Measurements: " & measurements
    )
)
```

The application should look like the following screenshot:

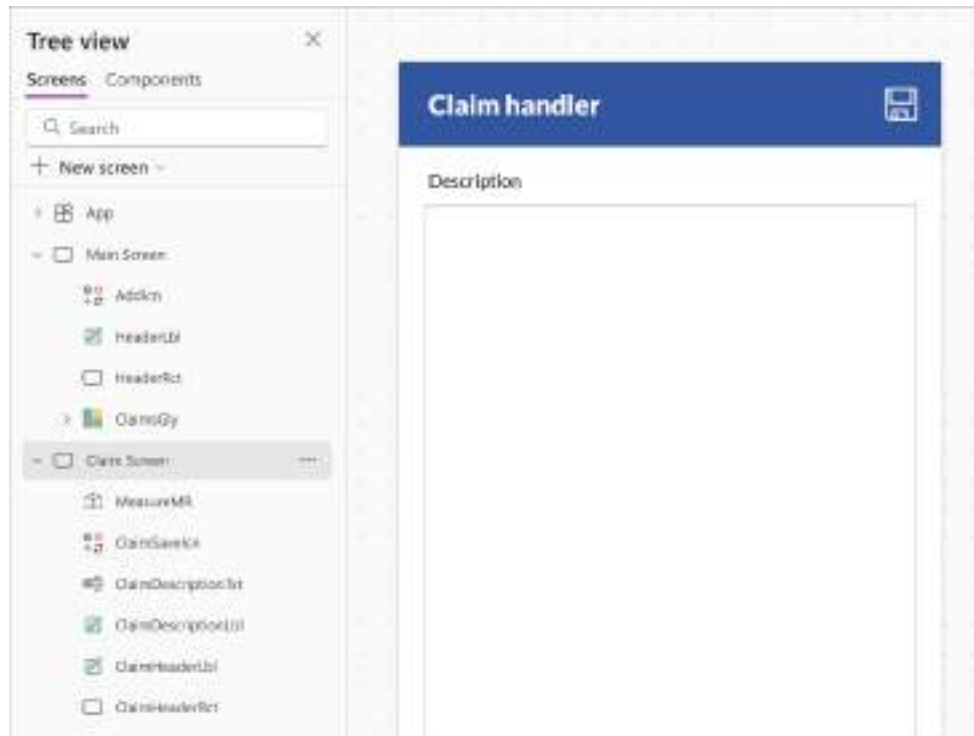


Figure 6.32: Claim handler app

20. Go to the **Data** panel on the left, click **+ Add data** to search, and select the **Measurements** table. Repeat the same for the **Images** table.
21. Select **ClaimSaveIcon** and change its **Icon** to the **Save** icon. Using the formula bar, change the **OnSelect** property to the contents of the **Save.txt** file available from the GitHub repository.
22. Click on **Save** to complete the application.

How it works...

After publishing the application, test it using a certified mobile device. Thanks to the mixed reality controls available for canvas apps, you will be able to add new claims, enter some descriptions for them, and include some measurements and pictures.

This solution can make life easier for insurance technicians visiting customers related to a claim. They can record the information directly on a claims system powered by Dataverse with extra data, including the measurements and images from the incident.

For my personal case, using the **Measure** control allowed me to get the size of my water heater to call my insurance company for a replacement, as you can see in the following screenshot:



Figure 6.33: Measure in Mixed Reality in action

The complete solution and the code file are available from the GitHub repository of this chapter.

There's more...

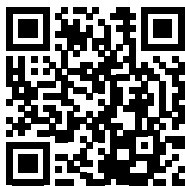
As mentioned at the beginning of this recipe, besides the **Measure** control in mixed reality, other controls allow an immersed and rich experience for your apps.

Each control aims to solve a particular need, so be sure to test them and see if they can improve the applications you created. There's more information regarding this at <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/mixed-reality-overview>.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



7

Improving User Experience

The process of creating applications is a complex workflow. You need to analyze requirements, design data sources, choose tools and platforms, and build the **User Interface (UI)**, among others.

Power Apps, a platform to build applications, is constantly evolving to help app makers create functional solutions quickly. However, this sometimes means sacrificing an appealing UI. As we will see in this chapter, improving the user experience of an application involves many aspects, such as performance, rich features, and UI improvement.

This chapter will show you how to enhance the UI of full-featured applications created from data, thus promoting their adoption. We will also discover how to improve our application-building workflow by creating one responsive application that works on mobile and desktop.

As for data, we will leverage the Microsoft Azure Key Vault service to build our application. Azure is a platform aiming to help organizations meet their business needs with a wide range of cloud services. We can use Azure to improve user experience by adding extra functionality for the user, such as by securely saving passwords.

The recipes in this chapter will help us go deeper into these concepts. Note that the second and third form a *two-part recipe* that provides a helpful service for users.

It also serves to demonstrate how we can make our app adaptive for desktop and mobile:

- Enhancing the design of our application interfaces
- Building a responsive password manager in Power Apps – setting up the Azure Key Vault service
- Building a responsive password manager in Power Apps – designing an adaptive application

Technical requirements

Depending on the type of connector needed for your data source, you might need a license for your applications. Standard connectors, such as the one used for SharePoint, don't require licensing apart from the existing Microsoft 365 license. However, premium connectors, such as the one needed for Azure Key Vault, do. Please refer to the book's *Preface* for more information about the licensing requirements.

All the applications we will use as examples in this chapter are available in the GitHub repository: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-07>

Enhancing the design of our application interfaces

A platform that automatically builds applications is an excellent feature of Power Apps. The main logic of the application gets created from the data we need to handle. Screens for activities such as listing records or manipulating data are already defined. This feature also acts as a great learning resource for app makers new to this platform on how to get things done for these tasks.

However, this doesn't mean that these ready-made applications achieve high scores in the design department. Remember that end-users expect appealing UIs for the apps they use.

With the help of this recipe, you will learn how to improve application screens by applying some design ideas to make them look better. These ideas will help you enhance your applications, like the one seen in the following figure:

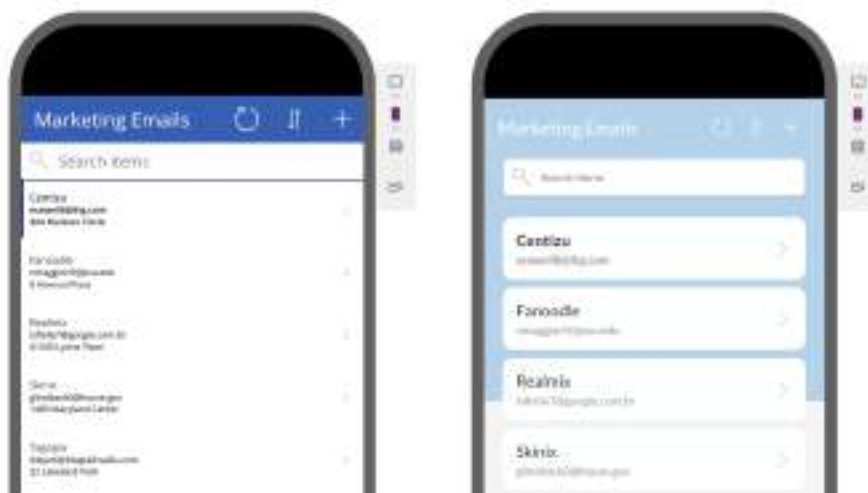


Figure 7.1: Application makeover

Let's discover how to adapt the ready-made application to be more user-friendly.

Getting ready

To apply the UI changes, we will need an application created from data, like the ones we learned about in *Chapter 2, Building Pixel-Perfect Solutions with Canvas Apps*. You can build your app using any data source of your choice.

For this particular example, I've used a simple SharePoint list with the following columns:

Column name	Column data type
Customer	A single line of text
Email	A single line of text
Address	A single line of text

Table 7.1: Customer stories list columns

As for the data, I've used a data generator service called **Mockaroo**, which helped me to create dummy data for the list. Check out their website for more info: <https://mockaroo.com>

How to do it...

When creating a Power App directly from a SharePoint list, the platform builds three screens to handle your data: **BrowseScreen1**, **DetailScreen1**, and **EditScreen1**. Let's follow these steps to improve each one's design. Keep in mind that the numbering of some controls might change.

Improving the Browse screen

1. Let's start by modifying the action bar called **RectQuickActionBar1**. Using the formula bar, change its height to `App.Height / 2` so that it takes up half the screen's size. Then, set its **Fill** color to `ColorValue("#B4D2E9")`
2. Select the **TextSearchBox1** control, and set its **X** location to 37 and **Y** to 114. Set the **Width** value `Parent.Width - 75`. Set the **Font** to **Lato** and its size to 16. Change the **Border radius** to 10
3. Delete **Rectangle11**, since we are not going to use this separator.
4. Change the **Y** property of **LblAppName1** to 11, the **Font** to **Lato**, and its size to 26
5. Using the *Shift* key, select the icons: **IconNewItem1**, **IconSortUpDown1**, and **IconRefresh1**. Change all these properties simultaneously: the **Y** to 23, the **Width** and **Height** to 64, and all padding properties to 16

6. Change the **X** property of each icon like so:
 - **IconNewItem1** to `Parent.Width - Self.Width - 33`
 - **IconSortUpDown1** to `IconNewItem1.X - Self.Width`
 - **IconRefresh1** to `IconSortUpDown1.X - Self.Width`

The header should look like this:



Figure 7.2: Header makeover

7. Select **BrowseGallery1** and set these properties: the **X** to 20, the **Y** to 208, the **Width** to `Parent.Width - 40`, the **Template size** to 117, and the **Template padding** to 15
8. Expand the gallery and delete **Rectangle1**, **Separator1**, and the **Body1** control.
9. Click on the pencil of the gallery to edit the contents, and from the insert toolbar, click on **Media | Image**. From the **Properties** pane, set **Image** to None and change its **Fill** color to `ColorValue("#FFFFFF")`. Set the **X** and **Y** to 0, and change the **Width** to `Parent.TemplateWidth` and the **Height** to `Parent.TemplateHeight`. Set the **Border radius** to 20, and then send the image to the back of the gallery canvas by right-clicking on the image control and selecting **Reorder | Send to back**.
10. Change the **NextArrow1** control to have **Width** and **Height** values of 64 and set all padding properties to 16. For the **Fill** color, set a lighter one, such as `RGBA(202, 202, 202, 1)`
11. Select **Title1** and set the **X** and **Y** property to 25, the **Font** to **Lato**, and the **Font size** to 22
12. As for the **Subtitle1** control, change its **X** property to 25, the **Y** property to 68, the **Font** to **Lato**, and the **Font size** to 16. Lastly, change the **Color** to `RGBA(149, 149, 149, 1)`
13. Finally, to obtain a better contrast, change the **Fill** color of **BrowseScreen1** to `RGBA(237, 237, 237, 0.6)`

Refining the Detail screen

1. Click on the **RectQuickActionBar2** control and change its **Color** property to `ColorValue("#B4D2E9")`

2. Select the icons, `IconDelete1`, `IconEdit1`, and `IconBackArrow1`, using the *Shift* key. Change all these properties at once: the **Y** to 12, the **Width** and **Height** to 64, and all padding properties to 16
3. Change the **X** property of each icon like so:
 - `IconEdit1` to `Parent.Width - Self.Width - 20`
 - `IconDelete1` to `IconEdit1.X - Self.Width`
4. Select `LblAppName2` and change its **Font** property to **Lato** and the **Font size** to 24
5. Change the **Y** property of `DetailForm1` to 112
6. Select `DetailForm1`, and using the **Properties** pane, click next to the **Fields** property and ensure that only the **Title**, **Email**, and **Address** fields appear on the list. Remove any other fields if necessary.
7. Expand `DetailForm1`, and using the *Shift* key, select all the data cards and change their **Height** properties to 130 and the **Display mode** to **Edit**.
8. Select `Title_DataCard1`, and from the **Properties** pane, go to the **Advanced** tab and click **Unlock to change properties**, as seen in the following screenshot:



Figure 7.3: Unlocking form cards

Still in the **Advanced** tab, change the **DisplayName** property to "TITLE"

9. Expand `Title_DataCard1` and select `DataCardKey1`, and then change the **Font** property to **Lato**, the **Font size** to 14, and the **Color** to `RGBA(149, 149, 149, 1)`. Untick the **Auto height** property and set the **Height** to 27



If the height doesn't update, you might need to manually change the dimensions by expanding the control and setting the value on the **Properties** pane.

10. Select **DataCardValue1**, and set the **Font** property to **Lato** and the **Font size** to 20. Untick the **Auto height** property and set the **Height** to 60 and all padding properties to 12. Using the formula bar, set the **Color** to `RGBA(0, 0, 0, 0.8)`, the **BorderThickness** to 1, the **Fill** to `Color.White`, and the **BorderColor** to `RGBA(237, 237, 237, 1)`. Set the **Y** property to `DataCardKey1.Y + DataCardKey1.Height + 10`
11. Repeat *steps 8, 9, and 10* for the rest of the data card controls. Note that you need to update the **Y** property formula on *step 10* to match each control's name.
12. Select **DataCardValue3** inside **Address_DataCard1**, change the **Vertical align** to **Top**, and set its **Height** to `App.Height - DetailForm1.Y - Address_DataCard1.Y - Self.Y - 30`
13. Select **Address_DataCard1** and change the **Height** to `Parent.Height - Self.Y`
14. Select **DetailScreen1** and set its **Fill** color to `ColorValue("#FBFAFB")`

The detail screen should look like the following screenshot:

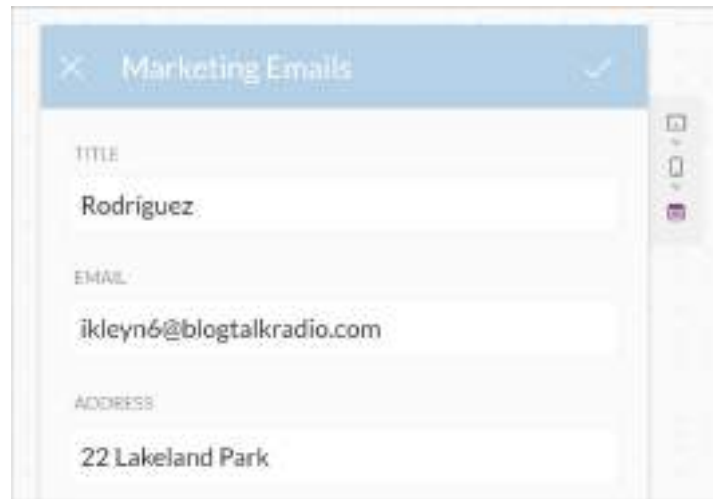


Figure 7.4: Detail screen makeover

Finishing touches for the Edit screen

1. Select **RectQuickActionBar3** and change its **Color** property to `ColorValue("#B4D2E9")`
2. Using the *Shift* key, select the **IconCancel1** and **IconAccept1** icons to change all these properties at once: the **Y** to 12, the **Width** and **Height** to 64, and all padding properties to 16. Change the **X** property of **IconAccept1** to `Parent.Width - Self.Width - 20`

3. Select **LblAppName3**, and change its **Font** property to **Lato** and the **Font size** to 24
4. Change the **Y** property of **EditForm1** to 112
5. Select **EditForm1**, and using the **Properties** pane, click on **Edit fields** and ensure that only the **Title**, **Email**, **Address**, and **Attachments** fields appear on the list. Remove any other fields if necessary.
6. Expand **EditForm1**, select all data cards except for **Attachments_DataCard1** using the *Shift* key, and change its **Height** property to 130
7. Select **Title_DataCard2**, and from the **Properties** pane, go to the **Advanced** tab and click **Unlock to change properties**, and then change the **DisplayName** property to "TITLE"
8. Expand **Title_DataCard2**, and select **DataCardKey4** and **StarVisible1** to change these properties: the **Font** to **Lato**, untick **Auto height**, and set the **Height** to 27. Change the **Color** to `RGBA(149, 149, 149, 1)`

Only for **DataCardKey**, set the **Font size** to 14

9. Select **DataCardValue4** and set these properties: the **Font** to **Lato**, the **Font size** to 20, the **Height** to 60, and all padding properties to 12. Using the formula bar, set the **BorderThickness** to 1, the **BorderColor** to `If(IsBlank(Parent.Error), RGBA(237, 237, 237, 1), Color.Red)`, the **Color** to `RGBA(0, 0, 0, 0.8)`, and the **Y** to `DataCardKey4.Y + DataCardKey4.Height + 10`
10. Repeat *steps 7, 8, and 9* for the rest of the data card controls. Note that you need to update the **Y** property formula on *step 8* to match each control name.
11. Select **DataCardValue7** inside **Attachments_DataCard1** and set its **Height** property to: `App.Height - EditForm1.Y - Attachments_DataCard1.Y - Self.Y - 30`
12. Expand **Title_DataCard2** and select the **ErrorMessage1** label. Change the **Font** property to **Lato** and the **Font size** to 14
13. Select **EditScreen1** and set its **Fill** color to `ColorValue("#FBFAFB")`

After applying the previous steps, the makeover is now complete.

How it works...

Making the changes described in the previous steps enhances a UI by creating an aesthetically pleasing and unified design across an application. Performing this task also helps adoption because users prefer to work on an interface that makes everyday tasks more appealing.

All screens follow the same pattern to create a seamless experience, as seen in the following screenshot:

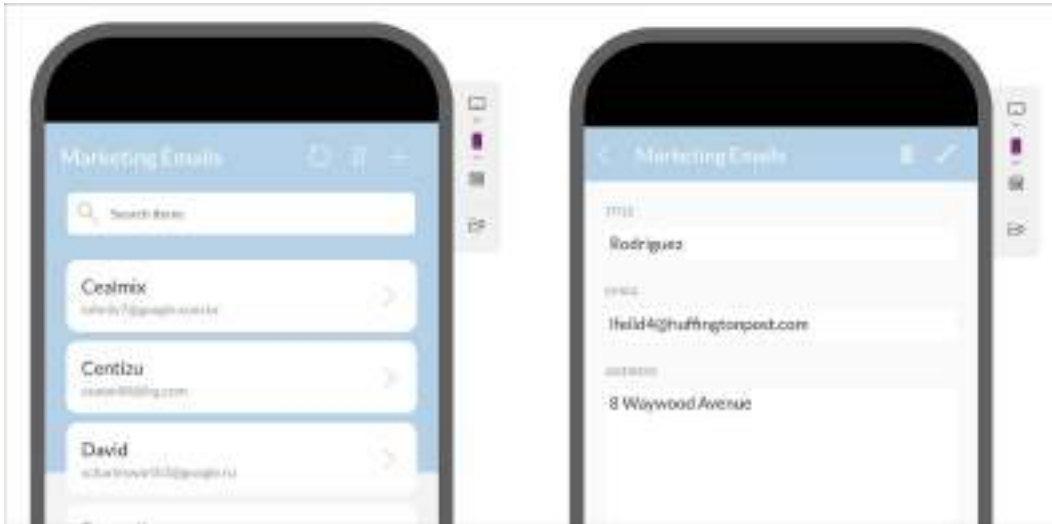


Figure 7.5: Seamless design in Power Apps

There's more...

Applications' UI designs are constantly evolving. You can reference *Chapter 6, Extending the Platform*, to review the rules when designing interfaces. For inspiration, you can also visit websites like **Dribbble** (<https://dribbble.com>), where designers showcase their website and application design work.

Designers are essential to an application development team, since they focus on building UIs, considering guidelines, UX rules, accessibility, and more. Power Apps offers a UI kit for **Figma**, one of the most popular prototyping tools among designers, to help build mock-ups of UIs that can then be transformed into Power Apps screens, waiting for some logic to be applied by makers. To get more information about this kit, visit: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/figma/design-using-kit>

Building a responsive password manager in Power Apps – setting up the Azure Key Vault service

Password managers are tools created to handle all your credentials from a central location, protecting them using a primary password and encryption. They can improve the user experience by saving personal and work-related credentials for services we use every day.

Microsoft Azure offers **Key Vault**, which provides a “safe” to securely hold credentials and certificates by encrypting this information in hardware security modules. For more information, see <https://azure.microsoft.com/en-us/products/key-vault/>

Getting ready

We will use the Azure Key Vault connector to access our data source, which is just one in the ever-growing list of connectors available for canvas apps. It will give us the ability to consume secrets stored in a vault. This recipe will teach you how to set up a key vault in your Azure subscription.

For the most up-to-date quick start on how to deploy a Key Vault, please refer to <https://learn.microsoft.com/en-us/azure/key-vault/secrets/quick-create-portal>

As well as the portal, you’ll also find many other ways to interact with Azure.

How to do it...

1. Go to the Azure portal at <https://portal.azure.com>, log in to your subscription, and click **Create a resource**. This action will open Azure Marketplace, where you can browse all available resources.
2. Search for Key Vault, and when found, click on **Create**:

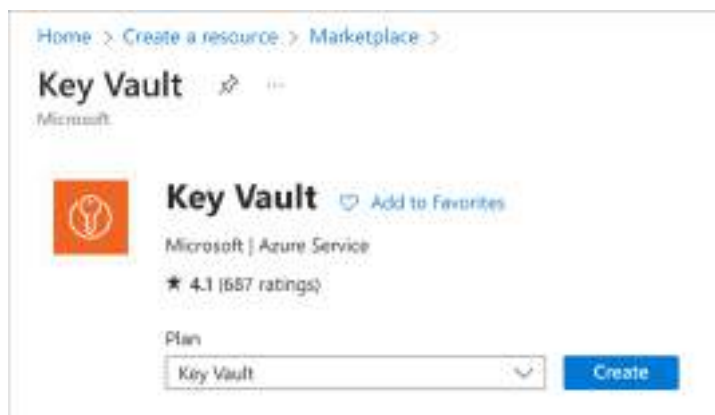


Figure 7.6: Key Vault service

The next step is to gather all the required information to create your resource.

3. Select an existing resource group or create a new one. A resource group acts as a container for your resources.
4. Select a key vault name. Remember that this name must be unique across all of Azure because this will then be available through the REST API:

```
https://<key-vault-name>.vault.azure.net/
```

As an example, I've used PowerVault

5. Select the region that is closest to you to get better network performance.
6. Select the **Standard** pricing tier.
7. Leave all other options as default, click **Review + create**, and then click on **Create** after the validation passes.
8. After a while, the deployment of your resource will be complete. Click on **Go to resource** to view your key vault.

How it works...

The current connector version only allows the consumption of secrets from a vault. We will add some credentials to our vault so that we can have them accessible from the app we are going to build, using the following recipe.

While still in the Azure portal, go to your resource, and under **Settings**, go to **Secrets**:

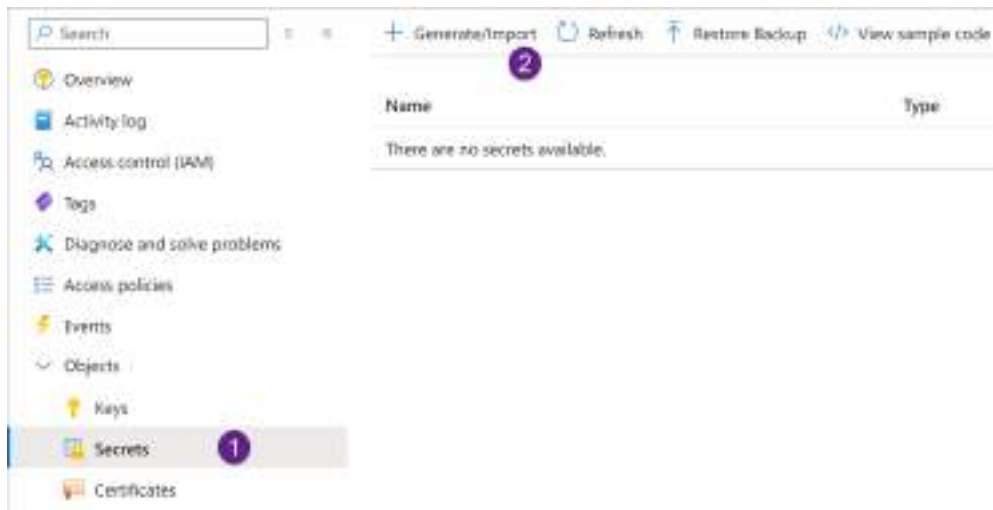


Figure 7.7: Secrets management

Click on **Generate/Import** to start adding secrets. In the new window, please input the name of the secret and its value. Create as many as possible to create a data set to shape your app.

Building a responsive password manager in Power Apps – designing an adaptive application

In the previous recipe, we completed setting up an Azure Key Vault instance to hold the secrets we will consume from our app. The application design will be responsive and adaptable when running on desktops or mobile devices.

Getting ready

Building a responsive application on any platform requires evaluating some design principles before making the app. When designing the UI, you must consider the different mediums your end-users will use to consume the app. You need to address the size and location of the controls for every device.

In Power Apps, many layout tools can help you build these types of applications. We aim to create one application that works the same on desktop and mobile devices.

You can find the complete application and the resources needed to build it in our GitHub repo: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-07>

Please have the resources handy to use them when building the application.

How to do it...

1. Go to the Power Apps maker portal at <https://make.powerapps.com> and create a solution.
2. Once the solution opens, click + **New | App | Canvas app** from the toolbar. Give the app a name and keep the **Tablet** format.
3. From the **Tree view**, choose **Split-screen** from the + **New screen** dropdown. Now, delete **Screen1** and rename **Screen2** Main Screen.
4. Once the Studio opens, from the left pane, select the **Data** pane (represented by a database icon). Click on **Add data**, and look for **Azure Key Vault** using the **Search** field.
5. Selecting this connection will open a configuration pane on the right. Leave the default **Authentication type**, enter the vault name used in the previous recipe, and click **Connect**.

6. Go back to the tree view, select the **App** element, and, using the formula bar, change the **OnStart** property to the following:

```
Set(
    CurrentTheme,
    {
        Background: ColorValue("#E9E9E9"),
        TextPrimary: ColorValue("#292A2D"),
        TextSecondary: ColorValue("#A4A4A4"),
        TextFill: ColorValue("#F3F3F3"),
        Divider: ColorValue("#E9E9E9"),
        Font: Font.'Segoe UI'
    }
)
```

Click on the ellipsis next to the **App** element and select **Run OnStart**.

7. On the left pane, select the **Media** panel (represented by a movie reel with a music note) and click + **Add media | Upload**. Select `safe.svg` from our GitHub repository.
8. Using the formula bar, change the **OnVisible** property of **Main Screen** to the following:

```
UpdateContext({selected: false});
ClearCollect(
    Secrets,
    AzureKeyVault.ListSecrets().value
)
```

9. Expand **ScreenContainer1** and select **LeftContainer1**. Configure it as follows:
 - Using the **Properties** pane on the right, select the last option in **Align (horizontal)**, called **Stretch**, set all padding to 20, and turn off **Flexible width**.
 - Using the formula bar, set the **Width** to the following:

```
If(
    MainScreen.Size = ScreenSize.Small,
    Parent.Width - Parent.LayoutGap * 2,
    280
)
```

- Using the formula bar, set the **Visible** property to the following:

```
If(  
    MainScreen.Size = ScreenSize.Small,  
    !selected,  
    true  
)
```

10. From the **Insert** panel on the left, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, change the **Text** property to Password Vault, the **Font size** to 14, and the **Font weight** to **Bold**
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary` and the **Font** to `CurrentTheme.Font`
 - Rename this text label `ApplicationNameLbl1`
11. From the **Insert** panel, select **Text input** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, clear the **Default** property and set the **Hint text** to Search, the **Font size** to 13, the **Minimum width** to 32, the **Border thickness** to 0, and the **Border radius** to 0
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`, the **Font** to `CurrentTheme.Font`, the **Fill** to `CurrentTheme.TextFill`, the **HoverFill** to `Self.Fill`, and the **FocusedBorderThickness** to 1
 - Rename this text input `SearchTxt`
12. From the **Insert** panel, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, change **Text** to Secrets, the **Font size** to 12, the **Font weight** to **SemiBold**, and the **Top padding** to 20
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary` and the **Font** to `CurrentTheme.Font`
 - Rename this text label `SecretsLbl1`
13. From the **Insert** panel, click on **Vertical gallery** under **Popular**. Set the rest of the properties as follows:
 - Using the **Properties** pane, turn off **Show scrollbar** and enable **Show navigation**. Set the **Minimum width** to 32 and the **TemplateSize** to 84

- Using the formula bar, set the **Items** property to:

```
Filter(Secrets, Find(SearchTxt.Text, name) > 0)
```

And **OnSelect** to the following:

```
UpdateContext({selected: true, secret: Blank()});  
UpdateContext({secret: AzureKeyVault.GetSecret(ThisItem.  
name)})
```

- Rename this gallery to **SecretsGly**
14. Expand **SecretsGly** and select **Separator1** (remember that the numbering might change). Set the properties as follows:
 - Using the **Properties** pane, change the **X** property to 48 and the height to 1
 - Using the formula bar, set the **Fill** property to `CurrentTheme.Divider` and the **Width** to `Parent.TemplateWidth - Self.X`
 - Rename this rectangle to **SG_SeparatorRct**
 15. Select **NextArrow1** inside **SecretsGly** and set the properties as follows:
 - Using the **Properties** pane, change the **Width** and **Height** values to 48, set all padding properties to 15, and set the **Focused border** to 0
 - Using the formula bar, set the **Color** property to `CurrentTheme.Divider`
 - Rename this text label **SG_ArrowIcn**
 16. Select **Image1** inside **SecretsGly** and set the properties as follows:
 - Using the **Properties** pane, change the **Image** property to safe, the **X** property to 0, the **Width** and **Height** to 48, and all padding properties to 10
 - Rename this image **SG_SafeImg**
 17. Select **Subtitle1** inside **SecretsGly** and set the properties as follows:
 - Using the **Properties** pane, set the **Font size** to 10 and the left **Padding** to 0
 - Using the formula bar, change the **Text** property to "name", the **Color** property to `CurrentTheme.TextSecondary`, the **Font** to `CurrentTheme.Font`, and the **Y** property to `Title1.Y - Self.Height - 5`. Remember to change the **Title1** name if yours is different.
 - Rename this label **SG_NameLb1**

18. Select **Title1** inside **SecretsGly** and set the properties as follows:

- Using the **Properties** pane, change the **Font size** to 14, the **Font weight** to Semibold, the left **Padding** to 0, and the **Y** property to 35
- Using the formula bar, set the **Color** to `CurrentTheme.TextPrimary`, the **X** to `SG_SafeImg.X + SG_SafeImg.Width + 10`, the **Font** to `CurrentTheme.Font`, the **Width** to `Parent.TemplateWidth - Self.X - 10`, and the **Text** to `ThisItem.name`
- Rename this label `SG_ValueLbl`

19. Select **Rectangle1** inside **SecretsGly** and set the properties as follows:

- Using the **Properties** pane, change the **Y** to 20 and the **Width** to 2
- Using the formula bar, set the **Color** to `CurrentTheme.TextPrimary` and the **Height** to `Parent.TemplateHeight - Self.Y * 2`
- Rename this rectangle `SG_DecorationRct`

The application, so far, should be like the following:



Figure 7.8: Vault under construction

20. From the **Tree view**, select **RightContainer1**. Configure it by using the **Properties** pane on the right. Select the last option in **Align (horizontal)**, called **Stretch**, and set the **Gap** property to 10, the **Top** and **Bottom** padding to 20, and the rest to 40:

- Using the formula bar, set the **Width** to `Parent.Width - LeftContainer1.Width - Parent.LayoutGap * 3` and the **Color** property to `CurrentTheme.Background`

21. From the **Insert** panel on the left, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, change the **Text** to **Name**, the **Font size** to 14, and the **Font weight** to **Semibold**
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary` and the **Font** to `CurrentTheme.Font`
 - Rename this label `NameLbl`
22. From the **Insert** panel, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, change the **Font size** to 14 and all padding properties to 10
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`, the **Font** to `CurrentTheme.Font`, the **Fill** to `CurrentTheme.TextFill`, and the **Text** to `secret.name`
 - Rename this label `NameValueLbl`
23. From the **Tree view**, select `NameLbl` and `NameValueLbl`, copy them to the clipboard, and paste them twice.
24. Select `NameLbl_1`, and using the **Properties** pane, change **Text** to `Secret`. Rename this label `SecretLbl`
25. Select `NameValueLbl_1` and rename it `SecretValueLbl`:
 - Using the formula bar, change the **Text** property to `secret.value`
26. Select `NameLbl_2`, and using the **Properties** pane, change **Text** to `Created`. Rename this label `CreatedLbl`
27. Select `NameValueLbl_2` and rename it `CreatedValueLbl`:
 - Using the formula bar, change the **Text** property to `secret.createdTime`
28. From the **Insert** panel located on the left, click on **Button** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, change the **Text** to `Go back`, **Align in container** to the first option, **Start**, the **Border radius** to 0, the **Font size** to 12, the **Text alignment** to **Left**, and the **Vertical align** to **Bottom**

- Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`, the **Font** to `CurrentTheme.Font`, the **Fill** to `Color.Transparent`, the **HoverColor** and **PressedColor** to `Self.Color`, the **HoverFill** and **PressedFill** to `Self.Fill`, the **OnSelect** to `UpdateContext({selected: false})`, and the **Visible** property to `MainScreen.Size = ScreenSize.Small`
 - Rename this button `BackBtn`
29. Select **ScreenContainer1** and disable **Wrap** from the properties pane. From the formula bar, change the **LayoutGap** property to:

```
If(
    MainScreen.Size = ScreenSize.Small && selected,
    0,
    16
)
```

30. Click on **Save** on the main menu to complete the app.

How it works...

After creating the application, you can test the responsive design by running it from a browser on the desktop, or the Power Apps application on a mobile device.

The application should look like this when viewed on a desktop browser:



Figure 7.9: Password manager on the desktop

When the application runs, the app will resize accordingly to window size changes. The controls resize and move thanks to the responsive containers.

Notice that the left section of the app remains unchanged when resizing the screen, but if you shrink the window too thinly, this section expands and becomes the only visible screen, as seen in the following screenshot:

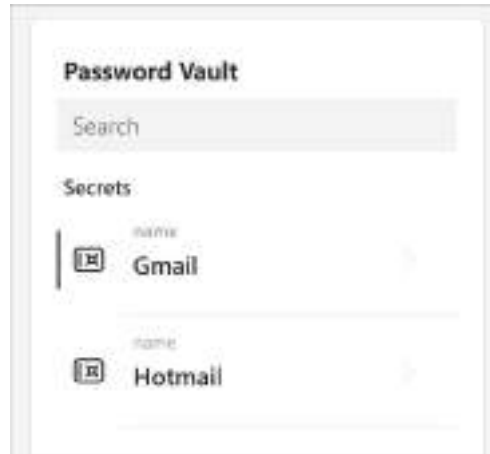


Figure 7.10: Left section expanded

This effect is the same when running the app on a mobile device – the left section becomes the main screen, and the detail section will only appear when we select a secret from the gallery:

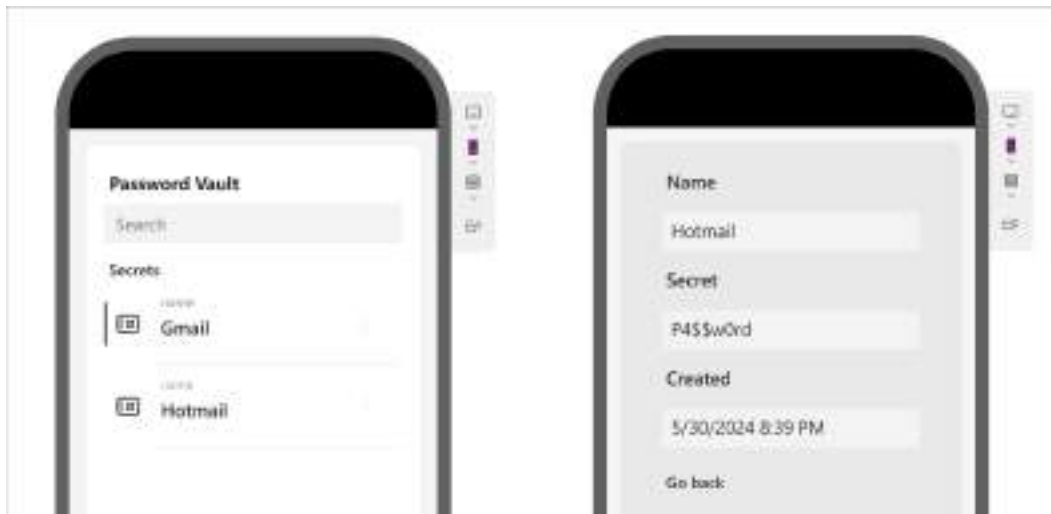


Figure 7.11: Password manager on mobile

Some testing might be required to get the desired design to work, primarily due to mobile devices' different resolutions. The responsive screens available in Power Apps help create a design template to build our apps.

There's more...

Power Apps offers many functional controls to build our apps, but these controls also need to be improved as the platform evolves. At the time of writing, some **modern controls and themes** are being introduced into Power Apps Studio to help makers create great-looking applications.

However, modern controls don't share all the functionality or the configuration capabilities as classic ones, so be on the lookout if there is a particular capability you might need before interchanging classic with modern controls.

An example is the Date Picker, one of the most controversial-looking controls in its classic form, which has beautifully evolved into a modern one. However, you can't change the date format or language as you can in the classic edition.

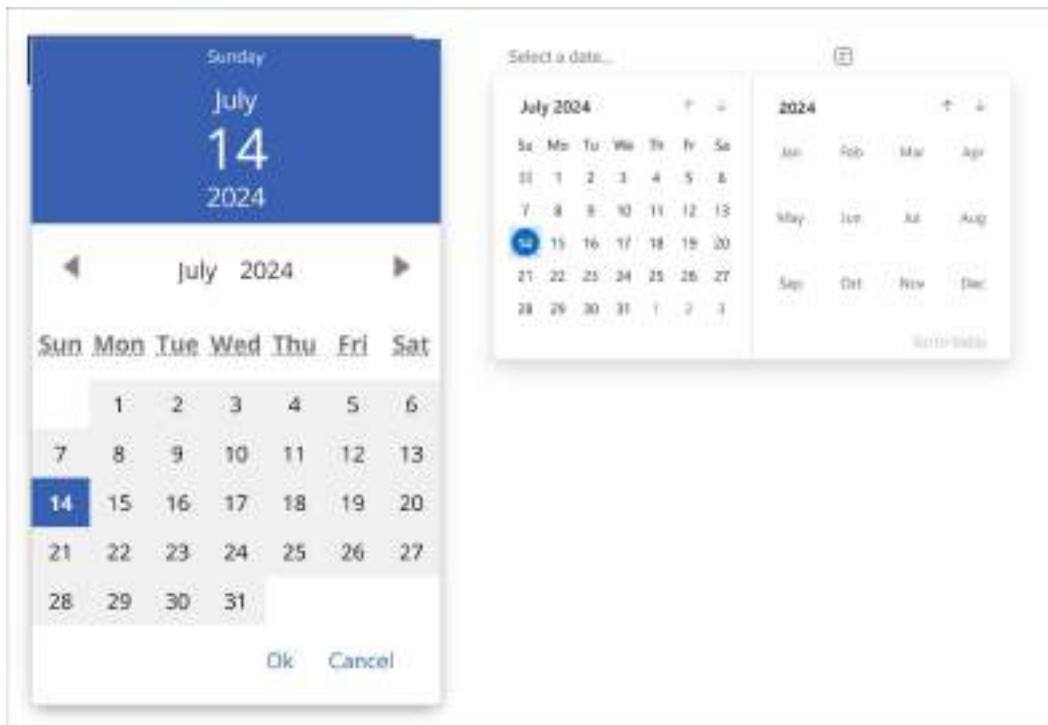


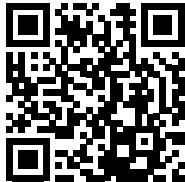
Figure 7.12: Classic Data picker versus the modern version

Be aware of these shortcomings before using them in your applications. Keep an eye out for improvements to modern controls at <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/controls/modern-controls/overview-modern-controls> and, especially, from the Power Apps blog, where they give us updates on these controls: <https://www.microsoft.com/en-us/power-platform/blog/power-apps/power-apps-category/building-power-apps/>

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



8

Power Apps Everywhere

Throughout this book, we have seen how valuable applications developed with Power Apps can be. Whether using multiple data sources or improving the application-building process through components, Power Apps allows makers to create solutions quickly and reliably for organizations.

Keep in mind that applications created with this platform are not just accessible through the Power Apps portal. There are many ways to use them, such as through embedding, desktop applications, mobile devices, and more.

In this chapter, we will discover all the possible ways to interact with Power Apps through different mediums with the help of the following recipes:

- Discovering applications through Power Apps mobile
- Leveraging modern browsers for Power Apps
- Improving SharePoint document libraries with Power Apps
- Embedding Power Apps in SharePoint pages
- Making Power BI reports interactive with embedded Power Apps
- Working with Power Apps in Microsoft Teams channels
- Integrating Power Apps inside Teams
- Building apps with Dataverse for Teams

Also, refer to *Chapter 3, Building from Data with Model-Driven Apps*, for another example of application embedding in Power Apps.

Technical requirements

The licensing model for this platform depends on the type of connector needed for your data sources. Standard connectors like the one used for SharePoint don't require an additional license besides Microsoft 365, but premium or custom connectors require a Power Apps license. For more information about the Power Apps licensing model, please refer to the *Preface* section.

The GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-08> holds the required files for each recipe.

Discovering applications through Power Apps mobile

This recipe will introduce how to interact with applications created in Power Apps besides using the maker portal at <https://make.powerapps.com>.

On mobile devices, the Power Apps application acts as a hub for all your solutions and is available in the most popular application stores:

- Google Play: <https://play.google.com/store/apps/details?id=com.microsoft.msapps>
- Apple's App Store: <https://apps.apple.com/us/app/power-apps/id1047318566>

Once you are logged in, this hub will list all the applications the user can access, whether using a phone or a tablet device.

This application offers a feature that allows the user to create an individual icon on the device's home screen to open a specific application. This feature is called **Pin to Home** for Android devices and **Add shortcut to Siri** for Apple devices. In this recipe, we will learn how to implement it to help with application adoption and discovery.

How to do it...

1. Install the Power Apps application from your device's application store and sign in using your organizational account.
2. From the list of applications, select the three dots next to the application you want to create an icon for, click **Pin to Home** on Android devices, and use **Add shortcut to Siri** on Apple devices. The following is the Power Apps application on each mobile device platform:

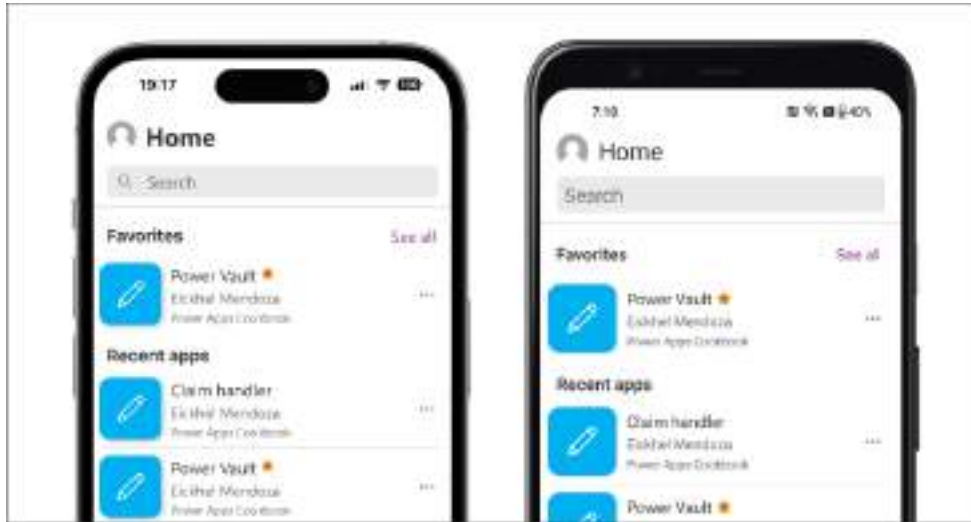


Figure 8.1: Power Apps application on Android (left) and iPhone (right) devices

3. On Android devices, a dialog will open to place the icon automatically or manually in the location of choice by using drag and drop, as shown in the following screenshot:

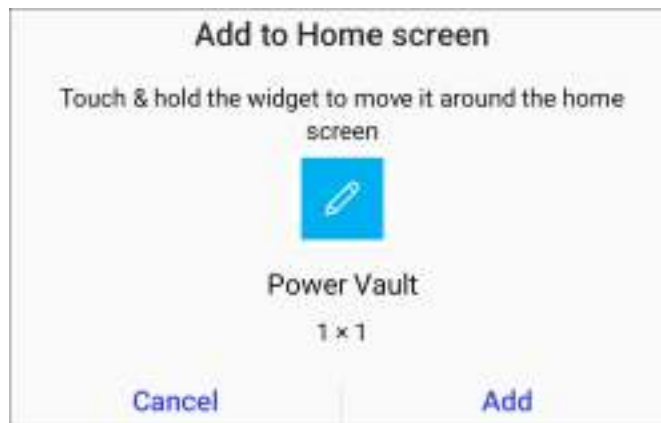


Figure 8.2: Android's Pin to Home dialog

Remember that this image might differ depending on your Android UI layers, such as Google's, Samsung's, or OnePlus'.

4. On Apple devices, the **Add shortcut to Siri** option will display a dialog confirming the inclusion of this action to Siri, as shown in the following screenshot:

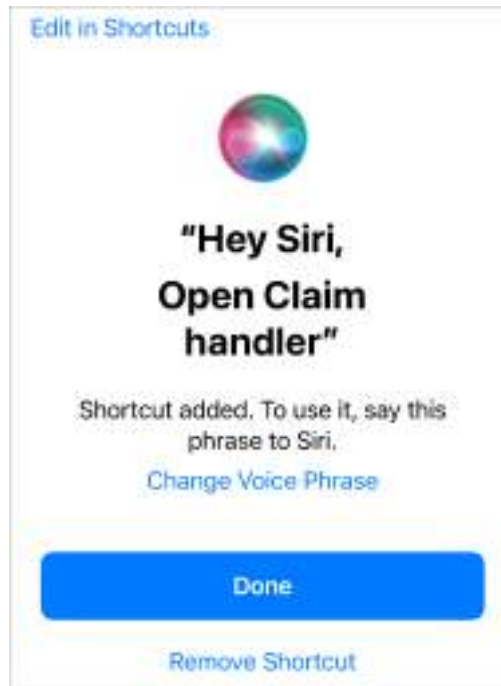


Figure 8.3: Apple's Pin to Home dialog

We can apply this shortcut to any application from the Power Apps application hub.

How it works...

The **Pin to Home** feature in Android adds a standalone icon, bypassing the Power Apps application by going directly to the desired app.

For Apple, it adds the ability to open the app using Siri. The operating system leverages this functionality in the native **Shortcuts** application. From there, you will find the recently created shortcut. You can also add this shortcut to the home screen. Click the three dots, then the shortcut's name, and finally, **Add to Home Screen**, as shown in the following figure:

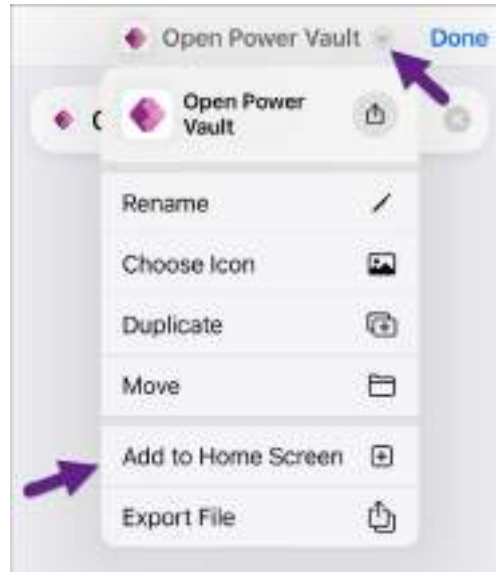


Figure 8.4: Shortcut's Add to Home Screen feature

The Power Apps application also offers the **Open at startup** feature, which allows you to run the selected application every time you open Power Apps.

Finally, remember the swipe gestures available on the application list. They're an excellent shortcut for sharing and adding to favorites, among other things.

There's more...

There is a Microsoft Store version of Power Apps that allows for app filtering depending on the environment, which is not available on other versions; however, it is missing the **Pin to Home** feature. Hopefully, this feature will be included in the future.

In the meantime, please refer to the *Leveraging modern browsers for Power Apps* recipe for an elegant workaround when using Power Apps on desktop systems. For more information, please visit <https://learn.microsoft.com/en-us/power-apps/mobile/windows-app-install>.

If you rely solely on Dynamics 365 apps, you can access your applications besides the Power Apps hub. Individual apps exist for **Dynamics 365 Sales**, **Dynamics 365 Business Central**, **Dynamics 365 Field Service**, and **Finance and Operations**. Head to your specific application store to try them, depending on the Dynamics 365 license available to your users.

These applications also lack Pin To Home but offer a great way to navigate between apps. They also provide the nice feature of always executing the last opened app.

Finally, it is also worth mentioning that the platform allows app makers to package canvas apps as custom-branded Android or iOS apps with the **Wrap** feature. You can find more information on this topic here:

<https://learn.microsoft.com/en-us/power-apps/maker/common/wrap/overview>

Leveraging modern browsers for Power Apps

Web browsers are the gateway for almost every online activity. They are not just for browsing websites anymore; they also serve apps, services, communication, and gaming. They do all this without the need for independent app installation.

That's why using a modern browser should be a common-sense rule for everyone; they use the latest web technologies and, most importantly, are up to date with security fixes.

In this recipe, we will leverage one of the latest features introduced in modern browsers to get faster access to the apps we create. As mentioned in the previous recipe, this feature will help us improve application discovery on desktop devices.

Getting ready

This workaround will require a Chromium-based browser like Microsoft Edge, Brave, or Google Chrome. For more information regarding which browsers use this technology, please refer to Browsers based on Chromium section at <https://en.wikipedia.org/wiki/Chromium>).

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com>, and from the left pane, click on **Apps** to view the list of applications in your environment of choice.
2. Click on the play icon that appears when you hover over the name of any application. This action will open a new window with the app running.
3. To improve the perception of your application as a standalone solution, click on the address section of the browser and include the following text, `&hiddenavbar=true`, as shown in the following screenshot:

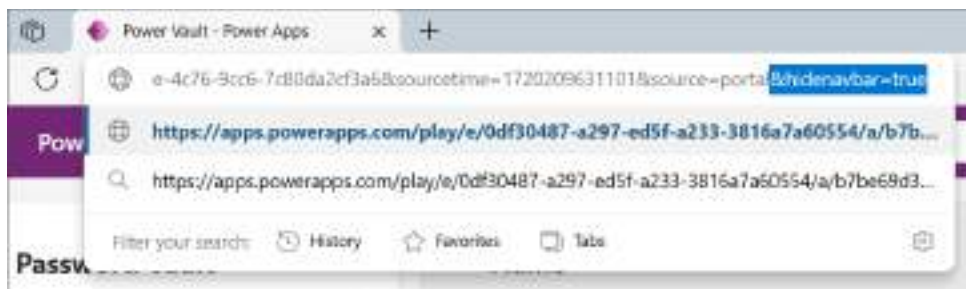


Figure 8.5: Address parameters for the Power Apps player

Adding this parameter will remove the Power Apps branding from the running application. Now, let's see how to run our applications as standalone solutions on Microsoft Edge and Google Chrome.

Microsoft Edge

1. After following *steps 1 to 3* in the preceding section, click on the browser's ellipsis dropdown and then select **Apps | Install this site as an app**, as seen in the following screenshot:

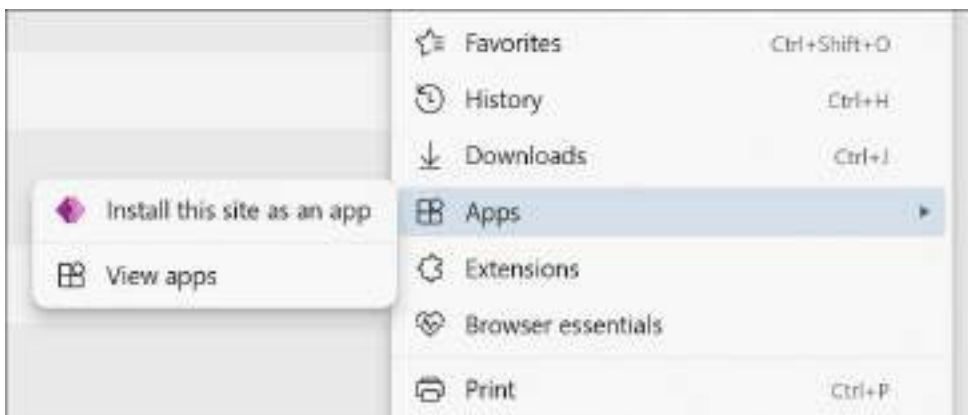


Figure 8.6: Installing Power Apps as an app in Microsoft Edge

2. A dialog will open asking for the name of this app. You can also change its icon by clicking **Edit** below the Power Apps icon. Leave the name as default or rename it as desired, and then click **Install**. Immediately after this, a new window will be created with the application running without any browser decoration.
3. A new dialog will appear from the ellipsis drop-down menu, giving you several options regarding the app's pin positions and the ability to enable auto-start on device login.

Google Chrome

1. After following *steps 1 to 3* in the *How to do it...* section of this recipe, click on the browser's ellipsis dropdown and then select **Save and share | Install page as app...**, as seen in the following screenshot:

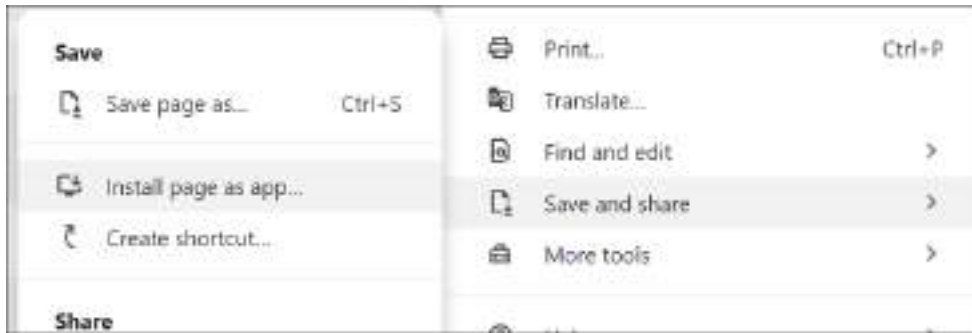


Figure 8.7: Installing Power Apps as a shortcut in Google Chrome

2. A dialog will open, asking for the shortcut's name. Leave it as default or rename it as desired, and click **Install**. Immediately after this, a new window with the application running without any browser decoration is created. You will also get a notification asking if you want to pin the application to the taskbar.

On both browsers

1. Resize the window to the desired dimensions. The app window will remember the size for future executions.

How it works...

Access to your application is now easier than ever; there's no more need to go to the Power Apps maker portal or the Office 365 home page. The application can be executed directly like any other on your system:

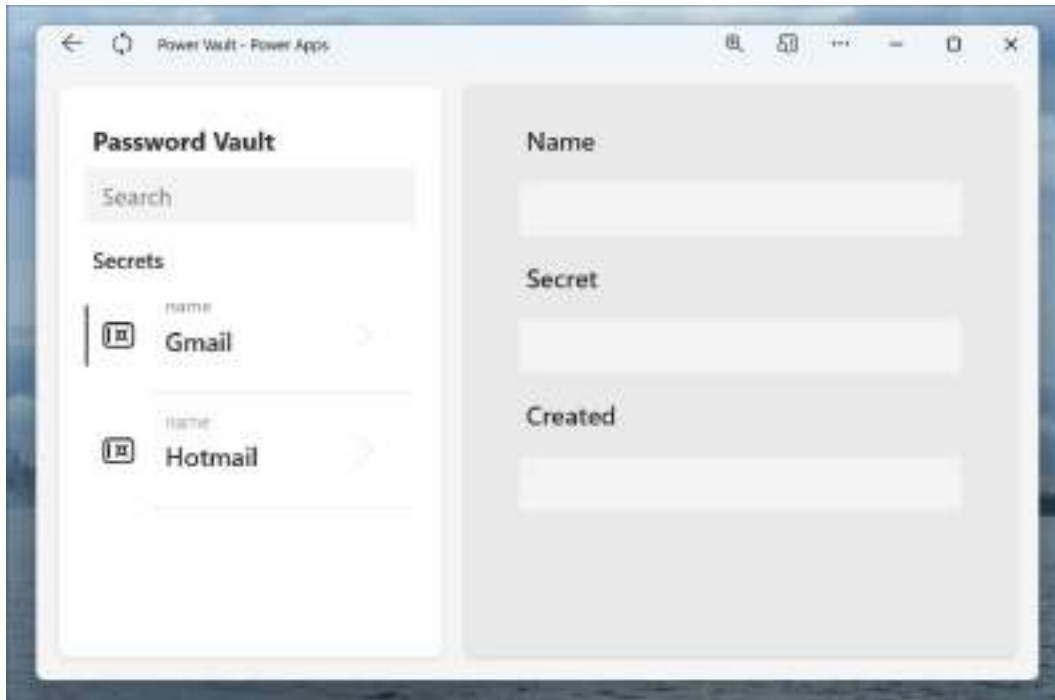


Figure 8.8: Power Vault running like a standalone application

You can reuse this technique with any other app in your organization (including model-driven apps), giving your users faster access to your solutions. Keep in mind that the `&hiddenavbar=true` parameter doesn't work on model-driven apps.

There's more...

Google Chrome also creates a desktop shortcut for apps using this technique. It might be helpful in other scenarios, such as distributing these shortcuts using an org-wide **Group Policy (GPO)**. This solution could deploy the specified shortcut on the desktop of all devices used by an organization.

Please refer to this blog for a step-by-step example of this technique:

<https://www.prajwaldesai.com/create-desktop-shortcut-using-group-policy-gpo/>

Microsoft Edge can also benefit from the GPO solution by using specific policies, especially **WebAppInstallForceList**, which allows the silent installation of apps without user intervention. Users will not be able to uninstall these apps.

For up-to-date documentation, please refer to <https://learn.microsoft.com/en-us/deployedge/microsoft-edge-policies#webappinstallforcelist>.

Improving SharePoint document libraries with Power Apps

In *Chapter 2, Building Pixel-Perfect Solutions with Canvas Apps*, we discovered the ability to embed Power Apps in SharePoint lists. This feature opens a wide range of possibilities to handle this data compared to SharePoint's standard forms by allowing you to use a full-featured canvas application.

We can also improve SharePoint document libraries. Since they inherit the functionality of SharePoint lists, the same Power Apps integration applies. However, since the focus is on handling files instead of items, this integration lacks usability from a user perspective.

This recipe will explain how to improve this integration by using a SharePoint feature called **column formatting**. We can change how columns get displayed using JSON code. For more information, visit the following site:

<https://learn.microsoft.com/en-us/sharepoint/dev/declarative-customization/column-formatting>

Getting ready

Since we are using SharePoint as our data source, there are no additional requirements besides a regular Microsoft 365 license. We will need a document library with some metadata columns to improve the library's categorization.

For this recipe, we will need a SharePoint document library with the following columns:

Type	Name
Multiple lines of text	Excerpt
Text	ISBN
Yes/No	Available

Table 8.1: Library columns

Please refer to *Chapter 1, App Maker Basics*, for instructions on how to build a document library.

The code needed for the column formatting is available from our GitHub repo at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-08>.

How to do it...

1. Go to the desired SharePoint site and create a document library named Book Requests.
2. Add files to the library by selecting **Upload | Files** from the toolbar or by dragging them. Once the upload is complete, click **Edit in grid view** to update the file's metadata in bulk, like when working with an Excel spreadsheet. Click on **Exit grid view** when finished.
3. Enter some test book data for each uploaded file, such as the following:
 - Excerpt:
"The Volunteer Chronicles" by Albert Castro: A heartwarming tale of a group of volunteers who come together to transform their community, discovering friendship and purpose along the way
 - ISBN:
202220232024
 - Available:
Yes
4. From the toolbar, click **Integrate | Power Apps | Customize forms**. This action will send the data structure of the document library to Power Apps to build an app.
5. Once Power Apps Studio completes the app's creation, you will get a screen called **Form-Screen1** to edit the library's metadata. Select the **SharePointForm1** form control and click on **Edit fields** on the right pane.

6. Leave the **Excerpt**, **ISBN**, and **Available** columns on the **Fields** pane. Remove the rest by selecting the ellipsis icon on each field and selecting **Remove**, as seen in the following screenshot:

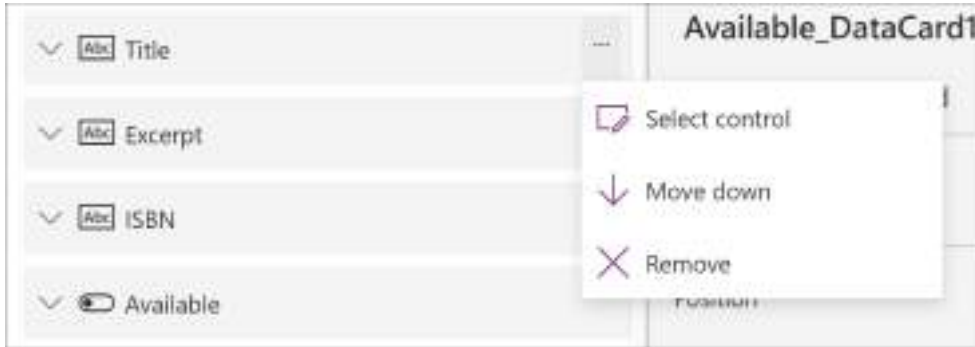


Figure 8.9: Remove field option

7. Click **+ Add field** in the **Fields** pane to add a **Thumbnail**. Drag the fields to arrange them in this order: **ISBN**, **Available**, **Excerpt**, and **Thumbnail**.
8. Expand **SharePointForm1** (keep in mind that the number might differ), select the excerpt text input control named **DataCardValue2**, and change the **Font size** to 12, the **Mode** to **Multiline**, all **Padding** properties to 10, and the **Height** to 200
9. Select the **Thumbnail_DataCard1**. From the properties pane on the right, click the **Advanced** tab and click **Unlock to change properties**. Using the formula bar, change the **Height** property to `Parent.Height - Self.Y`
10. Select the **DataCardValue1** inside **Thumbnail_DataCard1** and delete it. Do the same with the **DataCardSubvalue1**.
11. From the **+ Insert** dropdown, select **Media | Image**. Using the formula bar, change the **Image** property to `Parent.Default.Value1`, the **X** to 32, and the **Width** to `Parent.Width - 60`. Using the formula bar, change the **Height** property to `Parent.Height - Self.Y - 20`. Rename this control to **Thumbnail_Img**
12. Expand **ISBN_DataCard1** and select **DataCardValue3** to change the **Font size** to 12 and all **Padding** properties to 10
13. From the toolbar, click on **Save** and then **Publish**. On the dialog that appears, select **Publish to SharePoint**. Click on **Back to SharePoint** at the top left, and in the dialog that appears, click **Leave**.

14. Back in the document library, click on **+ Add column** again, select **Text**, and click **Next**. On the pane on the right, name the column **Actions**, check **Use calculated value**, enter **=""** as the **Default value**, and click **Save**.
15. Drag the **Actions** column just before **Name** and click on its drop-down arrow to select **Column settings | Format this column**, as seen in the following screenshot:

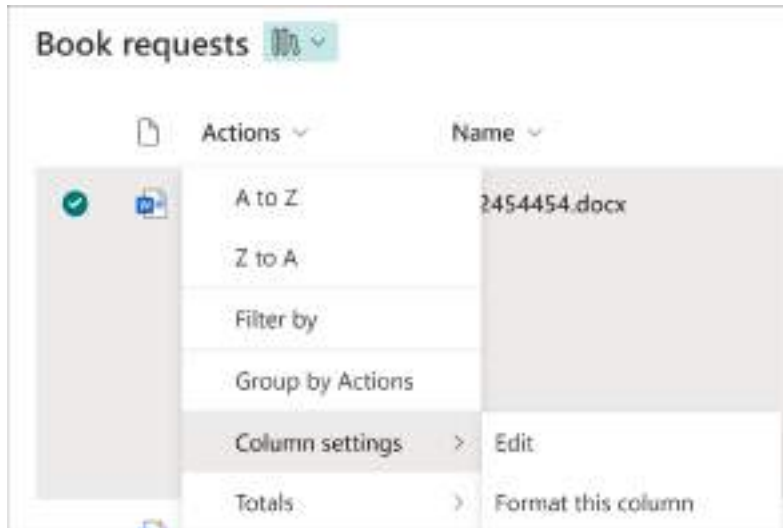


Figure 8.10: Format this column option

On the panel that opens, click on **Advanced mode** near the bottom. Paste the code found in our GitHub repo from the `ColumnFormat.json` file to prepare the action toolbar for each file. When finished, click **Save** and then close the pane.

How it works...

Working with Power Apps embedded in document libraries can help your users when they require more features besides what you get from default SharePoint forms. However, the steps needed to use these apps are very awkward in the current SharePoint implementation, mainly due to the need to open the properties pane to view the embedded app.

Right now, there are two ways to achieve this:

- Click on the ellipsis of a file, then select **More | Properties | Edit all**
- Select a file, click the info pane on the toolbar, and then look for the **Edit all** link

The following screenshot shows the first way to open the properties pane:

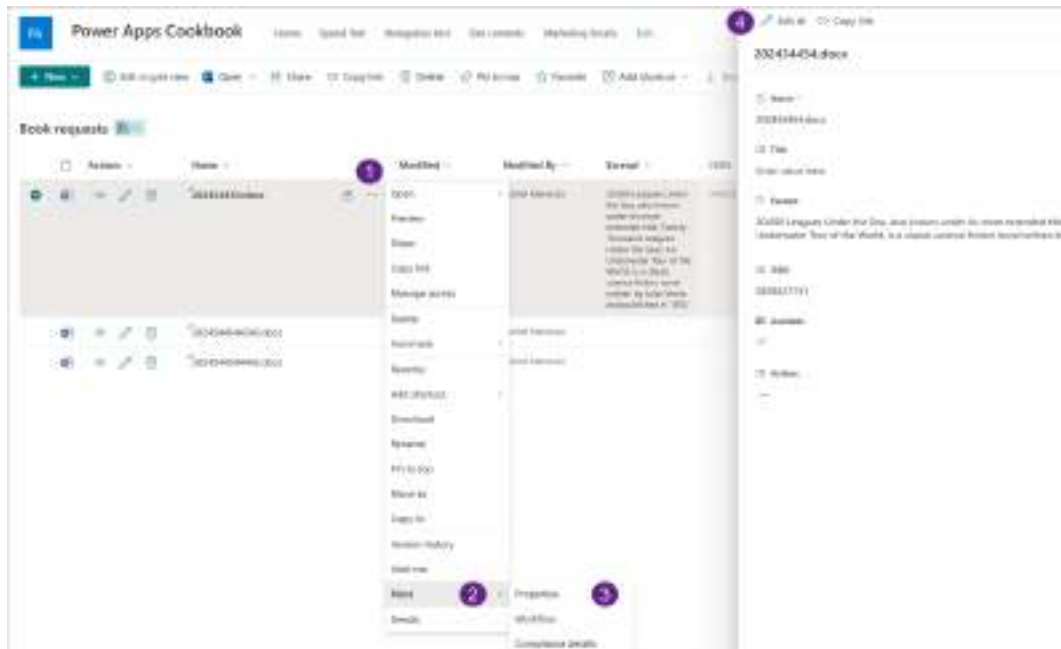


Figure 8.11: Steps to edit properties for a file

With the help of SharePoint column formatting, we have added three action buttons to interact with the library's files:

- **Eye icon:** Default click to open the file inside the library
- **Pencil icon:** This action acts as a shortcut to open the properties pane and, therefore, the embedded app
- **Trash icon:** Clicking on this performs the delete action

Now, we can easily access the application to edit the file properties by clicking the pencil icon, saving the need to find your way to the properties panel and even selecting the **Edit all** link since the Power App already opens the form in edit mode.

We have added the capability to include a file thumbnail for easy identification. Additionally, we may introduce more features based on our requirements, such as input field validation, cloud flow integration, and more.

However, it's important to note that these types of apps are restricted to SharePoint, meaning that they are not readily accessible from the Power Apps maker portal. They can only be viewed within the app list of the default environment from the Power Apps admin center.

There's more...

Using the Actions column to improve the use of embedded apps in document libraries is just an example of what you can achieve with **column formatting**. You can enhance the list visualization, apply color rules depending on columns' values, and even call a Power Automate cloud flow to interact with the file that triggered it.

For more examples of this feature, please refer to the SharePoint List Formatting Samples at <https://pnp.github.io/List-Formatting/>.

Embedding Power Apps in SharePoint pages

SharePoint pages are the tool of choice when using this platform for content management and delivery. Using content blocks, called **web parts**, you can build pages with text, images, videos, and dynamic content.

One of SharePoint's content blocks is the Power Apps web part, which allows embedding a canvas app inside a page for easier access. For more information on web parts, please refer to <https://support.microsoft.com/en-us/office/using-web-parts-on-sharepoint-pages-336e8e92-3e2d-4298-ae01-d404bbe751e0>.

Getting ready

We will need an existing application to use the Power Apps web part. The same rules for executing applications apply here:

- It needs to be shared with the users that will utilize the application.
- If a license is required to use the app, it will be enforced on the users accessing the SharePoint page.
- The users must grant the use of any connectors inside the app.

We will use the same application designed in *Chapter 7, Improving User Experience*, for this recipe.

How to do it...

1. Navigate to the Power Apps maker portal, <https://make.powerapps.com>, and select the **Apps** section on the left pane. Locate the mentioned app from the list, click on its ellipsis dropdown, and choose **Details**.

2. When the **Details** page appears, look for **App ID**, as seen in the following screenshot:

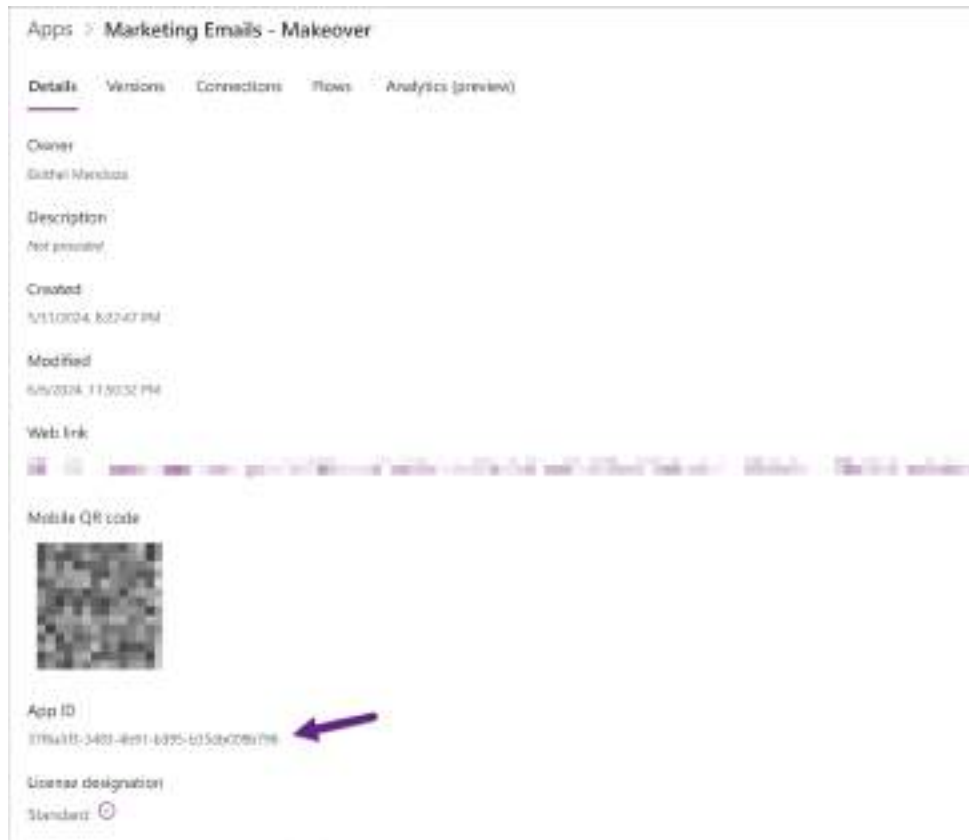


Figure 8.12: Marketing Emails - Makeover details

3. Copy the **App ID Globally Unique Identifier (GUID)** and save it to your clipboard.
4. Go to your SharePoint site and open the desired page to set up your app.

5. On the page's toolbar, click on **Edit**. Once the page is in edit mode, locate a space to embed the app and click the plus sign to add a web part, as seen in the following screenshot:

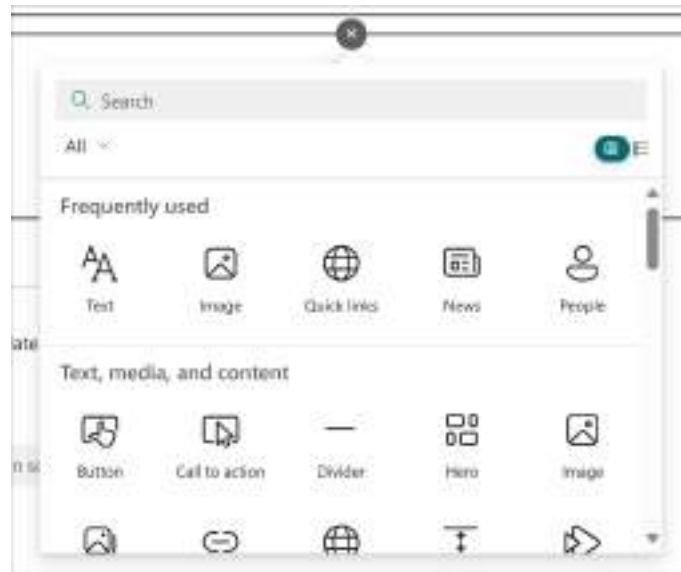


Figure 8.13: Web parts in SharePoint pages

6. Enter PowerApps (*without a space*) in the **Search** box and select it from the list.
7. Once the web part is inserted, it will open the properties pane to the right. Paste the collected app ID in the **App web link or ID** field. The application should appear immediately.
8. Click on **Show border** if desired. Drag to extend the web part to the appropriate size. Depending on the layout and responsiveness of the app, it should fill the web part correctly.
9. Once the changes are complete, click **Republish** on the page's toolbar.

How it works...

After following the web part configuration steps, you should have a fully functional application on your SharePoint page, as seen in the following screenshot:

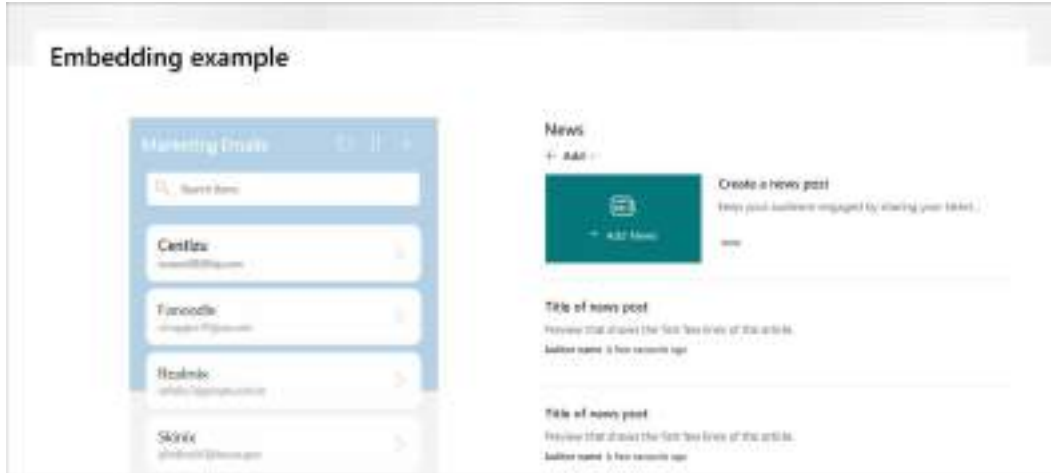


Figure 8.14: App embedded in a SharePoint page

Embedding a canvas Power App in a SharePoint page can help its discovery by placing it in a shared location. For example, an ideal place for a vacation request app would be the HR department site.

The **Marketing Emails** app is just an example. You can embed more complex applications using, for example, connections to Azure SQL, cloud flows, or any other service.

There's more...

This feature adds even more value when external users use SharePoint. Suppose your application doesn't need to be licensed by a Power Apps plan, and the application gets embedded in a site page shared with external users. In this case, you are automatically creating a tool for external users to interact with the organizational assets defined in your application.

An improved version of the Power Apps web part includes more features such as theme awareness, dynamic data handling, and more. Please refer to this [GitHub repository](https://github.com/pnp/sp-dev-fx-webparts/tree/main/samples/react-enhanced-powerapps) to get it:

<https://github.com/pnp/sp-dev-fx-webparts/tree/main/samples/react-enhanced-powerapps>

Please also see this instructional video on how to install it on your tenant:

<https://www.youtube.com/watch?v=EFhP4uu0r1o>

Making Power BI reports interactive with embedded Power Apps

Power BI is the definitive reporting tool. It allows the fusion of many data sources to build dashboards and reports that aid organizations in getting the knowledge they need from the insight received from this data.

Being part of Power Platform, we can expect integrations from other tools that compose this platform. For example, Power BI integrates with Power Automate via alerts. When an alert gets triggered, a cloud flow can execute notifications depending on the trigger's data or even record these alerts in a data source for auditing purposes.

Power Apps allows embedding applications inside reports to extend data visualization or information processing, thus improving the interaction with the analytics solution.

Getting ready

This recipe will use Power BI Desktop, the free report design tool. Installing it from the Microsoft Store is recommended because the system will update it automatically.

As for the data source, we will take advantage of the Azure SQL Database service, which allows you to use a direct connection instead of importing the data into the report.

To have an Azure SQL database, you will need an Azure subscription. To apply for this and other services from the Microsoft Cloud solution, sign up for a free account at <https://azure.microsoft.com/en-us/free/>. Please refer to *Chapter 1, App Maker Basics* for instructions on how to provision an Azure SQL database.

Regarding Power Apps, since we are using a premium connector, please refer to the *Preface* section of this book to get more information about the licensing requirements.

How to do it...

The following steps will help you design a Power BI report with an embedded app.

Installing and configuring Power BI Desktop

1. The following link will open the Power BI Desktop page on the Microsoft Store:
<https://apps.microsoft.com/store/detail/power-bi-desktop/9NTXR16HNW1T>
2. Install the application and launch it when it finishes.

3. On the startup screen, click the **SQL Server** icon, and on the dialog that opens, enter your database's **Server** address and **Database** name. Select **DirectQuery** for **Data Connectivity mode** and then click **OK**. These settings should appear as in the following screenshot:

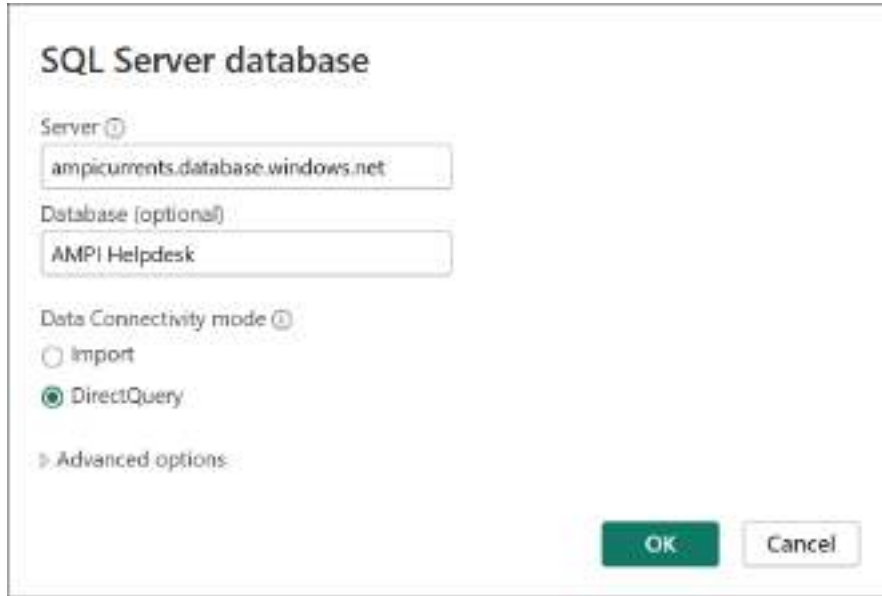


Figure 8.15: Azure SQL Database configuration in Power BI

4. The following dialog is where you can set the authentication type. Select the one you've configured for your database.
5. The **Navigators** dialog will open, displaying the contents of the database. Expand the database name and select the **SalesLT.Product** table, then hit **Load**. The Power BI report now has the data source configured for the Azure SQL database. You can start designing your report.
6. From the **Data** pane on the right, expand the table and select the following fields: **ProductNumber**, **Name**, and **ListPrice**. These fields will appear in a table visualization, as seen in the following screenshot:

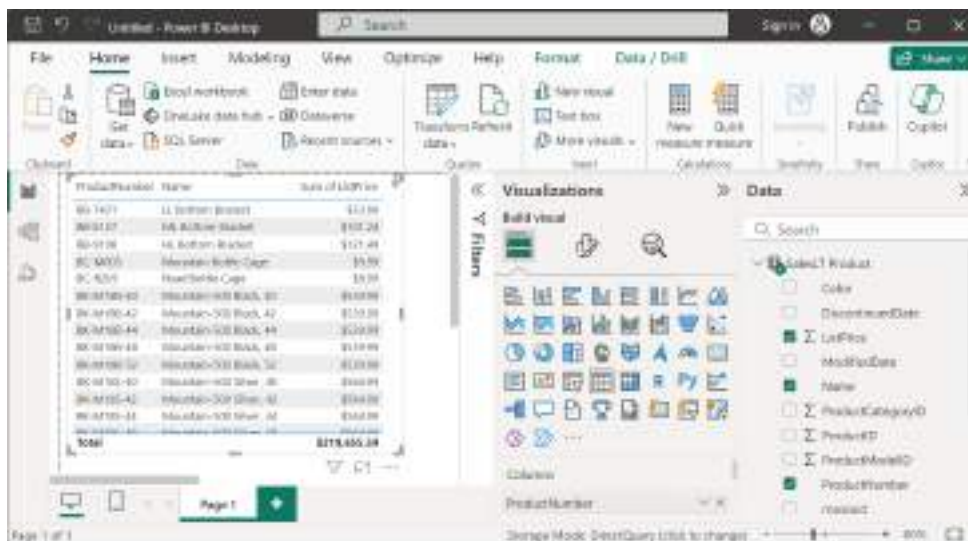


Figure 8.16: Data source fields visualization in Power BI Desktop

7. Log in using your organization's credentials by clicking **Sign In** on the app's title bar. On successful login, click the save icon at the beginning of the title bar. **Name** your report and click **Save**.
8. Click **Publish** on the **Home** toolbar. From the **Publish to Power BI** dialog, choose **My workspace** and click **Select**. Click **Got it** when the process completes.
9. Go to Power BI online, <https://app.powerbi.com/>, and from the left pane, click **Workspaces** and select **My workspace**. The list of elements contained in this workspace will appear. You will find two elements with the same name as your report. One is the actual report, and the other is the semantic model, which holds all the metadata. Click on the three dots at the end of the semantic model and choose **Settings** from the dropdown.
10. On the **Settings** page, expand **Data source credentials** and click **Edit credentials**. Set the **User name** and **Password** as you did on Power BI Desktop and click **Sign in**. This action needs to be done once because when you publish your report, the credentials are not included in this process.

11. Go back to **My workspace**. This time, click to open the report, and from the toolbar, click on **Edit**. Design the report to include the values, and then select the **Power Apps** icon from the **Visualizations** pane on the right. This action will add a visualization that will act as the container for the embedded app. Select the same fields as in *step 12*.
12. Once the field configuration is complete, the visualization will change to embed the app. Select the environment that will host your application and then click **Create new**. After a few seconds, Power Apps Studio will open on a new tab already connected to this report.

Building the Power App

1. By default, the integration creates an application with a gallery and the **PowerBIIntegration** control, which integrates Power BI and Power Apps.
2. Select the data icon on the left pane and click **Add data**. Search for **SQL Server** and select it. On the fly-out menu, click **Add a connection** if you don't have one already. On the pane that opens, choose **SQL Server Authentication** on the **Authentication type** dropdown. Set the same parameters to connect to the Azure SQL database from Power BI Desktop and click **Connect**.
3. Select **SalesLT.Product** from the list of tables and click **Connect**.
4. Return to the **Tree view** pane, select the gallery, and delete it.
5. From the **Insert** toolbar, insert a **Text label** three times:
 - Select **Label1** and change its **Text** property to `Product name`
 - Select **Label2** and, using the formula bar, change its **Text** property to `First(PowerBIIntegration.Data).Name` and the **Fill** property to `RGBA(242, 242, 242, 0.5)`. Set the **X** property to 40 and the **Y** property to 120
 - Select **Label3** and change its **Text** property to `List price`. Set the **X** property to 40 and the **Y** property to 240
6. From the **Insert** panel, click on **Text input**. Set the **X** property to 40 and the **Y** property to 320. Using the formula bar, change its **Default** property to `First(PowerBIIntegration.Data). 'Sum of ListPrice'` and the **Fill** property to `RGBA(242, 242, 242, 0.5)`. Change the **Format** property to **Number**.
7. Click on **Button** from the **Insert** panel and set the **X** property to 40 and the **Y** property to 440. Change the **Width** property to 560 and the **Text** property to `Update list price`. Using the formula bar, change the **OnSelect** property to the following:

```

Patch(
    'SalesLT.Product',
    LookUp(
        'SalesLT.Product',
        ProductNumber = First(PowerBIIntegration.Data).
        ProductNumber),
    {ListPrice: Value(TextInput1.Text)}
);
Back();
PowerBIIntegration.Refresh()

```

8. Select all controls and change the **Font** property to **Lato** and the **Font size** to 22.
9. On the main toolbar, click on **Settings**. Go to **Display**, turn off **Scale to fit**, and close the dialog.
10. Click on **Save**, give the app a name, and then click **Publish**.
11. Once the app gets saved, the Power BI online will update the report to reflect the embedded application, as seen in the following screenshot:

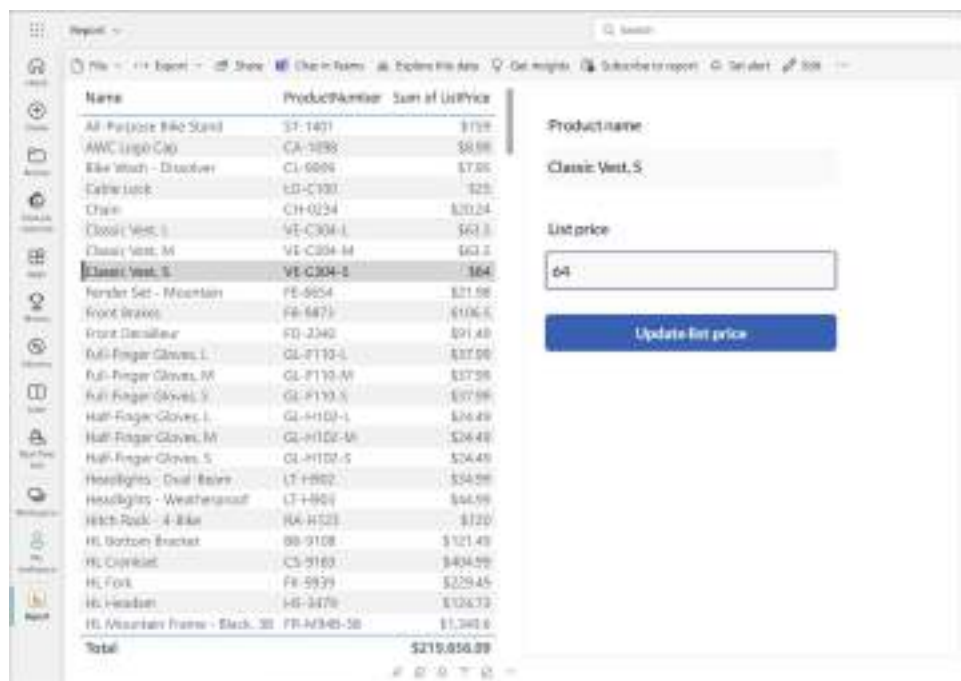


Figure 8.17: Power app integrated into a Power BI report

Now, we have the app fully embedded from the report we started locally and completed on the Power BI online platform.

How it works...

There are two main factors in this recipe, **DirectQuery** and **PowerBIIntegration**:

- **DirectQuery** allows direct connection to the data source. When a data refresh is requested, Power BI queries the data directly from the data source instead of importing the data for processing.
- The Power BI integration control (**PowerBIIntegration**) allows data interaction between Power BI and Power Apps. The **Data** property of this control receives a table of the fields defined in the visualization, as seen in *step 5*. For this reason, the code in Power Apps always uses the **First** function to get the first record of the **Data** property.

This solution offers a way to update the data directly from the report using an embedded application. Using the **Refresh** method of integration control, we can keep the data up to date in the report.

Working with Power Apps in Microsoft Teams channels

Microsoft Teams is the central hub of an organization's content. The platform offers a wide range of capabilities, including communication, collaboration, and content access.

One of its most valuable aspects is the opportunity to extend its features through customizations. For example, you can build custom applications, enhance messaging capabilities, build bots, and embed web content. Power Apps allows the creation of applications integrated within this collaborative platform.

Getting ready

This recipe will build an app for Microsoft Teams to handle the channels in a given team. To make this work, the user must have access to a team with proper permission to add and remove apps.

The canvas app will take advantage of the Microsoft Teams integration using the context parameters available when using an embedded app. For more information, please visit <https://learn.microsoft.com/en-us/power-apps/teams/embed-teams-app#use-context-from-teams>.

The resulting application and the code needed to build it are available in our GitHub repo, which is located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-08>.

How to do it...

The first step will be creating the canvas application to distribute through Teams channels.

Building the canvas app

1. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and create a solution for this recipe by navigating to **Solutions** and then clicking **+ New solution** from the toolbar. Name the solution **Teams Apps**.
2. Inside the solution, click **+ New | App | Canvas app** from the toolbar. On the dialog that opens, set the name to **Channel Manager**. Keep the **Tablet** format and click **Create**.
3. When Power Apps Studio opens, click **Settings** and fill the **Description** field with **Microsoft Teams channel management system**. Still on **Settings**, select **Display**, deactivate **Scale to fit**, and close the dialog.
4. On **Tree view**, select the **App** element. Using the formula bar, select the **OnStart** property and paste the code downloaded from the GitHub repository located in the **App - OnStart.txt** file. Right-click on the **App** ellipsis and select **Run OnStart** to execute this code, as seen in the following screenshot:

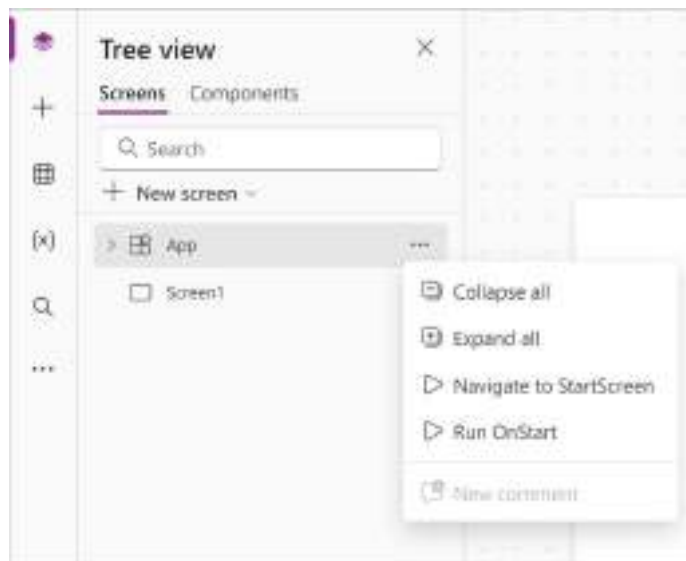


Figure 8.18: OnStart execution

5. From the toolbar, click on + **New screen** and choose **Sidebar**. Rename **Screen2** to Main Screen and delete **Screen1**.
6. From the data sources panel on the left (represented by a database icon), click **Add data** and use the **Search** field to look for Microsoft Teams. Select this connector from the list and select **Connect** from the panel that opens on the right.
7. From the **Tree view**, select **ScreenContainer1**. Set the properties as follows:
 - Using the formula bar, set the **Fill** to `CurrentTheme.AppBackground`
8. Repeat *step 7* for **BottomContainer1**.
9. Expand **ScreenContainer1**, select **HeaderContainer1**. Set the properties as follows:
 - Using the properties pane, change **Align (vertical)** to the last option, **Stretch**. Set the left and right **Padding** to 10
 - Using the formula bar, set the **Fill** to `CurrentTheme.MainColor`
10. While still selecting **HeaderContainer1**, from the **Insert** dropdown, click **Text label**. Set the properties as follows:
 - Using the properties pane, change the **Text** property to Channel Manager, the **Font** to **Segoe UI**, the **Font size** to 16, and the **Font weight** to **Semibold**. Enable **Flexible width**, and set the **Color** to white.
 - Name this label `MainTitleLbl`
11. From the **Insert** panel, click on the **Reload** icon located under **Icons**. Set the properties as follows:
 - Using the properties pane, change all the **Padding** properties to 20 and the **Color** to white.
 - Using the formula bar, set the **OnSelect** property to the code downloaded from the GitHub repository inside the `RefreshIcon - OnSelect.txt` file.
 - Name this icon `RefreshIcon`
12. Ctrl-click **RefreshIcon** to execute its code.
13. Select **MainScreen** from the **Tree view** panel and, using the formula bar, change the **OnVisible** property to `Select(RefreshIcon)`

14. Select **SidebarContainer1** and, using the properties pane, change **Align (vertical)** to the last option, **Stretch**. From the **Insert** panel, add a vertical gallery. Set the properties of the gallery as follows:
 - Using the properties pane, change the **Layout** to **Title and subtitle** and the **Align in container** to **Set by container**. Activate **Flexible height**, set the **TemplateSize** to 100, turn off **Show scrollbar**, and enable **Show navigation**.
 - Using the formula bar, set **Items** to `channelData` and the **Fill** to `CurrentTheme.Background`
 - Name this gallery `ChannelGly`
15. From the **Tree view**, expand **ChannelGly** and follow these instructions, keeping in mind that some of the numbers in the names might change:
 - a. Select **Separator5** and, using the formula bar, set the **X** value to 20, the **Height** to 1, the **Fill** to `CurrentTheme.Divider`, and the **Width** to `Parent.TemplateWidth - 40`
 - b. Name this control `SeparatorRct`
 - c. Select **NextArrow5** and, using the properties pane, set the **Icon** to **Message**, the **Width** and **Height** values to 48, and all **Padding** to 10
 - d. Using the formula bar, set the **Color** to `CurrentTheme.TextSecondary` and the **X** property to `Parent.TemplateWidth - Self.Width - 20`
 - e. Name this icon `MessageIcn`
 - f. Select **Subtitle5** and, using the properties pane, change the **Font** to **Segoe UI** and the **Font size** to 10
 - g. Using the formula bar, set the **Text** property to `$"Message count: {Text(ThisItem.messageCount.'@odata.count')}"` and the **Color** to `CurrentTheme.TextSecondary`
 - h. Name this label `MessageCountLbl`
 - i. Select **Title5** and, using the properties pane, change the **Font** to **Segoe UI**, the **Font size** to 16, and the **Y** property to 25
 - j. Using the formula bar, set the **Text** property to `ThisItem.displayName` and the **Color** to `CurrentTheme.TextPrimary`
 - k. Name this label `ChannelNameLbl`

The application should look as in the following screenshot:



Figure 8.19: Channel Manager application

16. From the **Tree view**, select **MainContainer1** and set the properties as follows:
 - Using the properties pane, change the **Align (vertical)** to the last one, Stretch, the **Gap** to 10, the left and right **Padding** to 20, and the top and bottom **Padding** to 10
 - Using the formula bar, set the **Fill** to `CurrentTheme . Background`
17. While still selecting **MainContainer1**, add the following from the **Insert** dropdown:
 - a. Add a **Text Label** control:
 - Using the properties pane, change the **Text** property to `Selected channel`, the **Font** to **Segoe UI**, and the **Font size** to 12
 - Using the formula bar, set the **Color** to `CurrentTheme . TextPrimary`
 - Name this label `ChannelLb1`
 - b. Add a **Text Label** control:
 - Using the properties pane, change the **Font** to **Segoe UI** and the **Font size** to 12
 - Using the formula bar, set the **Text** property to `ChannelGly.Selected.displayName`, the **Color** to `CurrentTheme . TextPrimary`, and the **Fill** to `CurrentTheme . BackgroundAlt`
 - Name this label `ChannelDisplayNameLb1`
 - c. Add a **Text Label** control:
 - Using the properties pane, change the **Text** property to `Subject`, the **Font** to **Segoe UI**, and the **Font size** to 12
 - Using the formula bar, set the **Color** to `CurrentTheme . TextPrimary`
 - Name this label `MessageSubjectLb1`

- d. Add a **Text input** control:
 - Using the properties pane, clear the **Default** property. Change the **Font** to **Segoe UI**, the **Font size** to 12, the **Border** to None, the **Border width** to 0, and the **Border radius** to 0
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`, the **Fill** to `CurrentTheme.BackgroundAlt`, the **HoverColor** to `CurrentTheme.TextPrimary`, and the **HoverFill** to `CurrentTheme.BackgroundAlt`
 - Name this text input `MessageSubjectTxt`
- e. Add a **Text Label** control:
 - Using the properties pane, change the **Text** property to `Message`, the **Font** to **Segoe UI**, and the **Font size** to 12
 - Using the formula bar, set the **Color** to `CurrentTheme.TextPrimary`
 - Name this label `MessageBodyLbl`
- f. Add a **Text input** control:
 - Using the properties pane, clear the **Default** property. Change the **Font** to **Segoe UI**, the **Font size** to 12, the **Mode** to **Multiline**, all **Padding** properties to 10, the **Border** to None, enable **Flexible height**, the **Border width** to 0, and the **Border radius** to 0.
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`, the **Fill** to `CurrentTheme.BackgroundAlt`, the **HoverColor** to `CurrentTheme.TextPrimary`, and the **HoverFill** to `CurrentTheme.BackgroundAlt`
 - Name this text input `MessageBodyTxt`
- g. An **Add icon** control:
 - Using the properties pane, change the **Icon** to **Send**, all **Padding** properties to 10, **Align in container** to the third option, **End**, and the **Minimum Width** and **Height** to 48
 - Using the formula bar, set the **Color** property to `CurrentTheme.TextSecondary` and the **OnSelect** to the code downloaded from the GitHub repository located in the `SendIcon - OnSelect.txt` file
 - Name this icon `SendIcon`

h. A **Text Label** control:

- Using the properties pane, change the **Font** to **Segoe UI**, the **Font size** to **10**, the **Text alignment** to **Right**, the **Minimum Width** to **200**, and the **bottom padding** to **7**
- Using the formula bar, set the **Text** property to "Sent at " & Text(Now()), the **Color** to CurrentTheme.TextSecondary, and the **Visible** property to varConfirmationVisible
- Name this label MessageSentLbl1

18. Select **ChannelGly** and, using the formula bar, set the **OnSelect** property as follows:

```
Reset(MessageSubjectTxt);
Reset(MessageBodyTxt);
UpdateContext(
    {
        varConfirmationVisible: false,
        containerVisible: true
    }
)
```

19. From the main toolbar, click **Save** and then **Publish**. Click on **Back**, and then close the **Studio** tab on the browser. By now, the application should look like the following screenshot:

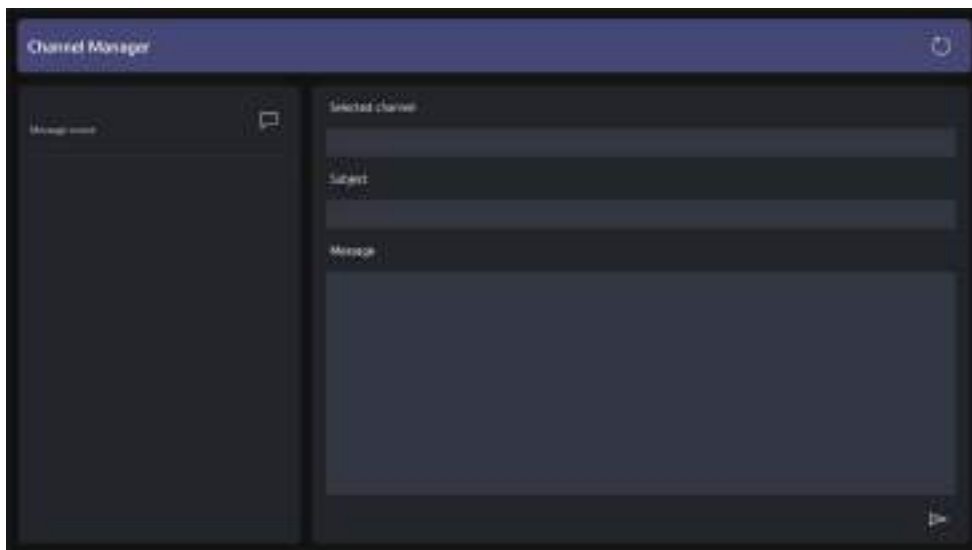


Figure 8.20: Channel Manager application

Microsoft Teams integration

1. Back in the Power Apps maker portal, open your solution, and from the **Apps** list, click on the **Channel Manager** ellipsis. Select **Add to Teams**. This action will open a pane on the right to export the application file. Expand **Advanced settings** if you want to fill in the optional developer information. Click on **Download app** to get the application file to your device, and then click **Close** on the panel.
2. Open the Microsoft Teams application or navigate to <https://teams.microsoft.com/>. Click on **Apps** in the lower section of the application's menu. An internal page of all available applications will open.
3. At the end of the **Apps** list, click on **Manage your apps** and then **Upload an app** on the toolbar, as seen in the following screenshot:

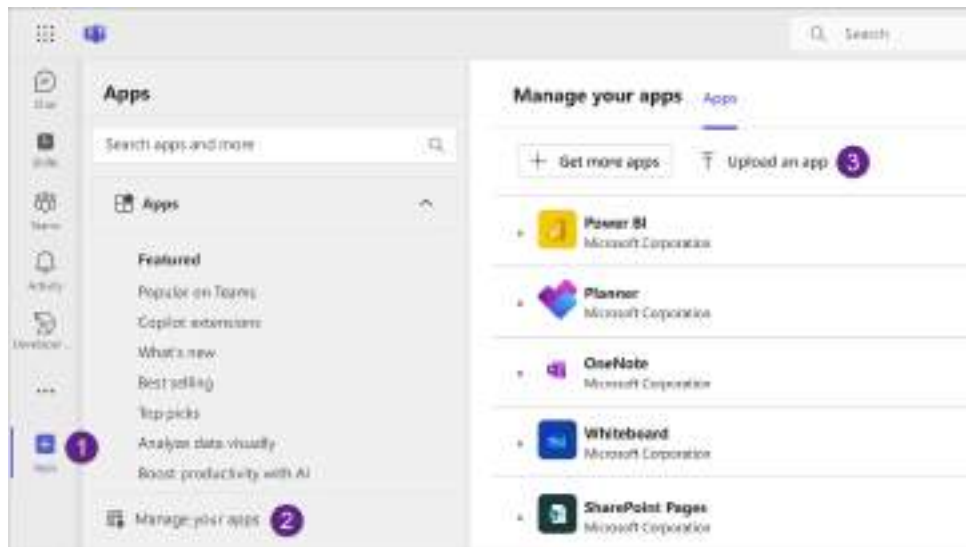


Figure 8.21: Custom application upload

4. On the open dialog, click on **Upload an app to your org's app catalog** (given that you are an admin) and upload the exported file to make the application available for the whole organization.
5. Go to any team you are part of and add a new tab on the **General** channel by clicking on the plus sign at the end of the list of tabs. Use the **Search** field to look for **Channel Manager** in the open dialog. Select it from the list, click **Add** on the application's information dialog, and then click **Save**.

How it works...

Using the Microsoft Teams context parameters, the application can integrate with any Team to query all the channels it contains. With this information and the help of the Teams connector's built-in functions, it can query the number of messages each channel has, as seen in the following screenshot:

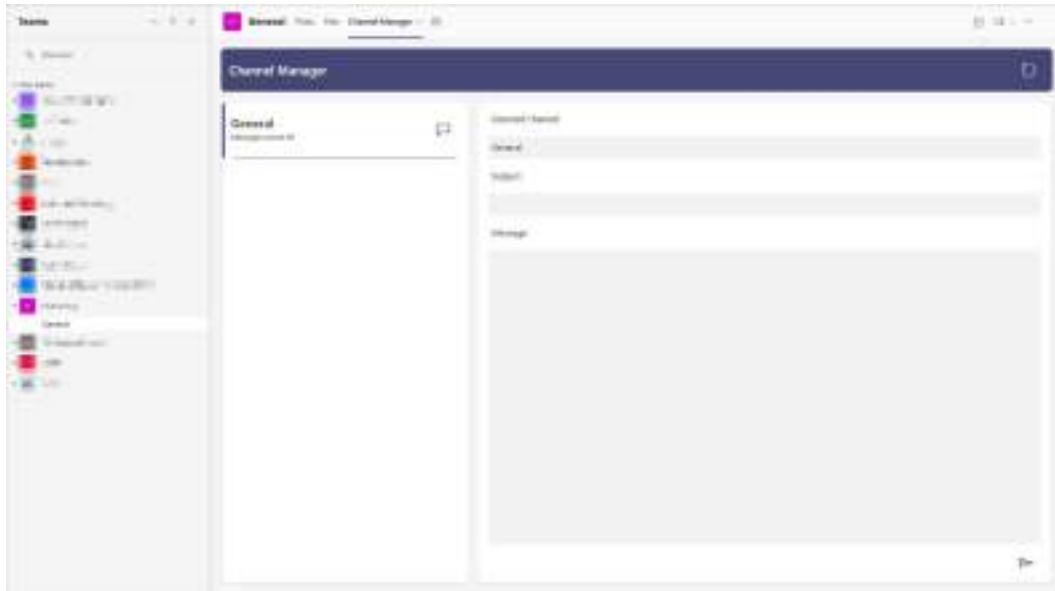


Figure 8.22: Channel Manager in action

Selecting a channel from the gallery allows sending a message to this channel, for example, to let them know they're over the quota on the number of messages.

The context parameters also provide a way for the application to be theme-aware, allowing the use of a matching theme:

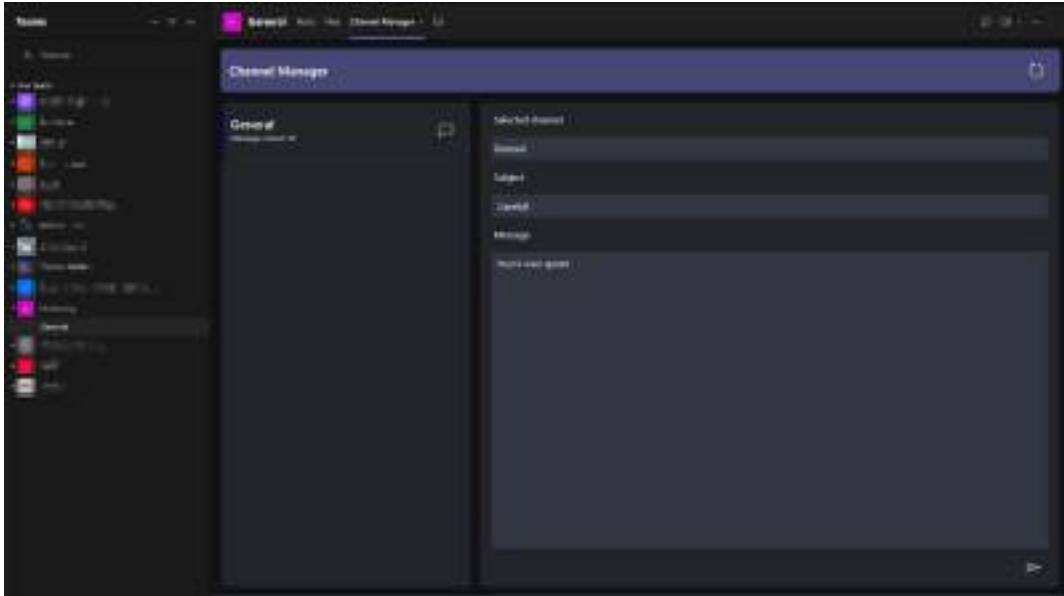


Figure 8.23: Channel Manager using the dark theme

Besides using the Teams integration, this application uses a code reuse technique by setting all the code to **RefreshIcon** and then using it from several app locations using the **Select** function. This method can improve the code maintainability of your applications.

There's more...

Using custom applications in Teams also extends to personal chats. Once a conversation has started, you can add an app deployed in the same way as we did in this recipe or by adding the Power Apps application itself and selecting an application from its list. Remember that this application must have been shared previously with all chat participants.

Integrating Power Apps inside Teams

When a custom application gets deployed to the **Apps** section in Microsoft Teams, it becomes available to the whole platform. You can use them in channels, chats, and even as a standalone application inside the Teams interface by pinning them to the main toolbar.

These methods allow you to configure Teams to match your needs. However, you might need to build an application for your entire organization that is quickly available for all users on the main toolbar without asking them to pin it manually.

We can achieve this using the Microsoft Teams admin portal. We will also learn how to automate the deployment of a Team through an application.

Getting ready

The Teams service admin role is required to manage the Microsoft Teams admin portal. An administrator can grant you this role through the Microsoft 365 admin center: <https://admin.microsoft.com/>.

There is a license requirement for the *Automatic deployment of Teams* section when communicating with the **Graph** API, which uses a premium connector. Please refer to the *Preface* section for more information regarding licensing.

The same cloud flow communicating with **Graph** will authenticate to the Microsoft Entra ID through application permissions defined in an app registration. An administrator can create this app for you and provide you with the required credentials. For more information about this, please refer to <https://learn.microsoft.com/en-us/entra/identity-platform/permissions-consent-overview#types-of-permissions>.

This recipe will use a Cloud flow to automate Teams deployments. At the time of writing, Power Automate does not use a new designer when building a cloud flow from a solution. The workaround is to create it, save it, and then open it again. Please refer to *Chapter 5's Technical requirements* section for more detailed instructions.

The resources needed to build this recipe are available on our GitHub repository, which is located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-08>.

How to do it...

We will configure the organization's policies to pin a previously created app to the Microsoft Teams main toolbar. Then, we will deploy a preconfigured team to the platform.

Getting application identifier

1. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and look for the **Channel Manager** app we created in the previous recipe, *Working with Power Apps in Microsoft Teams channels*.
2. Expand the ellipsis menu to click on **Details**. Copy the **App ID** as we will use it in a Power Automate cloud flow. Return to the **Apps** page.

Configuring Microsoft Teams policies

1. Go to the Microsoft Teams admin portal, <https://admin.teams.microsoft.com/>, and from the left pane, expand **Teams apps** and click on **Manage apps**.
2. On the list toolbar, use the **Search** field to look for Channel Manager. Once located, click on it to open its details. Look for and copy the **App ID**, which we will use in a Power Automate cloud flow.
3. Go to the **Setup policies** option in the **Teams apps** section on the left pane. Click on **Add** on the toolbar to include a new policy. Name it Channel Manager policy and use the following description: Application pinning for channel managers. Leave the rest of the options as default.
4. Click on **Add apps** under the **Installed apps** section. On the open pane, use the **Search by name** field and look for Channel Manager. When found, select it from the list by clicking on **Add**. Click on **Add** at the bottom of the pane.
5. Click on **Add apps** under the **Pinned apps** section. Use the **Search by name** field on the open pane and look for Channel Manager. When found, select it from the list by clicking on **Add**. Click on **Add** at the bottom of the pane.

6. Move it to the desired location in the pinned app list using the **Move up** or **Move down** toolbar options. You can also use drag and drop. We will locate it under the **Teams** option, as seen in the following screenshot:

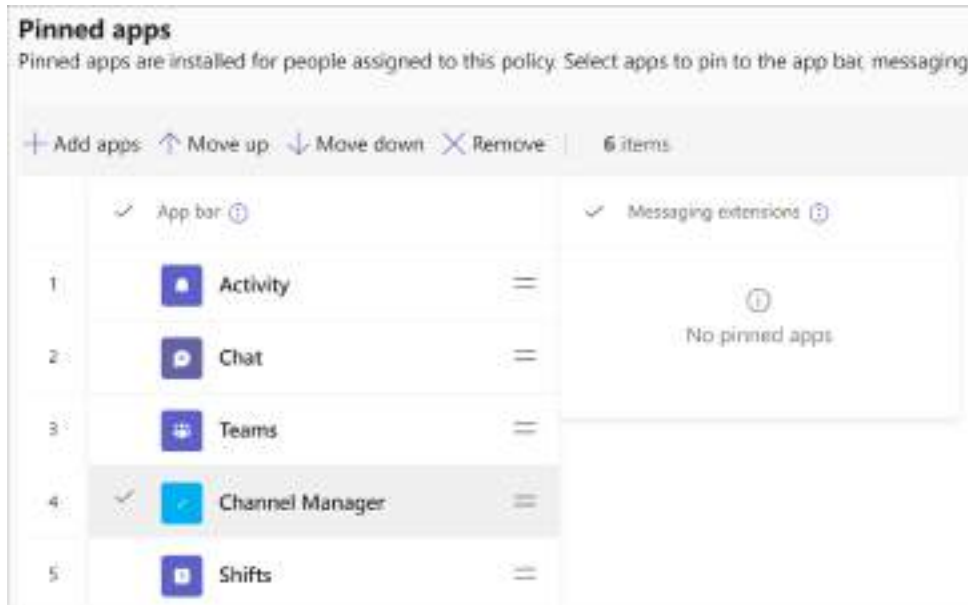


Figure 8.24: Recipe app pinned on Microsoft Teams

7. Click **Save** at the bottom of the screen and then **Save** again in the **User pinning** dialog.
8. Back in the list of policies, select **Channel Manager policy** and, on the toolbar, click on **Manage users | Assign users**. Add yourself to this policy using the **Search by display or username** field. Click on **Apply** to finish this section.

Building a secure app registration

1. Go to the Azure portal and open the Entra blade, or use this URL:
https://portal.azure.com/#blade/Microsoft_AAD_IAM/ActiveDirectoryMenuBlade
2. On the left pane, under **Manage**, select **App registrations**.
3. Click on **New registration**. On the **Register an application** page, enter **Team Access** as the name and leave **Accounts in this organizational directory only** selected. Leave the rest of the options as default and click on **Register**.

4. Once the registration finishes, click on **Certificates & secrets** on the left pane under **Manage**. Click on **New client secret**, and on the open dialog, add **Teams Access** secret as the description, select **730 days (24 months)** for the **Expires** dropdown, and click on **Add**. Copy the secret's value to a safe place, as you won't be able to see it again once you leave this page.
5. Click on **API permissions** on the left pane under **Manage**, and click on **Add a permission**. On the pane that opens on the right, select **Microsoft Graph** and then **Application permissions**.

Using the **Search** field, look for the following permissions and select the checkboxes next to them: `Channel.ReadBasic.All`, `TeamsAppInstallation.ReadWriteForTeam.All`, and `TeamsTab.ReadWrite.All`. Select these permissions, as shown here:

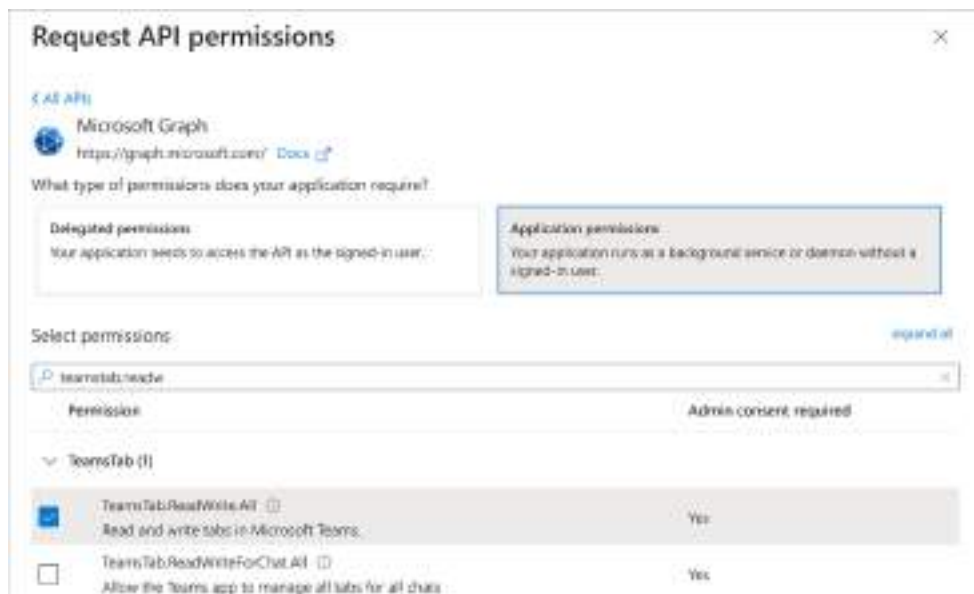


Figure 8.25: Microsoft Graph application permissions

6. Click on **Add permissions** to close the pane. Click on **Grant admin consent for your organization's name** in the permissions list. On the dialog that opens, select **Yes**.
7. Return to the main page by clicking **Overview** on the left pane. Copy the **Application (client) ID** and the **Directory (tenant) ID** from the **Essentials** section. We will use these and the secret to help authenticate the Power Automate cloud flow.

Automatic deployment of Teams

1. Go to the Power Automate portal, <http://make.powerautomate.com/>, and create a solution for this recipe by navigating to **Solutions** and then clicking **+ New solution** from the toolbar. Name the solution **Teams Automation**
2. Inside the solution, click **+ New | Automation | Cloud flow | Instant** from the toolbar. On the open dialog, enter **Team provisioning for** for **Flow name** and select **Manually trigger a flow** from the trigger list. Click **Create** to continue.
3. When the designer opens, click the plus sign to add a new action and look for **Create a team** under **Microsoft Teams**. Enter **Demo Team** for **Team Name** and add a description.
4. Add a new action and look for **Delay** under **Schedule**. Set **15** for **Count** and **Second** for **Unit**.
5. Add another action and look for **HTTP** under **HTTP**. Configure it as follows:
 - Set **URI** to `https://graph.microsoft.com/v1.0/teams/{outputs('Create_a_team')}['body/newTeamId']/primaryChannel`
 - For **Method**, select **GET**
 - Select **Authentication** from the **Advanced parameters** dropdown, then choose **Active Directory OAuth**
 - For **Tenant**, set the **Directory (tenant) ID** obtained in *step 7* of the *Building a secure app registration* section
 - For **Audience**, enter `https://graph.microsoft.com`
 - For **Client ID**, set the **Application (client) ID** obtained in *step 7* of the *Building a secure app registration* section
 - For **Secret**, enter the one collected in *step 4* of the *Building a secure app registration* section

The step configuration should look like the following:

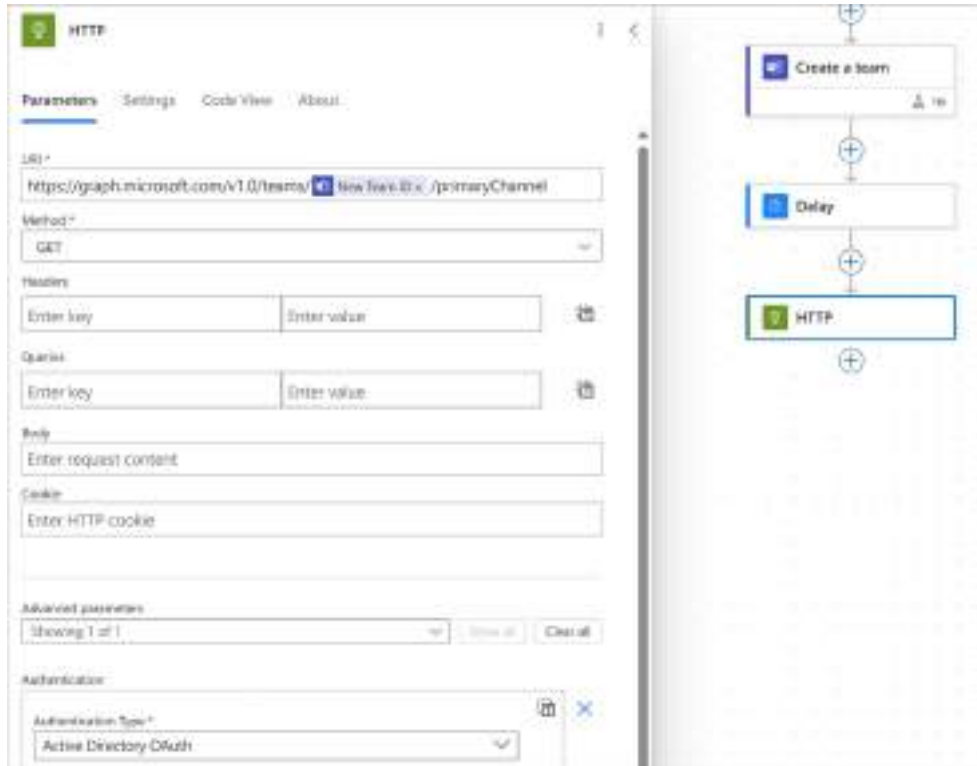


Figure 8.26: Graph request configuration

6. Rename this action to `Get primary channel`. Right-click on it and choose **Copy Action**.
7. Click on the plus sign and select **Paste an action**. Rename the new action as `Install Teams Tab` and set the following parameters:
 - Set **URI** to the following: `https://graph.microsoft.com/v1.0/teams/{outputs('Create_a_team')}['body/newTeamId']/installedApps`
 - Change **Method** to **POST**
 - Set the **Body** to the following:

```
{
  "teamsApp@odata.bind": "https://graph.microsoft.com/v1.0/appCatalogs/teamsApps/<TEAMS_APP_ID>"
}
```


- Replace <TEAMS_APP_ID> with the app ID collected in *step 4* of the *Configuring Microsoft Teams policies* section
 - Right-click on it and choose **Copy Action**
8. Click on the plus sign and select **Paste an action**. Rename the new action **Add Teams Tab** and set the following parameters:
- Set **URI** to the following: `https://graph.microsoft.com/v1.0/teams/{outputs('Create_a_team')}['body/newTeamId']}/channels/{body('get_primary_channel')}['id']}/tabs`
 - Change **Method** to **POST**.
 - For **Body**, insert the **Add Teams Tab.txt** file's contents from the GitHub repository. Replace <TEAMS_APP_ID> with the app ID collected in *step 4* of the *Configuring Microsoft Teams policies* section and replace <POWERAPPS_APP_ID> with the app ID collected in *step 2* of the *Getting application identifier* section.
9. Click on **Save draft** and then **Publish** on the Power Automate toolbar.

How it works...

When you run the cloud flow, it will create a demo team with the **Channel Manager** application installed. Then, it will add a new tab to the **General** channel, making the application available to the users, as seen in the following screenshot:

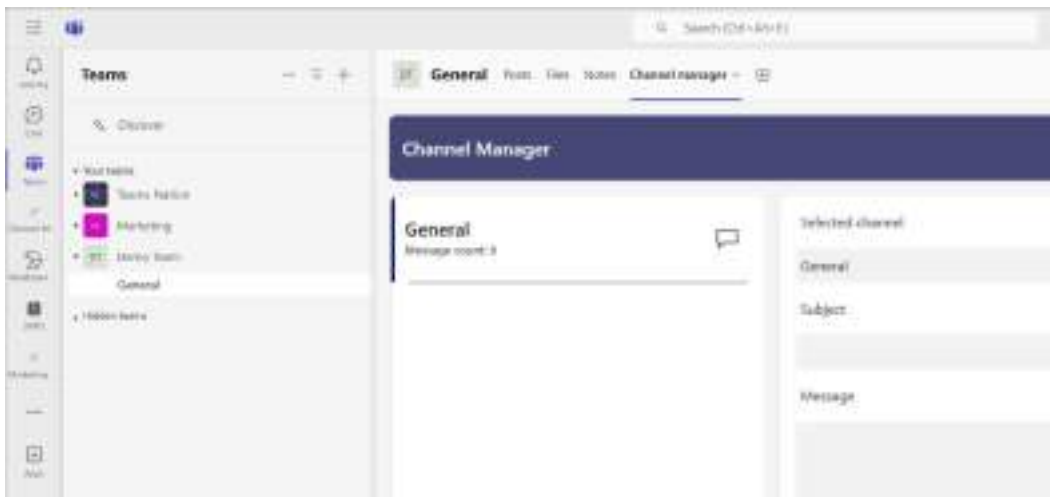


Figure 8.27: Demo team with the Channel Manager application installed

The app registration is needed to communicate with **Graph**. This API allows the programmatic handling of resources and services in the Microsoft 365 ecosystem. For more information, please refer to <https://learn.microsoft.com/en-us/graph/overview>.

The Teams policies allow setting rules for Microsoft Teams from a central location using configurations for specific users. To modify these rules for all users in the organization, you can edit the **Global (Org-wide default)** policy. Keep in mind that these policies might take a few hours to apply the changes. Once applied, you will see the changes, as shown in the following screenshot:

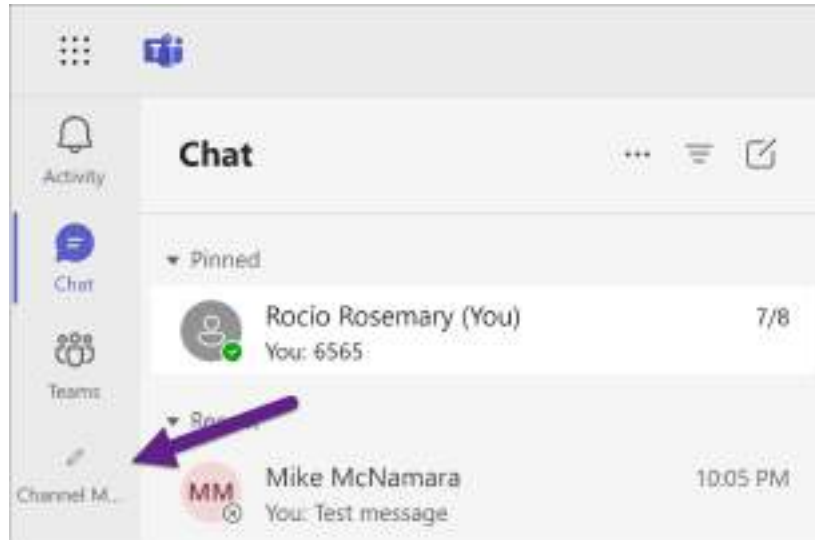


Figure 8.28: Teams policies applied

Finally, adding the delay to the cloud flow is required to give time for the platform to build the team completely. The concepts explained in this recipe can be used as a template to create more complex solutions when automating Microsoft Teams creation and configuration in your organization.

There's more...

When communicating with Graph from Power Platform, two connectors offer this capability without requiring a premium action, such as sending an HTTP request.

However, these actions are limited depending on the connector. The Microsoft Teams connector includes the Send a Microsoft Graph HTTP request action, which allows sending requests using these paths at the first level, /teams, /me, /users, and these at the second level, channels, chats, installedApps, messages, pinnedMessages

Here is an example: <https://graph.microsoft.com/v1.0/teams/channels>.

Alternatively, we can find Send an HTTP request V2 from the Office 365 Groups connector that only allows sending requests to the /groups level.

Remember that these connectors offer these options to avoid licensing costs by sending Graph requests, for example, from Power Automate.

For reference, please visit these links:

- Microsoft Teams connector: <https://learn.microsoft.com/en-us/connectors/teams/#send-a-microsoft-graph-http-request>
- Office 365 Groups connector: <https://learn.microsoft.com/en-us/connectors/office365groups/#send-an-http-request-v2>

Building apps with Dataverse for Teams

Throughout this chapter, there have been many interactions with Microsoft Teams, from embedding tabs to integrating directly into the Teams interface.

Microsoft also offers a way to build apps for this platform by introducing **Dataverse for Teams**. This approach allows the creation of applications inside a team that can be used through desktop and mobile devices. This team acts as an environment, just like the regular environments we have seen throughout the book.

Dataverse for Teams builds upon the idea of having several components that work together inside a team environment, including a built-in *light* version of Dataverse, Power Automate cloud flows, canvas apps, chatbots, and more. You'll find more information regarding this topic here: <https://learn.microsoft.com/en-us/power-apps/teams/overview-data-platform>.

This recipe will create an application using this new approach for building apps for Microsoft Teams.

Getting ready

The licensing model for Dataverse for Teams is simple: if users can access Teams, they can access apps created by this platform. As it already includes the data source part, there is no need for premium data connectors.

Keep in mind that if you need to use a premium connector to access other data sources, you and your users will require a premium connector. However, if this connection comes from a custom connector built by API Management, it will be included in the licensing for Dataverse for Teams.

More information on this regard can be found at <https://learn.microsoft.com/en-us/power-platform/admin/powerapps-flow-licensing-faq#are-the-existing-power-platform-use-rights-included-with-microsoft-365-licenses-changing>.

However, there are some limitations regarding the data source compared to the regular version of Dataverse, such as storage capacity or integrations with other systems. To get more information regarding this, please refer to <https://learn.microsoft.com/en-us/power-apps/teams/data-platform-compare>.

The resources needed to build this recipe are available at our GitHub repository located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-08>.

How to do it...

1. Start your Teams desktop application or navigate, using a modern browser, to <https://teams.microsoft.com/>.
2. From the platform's main toolbar, click on the ellipsis and, using the **Search for apps** field, search for Power Apps, as seen in the following screenshot:

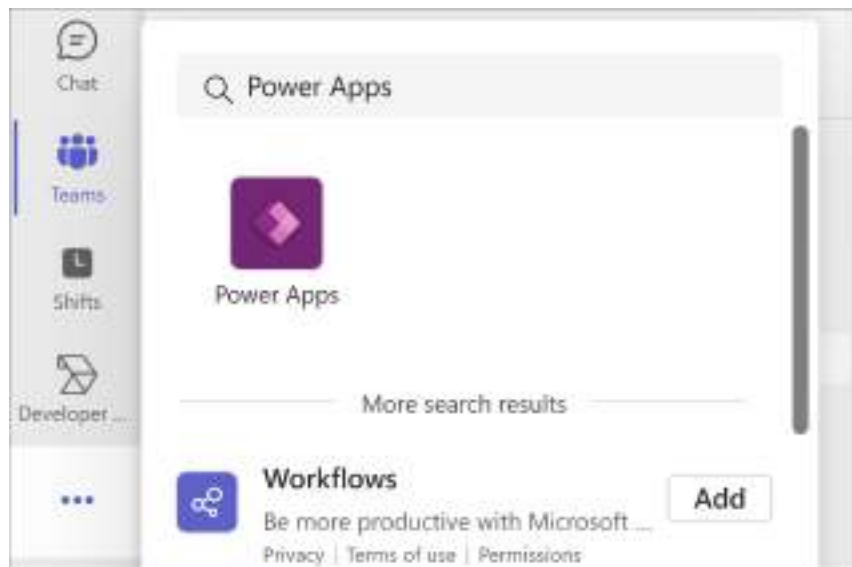


Figure 8.29: Power Apps application for Microsoft Teams

3. Click on the dialog that opens. The application now appears on the main toolbar. For easier access, right-click on its icon and select **Pin**.

4. On the Power Apps application, click on **Start now**. This action will prompt us to select a team where the application will be added. Select any of your teams and click on **Create**.



If this is the first time creating an app for this team, the process might take a while because it needs to provision this team's environment.

5. When the environment completes its configuration, Power Apps Studio will open, asking for the app's name. Enter `Time Tracking` and click **Save**.
6. Select the data panel on the left pane and click **Create new table**. On the dialog that opens, click on **Start with a blank table**.
7. Click on the pen near the text **New table**, enter `Time` entry for the **Display name**, and click **Save**. On the table designer, click on the **New column**, and from the drop-down menu, select **Edit column**. Enter `Title` for the **Display name** and click on **Update**.
8. From the toolbar, click the plus sign at the end of the column's header and include the following columns; when finished, click **Save and close**:

Name	Type
Description	Text
Hours	Number Decimal
Billable	Yes/No (Default Yes)

Table 8.2: Project table structure

9. Still in the data source panel, expand **Add data** and click **Create new table**. Repeat *steps 6* and *7* using `Project` as the table name and `Project` name for the first column.
10. Click on the plus sign at the end of the column's header and include the following columns:

Name	Type
Description	Text
Project start	Date and time Date only
Project end	Date and time Date only

Table 8.3: Project table structure

11. Select the first row and enter random data, just like when entering data in Excel. Use the following screenshot as an example:



Project name	Description	Project start	Project end
Technical support	On-site tech support	7/1/2024	12/31/2024
SharePoint migration	On-premises migration	9/1/2024	9/14/2024
Development	Low-code development	10/1/2024	10/4/2024
Enter text	Enter text	Enter or pick date	Enter or pick date

Figure 8.30: Project table data

Click **Save and close** when finished.

12. Click the **Tree view** on the left pane and select the **App** element. Using the formula bar, select the **OnStart** property and add the code from the App - OnStart .txt file available from the GitHub repository. Right-click on the **App** element's ellipsis to execute this code, as seen in the following screenshot:

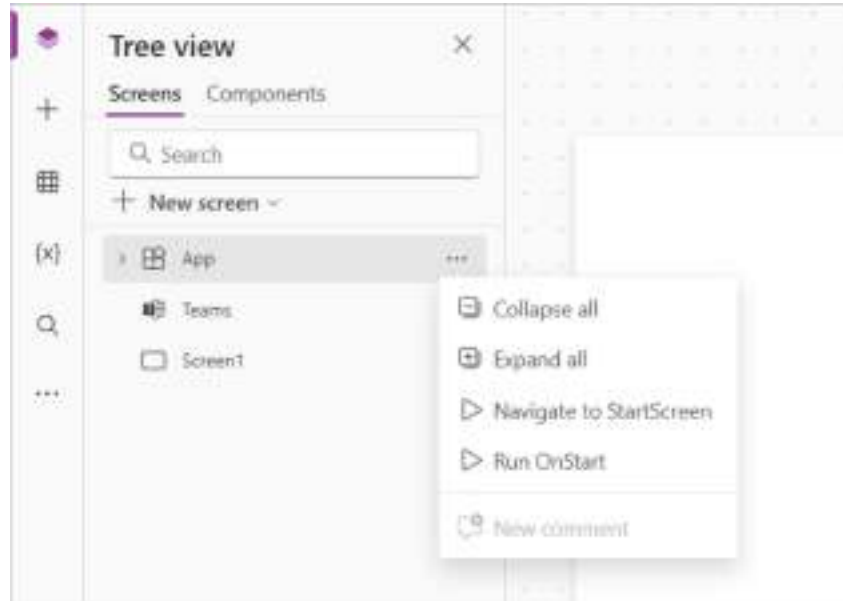


Figure 8.31: OnStart execution

13. From **Tree view**, select the ellipsis next to **Screen1** to rename it **Main Screen**
14. On the application canvas, click on **Start this screen... | With data**. Select **Time entries** from the list of tables. A form will be created from the table's metadata.
15. Expand **MainScreen | RightContainer1** and select **EditForm1** (keep in mind that the number might be different). On the properties pane on the right, set the following configuration:
 - Click on **Edit fields** and, using the ellipsis option on each field, reorder them as follows: **Title**, **Description**, **Billable**, and **Hours**. You can also use drag and drop.
 - Change the **Columns** property to 2
16. On the application canvas, select the **Title** card and drag its width to the screen's end. This action will expand the other cards as well.
17. Select the **Description** card and, using the formula bar, set the **Height** property to `(App.Height - Billable_DataCard1.Height) * 0.30`
18. Select the **DataCardValue2** text box inside **Description_DataCard1**. Using the formula bar, change the **Height** property to `Parent.Height - Self.Y - 20` and the **Mode** to `'TextBox.Mode'.Multiline`
19. Expand each data card and select each **DataCardValue** control to change its **Font size** property to 12 using the properties pane.
20. Select **MainScreen** from **Tree view**, and using the formula bar, set the **Fill** property to `CurrentTheme.Background`.
21. Using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary` for the following controls:
 - All **DataCardKey** labels inside **RightContainer1 | EditForm1**
 - **RightContainer1 | DeleteConfirmOverlay1 | DeleteText1**
 - **LeftContainer1 | BrowseGallery1 | Title1**
 - **LeftContainer1 | LeftControlContainer1 | IconButton_Add1 | IconButton_Add_Label1** and **IconButton_Add_Icon1**
22. Using the properties pane, change the **Color** property to `Color.White` for the following controls:
 - Apply this change to all icons inside **RightContainer1 | TopBar_RightContainer1**. You can use the *Shift* key to select and change their properties simultaneously.
 - Also, apply it to **LeftContainer1 | TopBar_LeftContainer1 | AppNameLabel1**.

23. Using the formula bar, set the **HoverColor** property for all icons inside **RightContainer1** | **TopBar_RightContainer1** to `ColorFade(Self.Color, -30%)`.
24. Using the formula bar, change the **Color** property of the **Subtitle1** label inside **LeftContainer1** | **BrowseGallery1** to `CurrentTheme.TextSecondary`
25. Using the formula bar, set the **Fill** property to `CurrentTheme.MainColor` for the following controls:
 - **RightContainer1** | **TopBar_RightContainer1**
 - **LeftContainer1** | **TopBar_LeftContainer1**
26. Select the **TopBottomDivider1** rectangle inside **RightContainer1** and delete it.
27. Select the **LeftRightDivider1** rectangle and, using the formula bar, change the **Fill** property to `CurrentTheme.Divider`, set the **Y** property to `TopBar_LeftContainer1.Height`, and set the **Height** property to `Parent.Height - Self.Y`
28. Select **Image1** inside **BrowseGallery1** and update the **Image** property using the formula bar by pasting the contents of the `Icon.txt` file downloaded from our GitHub repository.

After adding some test data, the application should look as shown in the following screenshot:

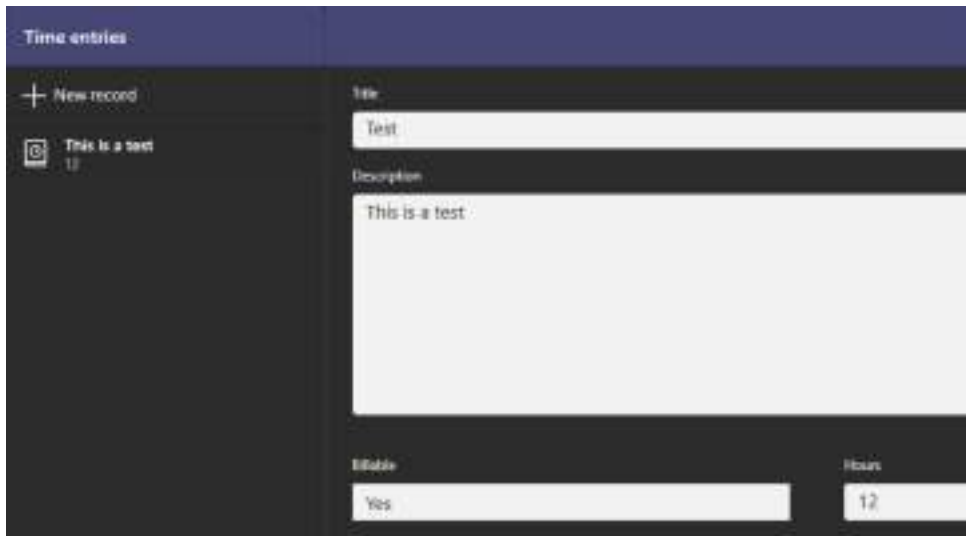


Figure 8.32: Time tracking app using Teams' dark mode

Now that the application design is complete, let's see how to modify the data source.

29. Go to the **Data** section on the left pane, click the ellipsis next to the **Time entries** table, and select **Edit data**.
30. Click on the plus sign at the end of the column headers. Set the name of the new column to **Project** and the type to **Lookup**. On the **Related table** dropdown, choose the **Project** table and click **Save**. Click **Close** to finish the table edits. Still in the **Data** section, click the three dots next to **Time entries** and select **Refresh**.
31. Go back to **Tree view** and select **EditForm1**. On the properties pane, click **Edit fields** and then **Add field** at the top of the **Fields** panel. Click on the check box next to the **Project** field and click on **Add**.
32. Using drag and drop, move the **Project** field until it is below the **Description** field. Close the **Fields** panel.
33. Inside **Project_DataCard1**, select **DataCardKey** and, using the formula bar, set the **Color** property to `CurrentTheme.TextPrimary`. Select **DataCardValue** and set the **Font size** to 12 using the properties pane.
34. On the top toolbar, click on **Save** and then **Publish to Teams**. On the dialog that opens, click on **Next**. The following dialog will display a list of the current team's channels to add the application as a tab. Click the plus sign next to the desired channel, then click **Save and close**.

How it works...

The Dataverse for Teams app template is responsive by default, easing the application-building process by giving us a preconfigured canvas for desktop and mobile devices. Besides this, the **Start this screen** option automatically creates the gallery and form controls for the desired data source.

Adding more data sources or applying a design to the application is all it takes to have a complete solution like the one shown in the following screenshot:

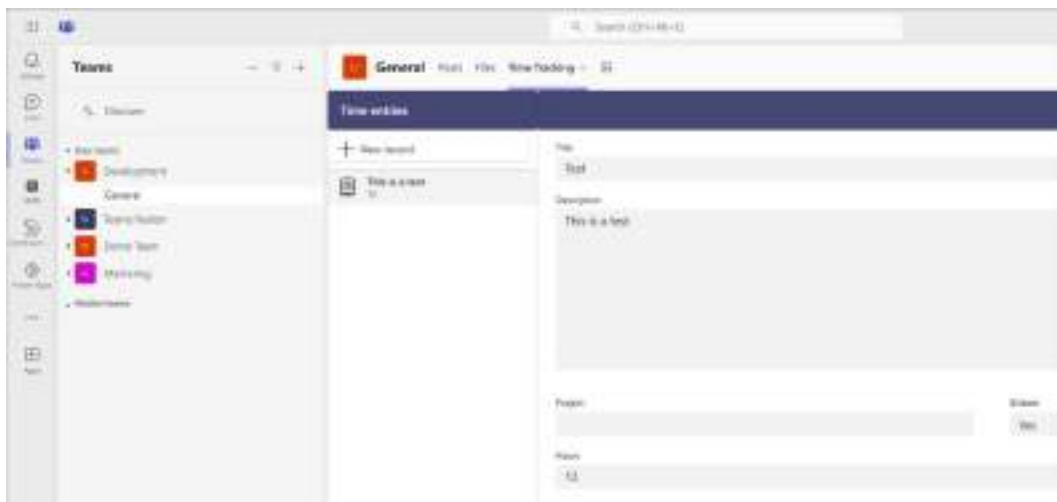


Figure 8.33: Dataverse for Teams as a tab

Using this application on the Microsoft Teams mobile client only requires navigating to the Teams channel and then clicking on the **Apps** tab to see the application in the list of tabs available. The following is the application as seen on a mobile device with the dark mode enabled:

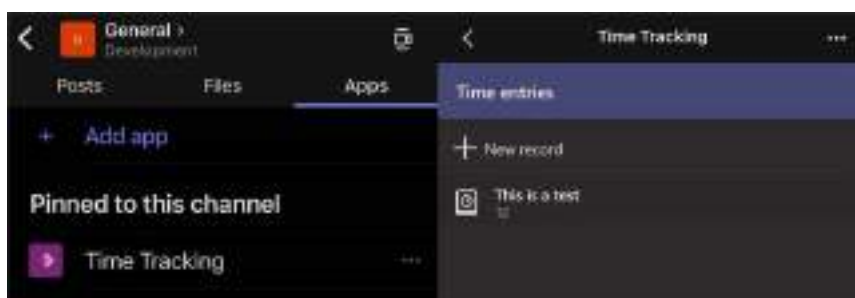


Figure 8.34: Dataverse for Teams on a mobile client

If you need to build a responsive solution that works on every device and can exist within the boundaries of the Dataverse for Teams limitations, such as 1 million rows or the lack of some column data types, this type of canvas app is a great solution.

There's more...

Dataverse for Teams also incorporates the ability to include Power Automate cloud flows and chatbots, so you can add more enhancements when you build a solution.

Also, when working on this platform, you can get more configuration options by clicking on the **Build** tab. Then, inside the list of items for a team, click **See all**. You will see the whole list of elements to build applications, as seen in the following screenshot:

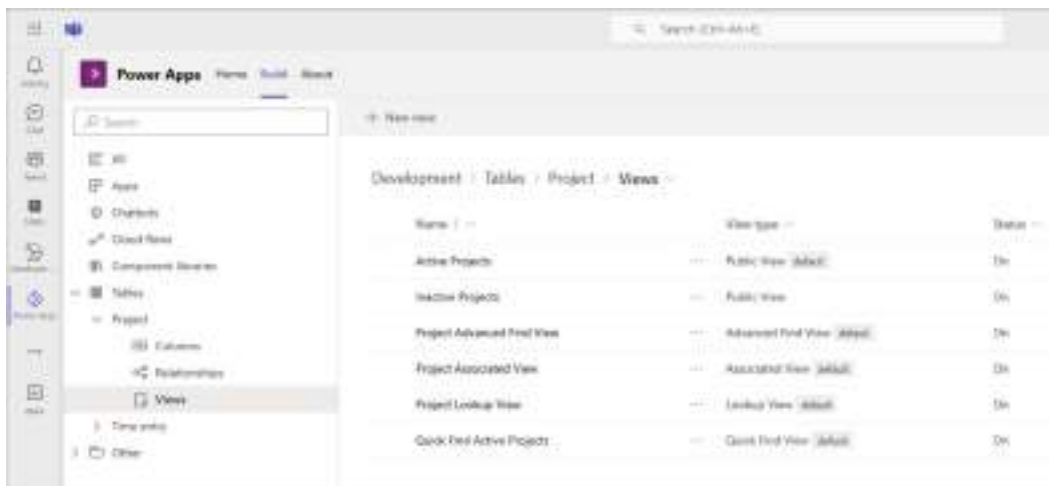


Figure 8.35: Dataverse for Teams solution elements

Going down this route allows you to configure a table's views or relationships, as these options are unavailable from the data source editor inside the canvas studio of Dataverse for Teams.

While working with Dataverse for Teams inside the Microsoft Teams desktop client, you might get notifications about messages or video calls that might interfere with your development session. As a recommendation, always use the **Open in new window** option by right-clicking on the Power Apps icon, as seen in the following screenshot:

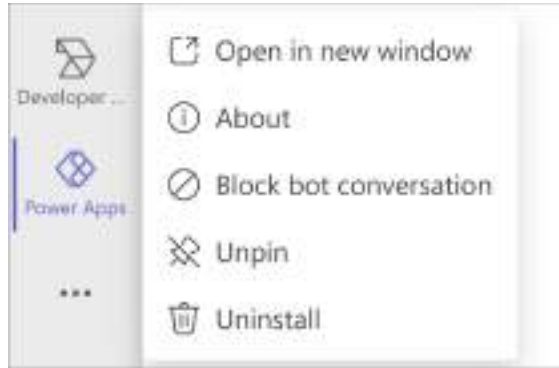


Figure 8.36: Pop-out app option

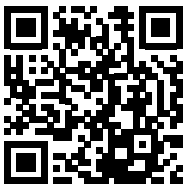
This way, you will get an independent Microsoft Teams window, allowing you to communicate with your peers easily while building your solutions.

Finally, it might be interesting to consider Microsoft’s guidelines on deploying modern applications for the Microsoft Teams platform. Visit this link to review them: <https://learn.microsoft.com/en-us/power-apps/teams/how-templates-built>.

Join our community on Discord

Join our community’s Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



9

Empowering your Applications with AI Builder

Artificial Intelligence (AI) describes when computers perform tasks such as problem-solving or learning using capabilities commonly found in intelligent beings. We can use the techniques found in AI to enhance business processes, seeing as that is our primary task when using Power Platform.

AI Builder is a Power Platform add-on that brings these techniques to the low-code platform with custom models like document processing and object detection and prebuilt models like text recognition and language detection. The model type gets chosen depending on the business problem it aims to solve.

Besides custom and prebuilt models, AI Builder offers capabilities through GPT prompts to help makers fulfill requirements such as summarizing or creating a response based on received content.

This chapter will use some of these models and prompt capabilities to solve specific business problems using the following recipes:

- Creating a customer success solution using sentiment analysis
- Building a text recognition system using canvas apps
- Gathering insights from data with the Copilot Answer component
- Using Power Automate to create an invoice classification system

Technical requirements

Regarding licensing, we can use included AI credits from regular Power Apps or Power Automate Premium licenses. Still, there is another licensing model that is different from the one we have come to know. A capacity add-on option exists, which can be purchased by an administrator to be allocated to the environment where AI Builder is required.

This add-on includes 1 million service credits. To get a better estimate on using these credits, Microsoft provides comprehensive documentation regarding capabilities, usage, and monitoring, among others. Visit this link for more information: <https://learn.microsoft.com/en-us/ai-builder/credit-management#number-of-ai-builder-credits-by-license>.

There is also a 30-day trial license that allows the usage of these features without requiring the capacity add-on or a standalone license. For more information, please refer to <https://aka.ms/tryaibuilder>.

You will find the resources from the recipes of this chapter in our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-09>.

Creating a customer success solution using sentiment analysis

Building solutions for business departments is an essential task for Power Platform makers. Our goal is to find manual procedures that need automation.

In this recipe, we will build an application to help the marketing department prioritize customer complaints with the help of AI Builder's sentiment analysis and then use the AIReply capability to create an appropriate response.

We will mix AI Builder's prebuilt capabilities to build our solution. A model for the sentiment analysis and a prompt will help create the response. To get more information about models, visit <https://learn.microsoft.com/en-us/ai-builder/model-types>, and to review available prompts in AI Builder, please navigate to <https://learn.microsoft.com/en-us/ai-builder/prebuilt-prompts>.

Getting ready

As mentioned in the *Technical requirements* section of this chapter, we will need to have AI Builder capacity either through Power Platform licensing, the capacity add-on, or simply by requesting a trial.

For this recipe, we will need a SharePoint list and a Microsoft form. Please refer to *Chapter 1, App Maker Basics* for instructions on how to build them.

How to do it...

This recipe will use a SharePoint list to store the customer complaints and a canvas app to manage them.

Creating the SharePoint list

1. Go to the homepage of your desired SharePoint site and create a list with the name of Customer complaints.
2. On the new list, add each of the following columns:

Column data type	Column name
Date and time	Date
Multiple lines of text	Message
Multiple lines of text	Response link

Table 9.1: Customer stories list columns

3. Rename the **Title** column to Customer name by clicking on it and then clicking **Columns settings | Rename**.

The new list should look like the following:

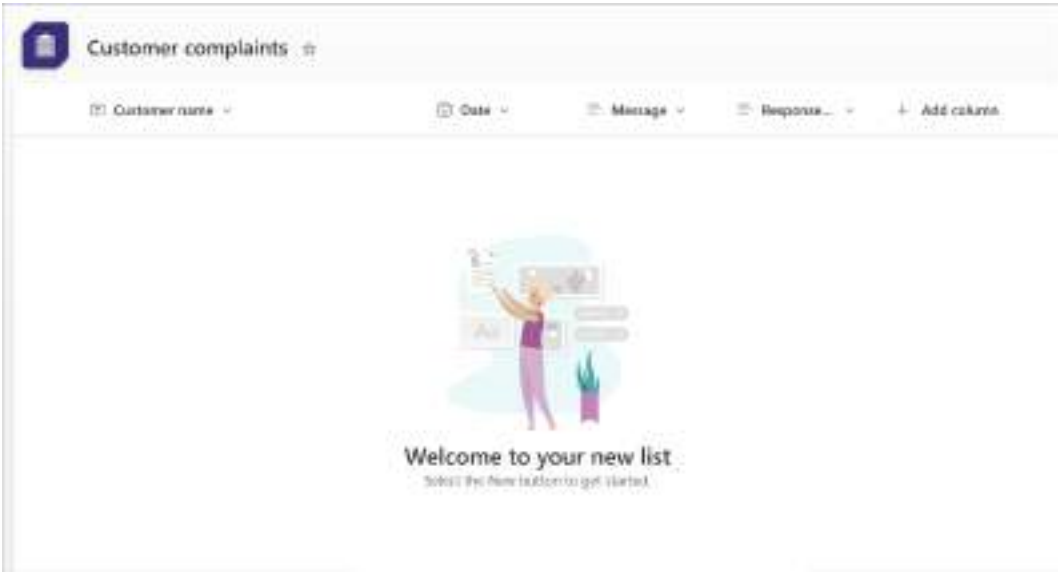


Figure 9.1: Customer stories list columns

The SharePoint data source is already complete. We will build a Microsoft form to receive the complaints.

Building the complaints form

1. Navigate to Microsoft Forms by typing this address: `https://forms.office.com`. Then, once there, click on the **New Form** button. This action will open the designer for you to start setting up the form.
2. Click on the **Untitled form** text to define the name, description, and image. You can also click **Style** on the toolbar to change to one of the built-in themes.
3. Create the following fields:

Field type	Required	Text
Text	Yes	Customer name
Text (Long answer)	Yes	Please describe your complaint

Table 9.2: Form fields configuration

4. Click on **Preview** on the toolbar and then on **Start now**. You should have a form just like the following:

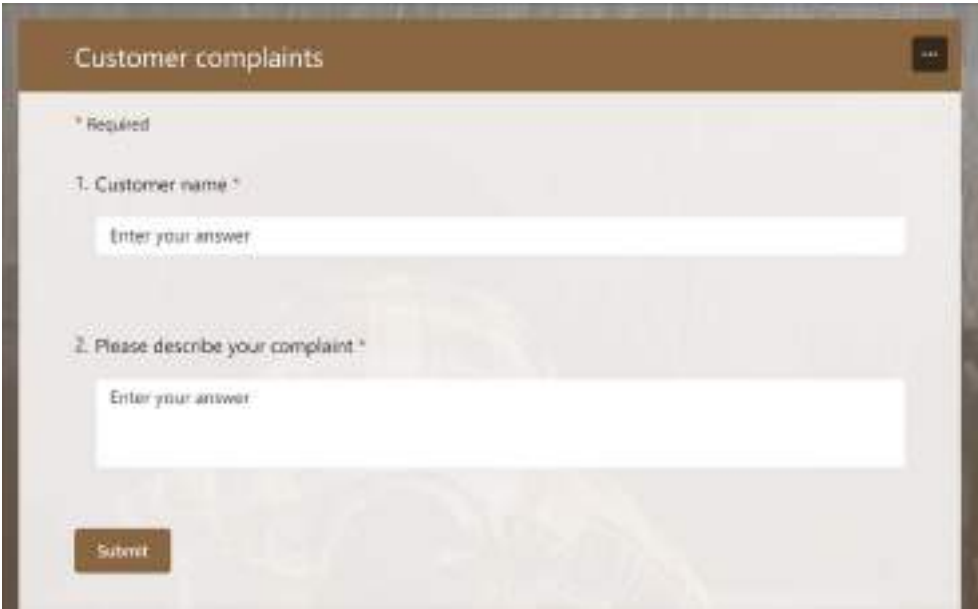


Figure 9.2: Customer complaints preview

Building the cloud flow

1. Go to the Power Automate portal at <https://make.powerautomate.com> and create a solution for this recipe by navigating to **Solutions** and then clicking + **New solution** from the toolbar. Name it **Complaints management**
2. From the toolbar, click + **New | Automation | Cloud flow | Automated**. Give it a name, such as **Complaints handler**, search for the trigger **When a new response is submitted on Microsoft Forms**, and then click **Create**.
3. When the designer opens, you will need to select the form from which we expect answers. Click on the first action of the designer and then choose the one you just created from the left pane.
4. The next step will be to get the actual response data by choosing the **Get response details** action using the previous step's result as the **Response Id** data.
5. Add a new action and look for **Create item on SharePoint**. Select the site and list you created and set the following properties:
 - **Customer name** for **Title**
 - Using the formula editor, set `utcnow()` for **Date**
 - Enter the following formula for **Response Link**:

```
concat(  
  'https://forms.microsoft.com/Pages/DesignPageV2.  
  aspx?id=', triggerOutputs()?['body/resourceData/formId'],  
  '&subpage=design&analysis=true&topview=SurveyResults&ridx='  
  sub(triggerOutputs()?['body/resourceData/responseId'],1)  
)
```

6. From the main toolbar, click **Save draft** and then **Publish**. Close the designer and go back to the solution.
7. Use the form to add random complaints to build our application.

Building the canvas app

1. Back in the solution, from the toolbar, click + **New | App | Canvas app**. Set the name as **Complaints manager** and click **Create**.
2. Once Power Apps Studio opens, click **New screen** from the toolbar and select **Sidebar**. Click the ellipsis next to **Screen1** and choose **Delete**. Rename **Screen2** to **MainScreen**.

3. Open the Data pane on the left and click + **Add data**. From the **Select data source** drop-down, search for SharePoint and select it. Select a previously created connection or create one, select the site and list from before, and click **Connect**.
4. Click on + **Add data** again, and this time, search for Sentiment analysis and select it.
5. Click again on + **Add data**, and this time, search for Environment and select it.
6. From the **Tree view**, expand **ScreenContainer1** and select **HeaderContainer1**. Set the properties as follows:
 - Set the last option for **Align (vertical)**, which is called **Stretch**.
 - Using the formula bar, set the **Fill** property to `RGBA(130, 141, 158, 1)`
7. Click the plus sign inside **HeaderContainer1** and choose **Text label**. Set the properties as follows:
 - Set **Text** to `Complaints manager`, the **Font** to **Segoe UI**, the **Font size** to 24, the **Font weight** to **Semibold**, enable **Flexible width**, set the **Left Padding** to 20, and the **Color** to white
8. Expand **BottomContainer1** from the **Tree view** and select **SidebarContainer1**. From the properties pane, set the last option for **Align (horizontal)**, which is called **Stretch**.
9. Click the plus sign inside **SidebarContainer1** and choose **Vertical gallery**. Set the properties of the gallery as follows:
 - Change the **Items** property of the gallery to **Customer complaints**, the **Layout** to **Title and subtitle**, reset the **Align in container** to **Set by container**, and enable **Flexible height**.
 - Using the formula bar, set the **Items** property to the following:


```
Sort(
    AddColumns(
        'Customer complaints',
        Confidence,
        'Sentiment analysis'.Predict(ThisRecord.Message)
    ),
    ThisRecord.Confidence.Document.TopSentiment.Confidence,
    SortOrder.Descending
)
```
 - Rename the gallery to `ComplaintsGly`

10. From the **Tree view**, select **MainContainer1** and set the properties as follows:
 - Using the properties pane, change the **Aling (horizontal)** to the last one, **Stretch**, the **Gap** to 20, the left and right **Padding** to 20, and the top and bottom **Padding** to 10
11. While still selecting **MainContainer1**, add the following from the **Insert** dropdown:
 - a. A **Text Label** control:
 - Using the properties pane, change the **Text** property to Complaint, the **Font** to **Segoe UI**, and the **Font size** to 12
 - Name this label ComplaintLbl1
 - b. A **Text Label** control:
 - Using the properties pane, change the **Font** to **Segoe UI**, the **Font size** to 12, and the **Vertical align** to **Top**
 - Using the formula bar, set the **Text** property to `ComplaintsGly.Selected.Message`
 - Name this label MessageLbl1
 - c. A **Text Label** control:
 - Using the properties pane, change the **Text** property to Subject, the **Font** to **Segoe UI**, and the **Font size** to 12, and enable **Flexible height**.
 - Using the formula bar, set the **Color** to `CurrentTheme.TextPrimary`
 - Name this label MessageSubjectLbl1
 - d. A **Text Label** control:
 - Using the properties pane, change the **Text** to Proposed response, the **Font** to **Segoe UI**, and the **Font size** to 12
 - Name this label ResponseLbl1
 - e. A **Text input** control:
 - Using the properties pane, change the **Font** to **Segoe UI**, the **Font size** to 12, the **Mode** to **Multiline**, all **Padding** properties to 10, enable **Flexible height**, and the **Border radius** to 0

- Using the formula bar, set **BorderColor** to `RGBA(219, 219, 219, 1)`, **Default** to `varReply.PreparedResponse` (don't mind the error), **HoverBorderColor** to `Self.BorderColor`, **HoverColor** and **HoverFill** to `Self.Fill`, and **FocusedBorderThickness** to 1
 - Name this text input `ResponseTxt`
- f. A **Button** control:
- Using the properties pane, set the **Text** to `Propose response`, the **Font** to **Segoe UI**, the **Font size** to 14, and the **Border radius** to 6
 - Using the formula bar, set the **Fill** to `RGBA(130, 141, 158, 1)`
12. From the **Insert** panel on the left, click on **Text label** under **Popular**:
- Leave the **Y** property as 40. Set the **Text** property to `Full Name`, the **Font** to **Lato**, and the **Font size** to 12. Change the left padding to 0
 - Using the formula bar, set its **X** value to `App.Width / 2.5 + 80`
 - Name this label `FullNameLbl`

How it works...

This application serves as a tool for integrating services available from Microsoft 365 and Power Platform to build a solution that includes AI capabilities into the mix.

When a customer files a complaint through Microsoft Forms, the cloud flow takes the responses into Dataverse to continue processing the complaints with the help of the canvas app. From there, we can help the representatives to compose the most proper response based on the complaint.

Regarding AI, the application uses two components: the prebuilt AI model for sentiment analysis, which orders the list of complaints based on the confidence level received from the model, and the PowerFx function called `AIReply`, which helps propose an appropriate response for the selected complaint.



The confidence score indicates the accuracy of the analysis, ranging from 0 to 1. This score is essential for sentiment analysis due to aspects of language that are difficult to judge accurately, such as double negatives.

The complete version of the application should look like the following figure:

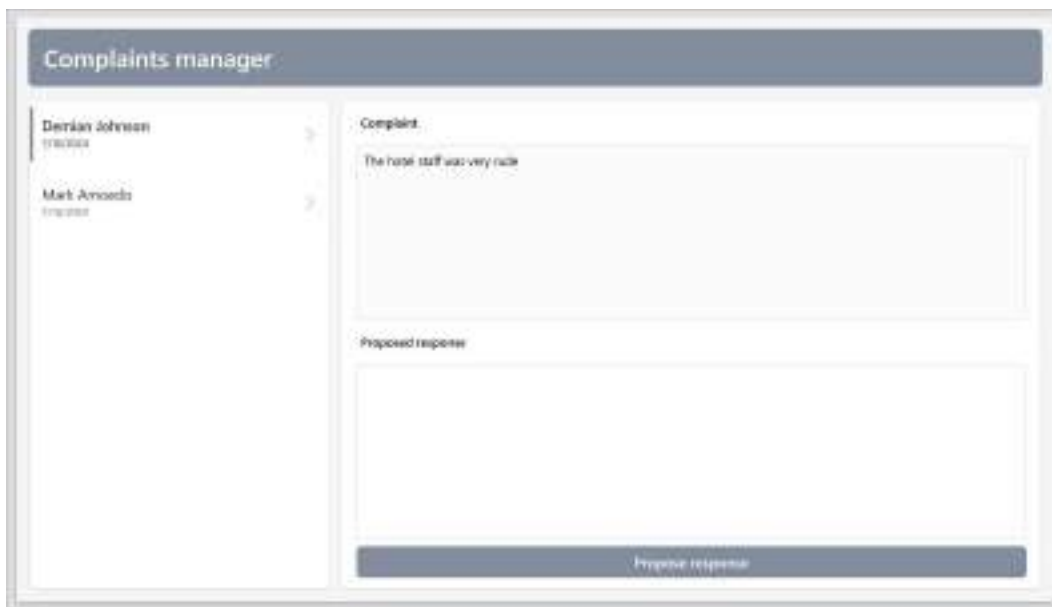


Figure 9.3: Customer complaints preview

These AI components are suitable for this recipe, but you can also utilize many other prebuilt models offered by AI Builder, such as the ones found here: <https://learn.microsoft.com/en-us/ai-builder/prebuilt-overview>. The same goes for the AI functions available for PowerFx. AIReply is just one example, so please review all the available options from this link to get more information: <https://learn.microsoft.com/en-us/power-platform/power-fx/reference/function-ai>.

To get a better understanding of the pricing of AI Builder, please review Power Platform Licensing Guide, where you will find the AI Builder rate card: <https://go.microsoft.com/fwlink/?linkid=2085130>.

Building a text recognition system using canvas apps

Document processing is an excellent tool for the data extraction of structured documents using a trained model. Such documents can be forms, invoices, quotes, and the like. However, there are some cases where the required information has no structure.

Text recognition is one of the features available in AI Builder. It processes text from images and PDF documents.

This recipe will create a solution to analyze photos taken from handwritten records and then include this information in a Dataverse table for archiving or further analysis.

Getting ready

This recipe will use some sample images from handwritten text. You can get them and the Power Platform solution created in this recipe from our GitHub repository. We will use a solution to package the table needed to store the sample images' data and the canvas app used to extract it.

How to do it...

1. Go to the Power Apps maker portal at <https://make.powerapps.com/> and create a solution for this recipe. Set the name to **Review Analysis**.
2. From the toolbar, click on **+ New | Table | Table**. On the pane that opens on the right, set the following values:
 - For the **Properties** tab, set the **Display name** to **Review** and leave the **Plural name** to **Reviews**
 - For the **Primary column** tab, set the **Display name** to **Full Name**
 - Click **Save** to start building the table.
3. Once the table has finished provisioning, click on **+ New | Column** from the toolbar and, on the pane that opens on the right, set the **Display name** to **Review Photo**, the **Data type** to **File | Image**, and click on **Save**.
4. Click on **+ New | Column** again and, on the pane on the right, set the **Display name** to **Review content** and the **Data type** to **Text | Multiple lines of text | Plain text**, and click on **Save**.
5. Click on **Forms** inside the **Data experiences** section and then click on the **Information** form of the **Main** form type:
 - From the **Table columns** section on the left pane, drag **Review content** to the form canvas below the **Full Name** column. Then, on the **Properties** pane on the right, expand **Formatting** and check **Use all available vertical space**.
 - Drag **Review Photo** just below the **Review content** column in the form canvas.
6. Click on the **Save and publish** button in the screen's top-right corner. Once it has been published, click the **Back** button in the screen's top-left corner.

Now that we have configured our data source, let's build the application.

Building the app

1. Open the solution again and click on **Apps**, and from the toolbar, click on **+ New | App | Canvas app**. Set the name to **Review Analyzer** and keep the **Tablet** form factor.

2. When Power Apps Studio opens, click the database icon on the left panel. Click on **Add data** and, using the **Search** field, enter Reviews and select it from the list.
3. Click the plus sign on the left pane, expand **AI Builder**, and select **Text recognizer**. Click **Got it** to dismiss the premium component warning.
4. In the **Properties** pane on the right, click **TextRecognizer1** to rename the control ReviewRecognizer:
 - Using the formula bar, set the **Width** to $\text{App.Width} / 2.5$ and the **Height** to $\text{App.Height} - \text{Self.Y} * 2$
5. From the **Insert** panel on the left, click on **Text label** under **Popular**:
 - Leave the **Y** property as 40. Set the **Text** property to Full Name, the **Font** to **Lato**, and the **Font size** to 12. Change the left padding to 0
 - Using the formula bar, set its **X** value to $\text{App.Width} / 2.5 + 80$
 - Name this label FullNameLbl
6. From the **Insert** pane on the left, click on **Text input** under **Popular**:
 - Using the **Properties** panel, clear the **Default** property, set the **Hint text** to Select the field, then the recognized content, set the **Font** to **Lato**, and the **Font size** to 16. Change the **Y** to 80, the **Border** width to 1, the **Border radius** to 0, and the **Hover fill** color to White
 - Using the formula bar, set the **X** value to $\text{App.Width} / 2.5 + 80$, the **Width** to $\text{App.Width} - \text{Self.X} - 40$, the **BorderColor** to `RGBA(202, 202, 202, 1)`, the **FocusedBorderThickness** to 2, and the **FocusedBorderColor** to `Color.DarkBlue`
 - Name this input FullNameTxt
7. Right-click **FullNameLbl** and click on **Copy**. Right-click anywhere on the canvas and click on **Paste**. Rename the copied label to ReviewLbl. Set the **Y** to 140 and change the **Text** to Review
8. Perform the same copy procedure with the **FullNameTxt** control. Rename the copied control to ReviewTxt, set the **Y** to 180, the **Mode** to **Multiline**, and change all padding to 10. Using the formula bar, set the **Height** to $\text{App.Height} - \text{Self.Y} - 100$
9. From the **Insert** panel, click on **Button** under **Popular**:
 - Using the **Properties** pane, set the **Text** to Save review, the **Border** width to 1, the **Border radius** to 0, the **Font** to **Lato**, and the **Font size** to 14

- Using the formula bar, set its **X** value to `App.Width / 2.5 + 80`, its **Y** value to `App.Height - 40 * 2`, and the **Width** to `App.Width - Self.X - 40`

Name this button `SaveBtn`. The designed interface should look like the following:



Figure 9.4: Review analyzer interface

10. From the **Insert** panel, expand **Media** and select **Image**. Name the control `ReviewImg` and set its **Visible** property to **Off**. Using the formula bar, change its **Image** property to `ReviewRecognizer.OriginalImage`
11. On the **Tree view**, click on **Screen1** and rename it to **Main Screen**. On the **Properties** panel, go to **Advanced**, and set the **OnVisible** property to `UpdateContext({varField: "Title"})`
12. Click the **ReviewRecognizer** control and, using the formula bar, set the **OnChange** property to the following:

```
If(
    !IsBlank(ReviewRecognizer.Selected),
    If(
        varField = "Title",
        UpdateContext({varTitleData: varTitleData & " " &
            ReviewRecognizer.Selected.Text}),
        UpdateContext({varReviewData: varReviewData & " " &
            ReviewRecognizer.Selected.Text})
    )
)
```

13. Select the **FullNameTxt** control and, from the **Properties** panel, go to **Advanced** and change the **OnSelect** property to `UpdateContext({varField: "Title"})`, set the **OnChange** property to `UpdateContext({varTitleData: Self.Text})`, and change the **Default** property to `varTitleData`
14. Click on the **ReviewTxt** control and, from the **Properties** panel, go to **Advanced** and change the **OnSelect** property to `UpdateContext({varField: "Review"})`, set the **OnChange** property to `UpdateContext({varReviewData: Self.Text})`, and change the **Default** property to `varReviewData`
15. Select the **SaveBtn** control and, using the formula bar, change the **OnSelect** property to the following:

```
Patch(
    Reviews,
    Defaults(Reviews),
    {
        'Full Name': FullNameTxt.Text,
        'Review content': ReviewTxt.Text,
        'Review Photo': {Value: ReviewImg.Image}
    }
)
```

16. From the main toolbar, click the **Publish** button. Select **Publish this version** on the dialog that opens. Finally, click on **Close** and then close the **Studio** tab on the browser.

The solution is now ready to recognize documents.

How it works...

This application allows the text recognition of images or PDF files. You can use the sample images found in the GitHub repository.

First, click on **+ New image** and select the file that will get recognized. Then, click on the field that will receive the recognized text. After that, click the image's bounding boxes (representing the recognized text) to insert its text into the selected field. Users can edit the received content to fix spelling errors or add more content.

Once the content gets verified, they can click on **Save review** to store the data, along with the image, in the **Reviews** table:

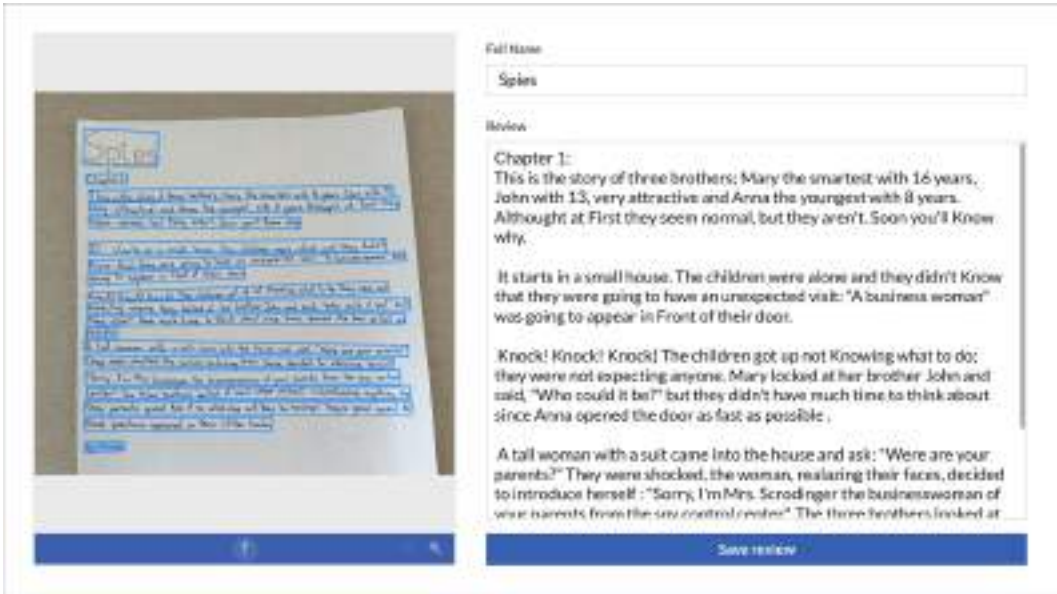


Figure 9.5: Short story review

The **OnChange** and **OnSelect** properties play a significant role in the application logic by dictating which field needs to be updated and keeping track of the user changes. The invisible image control allows the conversion from the base64 content used in the **Text Recognizer** control to an actual image format needed for the **Review Photo** data type.

The stored review can be seen in the Reviews table. See *Figure 9.6* for a general idea of the layout:

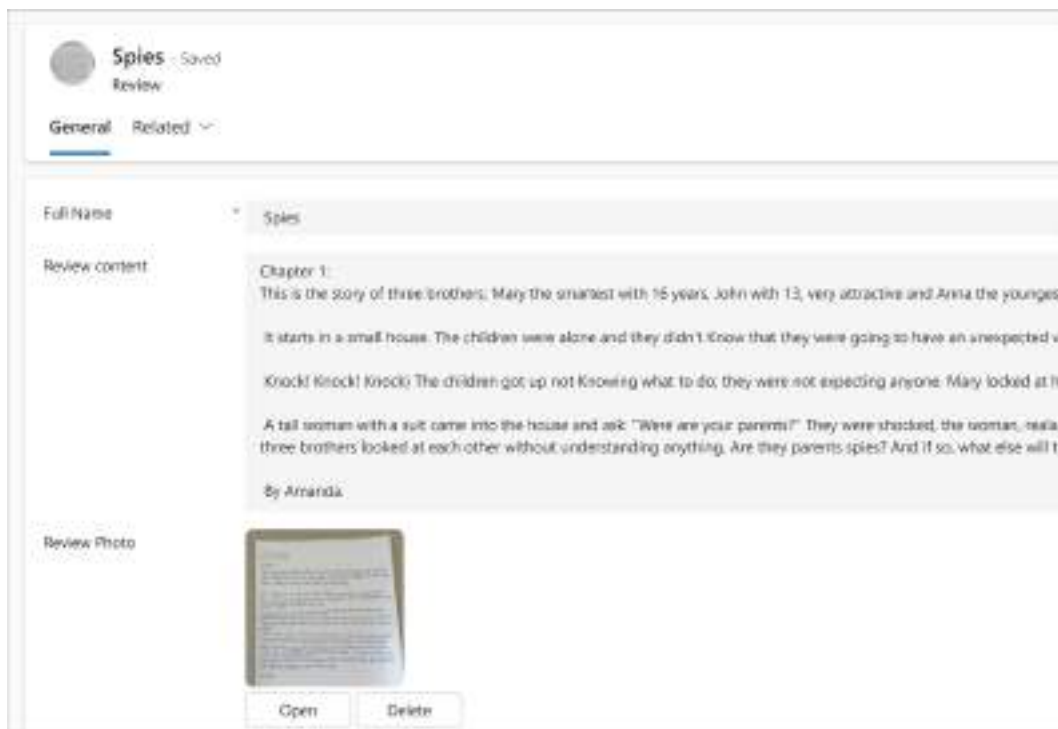


Figure 9.6: Short story review stored in the Reviews table

The information can be categorized and further analyzed by trained personnel or a custom-trained AI model for this purpose.

Gathering insights from data with the Copilot Answer component

When creating user interfaces for our users, we apply our experience in building applications using data structures to incorporate capabilities such as sorting, filtering, and searching.

No matter how we build and configure these capabilities, there's always room for improvement. Users might need new insights from the application's data, which would require developing this new capability and deploying a new version. What if we could add the ability to ask for data insights dynamically? This is where the Copilot Answer control can help us.

Getting ready

For this recipe, we will improve the application created from the previous recipe: *Building a text recognition system using canvas apps*. So, get it from the GitHub repository if you haven't built it already.

How to do it...

1. Go to the Power Apps maker portal and open the solution holding the app from the previous recipe.
2. Open the **Review Analyzer** canvas app. Once it opens, click on **Settings** from the main toolbar. Navigate to **Updates**, and from the search field, enter **Copilot answer** component. Make sure it's enabled.
3. From the insert panel on the left, click **Layout | Horizontal container**. Set its properties as follows:
 - Using the **Properties** pane, set the **Justify (horizontal)** to the last option, **Space between**, the **Gap** to 20, the **Height** to 42, the **Top** padding to 1, the **Border radius** to 0, and the **Drop shadow** to **None**.
 - Using the formula bar, set its **X** value to $\text{App.Width} / 2.5 + 80$, its **Y** value to $\text{App.Height} - 41 * 2$, and the **Width** to $\text{App.Width} - \text{Self.X} - 40$
 - Name this control **ButtonCnt**.
4. Go to the **Tree view** and select the **SaveBtn**. Right-click on it and select **Cut**. Now click on the **ButtonCnt**, right-click on it, and choose **Paste | Paste**.
5. Select the **SaveBtn** again and enable **Flexible width** from the **Properties** pane. Right-click on it and select **Copy**. Select the **ButtonCnt** and paste a copy of the button inside of it.
6. Select the new **SaveBtn_1** control and set these properties:
 - Using the **Properties** pane, set the **Text** to **Get insights**.
 - Using the formula bar, set its **Color** value to **Color.RoyalBlue**, the **Fill** to **Color.White**, and update the **OnSelect** to **Navigate('Insights Screen')**. Don't mind the error.
 - Name this control **InsightsBtn**.
7. From the **Tree view**, click on **+ New screen | Blank**. Rename this screen to **Insights Screen**.
8. From the **Main Screen**, copy the **FullNameLbl** and paste it to the **Insights Screen**. Update these properties:

- Set the **X** and **Y** to 40. Set the **Text** property to Review insights, the **Font size** to 24, and the **Width** to 250.
 - Name this label InsightsLb1
9. Repeat *step 8* with the **FullNameText** control and set these properties:
- Set the **X** to 40, the **Y** to 90, clear the **Default** property, the **Hint text** to Enter prompt, i.e., How many reviews there are?, the **Font size** to 14, and the **Border radius** to 4.
 - Using the formula bar, set the **DelayedOutput** to true
 - Name this text input PromptTxt
10. From the insert panel on the left, click **Input | Copilot answer**. If you don't see it, disable and re-enable this component from the app's settings. Set its properties as follows:
- From the **Select a data source** fly-out, select the **Reviews** table.
 - Using the **Properties** pane, set the **Title** to Ask Copilot, the **X** to 40, the **Y** to 150.
 - Using the formula bar, set the **Width** to `App.Width - Self.X * 2`, and the **Question** to `PromptTxt.Text`
 - Name this control CopilotAc

How it works...

We have just extended our previous recipe by adding the capability to get insight into the data already connected to the app. You can also use this approach to ask for data that is not already in the app. Simply add the Dataverse table, and you're done. Keep in mind that, at the time of writing, only Dataverse tables are supported, even if you configure it to a SharePoint list, for example.

The application should look like the following after asking about the first review and clicking the arrow icon in the Copilot component:



Figure 9.7: Copilot answer component in action

Other things to note from the recipe: setting the **Delayed output** from the text input avoids triggering the Copilot component on every keystroke, and changing the **Fields** property from the component allows it to get more inputs for the data gathering process.

There's more...

If you started your journey on Copilot Studio, you can leverage the work that you have done on it by connecting it to your canvas app. From the application settings, go to Copilot on the left pane and select from the list of Copilots available.

Using Power Automate to create an invoice classification system

Dealing with the invoice classification process is a tedious task for the financial department. Invoices usually come via email with little to no information regarding the provider, amount, or products involved, leaving no option for the financial staff to open each invoice to categorize them manually.

We will demonstrate one way to improve this process: creating a SharePoint document library where the staff can upload invoices without manually opening and reviewing them. A Power Automate cloud flow will monitor this library. When a new invoice gets uploaded, a trained **Document processing** model will extract the required information from these invoices to enrich its metadata and trigger an approval process, depending on the amount.

Other approaches to enhance this process include the following:

- Monitoring an email account with a Power Automate cloud flow to analyze the invoices attached to emails and store them in a specific location with the proper metadata
- Using Power Platform data gateways, connect to the on-premises network folder where the legacy financial system drops the invoices that need classification and storage

These are just some example scenarios that could be improved using Power Platform solutions.

Getting ready

This recipe will use some sample invoices to train the AI Builder model. You can get them from our GitHub repository, along with the Power Automate cloud flow created in this recipe.

We will also need to create the document library mentioned above. Go to your desired SharePoint site and create the Invoices library with these columns:

Column data type	Column name
Number	Invoice number
Text	Customer name
Date and time	Invoice date
Text	Department
Currency	Amount
Choice [Pending approval, Approved, Rejected]	Status
Multiple lines of text	Comments

Table 9.3: Invoices library columns

How to do it...

Let’s implement this recipe by configuring the AI model and the cloud flow.

Configuring the AI model

1. Go to the Power Automate maker portal and, on the left panel, click on **AI hub**.
2. If you plan to use a trial account, click the **Start free trial** button on the ribbon at the top of the screen. If you already have the AI Builder capacity, continue to the next step.
3. Click on **AI models** to view the list of models available. Select the **Documents** section, click the **Document processing** option, and click **Create custom model** in the opening dialog.
4. Once the model editor opens, click on the pencil in the upper-right corner of the screen next to rename it `Invoice model`, and then click on **Rename**.
5. Choose **Fixed template documents** and click **Next**.



From here, we could use the **Invoices** document type, which is a prebuilt model, but the idea of this recipe is to learn how to create custom models.

6. In the **Choose information to extract** page, click on **Add**, and include the following fields:

Field name	Field type	Format
Invoice number	Number field	Use dot (.) as a decimal separator
Customer name	Text field	-

Invoice date	Date field	Year, month, day
Department	Text field	-
Amount	Number field	Use dot (.) as a decimal separator

Table 9.4: Customer stories list columns

In the following screenshot, you can see the fields added:

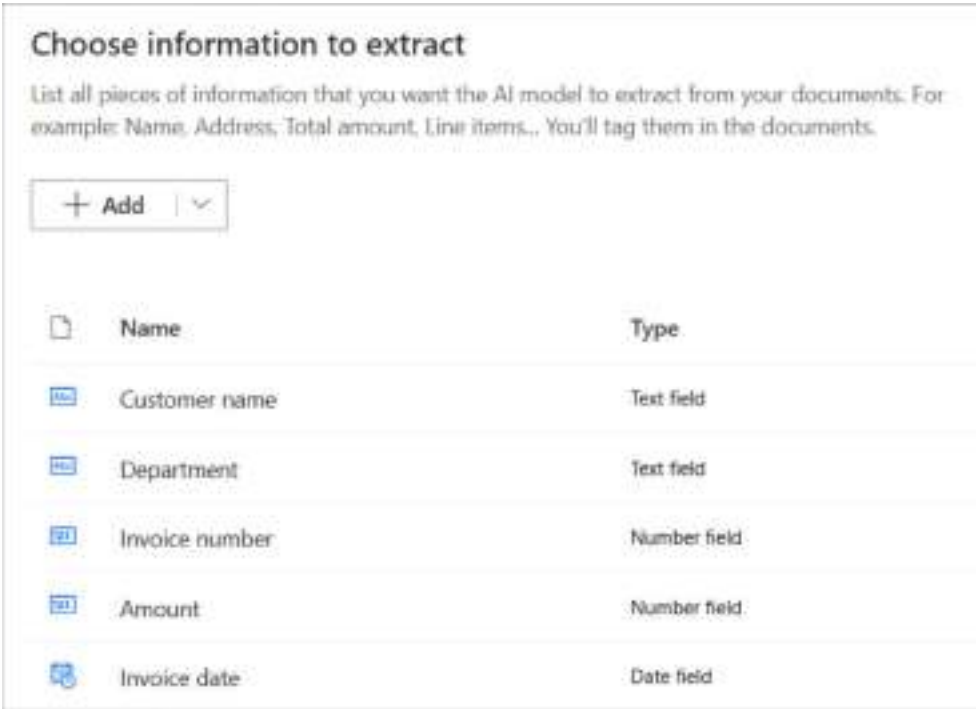


Figure 9.8: Fields configured for extraction

Click **Next** when you're finished.

- 7. On the **Add collections of documents** page, click **New collection**. Click on the **Collection 1** label to rename it as **AMPI Currents invoices**, and then click on the plus sign.
- 8. On the right panel, click **Add documents | My device** and select the five PDF files downloaded from the GitHub repository named **Invoice_0001.pdf** through **Invoice_0005.pdf**. Once they're selected, click on **Upload 5 documents**. When the upload process finishes, click **Done**.
- 9. Back on the **Add collections of documents** page, click **Next**.

10. You will land on the **Tag documents** page to tag the fields created in *Step 6*.
11. Click on each area of the invoice to tag the corresponding field, as shown in the following screenshot:

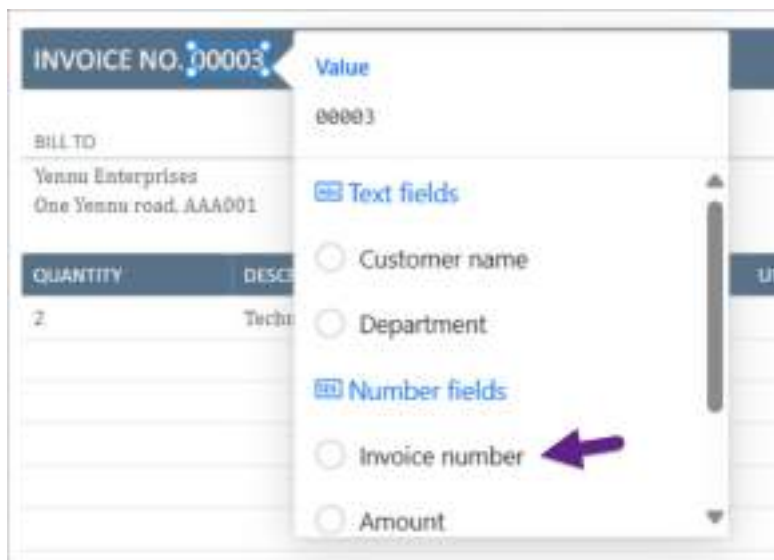


Figure 9.9: Field tagging

12. Draw a selection box to highlight several words, such as **Yennu Enterprises** in the preceding screenshot, and select **Customer name**.
13. When all fields get tagged in one invoice, a checkmark indicates that the tagging process is complete. Repeat the same tagging procedure with all the uploaded documents. Notice that some fields might get automatically tagged based on previous selections. Once the tagging process is complete, click on **Next** to continue.
14. On the **Model summary** page, click on **Train** to start training the model with the provided files. This process will take a while so click on **Go to models** to return to the **AI models** page. Keep an eye on the **Status** column to monitor the training process of the **Invoice model**. Once it finishes, the status will change to **Trained**.
15. When the training completes, click on **Invoice model** from the list to open its property page. To test the model, click **Quick test** and then **Upload from my device**. Select **Invoice_Test.pdf** from the files downloaded from the repository. If the test succeeds, all fields should be recognized in the file. Click on **Close** to finish.
16. Back on the **Invoice model** property page, click on **Publish** to make this model available to the platform.

Building the cloud flow

1. Go to the Power Automate maker portal and create a solution for this recipe. Include a connection reference for SharePoint, one for Dataverse, and one for Approvals.
2. From the solution's toolbar, click on **New | Automation | Cloud flow | Automated** to build a triggered flow. On the dialog that opens, enter **Invoice recognizer** as the **Flow name**, choose **When a file is created (properties only)** for **SharePoint** from the list of triggers, and click **Create**.



If you land on the classic designer, remember to save and reopen the cloud flow to get the modern one.

3. When the cloud flow gets created, the first action that appears is the trigger action. Enter the SharePoint location of the Invoices folder in **Site Address** and select it from the **Library Name** dropdown.
4. Add a new action and look for **Get file content using path** under **SharePoint**. Enter the same site address used for the trigger action. As for **File Path**, use the dynamic data selector and look for **Full Path**.
5. Include a new action and search for **Extract information from documents** under **AI Builder**. From the **AI model** dropdown, select **Invoice model**, select **PDF Document** from the **Form type** field, and choose **File Content** from the trigger, as shown in the following screenshot for the **Form** field:

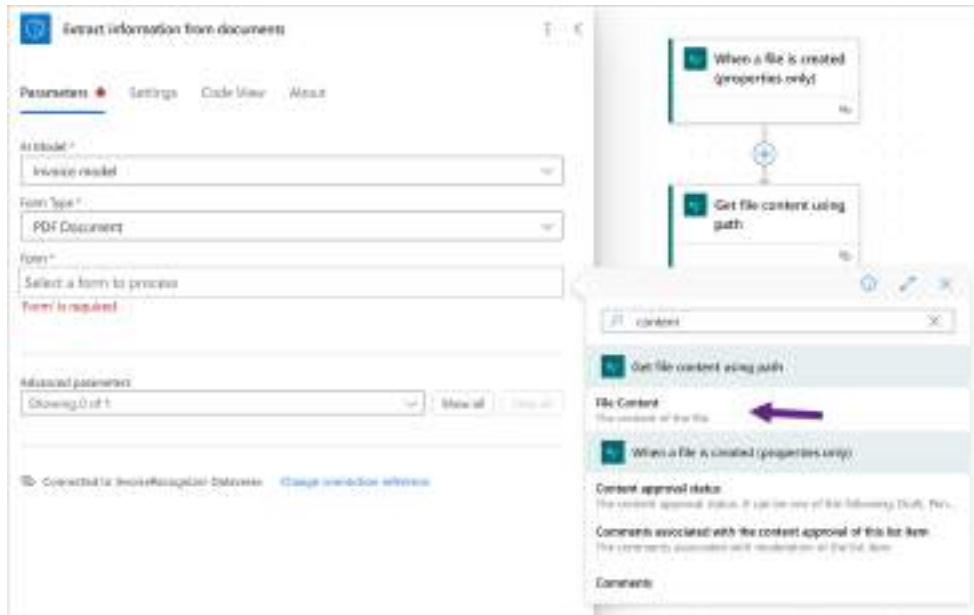


Figure 9.10: File content selection

6. Add a new action and look for **Update file properties** under **SharePoint**. Set the same **Site Address** and **Library Name** as in the trigger action. For **Id**, look for **ID** on the **Dynamic content** pane. Click **Show all** to see all properties displayed:
 - From the **Dynamic content** pane, select **Invoice number value** for **Invoice number**, **Customer name value** for **Customer name**, **Amount value** for **Amount**, and **Department value** for **Department**.
 - Select the **Invoice date** field and click on the **fx** button. Enter the following: `formatDateTime(, Go to the Dynamic content tab and select Invoice date value. Delete the closing parenthesis and add this content: , 'MMM dd yyyy'). Click Add to continue.`

- The complete formula should look similar to the following:

```
formatDateTime(body('Extract_information_from_documents')?['responsev2']?['predictionOutput']?['labels']?['Invoice_2d2b151ab11da7f1fe4e154b2d3d7d6d']?['value'], 'MMM dd yyyy')
```

- For the **Status Value** field, select **Pending Approval**.
7. Add a new action and look for **Create a sharing link for a file or folder** under **SharePoint**. Set the same **Site Address**, **Library Name**, and **Item Id** as in the previous steps. For the **Link Type**, select **View only** and choose **People in your organization** for the **Link Scope** field.
 8. Include a new action and search for **Approvals**. Add the **Start and wait for an approval** action. For **Approval type**, select **Approve/Reject – First to respond**. Set the following parameters:

- **Title:** Invoice approval request
- **Assigned to:** Enter any user's email in your tenant to receive the approval request. You can add several accounts separated by a semicolon.
- **Details:** This field is the body of the approval message. Use the following code as a template:

```
# Invoice approval
Invoice:
Customer name:
Date:
Amount: ****
[Click here to open the invoice]()
```

9. Complete the **Details** field template using the **Dynamic content** pane, selecting the following values for each field:
 - **Invoice:** Invoice number value
 - **Customer name:** Customer name value
 - **Date:** Use the same expression as in *Step 6*
 - **Amount:** Amount value (leaving two asterisks before and after)
 - **Between the parentheses:** Sharing Link

The complete action should look like the following:

Start and wait for an approval

Parameters Settings Code View Testing About

Approval Type *
Approve/Reject - First to respond

Title *
Invoice approval request

Assigned To * Switch to Advanced Mode
EM Eickhel Mendoza
Enter part of a name or email address to find more people

Details

```
# Invoice approval
Invoice: Invoice number v...
Customer name: Customer name v...
Date: formatDateTime(...)
Amount: Amount value
[Click here to open the invoice](Sharing Link)
```

Figure 9.11: Approval step configuration

10. Right-click the **Update file properties** action and select **Copy action**. Click the plus sign at the end and choose **Paste an action**.
11. Click the name of the new action to rename it to **Update file properties on response**. Click **Show All** to see all properties.
12. Expand the **Status Value** dropdown and select **Enter custom value**:
 - Hover over the **Status Value** field until the expression icon appears (denoted with *fx*) and click on it. Enter the following: `if(equals(`
 - Go to the **Dynamic Value** tab, look for **Outcome**, and select it.
 - Return to the **Expression** tab, delete the closing parenthesis, and enter this content: `, 'Approve'), 'Approved', 'Rejected')`

- Click **Add** to close the panel. The complete formula should be like the following:

```
if(equals(body('Start_and_wait_for_an_approval')?['outcome'],
'Approve'), 'Approved', 'Rejected')
```

13. Repeat the same for the **Comments** field, entering the following expression: `first(:`

- Go to the **Dynamic content** tab and select **body** from the **Start and wait for an approval** action. Delete the closing parenthesis and enter this content: `?['responses']?['comments']`

- Click on **OK**. The complete formula should look like the following:

```
first(body('Start_and_wait_for_an_approval')?['responses']?['comm
ents']
```

- Click on **Add** to close the panel.

14. From the main designer toolbar, click **Save draft** and then **Publish**.



If you've never used Approvals on your environment, you might get a warning telling you that it will get installed on the first run of the cloud flow. Be aware that this might take around five to ten minutes.

Now that we have completed the cloud flow that handles the approval, let's see how it works.

How it works...

As we mentioned at the beginning of this recipe, this cloud flow gets activated whenever a user uploads a file to the Invoices document library. It then analyzes the file's content with the help of the previously trained AI Builder model. The resulting data helps update the document metadata of the file in the document library.

Using the Power Automate approval feature, we create an approval request with the extracted invoice data, giving the approver upfront information to streamline the approval process.

The approval request sent to the approver is interactive and permits an immediate response, as shown in the following screenshot:

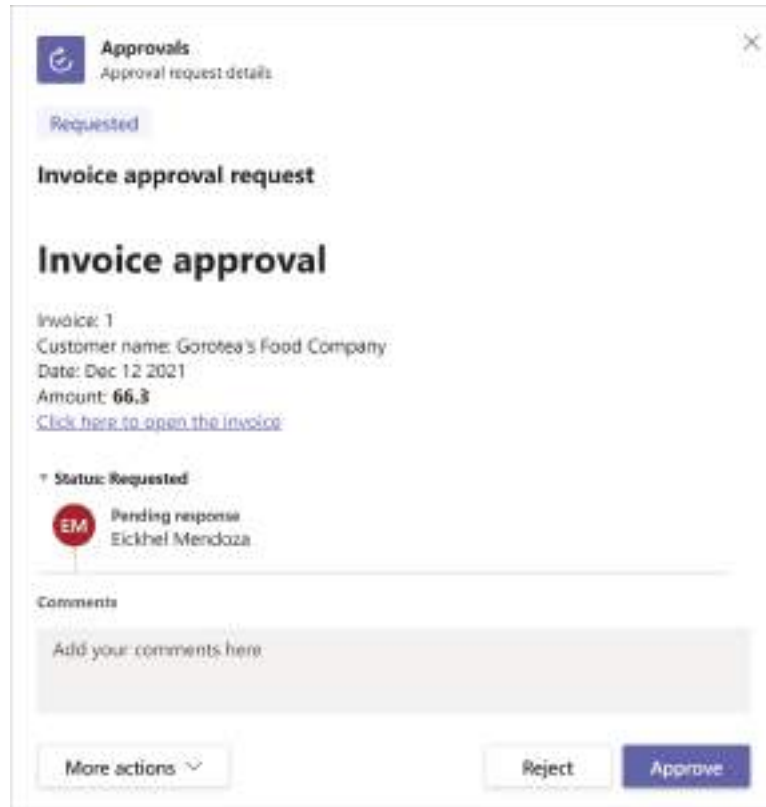


Figure 9.12: Approval example notification

The cloud flow pauses until the response gets collected through the approval process, and finally, the status metadata gets updated with the result.

Along with the invoice files needed to train the model, the GitHub repository also includes an `Invoice_Test.pdf` file, which can help you test the cloud flow.

There's more...

Extracting the data from the invoice enables us to create a more complex cloud flow where you could deal with conditions depending on this data. For example, if the invoice's amount is less than \$300, it could be that no approval is required, but if the department is finance, then maybe the approval process would require several approvers who all need to respond.

Using AI Builder collections (defined in *Step 7* of the *Configuring the model* section) allows using several invoice formats per model. This feature permits creating one model for all the invoice recognition needs instead of building one model per invoice format.

The platform also comes with prebuilt models such as invoice processing, receipt processing, and the identity document reader. These models detect standard fields, which helps us with the model configuration effort. However, they do have some limitations, such as a size of no more than 20 MB.

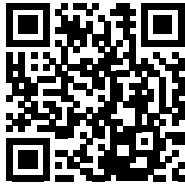
This recipe helps you learn to build a custom model specific to our example or the particularities of a custom invoice. To get more information about prebuilt models, please visit <https://learn.microsoft.com/en-us/ai-builder/prebuilt-overview>

Another solution offered by the platform is a complete set of tools to build a document automation solution with all the required elements. The **Document automation toolkit** guides you through preparing this automation in no time. Check it out here: <https://learn.microsoft.com/en-us/ai-builder/doc-automation>

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



10

Discovering the Power Platform Admin Center

Building solutions with Power Platform unleashes opportunities for business users to solve their needs, whether creating a supporting app, orchestrating automation, designing a business intelligence report, or even designing an AI-infused solution using Copilot Studio.

These platform solutions need a robust backend to support their functionality while also providing the supporting tools and configurations required to manage the platform correctly.

The Power Platform admin center is the central location for managing the backend of all components. This chapter will let you discover the essential tools and configurations available from the Power Platform admin center through the following recipes:

- Managing Power Platform environments
- Gathering Power Platform analytics
- Handling connectors through data policies

Technical requirements

From a security perspective, managing the Power Platform admin center requires at least the Power Platform admin role. If your organization includes Dynamics 365, this admin access role also gives you access to the admin center.

However, when a team gets provisioned with Dataverse for Teams in Microsoft Teams, it creates an environment for it. Users in the owners group can access the environment's configuration in the Power Platform admin center.

Managing Power Platform environments

Environments are containers of Power Platform components. They include all your solution components, such as applications, flows, and data. They also serve as a medium to isolate these components, whether for security reasons or to have independent development environments.

There are different types of environments. Each serves a particular functionality:

- **Production:** This environment is the one your end users will use when executing your solutions.
- **Sandbox:** This type of environment is a testing ground for your solutions. You can reset it to start from scratch.
- **Trial:** You can use these environments for rapid feature testing, as they get deleted after 30 days. There is also a subscription-based type focused on enterprise delivery. You can find more information here: <https://learn.microsoft.com/en-us/power-platform/admin/trial-environments>.
- **Developer:** This type of environment is created when an organization user applies for the Power Apps Developer Plan. You can find more information here: <https://www.microsoft.com/en-us/power-platform/products/power-apps/free>.
- **Microsoft Teams:** This type of environment gets provisioned when a Microsoft Teams user creates a solution using the Power Apps application (Dataverse for Teams) inside a team.
- **Support:** Microsoft creates these environments when dealing with a support ticket for your organization. You can find more information here: <https://learn.microsoft.com/en-us/power-platform/admin/support-environment>.
- There is also the **(default)** environment that gets created automatically for each tenant with a limited Dataverse database. More information is available here: <https://learn.microsoft.com/en-us/power-platform/admin/environments-overview#power-platform-environment-types>.

Managing these environments depends mainly on the capacity available in your organization, which, by default, is 1 GB of database capacity. Organizations can increase this by including add-ons or acquiring Power Platform licenses. The increment may vary based on the quantity and type of licenses.

For a more up-to-date increment definition, please refer to the *Dataverse default and accrued subscription capacity entitlements* section of the *Microsoft Power Apps and Power Automate Licensing Guide* available at <https://go.microsoft.com/fwlink/?linkid=2085130>.

How to do it...

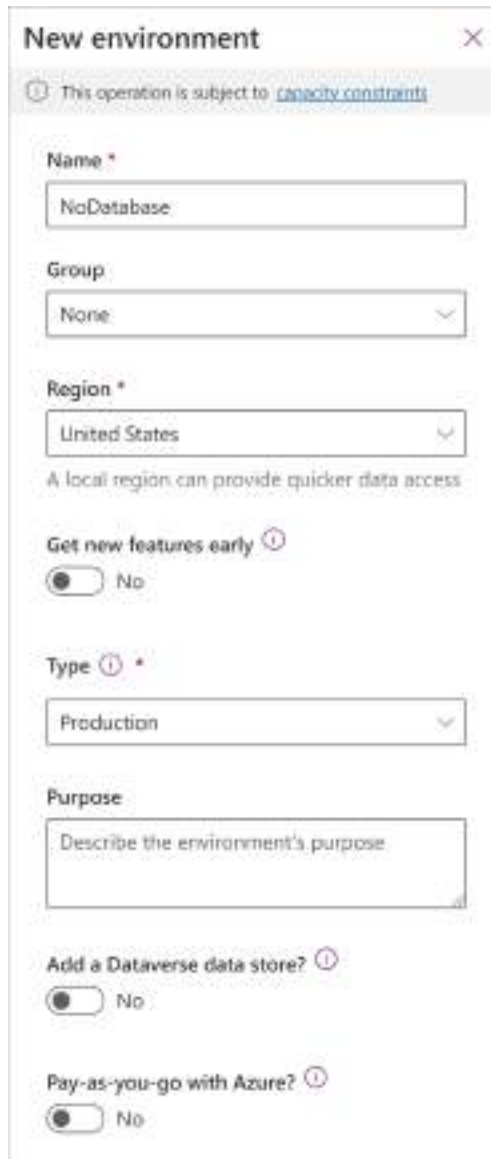
Provisioning an environment can be done with or without a database. In this recipe, we will learn the differences regarding security.

As a shared step for the following sections, go to the Power Platform admin center at <https://admin.powerplatform.microsoft.com>, or <https://aka.ms/ppac> for short.

Creating an environment without a database

1. From the left pane, click on **Environments** and then click on **New** from the toolbar. On the panel that opens on the right, enter NoDatabase for **Name**. Leave **Group** as the default.
2. For **Region**, select the one closest to your location.
3. Leave **Get new features early** unchecked and choose **Production** for **Type**.

4. Leave **Purpose** as default and leave the **Add a Dataverse data store?** and **Pay-as-you-go with Azure?** options unchecked. Your settings should look like the following:



New environment ✕

ⓘ This operation is subject to [capacity constraints](#)

Name *
NoDatabase

Group
None

Region *
United States
A local region can provide quicker data access

Get new features early ⓘ
☒ No

Type ⓘ *
Production

Purpose
Describe the environment's purpose

Add a Dataverse data store? ⓘ
☒ No

Pay-as-you-go with Azure? ⓘ
☒ No

Figure 10.1: Environment settings without database

5. Click **Save** when you're finished.

After a couple of seconds, the environment provisioning process will be complete.

Creating an environment with a database

1. Back on the toolbar, click on **New**. Repeat the same steps as before, but this time, enable the **Add a Dataverse data store?** option, and then click **Next**.
2. Set the default **Language** for the interface and the required **Currency** for reporting purposes. You must pre-define your environment's access by selecting a **Security group**. Please refer to this documentation for more info: <https://learn.microsoft.com/en-us/power-platform/admin/control-user-access>.
3. If you want, you can change the URL prefix of your new environment, decide whether you wish to enable Dynamics 365 apps (given that you have the required licenses), and choose if you want to deploy sample apps and data. Click on **Save** when you're finished.



The building process of this environment will take a while because it also needs to provision the Dataverse database.

How it works...

When you create an environment without a Dataverse database, the security gets handled at the environment level. Selecting **NoDatabase** from the list displays a detail page where you can add users as an **Environment admin** or an **Environment maker**, as shown in the following screenshot:



Figure 10.2: Database-less environment

In a nutshell, the admin can perform administrative tasks such as adding users while the maker can add resources to the environment. For more information on this, follow this link: <https://learn.microsoft.com/en-us/power-platform/admin/environments-overview#power-platform-environment-roles>.

When an environment gets a database, **Security roles** control its access, including more granularity on the access levels down to record-level security. To configure these roles, click on **Security roles** in the **Access** section, as shown in the following screenshot:



Figure 10.3: Dataverse environment

For an overview of these security roles, please navigate to <https://learn.microsoft.com/en-us/power-platform/admin/database-security>.

To take advantage of these improved security roles, you must know how to configure them properly. For a complete definition of the scope and privileges of the security roles, please visit <https://learn.microsoft.com/en-us/power-platform/admin/security-roles-privileges>.

There’s more...

When you’re working with Dataverse environments, the admin center allows the configuration of the **Capacity** add-ons for each environment.

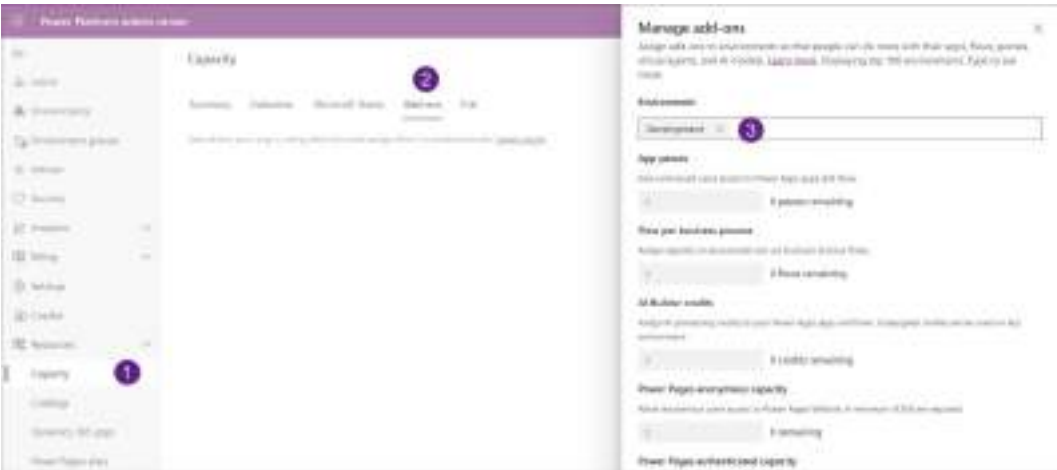


Figure 10.4: Dataverse analytics

From there, you can:

- Configure the licensing capabilities of the Power Apps per app plan: <https://learn.microsoft.com/en-us/power-platform/admin/about-powerapps-perapp>
- Refine the available AI Builder credits: <https://learn.microsoft.com/en-us/ai-builder/credit-management>
- Set up the Power Pages capacity levels: <https://learn.microsoft.com/en-us/power-pages/admin/capacity-management>

In summary, in this section, it is possible to configure all the capabilities that will be available in a particular environment. For more information on capacity management, please visit <https://learn.microsoft.com/en-us/power-platform/admin/capacity-add-on>.

Gathering Power Platform analytics

Analytics is a vital tool for measuring the performance of a given process. These measures can vary depending on their target. For example, website analytics offers information about the visitors to a website, such as location, cookie handling preferences, and the like. This information provides website owners with clear evidence that they might need to improve their infrastructure to handle the number of visitors or add another server near visitors' locations.

Similarly, this information is vital for the Power Platform because it allows you to learn about essential indicators such as the devices used to run your applications, how many runs are executed of a given flow, and, most importantly, how many errors your users are getting.

In this recipe, you will learn about all the ways we could get this information.

Getting ready

You will need administrative access, such as an Environment admin or Power Platform admin role, to gather the analytics information.

How to do it...

Analytics data can be collected from various locations depending on the platform you wish to review. Let's have a look at each of them.

Power Platform admin center

1. Go to the Power Platform admin center by opening this URL, <https://aka.ms/ppac>, and then expand **Analytics** from the left panel.

2. Select **Dataverse** and, from the **Home** tab, click on **Change filters**, as shown in the following screenshot:



Figure 10.5: Dataverse analytics

3. Select the environment from which you want to view the reports and the time frame.
4. Select **Power Automate** from the left pane to view the **Runs** tab of the selected environment. Click on **Change filters** to choose the environment and time range.
5. Click on **Power Apps** to get to the **Usage** tab. Click on **Change filters** to choose the environment and time.
6. Finally, from the left pane, click on **Copilot** to review its usage. Keep in mind that you can also update Copilot settings from the **Power Apps settings** section.

Power Apps maker portal

1. Go to this portal by visiting <https://make.powerapps.com> and clicking on **Apps** on the left pane.
2. Select an app from the list and, using its ellipsis dropdown, click on **Details | Analytics (preview)**, as shown in the following screenshot:



Figure 10.6: Usage view on Power Apps analytics

3. App-specific analytics will open, displaying **Usage** information. You can use the dropdown to view **Location**, **Performance**, and **Error** information.

SharePoint list Power Apps

1. Go to the SharePoint site holding the list or document library customized with an app. An example is, the Book requests document library created in *Chapter 8, Power Apps Everywhere*.
2. Open the library, click on the gear at the screen's top right, and select **Library settings**.
3. From the **Settings** page, click **Form settings** and **See versions and usage**. This action will open the embedded app's **Details** tab. Click on the **Analytics (preview)** tab to review its data.



SHAREPOINT LIST POWER APPS ANALYTICS: This feature is undocumented and might not work under some scenarios.

Power Automate

1. Go to the Power Automate maker portal, <https://make.powerautomate.com>, and click on **My flows** on the left pane.

2. Select a cloud flow from the list and select **Analytics** using its ellipsis dropdown. This action will open the flow-specific analytics, as shown in the following screenshot:



Figure 10.7: Cloud flow analytics

These are the current options to gather analytics from the Power Platform. Besides these, a new capability exists to collect and analyze these analytics using the Azure Application Insights service. More information can be found here: <https://learn.microsoft.com/en-us/power-platform/admin/analyze-telemetry>.

How it works...

Each analytic option displays specific information regarding each platform. While Power Apps and Power Automate provide data about the execution of each app or cloud flow, the admin center provides a complete overview of each environment as a whole. The following links provide insight into the data available in the Power Apps admin center regarding the following:

- Dataverse: <https://learn.microsoft.com/en-us/power-platform/admin/analytics-common-data-service>
- Power Apps: <https://learn.microsoft.com/en-us/power-platform/admin/analytics-powerapps>

- Power Automate: <https://learn.microsoft.com/en-us/power-platform/admin/analytics-flow>
- Copilot: <https://learn.microsoft.com/en-us/power-platform/admin/copilot/copilot-hub>



DATA COLLECTION TIME FRAME: Be aware that this data gets collected only for the last 30 days.

Another approach to gathering analytics for canvas apps is manually submitting statistics with the help of Azure's Application Insights. Provisioning this Azure resource allows you to store events collected from your application. Follow this documentation to configure it: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/application-insights>.

There's more...

In addition to the tools available within the Power Platform, Microsoft has introduced a set of components for gathering data across all environments to assist with your organization's governance and compliance regulations: the Microsoft Power Platform **Center of Excellence (CoE)** Starter Kit. This kit utilizes Power Platform tools like Power Apps, Power Automate, and Power BI, all seamlessly working together from a Dataverse database.

To get more guidance about this kit, please visit <https://learn.microsoft.com/en-us/power-platform/guidance/coe/starter-kit>.

A third-party solution also digs deeper into the analytics information gathered from Power Apps and Power Automate. Flow Studio enhances analytics data manipulation while adding tools like live flow documentation, migration, and monitoring. Visit the following link to find out more: <https://flowstudio.app>.

Handling connectors through data policies

The Power Platform has an ever-growing list of connectors available from different vendors. These connectors perform various tasks, such as handling files on cloud services or communicating data through APIs.

Keeping track of these communications would be extremely difficult without using data policies. These policies control sensitive data using three groups: Business, Non-Business, and Blocked. Placing connectors in these groups effectively builds a fence between them. If you place a connector, such as SharePoint, in the Business group and then add Dropbox to the Non-Business group, then your users won't be able to create an app or a flow that uses both connectors. For more detailed information on these policies, please refer to <https://learn.microsoft.com/en-us/power-platform/admin/wp-data-loss-prevention>.

This recipe will create a policy to block popular social media connectors and any new connector added to the Power Platform to prevent misuse of these services in the work environment.

Getting ready

To build this policy, you will need access to the Power Platform admin center via the Power Platform/Dynamics 365 admin role or the tenant admin role.

How to do it...

1. Go to the Power Platform admin center, <https://aka.ms/ppac>, then expand **Policies** and click on **Data policies** on the left pane. On the toolbar, click on **New Policy**.
2. Enter **Block social media** in the **Name your policy** field and click **Next**.
3. On the **Assign connectors** screen, look for the **X** connector. Click on the ellipsis next to it and choose **Block**.

Repeat this step for **Pinterest**, **LinkedIn**, **LinkedIn V2**, and **YouTube**. The list of blocked connectors should look like the following:

Assign connectors ⓘ

Business ID: Non-business (1242) | Default: **Blocked** (5)

Blocked connectors can't be used where this policy is applied.

☐	Name ▾	Blockable ▾	Endpoint config. ▾	Class ▾	Publisher ▾
	YouTube	Yes	No	Standard	Microsoft
	X	Yes	No	Standard	Microsoft
	Pinterest	Yes	No	Standard	Microsoft
	LinkedIn V2	Yes	No	Standard	Microsoft
	LinkedIn	Yes	No	Standard	Microsoft

Figure 10.8: Blocked connectors list

- 4. Click on **Set default group** at the screen's top right. On the dialog that opens, select **Blocked** and click **Apply**.
- 5. Click **Next** on the **Assign connectors** screen and **Next** again on the **Custom connectors patterns** section.
- 6. On the **Define scope** screen, leave **Add all environments** selected and click **Next**.
- 7. Click **Create policy** on the **Review and create policy** screen to finish the guided process.

How it works...

When the policy comes into action, these connectors will get blocked when a Power Apps or Power Automate solution tries to use them.

For example, if an app maker goes to the Power Apps maker portal and tries to create a canvas app using the LinkedIn V2 connector, the platform will block its use with the following dialog:



Figure 10.9: LinkedIn V2 connector blocked

This policy will also include any new connectors added to the Power Platform to the Blocked group until a user with an administrative role unblocks it by adding it to the Business or Non-Business group.

There's more...

The DLP capabilities of the Power Platform continue to evolve, adding more features to help secure the connectivity to and from your organization. Some of these new capabilities fine-tune data policies, allowing you to control the connectors at the action level.

In the following screenshot, we can see how to configure which actions are allowed for the X connector:

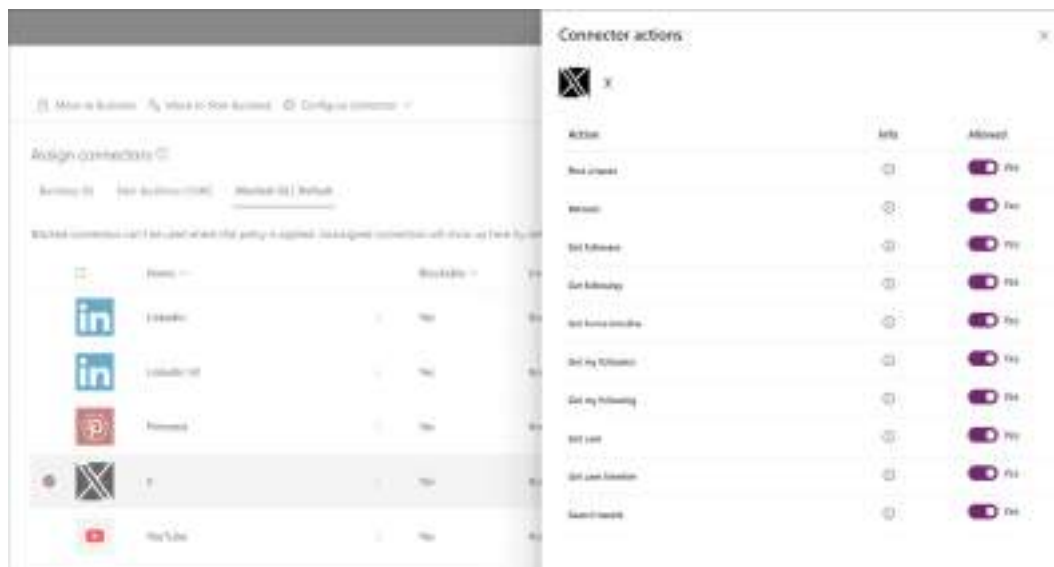


Figure 10.10: Connector actions configuration

To configure it, click the ellipsis next to the connector and select **Configure connector | Connector actions**.

Regarding the desktop flows, we can see some improvements specific to this type of flow, such as blocking the execution of scripts or Clipboard access. More information on this can be found here: <https://learn.microsoft.com/en-us/power-automate/prevent-data-loss#data-loss-prevention-for-desktop-flows>.

Lastly, custom connectors also leverage data policies by using URL patterns to secure the data flow. To get more insight into this, please visit <https://learn.microsoft.com/en-us/power-platform/admin/dlp-custom-connector-parity>.

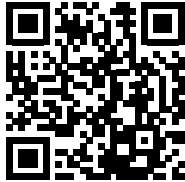
Remember these policies when securing your organization to prevent potential data loss incidents.

Don't forget to explore the admin center, where you'll find many settings to help you set up the right security and compliance measures for your tenants. These settings include **Managed environments**, **Tenant settings**, **Copilot configuration**, **Environment assignments**, and more. You can find reference information here: <https://learn.microsoft.com/en-us/power-platform/admin/>.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



11

Tips, Tricks, and Workarounds

When building applications, Power Apps provides a robust platform filled with features for rapid application building. These features include a responsive UI designer, connectors for data source handling, and a complete deployment toolset.

However, as with every development platform, not all the features we would like are included, or some are still a work in progress. Maybe some of the included features have limitations or are of a different scope from what we want to accomplish. For situations like these, developers often look for out-of-the-box solutions to meet their goals.

This chapter will present a series of workarounds that help solve some of the platform's limitations or improve the use of a feature that is already available.

The following recipes will explore these workarounds:

- Sharing OneDrive data between tenants
- Playing with vectors – SVGs in canvas apps
- Transferring SharePoint list apps from one site to another
- Troubleshooting using Power Apps Monitor
- Extending screen real estate using the canvas control
- Updating Azure SQL Server connections in Power Apps
- Renaming files in SharePoint document libraries

Technical requirements

Some of the recipes in this chapter require using a premium connector. Please refer to the *Preface* section of the book for more information.

The resources created in this chapter are available on our GitHub repository at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-11>.

Sharing OneDrive data between tenants

Sharing applications with internal users is a pretty straightforward task. Build the canvas app, publish it, and share it with specific users, groups, or the entire organization. If your app uses a data source, it might require more steps. For example, when using Dataverse, you will need to configure the security roles required to access the tables in your application.

There is also another form of sharing: **guest access**. You can invite users to your organization by making them guest users in the Azure Entra ID. For more information on this, go to <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/share-app-guests>.

Once the guest user gets included in the organization, you can share your app using the standard method. However, not all connectors support guest access because they work by requesting access to the user's own tenant.

Let's take the following scenario as an example:

A maker of the PowerNimbus tenant shares a canvas app using OneDrive for Business with a guest user of the Contoso tenant. When the Contoso user opens the canvas app, the application will request access to the OneDrive of the guest user in Contoso. The file used as a data source for tenant PowerNimbus won't exist for tenant Contoso.

This recipe will help you discover a workaround to solve the limitation in the previous example. You can extrapolate this solution to use it with other connectors that don't support guest access. The following link provides a list of connectors and how they deal with guest access: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/share-app-guests#which-connectors-create-connections-in-the-resource-tenant-by-default>.

Getting ready

When sharing an app with guest users, the licensing model depends on the app's connectors. Microsoft provides a table with specific sharing scenarios here: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/share-app-guests#what-license-must-be-assigned-to-my-guest-so-they-can-run-an-app-shared-with-them>.

To test this recipe, you will need a guest user of another organization to share this app. To add this external user, you will need an admin role or request this from an administrator using the steps described here: <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/share-app-guests#steps-to-grant-guest-access>.

Please refer to *Chapter 1, App Maker Basics*, for instructions on properly creating a Power Platform solution for this recipe.

How to do it...

We will leverage Power Automate's integration capabilities to solve our needs.

Uploading the data

1. Go to the Microsoft 365 portal, <https://www.office.com/>, and click the app launcher in the top-left corner of the screen. From the list of applications available, click **OneDrive** and open it.
2. Upload the `Sample Data.xlsx` file downloaded from our GitHub repository.

Building the cloud flow to get Excel data

3. Navigate to the Power Automate maker portal and create a solution. Once it opens, add a connection reference to **Excel Online (Business)**. From the toolbar, click **+ New | Automation | Cloud flow | Automated**. On the open dialog, click **Skip**.
4. Once the designer opens, in the **Search connectors and triggers** field, enter `When an HTTP request is received`. When found, click on it from the list.
5. Click `Untitled` in the upper-left screen to rename the flow to `Get Excel data`. If you landed on the classic designer, add a dummy action to be able to save the flow and then open it again to get the modern designer.
6. Click the trigger action named **manual** and select **Anyone** from the **Who Can Trigger The Flow?** dropdown.
7. Add a new action and look for **List rows present in a table** under **Excel Online (Business)**. Set the following properties:
 - **Location:** OneDrive for Business
 - **Document Library:** OneDrive
 - **File:** Use the file picker to select `Sample Data.xlsx`
 - **Table:** `DataTable1`

- Expand **Advanced parameters** and choose **Select Query**; enter `first_name`, `last_name`, and `email` in the **Select Query** field
8. Create a new action and look for **Select** under **Data Operation**. For the **From** field, click the lightning icon and select **body/value** from the previous action. In the **Map** section, do the following:
- Enter `fullname` for the **key** field and pick **first_name** and **last_name** for the **value** field (manually include a space in between both fields)
 - Enter `email` for the **key** field and choose **email** for the **value** field

This step should look like the following:

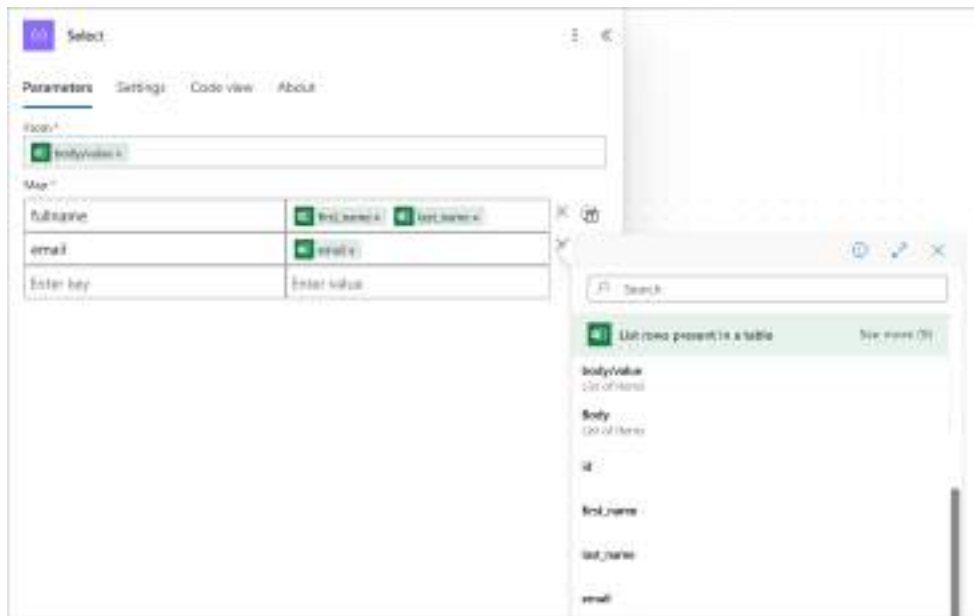


Figure 11.1: Flow Select action

9. Add a new action and look for **Response** under **Request**. Select the output of the **Select** action in the **Body** field.
10. Click **Save draft** and then **Publish**. Open the trigger action card to copy the **HTTP URL**. We are going to use it in another flow. Close the designer.

Building the cloud flow to communicate with Power Apps

11. Back in the solution, create an instant cloud flow by clicking **+ New | Automation | Cloud flow | Instant**. Enter **Get data from Flow** for **Flow name**, select **When Power Apps calls a flow (V2)** from the list of **Choose how to trigger this flow**, and then click **Create**. Repeat the previous steps to get the modern designer.
12. Add a new action and look for **HTTP**. Change the **Method** to **POST** and paste the **HTTP URL** from *step 10* of the previous section in the **URI** field.
13. Create a new action and look for **Respond to a PowerApp or flow** under **Power Apps**. Click on **Add an output** and choose **Text**. In the **Enter a name** field, enter **data**. Click on **Enter a value to respond with** and, using the **fx** icon, enter `join(body('HTTP'), ';')` and click **Add**.
14. Hit **Save draft** and **Publish**. Close the **Power Automate** tab.

Building the app

15. Back again in the solution, create a canvas app by clicking **+ New | App | Canvas app**. Enter **OneDrive shared for the App** for the **App name**, set **Format** as **Phone**, and click **Create**.
16. Once Power Apps Studio opens, go to the **Power Automate** pane on the left (represented by its logo) and click **Add flow**. A list of flows will appear on a floating pane. Click on **Get data from Flow**. You can use the search field to look for it.
17. Go to the **Tree view** pane on the left and select the **App** element. Using the formula bar, set the **OnStart** property to the contents of the **App - OnStart.txt** file downloaded from our GitHub repository. Click the ellipsis next to the **App** control and click on **Run OnStart**.
18. Go to the **Insert** pane on the left and click on the **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane on the right, set the **Text** property to **Excel data**, the **Font** to **Open Sans Condensed**, the **Font size** to **32**, the **Font weight** to **Semibold**, the **X** and **Y** properties to **0**, the **Height** to **100**, and the left **Padding** to **20**
 - Using the formula bar, set the **Width** to `App.Width`, **Color** to `Color.White`, and the **Fill** to `ColorValue("#1D455F")`
 - Name this label **HeaderLb1**

19. Back on the **Insert** panel, click on **Vertical gallery** under **Popular**. Set its properties as follows:

- Using the **Properties** pane, set the **Layout** to **Title and subtitle**, the **Template padding** to 15, turn off **Show scrollbar**, and enable **Show navigation**.
- Using the formula bar, set the **Items** to `ExcelDataCol1`, the **Width** to `App.Width, Y to HeaderLbl.Y + HeaderLbl.Height`, and the **Height** to `App.Height - Self.Y`
- Name this gallery `DataGly`

20. From the **Tree view**, expand **DataGly** and make these changes:

- Select the **Separator2** control (remember that these numbers might change). Using the formula bar, set its **Fill** color to `RGBA(235, 237, 242, 1)`, the **Height** to 1, and rename this rectangle as `SeparatorRct`
- Select the **NextArrow2** control and delete it. Do the same for the **Rectangle2**.
- Select the **Subtitle2** control and rename it as `SubtitleLbl`. Change the **Font** to **Segoe UI** and the **Font size** to 16. Using the formula bar, change the **Color** to `RGBA(150, 153, 162, 1)` and the **Text** property to `ThisItem.Email`
- Select the **Title2** control and rename it as `TitleLbl`. Change the **Font** to **Segoe UI**, and using the formula bar, change the **Color** to `RGBA(63, 64, 71, 1)` and the **Text** property to `ThisItem.FullName`

The design of your application should follow the one shown here:

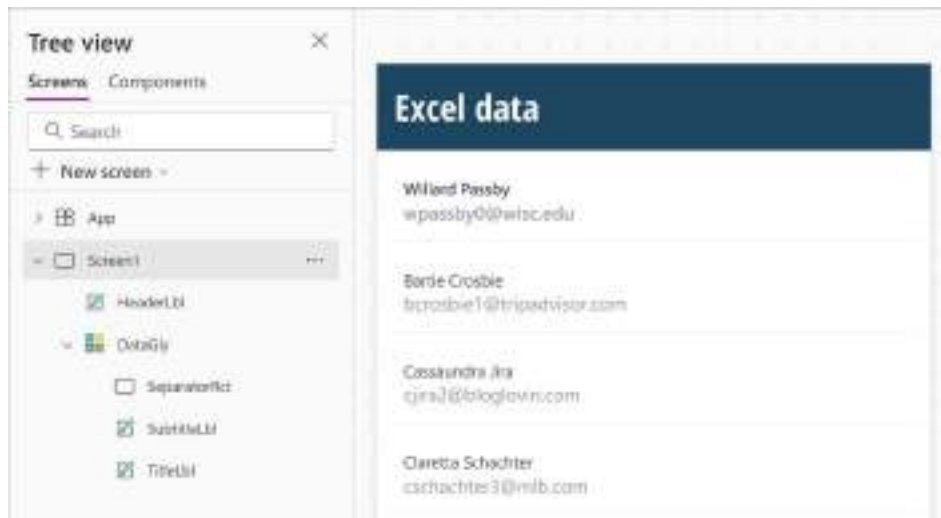


Figure 11.2: Power Apps application design

21. Click the **Publish** icon. After the publishing is complete, click on the **Share** icon.
22. Enter the guest user's name or email address and click the **Share** button at the bottom.
The guest user should receive an email like the following:

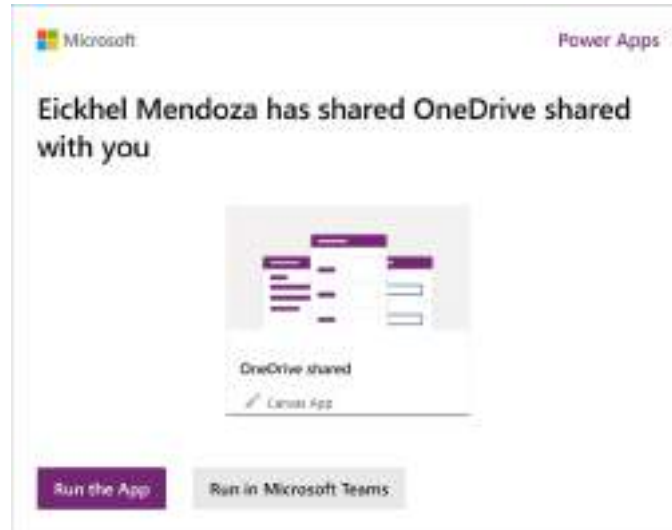


Figure 11.3: Shared app email

The guest user can then click on **Run the App** to execute the canvas app from their tenant.

How it works...

The application gets the Excel data from OneDrive using two cloud flows:

- **Get Excel data** requests the data from Excel using the regular connectors, but an HTTP request triggers it. An HTTP response handles the output.
- **Get data from Flow** gets the data from the previous cloud flow. It gets activated by the app, and the output gets sent back to the app.

The context of the connectors comes from the app's running user. That's why when a user executes an app that uses a connector, the platform requests that the user grant permission to use it.

If we used just one cloud flow, the app would ask permission to get the data from the running user's OneDrive on the Contoso tenant, making it impossible to share the file from the Power-Nimbus tenant, as we discussed at the beginning of this recipe.

Using two cloud flows breaks the context as we use HTTP requests and responses to communicate with each other. However, remember that the HTTP request action is a premium connector, as we see in the following image:

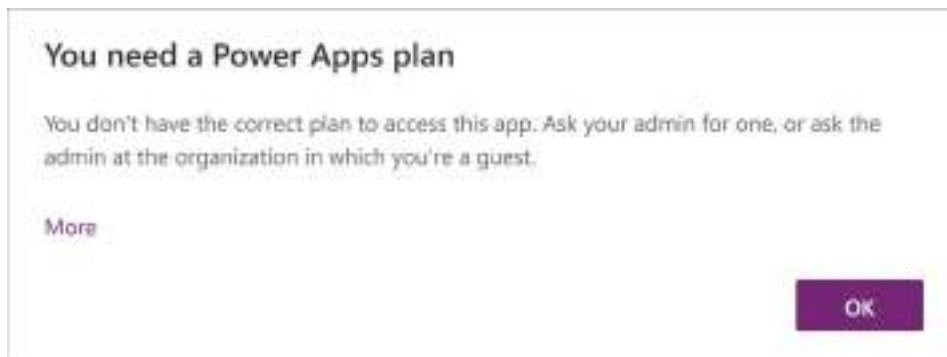


Figure 11.4: Canvas app requesting a license

You can provision per-app licenses to the environment hosting the app or assign a Power Apps Premium license to the guest user to properly license the app.

Using this approach also solves a scenario where you might want to share only some parts of an Excel file and not the complete file. The file owner can still work on the file; only a particular table gets shared with guest users.

Playing with vectors – SVG in canvas apps

Scalable Vector Graphics (SVG) is a vector image format that uses an **Extensible Markup Language (XML)** structure. It means that this is a text-based file format containing coordinates that create an image.

One particular advantage of this format is that the quality is not affected when you change its dimensions. Being a vector format, it draws the image according to its dimensions. One usage example is a company logo, where you need to maintain quality, no matter the size, in places such as business cards, websites, and document templates.

This format provides even more features, like animation or HTML content styling. For the complete documentation, please refer to <https://developer.mozilla.org/en-US/docs/Web/SVG>.

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and create a solution named SVG Templates
2. Once the solution opens, create a canvas app by clicking + **New | App | Canvas app**. Enter SVG Demo for the **App name**, keep the **Tablet** format, and click **Create**.
3. Go to the **Insert** pane on the left (represented by a plus sign), expand **Media**, and click on **Image**:
 - Using the **Properties** pane on the right, set the **X** and **Y** properties to 0 and the **Image position** to **Fill**.
 - Using the formula bar, set the **Width** to `App.Width`, the **Height** to `App.Height`, and the **Image** to the following code:

```
"data:image/svg+xml," & EncodeUrl("<svg xmlns='http://www.
w3.org/2000/svg' width='100%' height='100%' viewBox='0 0 1600
800'>
  <rect fill='#0f0f0f' width='1600' height='800' />
  <g>
    <polygon fill='#1d1e2e' points='1600 160 0 460 0
350 1600 50' />
    <polygon fill='#2c2e4d' points='1600 260 0 560 0
450 1600 150' />
    <polygon fill='#3a3d6c' points='1600 360 0 660 0
550 1600 250' />
    <polygon fill='#494d8b' points='1600 460 0 760 0
650 1600 350' />
    <polygon fill='#575caa' points='1600 800 0 800 0
750 1600 450' />
  </g>
</svg>")
```

4. From the **Insert** panel, click on **Text label** under **Popular**. Using the **Properties** pane, set the **Text** property to Power Apps Cookbook, the **Font** to Segoe UI, the **Font size** to 48, the **Font weight** to **Semibold**, the **X** and **Y** properties to 28, the **Width** to 390, and the **Height** to 160. Change the **color** to white.

The screen should look like the following:



Figure 11.5: SVG background image

5. Try resizing the image manually. You will see no loss in quality.
6. From the main toolbar, click on **New screen** | **Blank**. Using the formula bar, set the **Fill** property to `RGBA(39, 67, 125, 1)`
7. Go to the **Insert** pane on the left, expand **Media**, and click on **Image**. Using the formula bar, set the **X** property to `(App.Width - Self.Width) / 2`, the **Y** property to `(App.Height - Self.Height) / 2`, and the **Image** property to the contents of the `Image2 - Image.txt` file downloaded from our GitHub repository.

You should see a ring loader animation, as shown in the following screenshot:

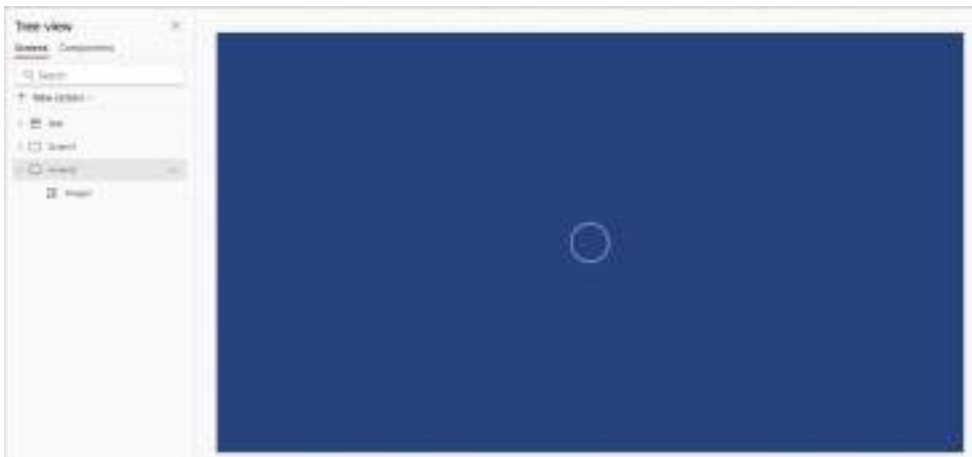


Figure 11.6: SVG loader image

8. From the main toolbar, click on **New screen** | **Blank**. Using the formula bar, set the **Fill** property to `RGBA(237, 237, 237, 1)`
9. Go to the **Insert** pane on the left, expand **Media**, and click on **Image**:
 - Using the **Properties** pane, set the **Image position** to **Stretch**, the **Width** to 790, and the **Height** to 100
 - Using the formula bar, set the **X** property to `(App.Width - Self.Width) / 2`, the **Y** property to `(App.Height - Self.Height) / 2`, and the **Image** property to the contents of the `Image3 - Image.txt` file downloaded from our GitHub repository.

You should see SVG shadow effects, as shown in the following screenshot:



Figure 11.7: SVG Text using CSS shadow effects

10. From the main toolbar, click on **File** | **Save**.

Now that we have used several SVG features inside Power Apps, let's see how it works.

How it works...

SVG images act as a container of many components, such as **Cascading Style Sheets (CSS)** styles, vector drawings, and text elements.

By adding the `"data:image/svg+xml,"` prefix to the **Image** properties, as we did in *step 3*, we inform the image control about the type of image format it will display. Encapsulating the SVG code inside the `EncodeUrl` function ensures correct encoding. For more info on this, please visit <https://learn.microsoft.com/en-us/power-platform/power-fx/reference/function-encode-decode>.

Keep in mind that we can introduce control properties or variables inside the SVG code to make it work for us. For example, in **Image3**, we are referencing `Label1.Text` as the text to display. The following is an extract from that code:

```
<text x='20' y='70' style='font-family: Segoe UI; font-size: 72; font-weight: 600; text-shadow: 4px 4px 0px #511B5190; fill: #511B51AA;'>" & Label1.Text & "</text>
```

Another added value of using SVG images is the file size. Being text-only files, the file size is minimal. For comparison purposes, the background image on **Screen1** occupies just 474 bytes. If we used regular formats, such as JPG images, the file size would increase to 10,717 bytes. In the end, the fewer resources our applications consume, the better.

There's more...

Many SVG resources are freely available on the internet, just like the loading animation used on **Screen2**, which you can find here: <http://samherbert.net/svg-loaders>. Just make sure to replace double quotes with single quotes before wrapping the code inside the `EncodeUrl` function.

Transferring SharePoint list apps from one site to another

SharePoint list apps created with Power Apps improve the standard SharePoint forms by embedding canvas apps into lists or document libraries, taking advantage of Power Apps's full potential to handle SharePoint data. However, even though this makes life easier for users, it is not the same for makers.

By default, you can only customize the form using Power Apps Studio. To get the complete maker portal experience, you will have to fiddle through the list or document library settings because these embedded apps are not visible from the list of apps available on the maker's portal.

To make matters worse, you can't take advantage of the **Export/Import** feature. If you build the application on one SharePoint site and then plan to export it to another site, it won't work.

With PowerShell's help, we will learn how to take an exported application package and modify it so we can import it to the destination site. PowerShell is a scripting language available for almost every operating system that allows scripts to be executed with the help of code libraries created for particular scenarios.

Getting ready

This recipe uses PowerShell to transform the exported package. Starting with Windows 7 SP1, **Windows PowerShell** comes preinstalled with Windows, but there is also a newer version called **PowerShell**, which is multiplatform and is available on Windows, macOS, and Linux. For more information on how to install it, please refer to <https://learn.microsoft.com/en-us/powershell/scripting/install/installing-powershell>.

Notice that PowerShell has an execution policy to invoke scripts. Use the following command for a quick bypass:

```
Set-ExecutionPolicy -ExecutionPolicy Bypass
```

For more information regarding these policies, please visit the following link: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies.

We are going to use a PowerShell script that is already available in our GitHub repository. If you want to learn more about how to include PowerShell in your toolbelt, please refer to <https://learn.microsoft.com/en-us/training/paths/get-started-windows-powershell>.

How to do it...

In this recipe, we will customize a list on the source site (development) with a canvas app, and then we will export this app to the destination site (production).

Building and customizing the list on the source site

1. Go to the development SharePoint site, click on the gear at the top right of the screen, and select **Site contents**.
2. From the toolbar, click on **New** and select **List**. For simplicity, click on **Blank list**. In the open dialog, enter Power Platform in the **Name** field, disable **Show in site navigation**, and click **Create**.
3. Once in the list, from the toolbar, select **Integrate | Power Apps | Customize forms**. This process will start building the app from the list structure.
4. The app gets created with a **SharePointForm1** control with the **Fields** pane already open on the right of Power Apps Studio. From the list of fields, click on the ellipsis next to **Attachments** and select **Remove**.

5. Go to the **Insert** pane on the left (represented by a plus sign) and click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane on the right, set the **Text** property to `Power Platform`, the **Font size** to 24, the **Font weight** to **Semibold**, the **X** and **Y** properties to 0, the **Height** to 60, and the **Left** and **Bottom Padding** to 10
 - Using the formula bar, set the **Width** to `App.Width`, the **Color** to `Color.White`, and the **Fill** to `RGBA(0, 84, 148, 1)`
 - Name this label `HeaderLbl1`
6. From **Tree view**, select the **SharePointForm1** control and, using the formula bar, set the **Y** property to `HeaderLbl1.Height` and change the **Item** property to `SharePointIntegration.Selected`

The application should look like the following:

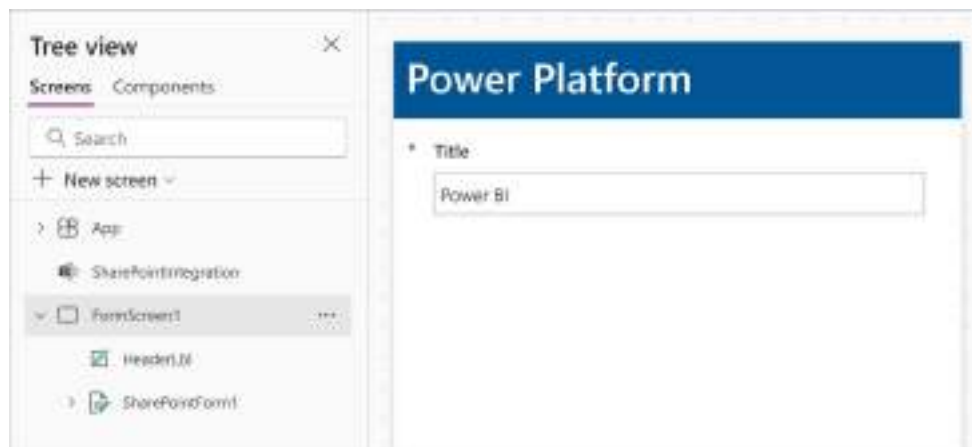


Figure 11.8: SharePoint list app

7. On the main toolbar, click on **Save** and then **Publish**. Click on **Publish to SharePoint** again on the open dialog. Finally, click the **Back to SharePoint** link in the top-left corner of the screen.
8. From the list's toolbar, click on **New** to verify it is using the embedded app. If you don't see it, refresh the browser a few times to get the changes.

Exporting the app

9. Still on the list, click on the gear in the top-right corner of the screen and select **List settings**. Once there, click on **Form settings** and then click on the **See versions and usage** link.

The **Details** page of the embedded canvas app will appear on the Power Apps maker portal. This is the only way to have the app visible on this portal.

10. From the toolbar, click on **Export package**, and on the page that opens, set the **Name** to Power Platform Dev. Leave the rest of the fields as the default.
11. From the **Review Package Content** section, click on the wrench icon next to the app. On the open pane, select **Create as new** and then click **Save**.
12. Click the **Download** button. After a while, it will download the package for this app. Save it to your computer and place `SPListTransform.ps1` from our GitHub repository in the same folder.

Configuring the destination site

13. Go to the production SharePoint site, click on the gear in the top-right corner of the screen, and select **Site contents**.
14. From the toolbar, click on **New** and select **List**. Given that you have access to both sites, click on **From existing list**. On the dialog that opens, select your development site from the list on the left and then choose **Power Platform** from the list on the right, as seen in the following screenshot:

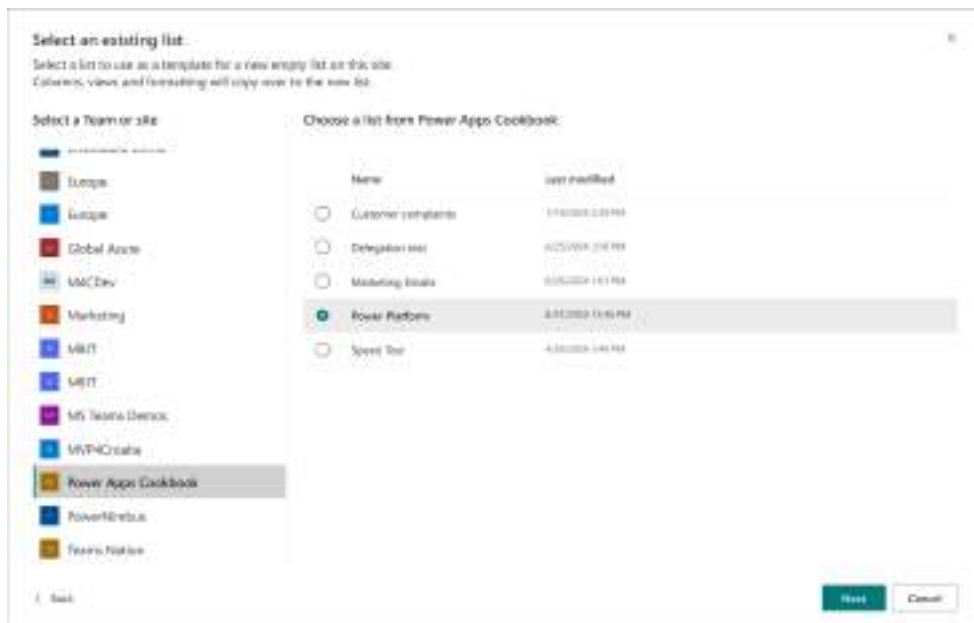


Figure 11.9: Existing list dialog

15. Click on **Next** and, on the dialog that opens, uncheck **Show in site navigation** and click **Create**.

If you don't have access to the development site from the production site, please create the list again, as we mentioned in *step 2* of the *Building and customizing the list on the source site* section.

16. Once the list gets created, go back to **Site contents** using the gear in the top-right corner of the screen. On the **Contents** list, click on the ellipsis next to the **Power Platform** list and select **Settings**.
17. Grab the following information:

- **siteUrl**: From the browser's address bar, copy the SharePoint site address. In our example, it is `https://ampicurrents.sharepoint.com/sites/Production`.

If you are working on the root site, then it should be something like `https://ampicurrents.sharepoint.com`.

- **listId**: From the browser's address bar, copy the GUID at the end of the URL. In our example, it is `6ddcd69c-9eb2-4a71-8924-1f392570fe5d`. Keep in mind that sometimes it might get wrapped between %7B and %7D. Remove them in that case.
- **listUrl**: From the **Settings** page, copy the **Web Address**. In our example, it is `https://powernimbus.sharepoint.com/sites/Production/Lists/PowerPlatform/AllItems.aspx`.

The data gets gathered from these locations:

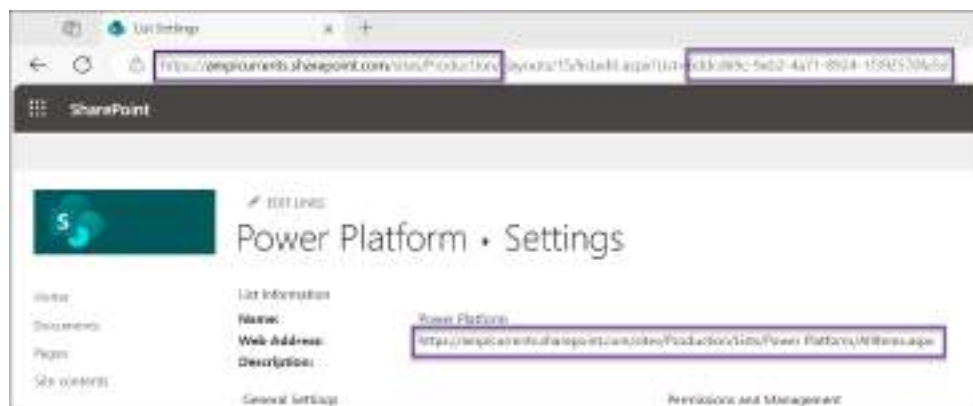


Figure 11.10: Transformation parameters

Keep this data safe, as we will use it later in the package transformation.

Transforming the exported package

18. On Windows devices, use File Explorer to navigate to the folder where the exported package and the script are located. Right-click anywhere in the folder to get the list of options available. Click on **Open in Terminal**, as seen in the following screenshot:

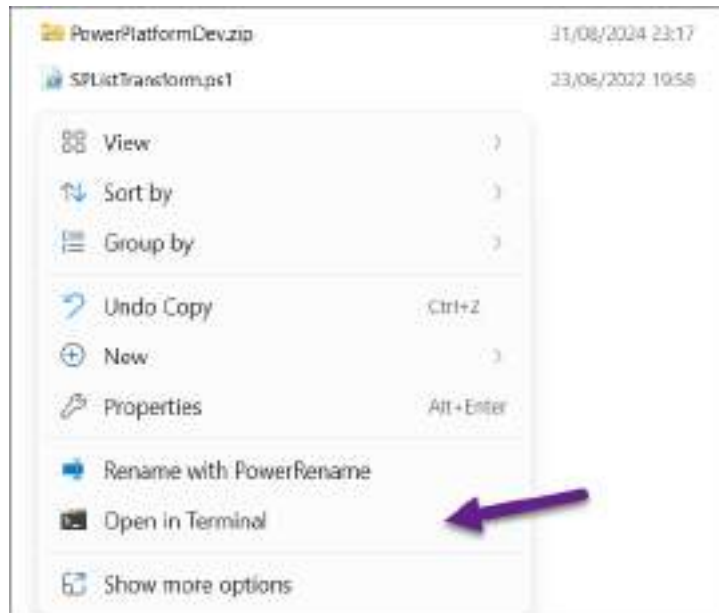


Figure 11.11: Terminal application available in Windows 11

19. The Terminal application will open by default with a Windows PowerShell tab. Enter the following command:

```
.\SPListTransform.ps1 -fileName "<package_filename>" -siteUrl  
"<siteUrl>" -listId "<listId>" -listUrl "<listUrl>"
```

Replace the text inside the <> placeholders with the values from your production site. Make sure not to forget the double quotes, especially when using URLs that might contain spaces. Using our example data, the command looks like the following:

```
.\SPListTransform.ps1 -fileName ".\PowerPlatform.zip" -siteUrl  
"https://ampicurrents.sharepoint.com/sites/Production" -listId  
"6ddcd69c-9eb2-4a71-8924-1f392570fe5d" -listUrl "https://  
ampicurrents.sharepoint.com/sites/Production/Lists/Power Platform/  
AllItems.aspx"
```

20. After entering the corresponding parameters, press *Enter*.



POWERSHELL EXECUTION POLICY

If you receive a security error about running scripts disabled on the system, you might need to change the execution policy. Please refer to the *Getting ready* section for instructions on bypassing this policy.

21. When the script completes, a new compressed file will appear with the name `Result.zip`

Importing the new package

22. Go to the Power Apps portal, <https://make.powerapps.com/>, and select **Apps** from the left pane.
23. From the toolbar, click on **Import canvas app**. On the **Import package** page, click **Upload** and browse for the `Result.zip` file.
24. Once it finishes uploading, in the **Review Package Content** section, click on the wrench action next to the app element.
25. In the **Resource name** field, enter `Power Platform Prod` and click **Save**.
26. Back on the **Import package** page, click on **Import**. After a couple of seconds, a message will appear letting you know that the import process has been successful, as seen in the following screenshot:



Figure 11.12: Successful package import

27. Click on the **Open app** link to complete the app configuration.
28. Once Power Apps Studio opens, go to the **Data sources** pane on the left. Click on the ellipsis near the existing SharePoint connection and select **Remove**.

29. Click **Add data** and enter SharePoint using the search field at the top of the **Data sources** pane. Once found, select it from the list and use one of the active connections, given that this connection has access to the production site, or click on **Add a connection** to configure it to the production site.
30. After setting up the connection, select the production site from the list of sites that appear on the right pane, and then select the **Power Platform** list. Click on **Connect** when finished.
31. Click the **Save** icon and then **Publish**. Click **Publish to SharePoint** again on the open dialog, and then click on the back arrow at the top left of the screen.
32. Close Power Apps Studio and go to your list on the production site.
33. Refresh a couple of times until you get the embedded canvas app when you click on **New** to enter data, as seen in the following screenshot:



Figure 11.13: SharePoint list app

Now, let's discover how the script works.

How it works...

Exporting a canvas app using the Power Apps maker portal creates a compressed ZIP file with all the configuration and resources needed. It contains elements such as connections, components, and image resources.

The script takes this package and looks for the main configuration file, which connects the app with the list. Using the script with the production parameters changes this link to the production list.

You can use this technique to replicate embedded apps on different sites or even another list on the same site.

The destination lists can have different structures, and when you change the connection at the end of the import procedure, you can adapt the app for the list.

There's more...

The Power Platform comes with its own set of libraries for PowerShell, letting you extend the available scripting capabilities. One is specific when dealing with canvas apps, **pac canvas**, which allows you to pack, unpack, and even download applications from environments.

To get more information, please visit <https://learn.microsoft.com/en-us/power-platform/developer/cli/reference/canvas>.

If you're interested in learning more about PowerShell, consider looking into the fifth edition of *Mastering PowerShell Scripting*, by Chris Dent.

Troubleshooting using Power Apps Monitor

Monitoring application performance is a must for business solutions built for an enterprise. As a Power Platform maker, you need to take care of application slowdowns, connectivity issues, or any other incident that might arise.

As we mentioned in *Chapter 10, Discovering the Power Platform Admin Center*, you can use the **Performance** page from the **Analytics** metrics to measure execution variables for a specific app.

The Power Apps portal provides a monitoring tool that gathers performance indicators in real time. We will learn how to take advantage of this tool in this recipe.

Getting ready

We will need a SharePoint list with data already loaded to build this recipe. We provide an Excel file to create this list as well as the completed app in our GitHub repository located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-11>.

How to do it...

For this recipe, we will need to provision test data into a SharePoint list and then use this list as the data source for the canvas app that we will use to discover the capabilities of Monitor.

Provisioning the SharePoint list

1. Go to the desired SharePoint site, click on the gear in the top-right corner of the screen, and select **Site contents**.
2. From the toolbar, click on **New** and select **List**. Select **From Excel** and then click on the **Upload file** button. Browse to choose `MonitorData.xlsx`, which was downloaded from our GitHub repository.

3. On the **Customize** dialog, leave the default settings and click on **Next**. SharePoint will provide a name for the list, change the **Name** field to **Monitor Data**, and uncheck **Show in site navigation**. Click **Create** when finished.
4. Wait a few seconds until the list gets created before moving to the next step. You will get redirected to the list when finished.

Building the app

5. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and create a solution. Once it opens, add a connection reference to **SharePoint**.
6. From the toolbar, click **+ New | App | Canvas app**. In the open dialog, enter **Monitor Data** for the **App name**, set **Phone** as **Format**, and click **Create**.
7. Once Studio opens, go to the **Data sources** pane on the left (represented by a database icon), click **Add data**, and look for **SharePoint**. Configure the connection using your credentials or use an existing one. From the right pane, select the site and list used in the previous section and click **Connect**.
8. Go to the **Insert** pane on the left (represented by a plus sign) and select **Button** under **Popular**:
 - Using the **Properties** pane on the right, set the **Text** property to **Non-Delegable** and the **Y** property to **100**
 - Using the formula bar, set the **Width** property to `App.Width - Self.X * 2`, and the **OnSelect** property as follows:

```
UpdateContext(  
  {  
    result: First(  
      Filter(  
        'Monitor Data',  
        IsBlank(name)  
      )  
    ).name  
  }  
)
```

9. Right-click **Button1** on the canvas, select **Copy**, then right-click anywhere on the canvas and choose **Paste**. Set the new button's properties, **Button1_1**, as follows:
 - Using the **Properties** pane, set the **Text** property to **Delegable**, **X** to **40**, and **Y** to **200**

- Using the formula bar, set the **OnSelect** property to the following:

```
UpdateContext(  
  {  
    result: First(  
      Filter(  
        'Monitor Data',  
        name = Blank()  
      )  
    ).name  
  }  
)
```

10. Right-click anywhere on the canvas again and select **Paste**. Set the **Button1_2** properties as follows:

- Using the properties pane, set the **Text** property to **Collection**, **X** to 40, and **Y** to 300
- Using the formula bar, set the **OnSelect** property to the following:

```
ClearCollect(  
  dataCol,  
  'Monitor Data'  
);  
UpdateContext(  
  {  
    result: First(  
      Filter(  
        dataCol,  
        IsBlank(name)  
      )  
    ).name  
  }  
)
```

11. Go to the **Insert** pane on the left and select **Text label** under **Popular**. Set its properties as follows:

- Using the **Properties** pane, set the **Text alignment** to **Center**, **X** to 40, and **Y** to 400
- Using the formula bar, set the **Text** property to **result**

The completed application should look like the following:

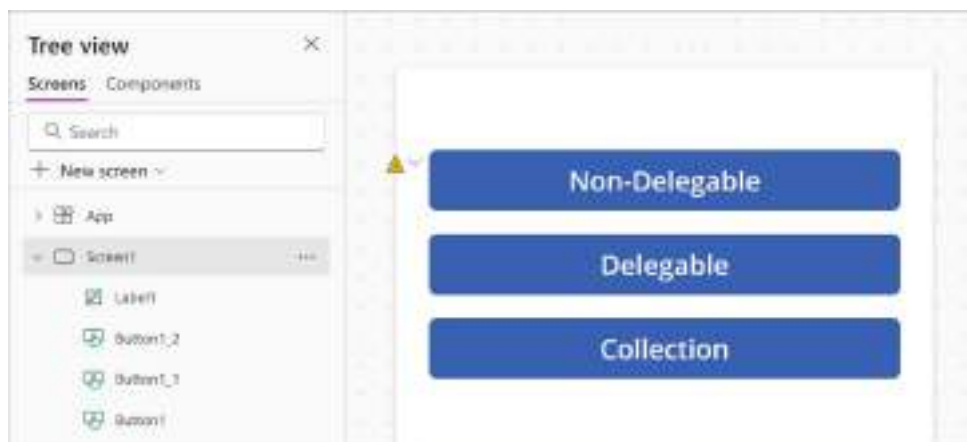


Figure 11.14: Canvas app ready to monitor

12. From the main toolbar, click on **Save**. Go back using the back arrow in the top-left corner of the screen.
13. From the solution, click on the ellipsis next to the **Monitor Data** app and click **Monitor**. This action will open a new window with the Monitor tool. From its toolbar, click **Play published app**.

Now that everything is set up, let's see how the Monitor tool helps discover how the Power Apps backend works.

How it works...

The Monitor tool connects to the Power Apps backend to observe every interaction with the application, from button clicks to the screen's **OnVisible** executions.

Place each window next to the other for best results, as seen in the following screenshot, or better yet, use two computer monitors if available.



Figure 11.15: Power Apps Monitor side by side with the Monitor Data app

Click the buttons in this order: the **Non-Delegable** button, the **Delegable** button, and finally, the **Collection** button.

From the Monitor window, expand the **Result info** column to see what happens behind the scenes when requesting data.

The following is the result of this execution:

	Id	Time	Category	Operation	Result	Result info
	1	00:37:20.167	Performance	App.OnStart	Success	
	2	00:37:20.192	ScreenLoad	Navigate (Screen1)	Success	
	3	00:37:20.192	ScreenLoad	LoadScreen (Screen1)	Success	
	4	00:37:19.373	UseAction	Select	Success	
	5	00:37:20.527	Network	getFlows	Success	
	6	00:37:20.865	Network	getFlowsCount	Success	Requested 500 rows. Received 500 rows.
▲	7	00:37:20.828	Delegation	Filter	Warning	Formula not delegated. Partial dataset of 500 rows scanned.
	8	00:37:20.838	Function	Filter	Success	
▲	9	00:37:20.831	Delegation	Find	Warning	Formula not delegated. 0 rows scanned.
	10	00:37:24.808	UseAction	Select	Success	
	11	00:37:24.991	Network	getFlows	Success	
	12	00:37:24.992	Network	getFlowsCount	Success	Requested 1 rows. Received 0 rows.
	13	00:37:29.328	UseAction	Select	Success	
▲	14	00:37:29.371	Function	ClearCollect	Warning	Partial dataset. 500 rows collected.
	15	00:37:29.393	Function	Filter	Success	

Figure 11.16: Power Apps Monitor session

The execution shows the following results:

- The lines with an **Id** of 4 to 9 represent the request for the **Non-Delegable** button. As you can see, even though we request the first element with an empty name, the data source gets scanned to the default delegation limit of 500, thus sending the warning.
- Lines 10 to 13 represent the execution of the **Delegable** button. The code executes the same type of request as the **Non-Delegable** button but with a twist that permits delegation on SharePoint. For more information, please visit <https://learn.microsoft.com/en-us/connectors/sharepointonline/#power-apps-delegable-functions-and-operations-for-sharepoint>.
- Finally, lines 13 to 15 represent one common workaround many makers tend to do to avoid delegation warnings: using collections to get the data and perform the filter. However, as you can see, it hits the delegation limit and throws an alert on Monitor.

These are vital indicators of how performant your app can be. For more information regarding the capabilities of this tool, please refer to <https://learn.microsoft.com/en-us/power-apps/maker/monitor-canvasapps>.

There's more...

Canvas apps are not the only ones that can benefit from the Monitor tool. Model-driven apps can also take advantage of it to review events gathered from an app. For more information on how to use this tool on these types of apps, please refer to <https://learn.microsoft.com/en-us/power-apps/maker/monitor-modelapps>.

Extending screen real estate using the canvas control

When building apps, you must consider your app's specific UI needs. You can design many screens, use tabbed menus, or use forms to organize the input data.

However, one control remains hidden from the **Insert** pane in Power Apps Studio, which only appears when a specific type of screen gets created: **canvas**. This control generates a container that scrolls vertically to the dimensions you require, just like a website on the internet. This recipe will show you how to use it.

Getting ready

The completed app, as well as the image resources, are available in our GitHub repository.

How to do it...

The following steps will use the canvas control to replicate a website:

1. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and create a solution.
2. From the solution's toolbar, click + **New | App | Canvas app**. In the open dialog, enter Power Apps Homepage for the **App name**, keep **Tablet** as **Format**, and click **Create**.
3. Once Power Apps Studio opens, from the toolbar, click on **Settings | Display**, turn off **Scale to fit**, and close the dialog.
4. Click on the **Media** panel on the left (represented by a movie reel) and then click on **Upload**. Browse to select the **hero.jpg**, **Power Apps.svg**, and **Microsoft.png** files downloaded from our GitHub repository.
5. From the toolbar, click on **New screen** and select **Scrollable**.
6. From the list of screens, click the ellipsis next to **Screen1** and choose **Delete**.
7. Click the ellipsis next to **Screen2** and click **Rename**. Set the name as **HomepageScreen**.

8. From the **Tree view** pane, select **LblAppName1**, and using the ellipsis menu next to it, choose **Delete**.
9. From the **Tree view** pane, select **RectQuickActionBar1** and rename it as **HeaderRct**. Right-click on this control and choose **Reorder | Bring to front**. Using the **Properties** pane on the right, change the **Height** to 60 and the **Color** to **White**
10. From the **Insert** panel on the left (represented by a plus sign), expand **Layout** and click on **Container**. Set its properties as follows:
 - Using the **Properties** pane, set the **Y** property to 0, the **Height** to 60, and **Drop shadow** to **None**
 - Using the formula bar, set the **Width** property to $\text{App.Width} * 0.85$ and the **X** property to $(\text{App.Width} - \text{Self.Width}) / 2$
 - Name this container **HeaderCnt**
11. Keep **HeaderCnt** selected and, using the **Insert** panel, click on **Image** under **Media**:
 - Using the **Properties** pane, set the **Image** property to **Microsoft**, **X** to 0, the **Width** to 120, and all **Padding** properties to 10
 - Name this image **MicrosoftLogoImg**
12. Keep **HeaderCnt** selected and, using the **Insert** panel, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, set the **Text** property to **| Power Platform**, the **Font** to **Segoe UI**, the **Font size** to 14, the **Font weight** to **Semibold**, **X** to 120, **Y** to 0, the **Width** to 160, and the **Height** to 60
 - Name this label **PowerPlatformLbl**
13. Right-click **PowerPlatformLbl** and choose **Copy**. Right-click anywhere inside **HeaderCnt** and select **Paste**. Select **PowerPlatformLbl_1** and set the properties as follows:
 - Using the **Properties** pane, set the **Text** property to **Power Apps**, the **Font size** to 12, **X** to 300, and the **Width** to 100
 - Name this label **PowerAppsLbl**
14. Right-click **PowerAppsLbl** and choose **Copy**. Right-click anywhere inside **HeaderCnt** and select **Paste**. Select **PowerAppsLbl_1** and set the properties as follows:

- Using the **Properties** pane, set the **Text** property to About Power Platform Products Pricing Partners Resources (maintain the four spaces between each option), the **Font weight** to **Normal**, **X** to 440, the **Font size** to 10, and the **Width** to 400
 - Name this label MenuLb1
15. Keep **HeaderCnt** selected and, using the **Insert** panel, click on **Button** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, set the **Text** property to Buy now, **Y** to 15, the **Width** to 60, the **Height** to 30, the **Color** to Black, the **Fill** color to White, the **Border width** to 1, the **Border color** to Black, the **Border radius** to 4, the **Font** to Segoe UI, the **Font size** to 10, and the **Font weight** to **Normal**
 - Using the formula bar, set **X** to `Parent.Width - 255`
 - Name this button BuyNowBtn
 16. Right-click **BuyNowBtn** and choose **Copy**. Right-click anywhere inside **HeaderCnt** and select **Paste**. Select **BuyNowBtn_1** and set the properties as follows:
 17. Using the **Properties** pane, set the **Text** property to Start Free, **Y** to 15, the **Color** to White, and the **Fill** color to Black
 18. Using the formula bar, set **X** to `Parent.Width - 165`
 19. Name this button StartFreeBtn
 20. Right-click **BuyNowBtn** and choose **Copy**. Right-click anywhere inside **HeaderCnt** and select **Paste**. Select **BuyNowBtn_1** and set the properties as follows:
 - Using the **Properties** pane, set the **Text** property to Sign in, **X** to 1220, and **Y** to 15
 - Using the formula bar, set **X** to `Parent.Width - Self.Width - 5`
 - Name this button SignInBtn

Now that we have completed the header of the home page, your app should look like the following:



Figure 11.17: Header component complete

21. From **Tree view**, select the **Canvas1** control and rename it as **HomeCvs**. Expand it, select **DataCard1**, and rename it as **HeroDtc**. Using the formula bar, set the **Height** to **1200**. To insert the following controls, always keep **HeroDtc** selected.
22. Using the **Insert** panel, click on **Image** under **Media**. Set its properties as follows:
 - Using the **Properties** pane, set the **Image** property to **hero**, the **Image position** to **Fill**, the **X** and **Y** properties to **0**, and the **Width** to **600**
 - Using the formula bar, set the **Width** to **Parent.Width**
 - Name this image **HeroImg**
23. Using the **Insert** panel, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, set the **Text** to **Power Apps**, the **Font** to **Segoe UI**, the **Font size** to **28**, the **Font weight** to **Semibold**, **X** to **250**, **Y** to **90**, the **Width** to **250**, and the **Height** to **100**
 - Name this label **HeroTitleLbl**
24. Add another **Text label**. Set its properties as follows:
 - Using the **Properties** pane, set the **Text** to **Deliver apps embedded with AI to modernize your legacy applications and systems fast.**, the **Font** to **Segoe UI**, the **Font size** to **14**, **X** to **250**, **Y** to **160**, the **Width** to **600**, and the **Height** to **60**
 - Name this label **HeroSubTitleLbl**
25. Using the **Insert** panel, click on **Container** under **Layout**. Set its properties as follows:
 - Using the **Properties** pane, set **X** to **250**, **Y** to **240**, the **Width** to **640**, the **Height** to **200**, the **Border radius** to **16**, and the **Drop shadow** to **Regular**
 - Name this button **HeroCnt**
26. Select **HeroCnt**, and using the **Insert** panel, click on **Image** under **Media**. Set its properties as follows:
 - Using the **Properties** pane, set **Image** to **Power Apps**, the **X** and **Y** properties to **30**, the **Width** and **Height** to **50**, all **Padding** to **10**, the **Color** to **white**, and the **Border radius** to **8**
 - Name this label **PowerAppsImg**

27. Select **HeroCnt**, and using the **Insert** panel, click on **Text label** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, set **Text** to Start from a table, a sketch, a sentence, or your own code and build solutions your way., the **Font** to **Segoe UI**, the **Font size** to 12, the **Font weight** to **Semibold**, **X** to 80, **Y** to 35, and the **Width** to 540
 - Using the formula bar, set the **X** property to $(\text{App.Width} - \text{Self.Width}) / 1.8$ (don't mind the overlap)
 - Name this label **GuidedTourLbl1**
28. Select **HeroCnt**, and using the **Insert** panel, click on **Button** under **Popular**. Set its properties as follows:
 - Using the **Properties** pane, set **Text** to Try for free, **X** to 20, **Y** to 125, the **Width** to 100, the **Height** to 50, the **Border width** to 0, the **Border radius** to 6, the **Font** to **Segoe UI**, the **Font size** to 12, and the **Font weight** to **Semibold**
 - Using the formula bar, set the **Fill** property to `ColorValue("#006dc1")`
 - Name this label **TryForFreeBtn**
29. Right-click **TryForFreeBtn** and choose **Copy**. Right-click anywhere inside **HeroCnt** and select **Paste**. Select **TryForFreeBtn_1** and set the properties as follows:
 - Using the **Properties** pane, set the **Text** property to Take a guided tour, **X** to 140, **Y** to 125, the **Width** to 150, **Y** to 95, and the **Border width** to 1
 - Using the formula bar, set the **Fill** property to `Color.Transparent`, the **Border-Color** to `ColorValue("#006dc1")`, and the **Color** to `RGBA(0, 66, 117, 1)`
 - Name this button **GuidedTourBtn**
30. From the main toolbar, click on **Publish**. When saving and publishing finishes, you can close the browser tab.

These steps complete the first section of the Power Apps home page simulation.

How it works...

Back in the Power Apps portal, click on **Apps**, and look for **Power Apps Homepage**. Click on it to run it. In the new browser tab, you will see the application acting as the Power Apps home page. You have created a replica of the first section of the Power Apps home page (September 2024 edition).

Using the canvas control, we can imitate the scrolling effect websites have when they have more content, as you can see in the following screenshot:

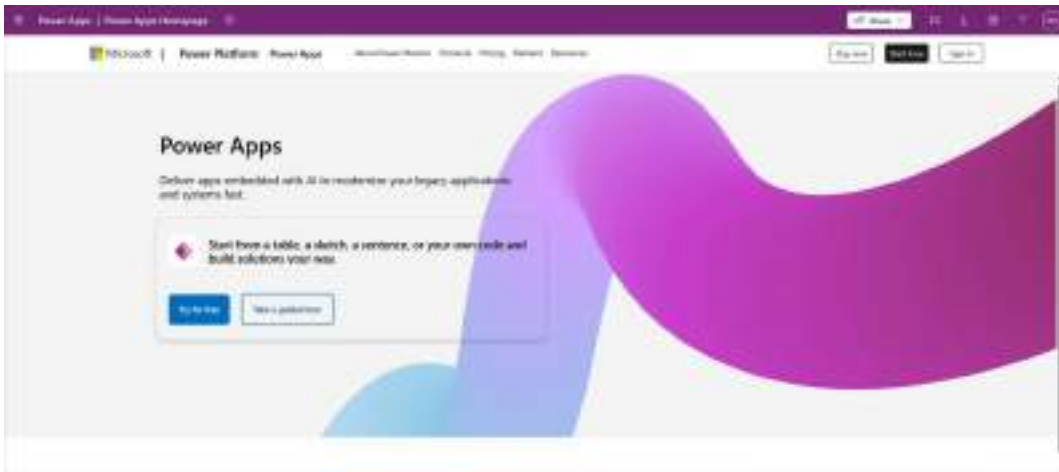


Figure 11.18: Power Apps home page replicated on a canvas app

If you scroll, the header will remain on top, and the rest of the contents will move accordingly. For best results, add `&hiddenavbar=true` at the end of the URL in the browser's address bar to hide the top Power Apps navigation bar.

You can use the provided app to continue replicating the Power Apps home page as an exercise.

Updating Azure SQL Server connections in Power Apps

Building an app using an Azure SQL database is a great approach when creating a solution to solve a complex business need. This data source provides many advantages, such as the following:

- Views, which can help perform tasks such as data joining and transformations at the database level
- Stored procedures, which consist of a series of code routines that can be executed in the database to help with data processing and validation tasks
- Scalability, as you can improve the database engine's performance on-demand or by setting a set of rules

These are just some examples of your application's instant capabilities when using this data source.

When building solutions for the enterprise, using environments such as development, testing, and production is a good practice. These environments help developers iterate application versions without affecting end-users daily tasks when using your solution.

However, when using Azure SQL canvas apps, transferring apps between environments is not as straightforward as it should be. This is because the data source connection is linked to the app. When you move it to another environment, it remains connected to the development database. There is no easy way to change it to a production database other than deleting all connections and adding them again, pointing to the production database.

Getting ready

In this recipe, we will explore a solution to simplify the transfer process using PowerShell. If you need instructions on how to prepare your device to work with PowerShell, please refer to the *Getting ready* section of the *Transferring SharePoint list apps from one site to another* recipe.

The complete script can be found in our GitHub repository. We will use it with specific parameters to help us transfer our application.

Azure SQL databases

For this scenario, we will need two databases: one for the development environment and another for the production environment. Refer to *Chapter 1, App Maker Basics*, or follow this quick-start guide to help you build the required databases: <https://docs.microsoft.com/en-us/azure/azure-sql/database/single-database-create-quickstart?tabs=azure-portal>.



Be sure you select to use sample data when creating the databases.

How to do it...

To demonstrate the use of this recipe, first provision the Azure SQL databases. For simplicity, we will assume the names devDatabase and proDatabase.

We will also need two environments (Dataverse is not required): Development and Production. If you already have these environments, skip to the *Building the app* section.

Creating the environments

1. Go to the Power Platform admin center by opening this URL: <https://admin.powerplatform.microsoft.com> or <https://aka.ms/ppac> for short. Once in the admin center, select **Environments** on the left panel.
2. From the toolbar, click **New** and, on the panel that opens on the right, enter **Development** for the **Name**. Select a **Region** closer to your location; for **Type**, choose **Sandbox**. Click **Save** when finished.
3. Repeat *step 2*, but set **Name** as **Production** and **Type** as **Production**.

Building the app

4. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and ensure you are in the **Development** environment (or your environment of choice) by using the environment selector near the top-right corner of the toolbar.
5. If you have never created an SQL Server connection in the current environment, add one by clicking **Connections** from the left pane.

From the toolbar, click + **New connection**, select **SQL Server**, and provide the required parameters depending on the authentication type.

6. Return to the maker home page by clicking **Home** from the left pane. Click **Start with data** from the **Other ways to create an app** section.
7. From the **Create an app** wizard, click **Select external data** and then click **From SQL**.
8. Select the previously configured connection, then enter the **Server** name and the **Database name** and click **Connect**. Choose the **SalesLT.Customer** table and click **Create app**, as seen in the following screenshot:

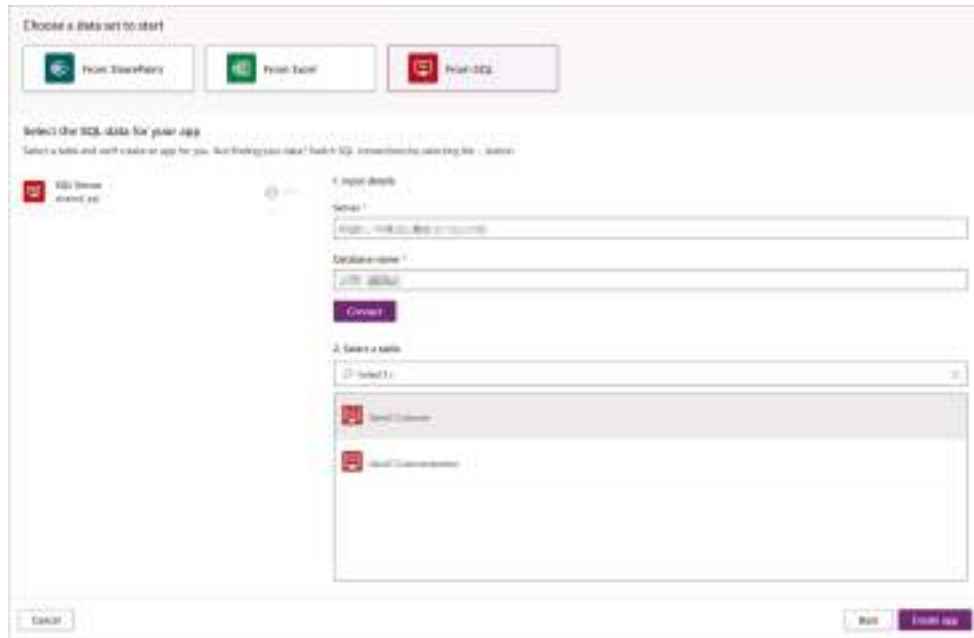


Figure 11.19: SQL Server parameters

9. You will arrive at Power Apps Studio with a complete application to maintain the database's customers. Click the **Publish** icon on the main toolbar. Set the application **Name** to Customer Management and click **Save**. You can close the designer now.
10. Back in the maker portal, click on **Apps** and look for the newly created app. Click the three dots next to it and select **Export package**.
11. On the **Export package** page, enter Customer Management Solution for **Name**, click on the wrench icon next to the **App** resource, and select **Create as new** on the panel that opens on the right. Hit **Save** afterward.

Click **Export** and download the application package to a folder on your device.

18. Invoke the script using the configuration properties and the connection IDs gathered from the previous steps. The following is an example of the script invocation:

```
.\SQLMigrator.ps1 '  
-fileName .\CustomerManagementSolution.zip '  
-sourceServer devServer.database.windows.net '  
-sourceDatabase devDatabase '  
-sourceConnectionID 3a95ce6c2c6e430a82ca37b333943edc '  
-destinationServer proServer.database.windows.net '  
-destinationDatabase proDatabase '  
-destinationConnectionID 1196004e99734f52ac5794eac172333f
```



POWERSHELL BACKTICK

Using the backtick symbol (`) allows you to write commands using multiple lines. It improves readability on scripts that require many or too-long parameters.

19. Enter the corresponding parameters and then press *Enter*.
20. When the script finishes, a new file will appear with the `-migrated` suffix.
21. Return to the Power Apps maker portal and ensure you are in the **Production** environment. On the left pane, click on **Apps** and then click on the toolbar's **Import canvas app** button. Click on **Upload** and browse to select the migrated version of the package.
22. When the **Import package** page opens, click on the wrench icon next to the **SQL Server Connection** resource and, on the pane that opens on the right, select the production connection and click on **Save**. Click on **Import** when finished.
23. When the platform completes the import, an **Open app** link will appear in the **Next steps...** section. Click on it to open the app.

Since this will be the first time you open the app, you will get prompted to consent to the Azure SQL connector's use.

24. Go to the **Data sources** pane on the left (represented by a database icon) and verify that the application now uses the destination database by hovering over the connection, as seen in the following screenshot:

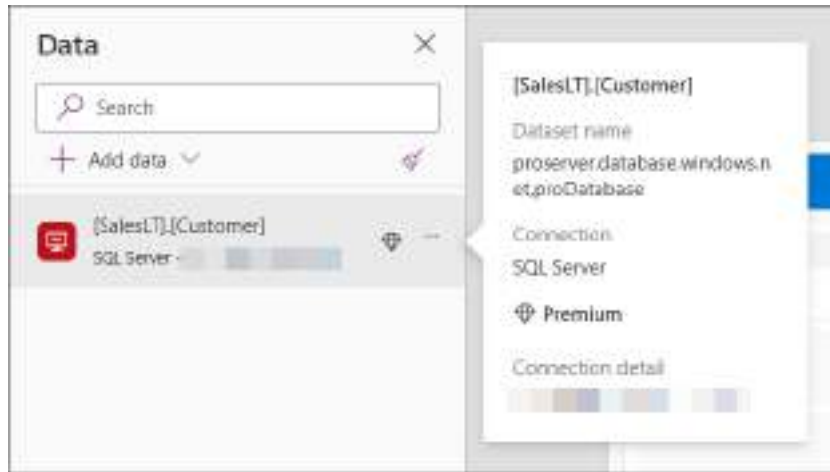


Figure 11.21: Production database connection

25. Click on the connection's ellipsis and select **Refresh**. If it refreshes successfully, the connection transformation process is a success.

How it works...

Using the provided script, you can change the connections included in the exported package. It takes care of the package configuration properties and the included app by replacing the source connection configuration with the destination one:

- **fileName:** This is the exported “base” solution.
- **sourceServer:** This represents the source Azure SQL Server name.
- **sourceDatabase:** As before, this is the source Azure SQL database name.
- **sourceConnectionID:** This is the GUID of the source connection.
- **destinationServer:** This is the destination Azure SQL Server name.
- **destinationDatabase:** This is the destination Azure SQL database name.
- **destinationConnectionID:** This is the GUID representation of the destination connection.

The destination parameters represent the identifiers of the destination Azure SQL Server, database, and connection ID.

The previous example transforms a simple application with just one database connection configured. Complex solutions tend to have multiple connections for each table or view used in the app. Manually changing them when transferring the app from different environments would be tedious.

You can use this technique not only in different environments of the same tenant but also to transfer an app you've been building in your organization to an external third party. Each time you complete a milestone, export the app, execute the script on the package, and install it in your client's environment.

There's more...

The transformation script uses a native ZIP file unpack mechanism available in PowerShell. The Power Platform CLI introduces native capabilities to allow unpacking to a YAML format for improved readability. Besides this, you can do more tasks, such as downloading a canvas app, packing it, and even deploying it. At the time of writing, these capabilities are in preview. You can find more information here: <https://learn.microsoft.com/en-us/power-platform/developer/cli/reference/canvas>.

Renaming files in SharePoint document libraries

SharePoint is the ideal choice for document classification systems. This platform provides a standard set of metadata properties while allowing us to create custom properties to improve data categorization.

For example, let's take a document library to hold company contracts. We can enhance this library by adding extra columns, such as the contractor's name, the expiration date, and the department that requested this contract. Besides using this information for ordering and classification, you can also act on this data:

- Build a Power Automate cloud flow that sends a Teams notification to a specific department warning them about an expiring contract.
- Design dashboards based on this data to understand the number of contracts per contractor or which department requests more contracts.
- Create a Power Apps app to gather data while on the go. This app can even collect signatures for these contracts.

Throughout this book, several recipes use Power Automate to handle SharePoint metadata in many ways. However, a particular metadata property is not editable by default: the filename.

Let's focus on this example. At the end of the year, you need to send contracts to an attorney who uses a legacy platform to handle the company's legal issues. If you extract these documents from SharePoint, you might lose some metadata.

In this recipe, we will change the names of the files inside a document library to prepare them for extraction, giving them meaningful names from the stored metadata.

Getting ready

The Power Automate cloud flow created in this recipe, as well as some sample files and code, are available in our GitHub repository.

For this recipe, we will need a SharePoint document library called Contracts. Please refer to *Chapter 1, App Maker Basics*, for instructions on how to create one using the following columns:

Column data type	Column name
Text	Contract number
Text	Reference
Date and time	Expiration

Table 11.1: Contracts library columns

How to do it...

Let's build this recipe by creating the SharePoint document library and the Power Automate cloud flow.

Creating the SharePoint document library

1. Go to your desired SharePoint site and create the document library.
2. From the toolbar, click on **New | Folder**. On the dialog that opens, enter **AMPI Currents** and click **Create**. Click on the name of this folder in the document library to open it.
3. Upload some sample files to this folder using the **Upload** button on the toolbar or by dragging them from the file explorer to this document library. You can use the sample files provided by our GitHub repository.
4. Update the column data of these files using test data. For easier updates, click **Edit in grid view** from the toolbar to edit data in bulk. The **AMPI Currents** folder should look similar to the following:



Name	Contract number	Reference	Expiration
Contract.docx	88542	315487	3/16/2025
Data report.xlsx	88543	154454654	12/31/2024
Data request form.pdf	88544	49546	12/31/2025

Figure 11.22: AMPI Currents folder inside the Contracts library

Now that we have created the document library, let's build the cloud flow.

Building the cloud flow

5. Navigate to the Power Automate maker portal and create a solution. Once it opens, add a connection reference to SharePoint. For the next step, please refer to *Chapter 1, App Maker Basics*, for instructions on how to build a cloud flow using the new designer.
6. From the toolbar, click **+ New | Automation | Cloud flow | Instant**. In the open dialog, enter **Process Contracts** for **Flow name**, select **For a selected item** under **SharePoint**, and click **Create**.
7. On the trigger action, open the dropdown on **Site Address**, and select **Enter custom value**. Enter **No data**. Do the same for **List Name**.



This configuration is necessary to unlink this cloud flow from any list. If not, anyone could execute it from a selected item.

8. Add a new action and look for **Get files (properties only)** under **SharePoint**:
 - For **Site Address**, select the site where you created the Contracts document library and select it from the **Libray Name** dropdown.
 - Expand **Advanced parameters**, look for the **Limit Entries to Folder** field, enter **/**, and select **Insert dynamic content**. Select **fileName** from the list.
9. Create a new action and look for **Send an HTTP request to SharePoint** under **SharePoint**:

- Choose the same site as before for **Site Address**, select **PATCH** for **Method**, and enter the following code for **URI**:

```
_api/web/lists/getbytitle('Contracts')/items()
```

- Put the cursor between the last parentheses and select the **ID** from the previous step using the **Dynamic content** tab. This action will automatically add an **Apply to each** action so it can process all files found in this document library. Click **Add when done**.
- Expand **Advanced parameters**, look for the **Headers** field, and enter If-Match for the key and * as its value.
- Expand **Advanced parameters**, look for the **Body** field, and copy the following code to paste it:

```
{
  "FileLeafRef": "@{items('For_each')?['ID']}-@
{items('For_each')?['Contractnumber']}-@{items('For_
each')?['Reference']}-@{triggerBody()?['fileName']}"
}
```

This action should look like the following screenshot:

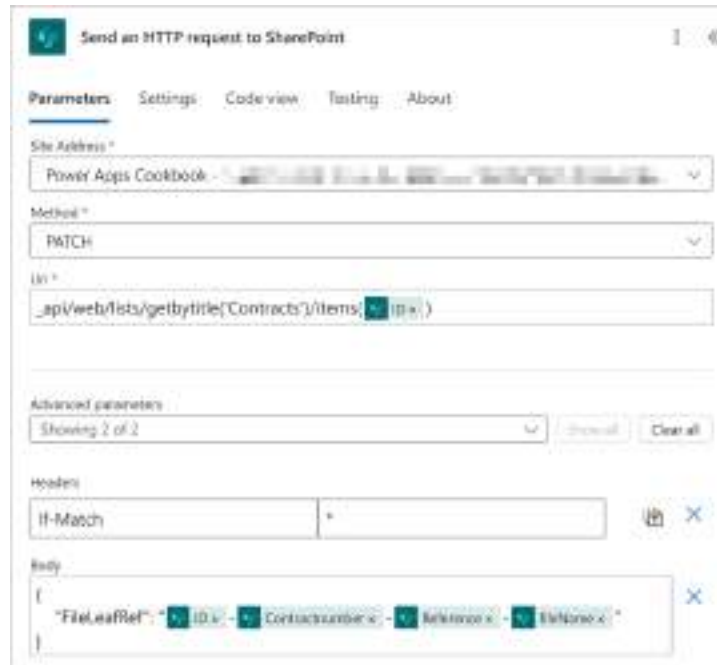


Figure 11.23: HTTP request to SharePoint action

10. From the toolbar, click on **Save draft** and then **Publish**. Click on the back arrow located on the left.
11. On the flow details page, click on **Export | Get flow identifier**. Copy the cloud flow's identifier on the pane that opens on the right.

Configuring the document library

12. Go back to the **Contracts** document library on SharePoint. Click on the header of the **Name** column, and select **Column settings | Format this column**.
13. On the pane that opens on the right, click on the **Advanced mode** link near the bottom. Paste the contents of the `Column_formatting.json` file, which is available from our GitHub repository. Look for the text `<FLOW_ID>` and replace it with the cloud flow ID collected in *step 11* of the previous section. When finished, click on **Save** and close this pane.
14. With the column format applied, the **Name** column will display a **Rename all** button next to each folder in this library, as seen in the following screenshot (adapt the column width to view the button correctly):

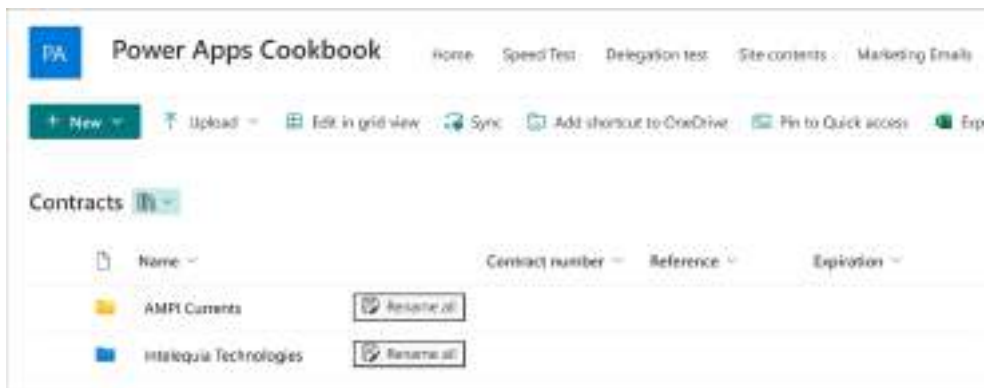


Figure 11.24: Name column formatted

Let's discover how the complete solution works.

How it works...

We have integrated a cloud flow into a document library to process the files inside folders using a SharePoint feature called column formatting. This solution will adapt the name of the files using their metadata so they can include this information when extracted from SharePoint.

Let's test this functionality by clicking on **Rename all** in the **AMPI Currents** subfolder. The first time you click on it, the right pane will ask for consent to use the SharePoint connector. Click on **Continue** to proceed to the confirmation to run the cloud flow. Click on **Run flow** to start it.

After a few seconds, open the **AMPI Currents** subfolder to view its contents. The filenames should have changed using the metadata gathered from the file **SharePoint ID**, the **Contract number**, and the **Reference** column, as seen in the following screenshot:

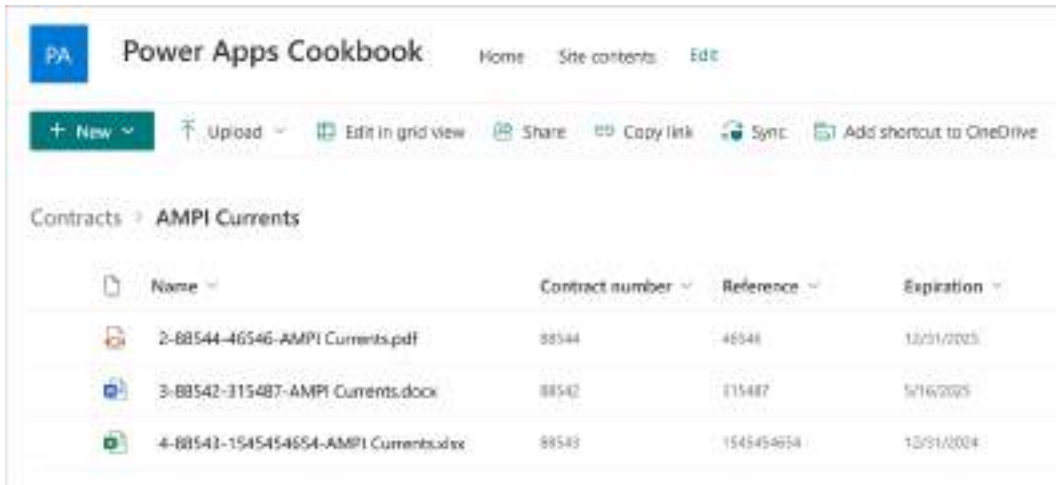


Figure 11.25: File name changes

You can change the filename pattern by updating the **Body** section of the **Send an HTTP request to SharePoint** action.

Some things to notice:

- We used a custom **Site Address** and **List Name** for the trigger action, so you can't invoke this cloud flow from a selected item in the document library.
- The button only appears in subfolders of this document library by setting a CSS display property inside the JSON column formatting, like so:

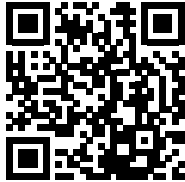
```
"display": "=if([$File_x0020_Type] == '', 'inherit', 'none')"
```

SharePoint column and view formatting are great tools to improve the views used in SharePoint. For more in-depth information about these features, please visit <https://pnp.github.io/List-Formatting/>.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



12

Advanced Techniques with the Power Apps Component Framework

Low-code platforms offer many possibilities for building applications, focusing on easy-to-learn languages, ready-to-use solutions, and a robust backend to support almost all business requirements.

However, some scenarios might require a component to solve a particular task with tools created using traditional coding technologies. Thankfully, the Power Platform is open to these approaches as well, and that's why we can leverage the advanced techniques of professional development using the **Power Apps Component Framework (PCF)**.



MONSTERS AHEAD

Through out this book, we have been working closely with the low-code portion of the Power Platform. This chapter will focus more on the pro-code side by using well-known web technologies to build a canvas component. Some knowledge of these technologies will be required to get the most out of this chapter.

This framework consists of code components that work just like regular Power Apps components. Using standard web technologies such as HTML, CSS, and TypeScript paired with extensive framework APIs creates reusable components built in a single solution package.

To get more information regarding these code components, please visit <https://learn.microsoft.com/en-us/power-apps/developer/component-framework/custom-controls-overview>.

Through a series of recipes, we will discover how to build a PCF solution using **Font Awesome** resources inside your canvas apps. This service offers an extensive icon library to help in web-based projects.

These recipes will work together to deliver a complete solution:

- Configuring your development environment
- Setting up Font Awesome
- Creating the project
- Building the application logic
- Deploying the solution
- Testing the component on a canvas app

Technical requirements

When using code components, the licensing requirements depend on the component's capabilities. If it doesn't connect to external services or data sources, it will only require a minimum license of Microsoft 365. On the other hand, connecting external services, including Dataverse for model-driven components, involves using a Power Apps license.

For more information on this regard, please refer to <https://learn.microsoft.com/en-us/power-apps/developer/component-framework/overview#licensing>.

The complete code solution is available from our GitHub repository located at <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-12>.

Configuring your development environment

Building code components requires configuring your local development device and, in the case of canvas apps, your Power Platform environments as well. These configurations involve installing specific tools to aid in component life cycle management.

The Microsoft Power Platform **Command-Line Interface (CLI)** is the primary tool for building the project structure for code components, and it requires several tools to perform its tasks. This recipe will cover the detailed steps to prepare the CLI installation, separating each tool installation into different sections.

Getting ready

The required tooling to build PCF projects supports Windows 10/11 or macOS. We will leverage the JavaScript runtime engine, **Node.js**, to serve on the scripting side and its package manager, **npm**, to gather all the required references.

When choosing a development tool to build these components, you have two options: **Visual Studio (version 2017 or later)** or **Visual Studio Code (VS Code)**. I suggest using **VS Code** because it is a lightweight editor focused on web technologies.

Regarding the Power Platform environment, it needs to include Dataverse to access the configuration settings to enable PCF use.

How to do it...

Before creating the code component, we will dive into the detailed installation instructions to configure the development environment. We will use Windows as the default system for this recipe.

Node.js/npm

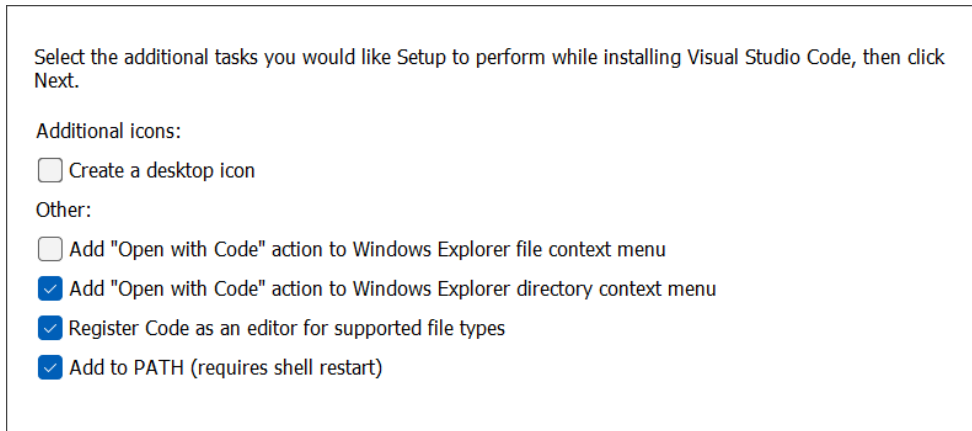
1. Go to the Node.js website, <https://nodejs.org/en/>, and look for the download button with the **Long Term Release (LTS)** tag. At the time of writing, the current version is **v20.17.0 LTS**.
2. Once downloaded, open the **.msi** file to begin the installation wizard:
 - Click **Next** on each step, ensuring that the **npm package manager** gets included in the installation. It gets included by default.
 - Don't include **Tools for Native Modules** unless you want to, as we will not use them for the solution.
3. When the install process finishes, right-click on the **Start** menu and select **Windows PowerShell | Windows PowerShell (Terminal on Windows 11)** to verify the Node.js/npm installation:
 - Enter `node -v` and press **Enter**. You should receive a message like **v20.17.0**.
 - Enter `npm -v` and press **Enter**. You should receive a message like **10.8.2**.

These messages verify the correct installation of this tool. Keep in mind that these versions will get updated over time.

Editor – VS Code

This editor is the preferred development tool for creating PCF components:

4. Navigate to <https://code.visualstudio.com/download> to get the VS Code installer. Click on the **Windows** button to download the installation package.
5. Once downloaded, run the executable. Click on **Next** on each screen of the installation wizard. On the **Select Additional Tasks** screen, we recommend checking all options in the **Other** section as they make VS Code easier to access from the whole system:



Select the additional tasks you would like Setup to perform while installing Visual Studio Code, then click Next.

Additional icons:

☐ Create a desktop icon

Other:

☐ Add "Open with Code" action to Windows Explorer file context menu

☒ Add "Open with Code" action to Windows Explorer directory context menu

☒ Register Code as an editor for supported file types

☒ Add to PATH (requires shell restart)

Figure 12.1: Select Additional Tasks screen

6. Remember that if you choose this editor, you need to install **Build Tools for Visual Studio**. There is a lightweight version that doesn't install the core version of Visual Studio. Use this URL to download it:

https://aka.ms/vs/17/release/vs_buildtools.exe

7. Click **Install** and close the installer when finished.

Editor – Visual Studio 2017 or later

If you plan to use this editor, I recommend using the 2022 edition, as it is the latest version to date:

8. Navigate to <https://visualstudio.microsoft.com/downloads/> and click on the **Free download** button just below the **Community** section of the page. This action will initiate the download of the Visual Studio installer. Once downloaded, run the executable.
9. The installer will download the required components, and then it will display the **Workloads** tab. Select **Node.js development**.

10. Go to the **Individual components** tab and search for **nuget targets**, then check this component to include it in the installation, as seen in the following screenshot:

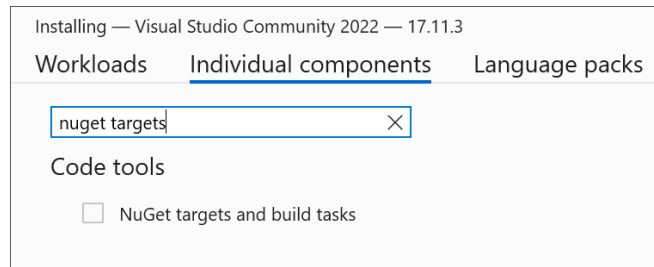


Figure 12.2: Individual components tab

11. Click on **Install** and wait for the installation process to complete.
12. Visual Studio will start automatically when the installation finishes.

Microsoft Power Platform CLI

13. Go to the following page, <https://aka.ms/PowerPlatformCLI>, to download the CLI installer. The preferred method is to integrate it into VS Code as an extension.
14. Follow the steps to use this method or navigate directly to this page to follow the installation steps:
<https://learn.microsoft.com/en-us/power-platform/developer/howto/install-vs-code-extension>
15. When the extension installation process is complete, open a new Terminal session from VS Code, enter `pac`, and press `Enter`. You should be able to see the Microsoft Power Apps CLI usage options.

Configuring the Power Platform environment

16. Go to the Power Platform admin center by opening this URL: <https://admin.powerplatform.microsoft.com> or <https://aka.ms/ppac> for short.
17. From the list of **Environments**, open the one that will host the code component.
18. From the toolbar, click on **Settings**, expand the **Product** section, and click **Features**.
19. Enable the **Allow publishing of canvas apps with code components** option and click **Save**.

How it works...

All the tools installed using this recipe play a different role in the PCF building process:

- `npm` manages the modules needed for this tooling

- Build Tools helps with the application assembly
- Visual Studio/VS Code serves as the full-fledged code editor

Remember that all program versions noted in this recipe might change with updates.

There's more...

Whether you choose to work with Visual Studio or VS Code, you will always need to install the Microsoft Build Tools. These tools are only available when you invoke **Developer PowerShell for VS 2022** from the **Start** menu. To allow access through the whole system, you can add the location of these tools to the global system path by following these steps:

1. Right-click the **Start** menu icon and select **Run**. You can also use the keyboard shortcut *Win + R*.
2. Enter `sysdm.cpl` and click **OK**. When the **System Properties** dialog opens, go to the **Advanced** tab and click **Environment Variables...**
3. Look for the **Path** variable in the **System variables** section and click **Edit...**
4. On the **Edit environment variable** dialog, click on **New** and enter the following path:

```
C:\Program Files (x86)\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin
```

If you used the lightweight version installer of Build Tools, then the path should be the following:

```
C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\MSBuild\Current\Bin
```

To be safe, you can also click on **Browse...** to navigate to the folder path. Click **OK** when finished, and then click **OK** on all remaining dialogs.

You can now invoke the `msbuild` command-line tool from any console in your system.

Setting up Font Awesome

Font Awesome is a well-known service among web developers that provides a massive variety of icons and social logos for websites worldwide.

This service provides users with **kits** to use its features. You can use these kits for different web projects as they act as containers with specific configurations.

Among the highlights of this service, we can find the following:

- A **Content Delivery Network (CDN)** for faster delivery of assets to use in your solutions
- Domain-level security to limit which sites can use your kits
- The ability to create icon subsets to load only the icons you need for your project

How to do it...

1. Navigate to the Font Awesome website, <https://fontawesome.com/>, and click the **Start for Free** button.
2. On this page, you will need to enter your email account to create your first kit. Once entered, click **Send Kit Code** and wait for the confirmation email to arrive in your inbox.
3. When the email arrives, click the confirmation message. You will then navigate to the Font Awesome website and complete your account configuration. Set it and continue until you reach the **How to Use** section of your first kit, as seen in the following screenshot:



Figure 12.3: Font Awesome kit configuration

4. Keep the URL of your kit in a safe place as we will need to use it as a parameter for our PCF component. In the previous screenshot, the URL inside the `src` is as follows:

```
https://kit.fontawesome.com/<XXXXXXX>.js
```

This completes the account configuration and the creation of your kit.

How it works...

The kit provides an excellent solution for adjusting the service to the specific requirements of each project. You can complete the kit's configuration using the **Settings** tab, modifying details such as its name, the pricing tier, and the allowed domains.

Using this approach, you can configure your code component using just the kit's URL. Later on, you can make changes to the kit without redeploying modifications to your project.

Creating the project

When working with PCFs, the Power Platform CLI is responsible for creating your component's project structure. It also manages the project's packaging as a Power Platform solution that can be deployed to your environments.

This command-line tool also performs other tasks, such as the following:

- Managing plug-in projects that harness the event framework of Dataverse
- Handling authentication to deploy solutions to the Power Platform environments directly from the command line
- Packing and unpacking of canvas apps

Please refer to the official documentation to discover these and other functionalities: <https://learn.microsoft.com/en-us/power-platform/developer/cli/introduction>.

Getting ready

Let's continue by building the project structure for our code component and configuring the project's *manifest*. This file defines your component by describing its metadata with information such as the name, version, properties, or required resources.

For a detailed reference on this file, please refer to <https://learn.microsoft.com/en-us/power-apps/developer/component-framework/manifest-schema-reference/manifest>.

We will continue the recipe using VS Code, as it is the ideal editor for this type of project. We will also mention how to start creating a component using Visual Studio.

How to do it...

1. Open VS Code, and if you don't have an active terminal console, click **Terminal | New terminal** from the menu bar.

2. From this console, navigate to the desired path for your project. For this recipe, we will use the following instructions to create our path:
 - Move to the root of the drive by entering `CD \` and pressing *Enter*
 - Create the `repos` folder and the `FontAwesomePCF` subfolder by entering `MD repos\FontAwesomePCF` and pressing *Enter*
 - Enter the project folder by typing `CD repos\FontAwesomePCF` and pressing *Enter*
3. Let's build the project structure using the Power Platform CLI using the following command:

```
pac pcf init --namespace PowerNimbus --name FontAwesomeComponent  
--template field
```



NAMESPACE: This term refers to a logical container to hold identifiers of this component.

4. Once we receive the successful project creation message, we need to install the external modules by typing in the `npm install` command and pressing *Enter*.

You will receive a series of console messages, including warnings, which are safe to ignore.

5. When the installation of the modules is complete, open the project with your editor of choice:
 - **Visual Studio:** Open the folder by clicking on **File | Open | Folder...** and continue the code editing from there. You will need to keep the console open to issue the required commands.
 - **VS Code:** Enter the code `.` command on the console to open the editor in the current folder. After this, you can close the original VS Code window.



DEVELOPMENT EDITOR: The rest of the chapter will continue using VS Code.

6. Using the **Explorer** pane on the left, locate `ControlManifest.Input.xml` inside the `FontAwesomeComponent` subfolder and open it:

- Locate all code comments and remove them. The representation of these comments is blocks that start with `<!--` and end with `-->`.
- Locate the control tag and change the description-key attribute to Font Awesome loader.
- Find the property tag and change it to the following:

```
<property name="FontAwesome_KitURL" display-name-
key="FontAwesome_KitURL_Display_Key" description-
key="FontAwesome_KitURL_Desc_Key" of-type="SingleLine.URL"
usage="bound" required="true" />
```

With the preceding changes, the complete manifest should look like the following:

```
<?xml version="1.0" encoding="utf-8" ?>
<manifest>
  <control namespace="PowerNimbus"
constructor="FontAwesomeComponent" version="0.0.1" display-
name-key="FontAwesomeComponent" description-key="Font Awesome
loader" control-type="standard" >
    <external-service-usage enabled="false">
    </external-service-usage>
    <property name="FontAwesome_KitURL" display-name-
key="FontAwesome_KitURL_Display_Key" description-
key="FontAwesome_KitURL_Desc_Key" of-type="SingleLine.URL"
usage="bound" required="true" />
    <resources>
      <code path="index.ts" order="1"/>
    </resources>
  </control>
</manifest>
```

7. Click on **File | Save** or use the shortcut *Ctrl + S*.

How it works...

As you can see, the Power Platform CLI takes care of creating the project's skeleton, leveraging the rest of the tooling to complete the body by installing all the required modules.

The manifest file acts as the control center by defining the capabilities and interaction of this component:

- The setting of `external-service-usage` to `false` allows the platform to acknowledge that this component will not use external services or data sources, thus labeling it a non-premium component.
- The `property` tag indicates that we will receive parameters for this component. In this case, that's the Font Awesome kit's URL.
- The `resources` section specifies all resource files the component will use, such as code, images, or CSS styles.

Building the application logic

Now that we have completed the project structure and manifest configuration, we can start building the application logic in the provided code resource file: `index.ts`.

This file will be the component's entry point and will be in charge of the control initialization, event handling, and control removal.

How to do it...

1. From the VS Code menu, click on **Terminal** | **New terminal**. This action will open the integrated PowerShell console.
2. From this integrated console, enter `npm run build` and press *Enter*. This command will execute the component's build process, including the manifest configuration.
3. Using the **Explorer** pane, expand **FontAwesomeComponent** and double-click the `index.ts` file to open it.
4. Enter the following code just below the `export class` sentence:

```
private _URL: string;
```

5. Find the `init` function and enter the following code between the curly brackets:

```
this._URL = context.parameters.FontAwesome_KitURL.formatted ?
context.parameters.FontAwesome_KitURL.formatted : "";
```

```
// Create the request for the Font Awesome Kit
const script = document.createElement("script");
script.id = "FontAwesomeKit";
script.type = "text/javascript";
script.crossOrigin = "anonymous";
```

```
// val is the default when debugging
if (this._URL.length > 0 && this._URL !== "val") {
```



```

    script.src = this._URL;
  }

  document.getElementsByTagName("head")[0].appendChild(script);

```

6. Look for the `updateView` function and replace the contents between the curly brackets with this code:

```

this._URL = context.parameters.FontAwesome_KitURL.formatted ?
context.parameters.FontAwesome_KitURL.formatted : "";
if (this._URL.length > 0 && this._URL != "val") {
  // We get the previously created link element
  const script = <HTMLScriptElement>document.getElementById
    ("FontAwesomeKit");
  script.src = this._URL;
}

```

7. Save the file by pressing **Ctrl + S**.
8. From this integrated console, enter `npm run build` again, and press **Enter**. Once the code compilation process is complete, enter `npm start watch` and press **Enter**.

This command will invoke the PCF test environment in a browser window, as seen in the following screenshot:

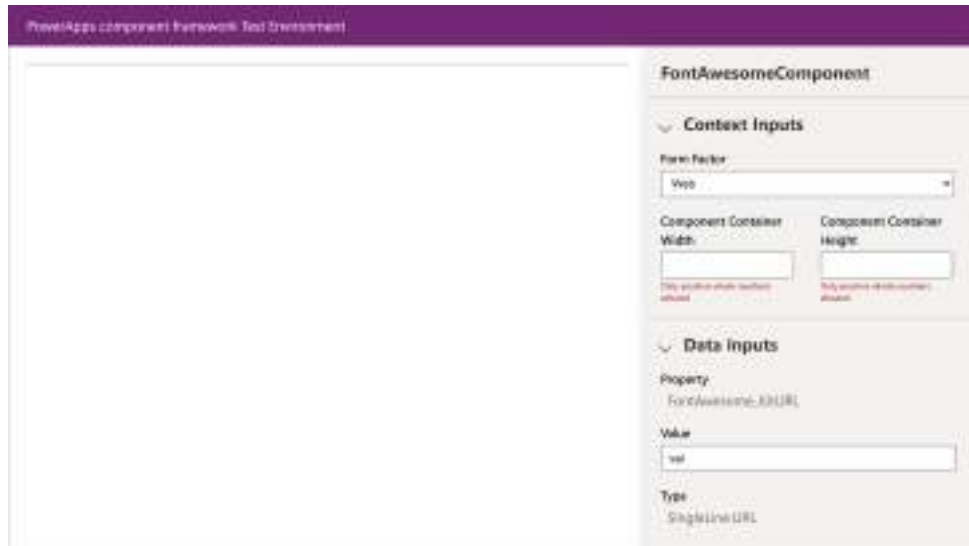


Figure 12.4: *FontAwesomeComponent* loaded in the test environment

9. Go back to VS Code, and from the integrated console, press *Ctrl* + *C*, *Y*, and *Enter* to stop the debugging session by confirming that you want to **Terminate batch job (Y/N)**. Close the browser window.

How it works...

Through this recipe, we have completed the required application logic to load the Font Awesome script inside our code component.

From the `index.ts` file:

- The `init` function creates the script embedding placeholder in the web page's header section serving the component. As Power Apps runs on a browser, it is also considered a web app.
- The `updateView` function will trigger when the Font Awesome kit's URL value gets entered in the `FontAwesome_KitURL` property.

The PCF component uses this technique to load the Font Awesome script inside the app.

One important thing to notice is that every modification of the manifest file will need a code compilation using `npm run build`. This action is required to make the elements of this file available in our code. An example would be using the `FontAwesome_KitURL` property in the `updateView` function as `context.parameters.FontAwesome_KitURL`.

We also discovered how to debug our code component using the PCF test environment. This environment allows the loading of the code components to see how they will react when deployed to an app using the manifest's properties.

There's more...

It can be daunting to deal with pro-code languages such as TypeScript, JavaScript, and so on. In situations like this, a specific type of Copilot can aid in the coding tasks or when troubleshooting. GitHub Copilot can be acquired as an add-in for VS Code or Visual Studio to help develop PCF components. For more information, please visit <https://github.com/features/copilot>.

Deploying the solution

When your project's development and testing phase is complete, you can package and ship it as a solution. The Power Platform CLI can help you create a solution from your project and even deploy it to the desired environment.

In this recipe, we will create the solution package using the CLI and then deploy it to our Dataaverse environment.

How to do it...

First, let's configure and build the solution package and then import it using the Power Apps maker portal.

Building the solution

1. Using VS Code's integrated console, create a subfolder to hold the solution components by entering the following command: `MD FontAwesomeSolution`. Once created, navigate to this subfolder by entering `CD FontAwesomeSolution`.
2. Execute the following command to build the solution structure:

```
pac solution init --publisher-name PowerNimbus --publisher-prefix  
pwn
```

3. Enter the following command to add a reference to our code component:

```
pac solution add-reference --path ..
```

4. From VS Code's **Explorer** pane, navigate to **FontAwesomeSolution | src | Other** and double-click to open the `Solution.xml` file.
5. Change the `LocalizedName` of the `SolutionManifest` from `FontAwesomeSolution` to `Font Awesome Solution`. Press `Ctrl + S` to save this file.
6. Back in the integrated console, execute the `msbuild /t:restore` command to build the package, and when it finishes, execute `msbuild` to build the solution.
7. You will find the solution package in the `FontAwesomeSolution\bin\debug` subfolder.

Deploying to the environment

8. Go to the Power Apps maker portal, <https://make.powerapps.com/>, select your environment of choice, and from the left pane, click on **Solutions**.

9. From the toolbar, click on **Import**. On the pane that opens on the right, click on **Browse** and navigate to select the solution package deployed in the previous section. Click on **Next** and then on **Import** to start this process.
10. After a couple of seconds, the Font Awesome solution should appear in the list of **Solutions**.

How it works...

Just as when creating a solution in the Power Apps maker portal, you will need to define the publisher and its prefix, and they need to be unique in your environment. The next step is to add an existing component, and that's what the `add-reference` command is doing.

MSBuild completes this process by doing precisely what the export solution does in the maker portal, creating a package to be deployed to an environment.

As you can see, the Power Platform CLI is a great alternative when performing manual tasks in the portal. It is an essential tool when automating development processes or applying **Application Life Cycle Management (ALM)** to your projects: <https://learn.microsoft.com/en-us/power-platform/alm/basics-alm>.

Now, let's test this code component.

Testing the component on a canvas app

The final step of our deployment process is to import the PCF component to a canvas app and verify the result. We will do this by creating an app from scratch.

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com/>, and select the environment where we deployed the code component.
2. Create a solution and select **New | App | Canvas app** from its toolbar. In the opening dialog, enter **Font Awesome Test for App name**, keep **Tablet** as **Format**, and click **Create**.
3. Once Power Apps Studio opens, from the **Insert** panel on the left, click on the little folder icon just below the search field.

4. On the pane that opens on the right, look for the **Code** tab at the top and select it. From the list of elements to import, you should see **FontAwesomeComponent**, as seen in the following screenshot:

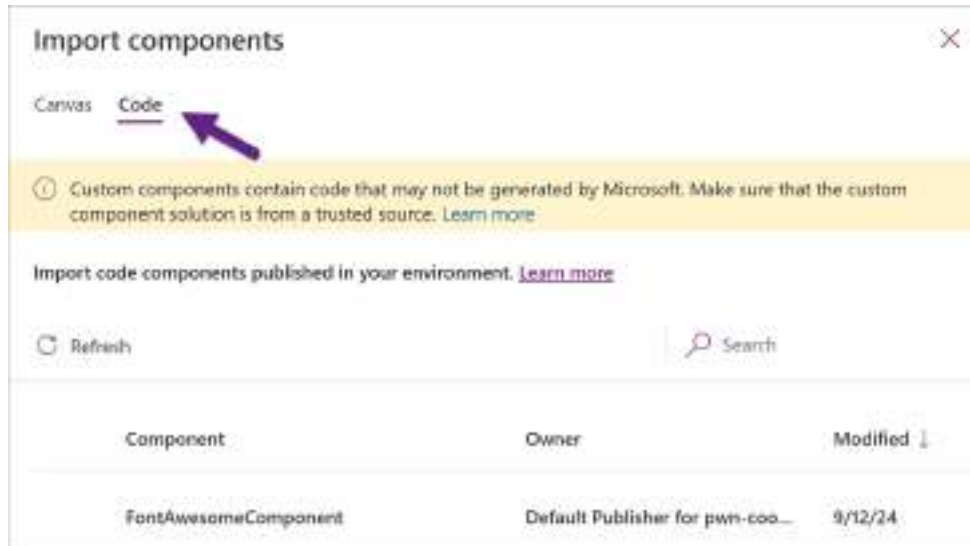


Figure 12.5: FontAwesomeComponent in the Component section

Select it and click on **Import**.

5. You should now see a new section called **Code components** from the **Insert** panel on the left. Expand it and click on **FontAwesomeComponent**.
6. Once added to the canvas, select it and, using the formula bar, set the `FontAwesome_KitURL` property to your kit's URL.
7. From the **Insert** panel on the left, expand **Display** and click on **HTML text**. Set its properties as follows:
 - Using the **Properties** pane on the right, change the **Font size** to 64, the **Width** to 130, and the **Height** to 140
 - Using the formula bar, set the **X** property to $(App.Width - Self.Width) / 2$, the **Y** to $(App.Height - Self.Height) / 2 - 60$, the **Color** to `RGBA(50, 86, 160, 1)`, and the **HTMLText** to the following:


```
"<i class='fa-solid fa-robot fa-solid'></i>"
```
 - Name this control `RobotHtml`

8. Back in the **Insert** panel, expand **Display**, and click on **HTML text**. Set its properties as follows:
 - Using the **Properties** pane on the right, change the **Font** to **Segoe UI**, the **Font size** to 24, the **Width** to 400, and the **Height** to 64
 - Using the formula bar, set the **X** property to $(\text{App.Width} - \text{Self.Width}) / 2$, the **Y** to $(\text{App.Height} - \text{Self.Height}) / 2 + 20$, the **Color** to `RGBA(130, 141, 158, 1)`, and the **HTMLText** to the following:


```
"<i class='fa-solid fa-hand-point-right fa-solid'></i>
Welcome to our app <i class='fa-solid fa-hand-point-left fa-
solid'></i>"
```
 - Name this control `WelcomeHtml`

The completed application should look like the following:



Figure 12.6: Canvas app using Font Awesome icons

9. Click on the **Publish** icon, and then you can close the designer.

How it works...

The code component works without a user interface. It's a code-only component that helps load the script required to use Font Awesome icons in Power Apps.

As the embedding occurs at the head level of the web page's document element, it becomes available to all controls using the provided classes. This is when the HTML text control becomes handy, as it can handle HTML code to render its content. Keep in mind that, for this specific scenario, we need to modify the HTML classes to make it work.

From the Font Awesome documentation, <https://fontawesome.com/icons/hand-point-left?f=classic&s=solid>, the usage should be `<i class="fa-solid fa-hand-point-left"></i>`, but we repeat the `fa-solid` section at the end of the class since the Power Apps engine modifies some classes to make the HTML code work without issues.

For more information, please refer to the official documentation regarding the process of building a code component at <https://learn.microsoft.com/en-us/power-apps/developer/component-framework/create-custom-controls-using-pcf>.

There's more...

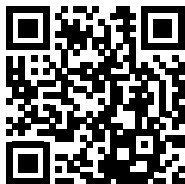
Code components are also available for model-driven apps to improve the rendering of elements in forms, dashboards, and views. For more information, please visit <https://learn.microsoft.com/en-us/power-apps/developer/component-framework/add-custom-controls-to-a-field-or-entity>.

As an added resource, please visit the PCF gallery, <https://pcf.gallery/>, for a massive collection of components created by the community to extend canvas and model-driven apps.

Join our community on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/powerusers>



13

Reaching Beyond the Organization with Power Pages

Throughout this book, we have learned about the capabilities that Power Apps offers to all kinds of users of an organization, from low-code makers to pro-code developers. These skills are often geared toward handling data internally, but what about when an organization needs contributions from external users? That's when the latest component of Power Platform comes into play: Power Pages.

This component enables us to expand the capabilities of Dataverse beyond the organization, allowing us to provide solutions such as:

- A self-service solution for customers to manage, update, and review their financial data so that the legal department can build contracts
- A support portal to allow external users to manage their tickets
- An internet presence to offer all company services

Power Pages is the evolution of the previous method for collaborating with external users: Power Apps portals. It provides enhanced capabilities with a redesigned studio, templates, and site creation experience. For more information, please visit <https://learn.microsoft.com/en-us/power-pages/introduction>.

In this chapter, we will extend the help desk solution developed in *Chapter 3, Building from Data with Model-Driven Apps*, to allow customers to create support tickets using the following recipes:

- Setting up Power Pages
- Extending the Dataverse solution

- Building the help desk interface
- Configuring user access and security
- Automating record attributes
- Adding custom fonts to the portal



Keep in mind that we won't be covering the authentication methods that external users would use to log in to your portal since the appropriate method depends on the security requirements of the organization using this solution.

Having said that, from my experience, I would recommend using Azure Active Directory B2C, since it creates a subset of Active Directory to separate your customers from your regular users. More information can be found here: <https://learn.microsoft.com/en-us/azure/active-directory-b2c/overview>.

Technical requirements

Power Pages uses a particular licensing requirement with a clear differentiation of internal and external users. For more information, please refer to <https://learn.microsoft.com/en-us/power-platform/admin/powerapps-flow-licensing-faq#power-pages>.

Regarding the approach of this chapter, as in *Chapter 3*, the following recipes need to be addressed in sequence to build a complete help desk solution. For easier access, here's the link to *Chapter 3*'s GitHub repository: <https://github.com/PacktPublishing/Microsoft-Power-Apps-Cookbook-Third-Edition/tree/main/Chapter-13>.

Setting up Power Pages

We must use a specific maker portal when creating websites in Power Platform: <https://make.powerpages.microsoft.com>. This differs from the regular maker portals since it only covers website templates and solutions. For everything else, you need to resort to the Power Apps or Power Automate maker portals.

Getting ready

Since we will be extending the help desk solution from *Chapter 3*, *Building from Data with Model-Driven Apps*, you need to have it deployed in the Power Platform environment of your choice. You will need to use the same one to build the support portal.

How to do it...

1. Go to the Power Pages maker portal and select your desired environment from the list at the top right of the screen. If this is your first time accessing this portal, click on **Get started** to arrive in the maker portal:

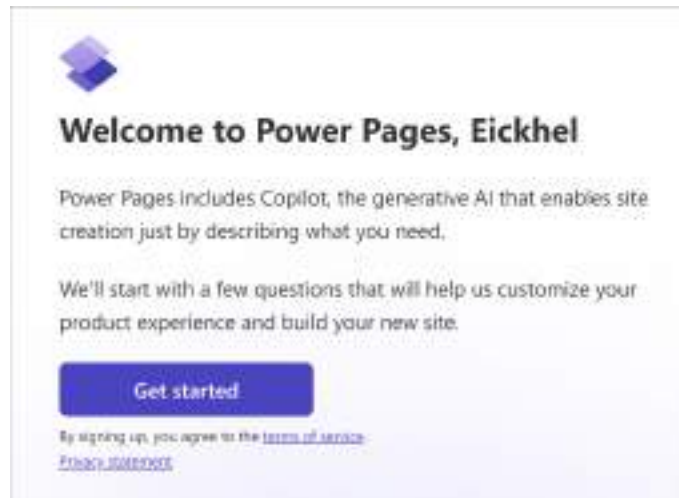


Figure 13.1: Power Pages introduction

2. Click the **Start with a template** button in the middle of the page.
3. On the next screen, a list of templates will appear to help you choose the look and feel of your new site. Select **Starter layout 1** and click **Choose this template**.
4. Enter the desired site name in the **Give your site a name** field—for example, **AMPI Helpdesk portal**. Change the web address to a more meaningful name as well. Click **Done** when you're finished. The portal will start provisioning.
5. After a while (around 5 minutes if it is the first portal in the environment), the created site will open in the design studio, the designer's name for Power Pages.

How it works...

The Power Pages maker portal allows you to quickly provision websites in a truly no-code way. Simply select the desired template, and all the necessary coding will get packaged on a `powerappsportals.com` subdomain. In the following recipes, we will adapt this site to the specific needs of a help desk solution.

There's more...

Another option to consider when creating sites is to use an AI prompt from the maker portal to request the Power Pages Copilot to build a site that covers our needs.

Depending on the request, Copilot will prepare a home layout for your website, and then it will offer a set of pages, including relevant content, covering some of the requirements from the prompt. Some will be standard pages, such as About Us, Contact Us, and FAQ, but others will focus on the prompt's topic. For example, if you're building an onboarding website, you will get pages covering new hire resources or training programs. This will definitely help you get up to speed when using Power Pages.

Extending the Dataverse solution

Working from a portal often requires different approaches to data manipulation. When working on a model-driven application, we can take advantage of all sorts of data controls to manage our process efficiently. Still, a more straightforward approach can sometimes help specific scenarios, such as when an external user creates support tickets through your Power Pages portal.

To improve the existing help desk solution to be used on Power Pages, we will create new forms for the **Tickets** table.

Getting ready

As mentioned, you will be extending the help desk solution from *Chapter 3, Building from Data with Model-Driven Apps*, so you must have it deployed into your environment.

How to do it...

1. Go to the Power Apps maker portal, <https://make.powerapps.com>, and click **Solutions** on the left pane.
2. Look for your help desk solution, as per our example, **AMPI Helpdesk**, and click on it to open it.
3. Expand the **Tables** section of the **Objects** list, and select the **Ticket** table. Click **Properties** from the **Table properties** section. Click **Enable attachments (including notes and files)** and click **Save** to continue.
4. From the left pane, expand **Columns**:
 - Click **+ New column** from the toolbar, and on the pane that opens, set **Contact** for **Display name**, **Lookup | LookUp** for **Data type**, and **Contact** for **Related table**. Click on **Save** when you're finished.

- From the list of **Columns**, look for **Ticket Status**. Change the **Default choice** to **Pending**. Click on **Save** when you're finished.
5. Still on the **Ticket** table, click on **Forms** on the left pane.
 6. Click on **+ New form | Main form** from the toolbar. Make the following changes:
 - Set **Display Name** on the right pane to **Web form**
 - Delete all data controls. Ensure the **Title** and **Description** fields remain. Rearrange them as you see fit.
 - From the toolbar, click on **Save and publish**.

The form should look like the following:

Figure 13.2: Ticket Web form

7. From the toolbar, click on **Back**.
8. Repeat *step 6* to create a new form:
 - Set **Display Name** on the right pane to **Web form - Details**.
 - Besides **Title** and **Description**, include the **Ticket Status** field. Set it as **Read-only** from the **Properties** panel.
 - From the **Components** section on the left pane, expand **Grid** and select **Subgrid**. From the popup that opens, enable **Show related records**, and choose **Ticket Operations (Ticket)** from the **Table** list. Leave **Active Ticket operations** as **Default view** and click **Done**.
 - Still in the **Components** section, expand **Display** and select **Timeline**.
 - Rearrange the form as you see fit. You might need to allow proper spacing for the **Timeline** control and the list of **Ticket Operations**.
 - From the toolbar, click on **Save and publish**.

Here's an example of the form layout:

Figure 13.3: Ticket's Web form – Edit

9. You can close the form designer and the solution.

How it works...

The changes made in this recipe are needed to create a better experience on the support portal. Adding **Contacts Lookup** to the **Tickets** table allows users to view only the associated tickets whenever they log in to the portal.

Building the help desk interface

Power Pages offers a portal solution like any other **Content Management System (CMS)** platform, such as WordPress (<https://wordpress.com>) or DNN (<https://www.dnnsoftware.com>). The difference lies in the built-in integration with Dataverse, not only with data but also with security and integrations.

Taking advantage of this platform's website-building features, we will create an external user-facing portal to build an internet presence that allows registered users to create support tickets.

Similar to using the model-driven designer, modifying pages is just like changing a Microsoft Word document with a predefined structure. Moving the mouse over the page hovers the sections and elements of the page. Clicking on them allows you to edit them or remove them altogether. If you have some experience building SharePoint pages, you will feel right at home.

Getting ready

We will continue extending the help desk solution by exposing the **Ticket** table to enabled portal users.

How to do it...

1. Go to the Power Pages portal, <https://make.powerpages.microsoft.com>, and look up the previously created portal from the list. Click on **Edit** to open it.
2. The portal opens on the **Pages** section, displaying the website's **Main navigation** on the left:
 - Expand **Pages**, click the ellipsis (...) menu next to **Subpage 2**, and choose **Delete**. On the dialog that opens, confirm the action.
 - Select the ellipsis next to **Subpage 1** and click on **Promote this subpage**. Click on the ellipsis again and select **Page settings**. On the open dialog, set **Name** to **Tickets** and **Partial URL** to **tickets**. Click **OK** when you're finished.
 - Click on the ellipsis next to **Pages** and select **Page settings**. On the open dialog, set **Name** to **Services** and **Partial URL** to **services**. Click **OK** when finished.
3. Update the **Home** page to match your needs. An example of these changes can be seen in the following screenshot:

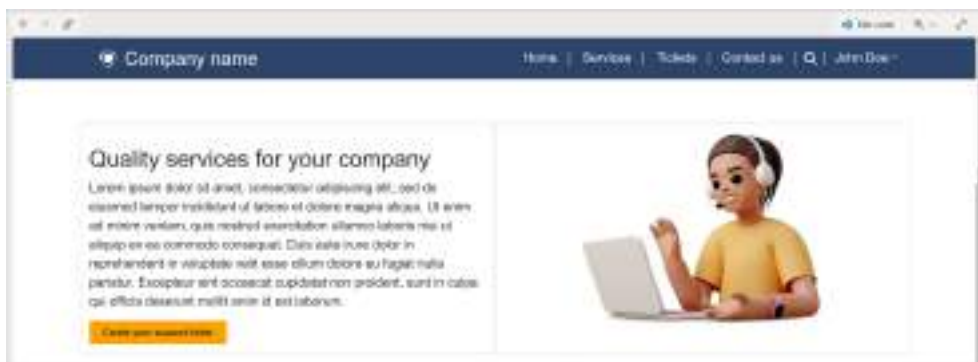


Figure 13.4: Company home page

4. In the previous image, the background is white. To change the background color of a section, click on it, and a popup will appear with options to set. Click **Edit background** to set predefined colors or an image. You can also change these colors in the **Styling** section of the left pane. From this section of the editor, you can change various settings of the selected theme:
 - In the **Color palette** section at the top, click on a circle with a plus sign to add a new color.
 - From the **Fonts** section, you can change all presets with an instant preview on the right side.
5. Return to the **Pages** section with the **Home** page selected and click the **Create your support ticket** button. On the open popup, click **Edit** and select **Tickets** from the list of pages. Click **OK** when finished.
6. From the **Pages** section, click on the + **Page** button. On the open dialog, set **Page name** to Ticket – Details, deselect **Add page to main navigation**, keep the **Start from blank** layout, and click on **Add**:

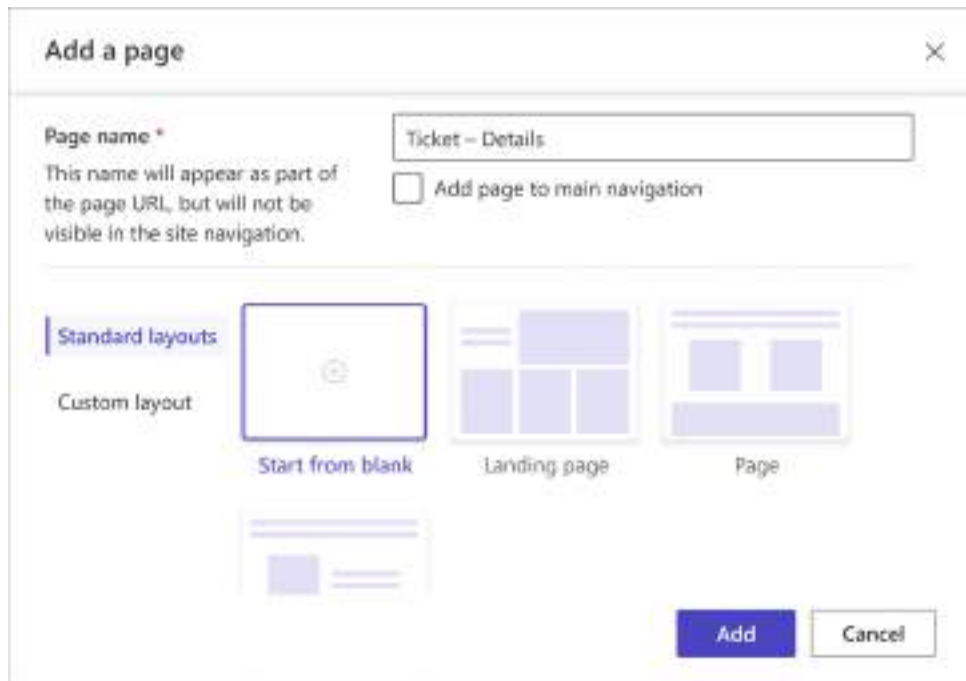


Figure 13.5: New page dialog

7. From the options under **Choose a component to add to this section**, select **Form**:
 - Click on **New form**.
 - Set **Ticket** for the **Choose a table** field and **Web form - Details** for the **Select a form** dropdown. Set **Name your copy of the selected form** to **Ticket detail form**.
 - Click on the **Data** section and select **Is read-only** from the **Data from this form** dropdown.
 - Click on the **On submit** section. Select **Redirect to a page** on the **When the form is submitted** dropdown and select the **Tickets** page on the adjacent dropdown.
 - Finally, click on **CAPTCHA** and deselect both options for anonymous and authenticated users.
 - Click **OK** when you're finished.
8. Repeat *steps* 6 and 7 to create a new page, but make these changes:
 - Set **Page name** to **Ticket - New**
 - Set **Ticket** for the **Choose a table** field and **Web form** for the **Select a form** field. Set **Name your copy of the selected form** as **Ticket new form**
 - Set **Data from this form** to **Creates a new record**.
9. Select the **Tickets** page from the **Main navigation** section. Select the bottom area, the one with the **Product or service one** paragraph, and select the ellipsis to choose **Delete**, as seen in the following screenshot:

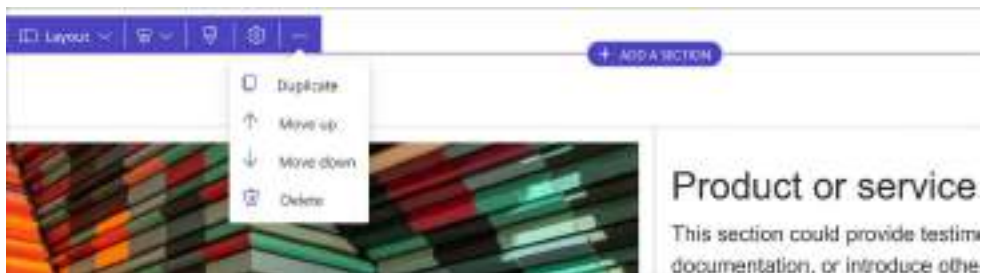


Figure 13.6: Section options

10. Now select the text control with this content: **This page can be customized by adding new sections and components**. Remove it using the ellipsis menu.

11. Select the **Subpage one** text control and change it to **Pending tickets**. Click the plus sign just below this control and select **List**. From the opened dialog:
 - Set **Ticket** for the **Choose a table** field and **Pending Tickets** for the **Select data views** field.
 - Click on the **Actions** section and turn on **Create a new record**. For **Target type**, select **Webpage** and choose **Ticket - New**. Do the same for **Edit record**, selecting **Ticket - Details** as the web page.
 - Finally, click on **More options** and click on **Enable search in this list**.
 - Click **Done** when you're finished.
12. Practice by updating the **Services** and **Contact us** pages to match your needs.

How it works...

This exercise will help you familiarize yourself with the low-code approach of Power Pages when building web pages. It will also serve to prepare the **Tickets** page, which will be helpful for registered users on the site to view their active tickets with our help desk company.

The **Tickets** page and the accompanying pages should look like the following screenshot:

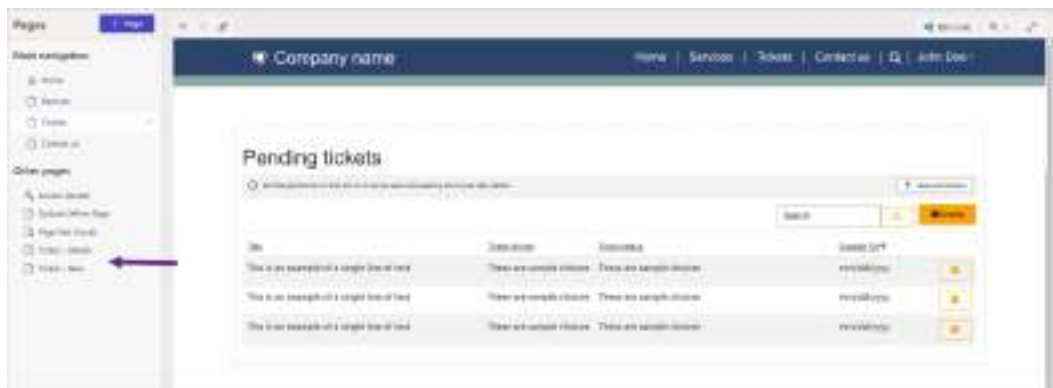


Figure 13.7: Viewing ticket pages from other pages

You can test the whole website as an anonymous user by clicking on **Preview | Desktop** on the main toolbar. To view it as a registered user, click **Sign in** while previewing the site and select **Azure AD** in the **Sign in with an external account** section. The following recipe will look at setting up permissions to this portal.

There's more...

When you land on your newly created site, you will find that the Power Pages Copilot is ready to help you build content and functionality. One example is the ability to create forms to handle table records from Dataverse.

Following our previous example of creating an onboarding site using a prompt, we could ask to Create a new form for new hires to build a profile. From this prompt, Copilot will request that you specify if you want to build it from an existing table or if you want it to create a new table for you.

The following image shows how Copilot builds the table and the form for you:

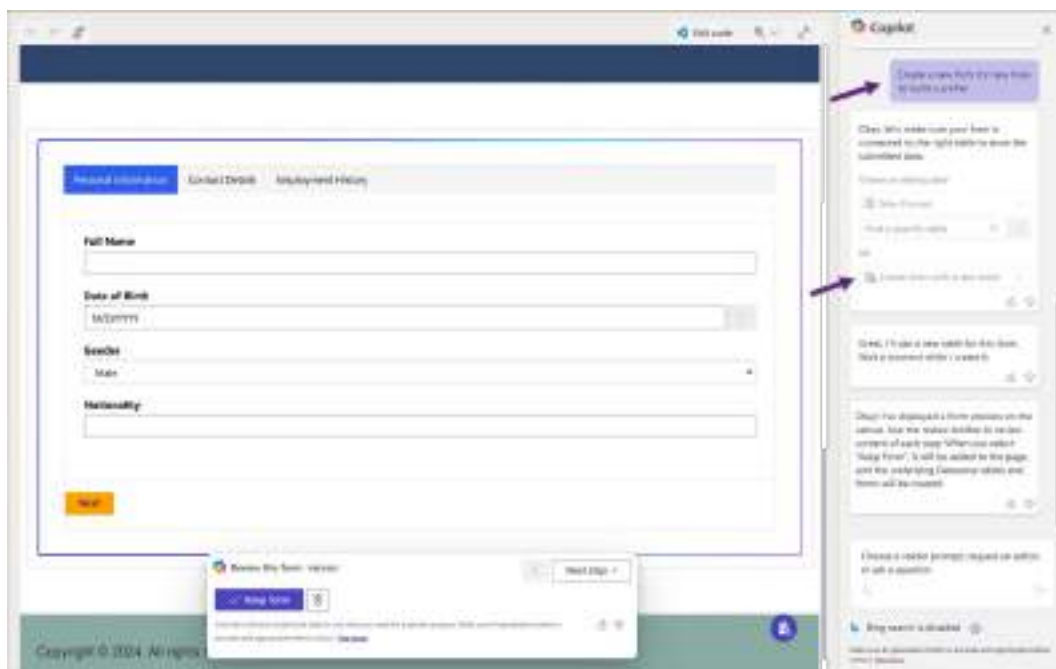


Figure 13.8: Form preview from a prompt

Using this feature from an existing table works just as well. Again, this could help speed up the website-building process.

Configuring user access and security

Building a website to acquire an internet presence is vital for any organization. These websites can add value by implementing services to registered users, allowing them to interact with company data using proper permissions.

Dataverse offers a security model to interact with its data. Power Pages provides another layer, allowing makers to safeguard certain portal areas. This layer comprises several components, including authenticated users, web roles, table permissions, and page permissions.

For more information, please visit <https://learn.microsoft.com/en-us/power-pages/security/power-pages-security>.

For this recipe, we will take advantage of table permissions for the base tables and related child tables. These permissions will be associated with authenticated users. We will also cover how to use code snippets to improve the experience for users not authenticated to the portal.

Getting ready

We will continue extending the help desk solution by exposing the **Ticket** table to enabled portal users.

How to do it...

We will approach this recipe in two parts. First, we will configure the table so that only authenticated users can use it. Then, we will secure access to the page by implementing a code snippet that checks the user's registration status.

Setting up permissions

1. If you haven't already done so, open your site from the Power Pages portal, <https://make.powerpages.microsoft.com>, by clicking **Edit**.
2. Click the **Security** option from the left panel and select **Table permissions**.
3. From the toolbar, click on **New permission** and set the following options:
 - On the **Basic** tab, enter **Ticket** for **Name**.
 - Choose the **Ticket** table from the **Table** list.
 - Set **Access type** to **Contact access**, and choose the **Relationship** created from the *Extending the Dataverse solution* recipe when you added the **Lookup** to the **Contacts** table.
 - In the **Permission to** section, check **Read**, **Update**, **Create**, **Append**, and **Append to**.
 - Under **Roles**, click on **Add roles** and check **Authenticated Users**.
 - Click on **Save** when you're finished.

The following is an example of the **Ticket** permission:

Ticket [X]

Basic Child permissions

Name *
Ticket

Table *
Ticket

Access type *
Contact access

Show rows in the table associated to the signed-in user (contact). [Learn more](#)

Relationship *
apc_ticket_Contact_contact

Related tables

- apc_ticket_Contact_contact
- Contact

Permission to

☒ Read
 ☒ Update
 ☒ Create
 ☐ Delete
 ☒ Append
 ☒ Append to

Roles

+ Add roles

Authenticated Users

Figure 13.9: Basic fields for the Ticket permission

4. Click to open the **Ticket** permission from the list, and when it opens, go to the **Child permissions** tab and click on + **New**. Set the following options:
 - Enter **Contact** for **Name**.
 - Select the **Contact** table from the **Table** list and choose the **Relationship** between the **Ticket** and the **Contact** table.
 - In the **Permission to** section, check **Append** and **Append to**.
 - Click on **Save** when you're finished.

5. Add another child permission to the **Ticket** table:
 - Enter Operation for **Name**.
 - Select the **Ticket operation** table from the **Table** list and select the existing **Relationship** between these tables.
 - Enable **Read** in the **Permission to** section and click on **Save** when you're finished.
6. Add one last child permission to the **Ticket** table:
 - Enter Notes for **Name**.
 - Select the **Note** table from the **Table** list and select the existing **Relationship** between these tables.
 - Enable **Read, Update, Create, Append, and Append to** on the **Permission to** section and click on **Save** when you're finished.

The permission configuration should look like the following:

Name	Status	Table	Access Type	Role	Relationship	Read	Update	Create	Delete	Append	Append to
Feedback	Active	Feedback	Read-only	Authenticated Users - Global		☒	☒	☒	☒	☒	☒
Email	Active	Email	Read-only	Authenticated Users - Global	app:Email:Email:Email	☒	☒	☒	☒	☒	☒
To Contact	Active	Contact			app:Contact:Contact:Contact	☒	☒	☒	☒	☒	☒
To Notes	Active	Notes			app:Notes:Notes:Notes	☒	☒	☒	☒	☒	☒
To Operations	Active	Operations			app:Operations:Operations:Operations	☒	☒	☒	☒	☒	☒

Figure 13.10: Configuration of table permissions

Now that the permissions are ready, let's add a code snippet to check whether a user is registered.

Securing page access

7. Go back to the **Pages** section and select the **Tickets** page. Click on the code editor icon near the top-right corner of the screen, as seen in the following screenshot:

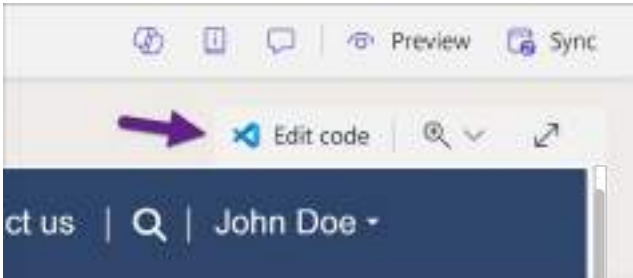


Figure 13.11: Button to open the code editor

8. This action will open an instance of Visual Studio Code for the web, an online version that is already linked to the Power Pages portal to make code changes to the web page components.
9. On the open dialog, click **Allow** to permit the usage of the **Power Platform Tools** extension.
10. The editor will arrive at the code behind the **Tickets** page. Include the following code at the top:

```
{% if user == Null %}  
<script>  
  window.location.href = './SignIn/';  
</script>  
{% endif %}
```

11. Close the **VS Code** editor and click **Sync** on the open dialog to merge the code changes into the page.

How it works...

After setting up the pages to manage the support ticket process, we need to prevent anonymous users from creating tickets without at least registering on the portal.

First, we secure access to the **Ticket** table at the Power Pages level, so only authenticated users can access the records from this table, even if they already know the URL to create tickets or view their details. We also configured child permissions to tables related to the **Ticket** table so the data is available for controls such as the **Timeline** and the **Ticket operations** subgrid.

Second, we added custom code to the **Tickets** page to check for the current user. If we don't have one, we redirect to the sign-in/register page.

There's more...

The code used to redirect users is called **Liquid**. It's an open-source template language to create dynamic content. Please refer to the official documentation to review all the possibilities that this language provides to add custom template logic to your sites: <https://learn.microsoft.com/en-us/power-pages/configure/liquid/liquid-overview>

Liquid is one of the ways to use **code components**, which we covered in *Chapter 12, Advanced Techniques with the Power Apps Component Framework*. More information is available here: <https://learn.microsoft.com/en-us/power-pages/configure/liquid/component-framework-liquid>

Another approach is to leverage the Power Pages designer and configuration to allow the usage of code components. Please visit this link to gather more information in this regard: <https://learn.microsoft.com/en-us/power-pages/configure/component-framework>

Regarding security, make sure you visit the Power Platform admin center and then **Resources** | **Power Pages sites** to review even more features available in the **Security** tab.

Automating record attributes

Every help desk solution follows a similar process flow. A user logs in to the portal, creates a ticket, and then interacts with it until it resolves. The idea is that every new ticket gets associated with the logged-in user, so they can see their status when they log in again.

To accomplish this, we will leverage functionality available from the **Power Pages Management**. This application allows the configuration of all Power Pages websites in a specific environment. It holds every configuration setting you can think of, from the forms you build in the Power Pages editor to the website analytics settings of your provider of choice, such as Microsoft Clarity (<https://clarity.microsoft.com>) or Google Analytics (<https://analytics.withgoogle.com>).

Getting ready

We will access the Power Pages Management application from the Power Pages editor to configure the ticket form metadata. With our site open, we will continue this recipe from the Power Pages editor.

How to do it...

1. From the left pane of the Power Pages editor, click on the ellipsis and select **Power Pages Management**. A new tab will open, displaying the configuration details of the current website. It is a regular model-driven app that handles the portal's Dataverse configuration.
2. From this new tab, click on **Basic Forms** in the **Content** section of the left pane. You should see a list of forms available on your site, like the ones in the following screenshot:



Figure 13.12: Basic Forms list

3. You should see the **Ticket new form** we created in a previous recipe, *Building the help desk interface*. Click on it to view its details.
4. Go to the **Additional Settings** tab and set the following configuration:
 - Scroll down to the **Associate Current Portal User on Insert** section and check the **Associate Current Portal User** checkbox. Keep in mind that there is a large section of whitespace at the top of this page.
 - Right of the **Portal User Lookup Column** field, select **Contact (apc_contact)** from the dropdown. Remember that the internal name between parentheses will be different in your environment.
 - When you're finished, click on **Save & Close** from the main toolbar.
5. Open the **Ticket detail form** from the **Active Basic Forms** list.
6. Go to the **Basic Form Metadata** tab, click **New Basic Form Metadata** from the subgrid's toolbar, and set the following:
 - Select **Timeline** from the **Type** dropdown.
 - From its settings, change **Create Enabled** to **true**.
 - When you're finished, click on **Save & Close** from the main toolbar.
7. Repeat *step 6*, but select **Notes** from the **Type** dropdown.
8. Close the **Power Pages Management** application browser tab and, back in the Power Pages editor, click on **Sync** at the top right of the screen.

How it works...

After updating additional settings of the **Ticket** forms, we accomplished several features:

- Every new ticket gets the current user associated with it. The next time the user visits our site, they will see their pending tickets.
- Each ticket will display a list of related operations. This way, the user can see what actions are being made by the support team.
- By including the **Timeline** control, the user can also interact by creating notes, even with attachments, to supply more information to the support team about the ticket.

As an example, in the following screenshot, you can see a ticket with all the related information:



Figure 13.13: Ticket details

There's more...

The Power Pages Management application allows the configuration of many features to improve the capabilities of your site. Examples can be multi-language support, model-driven charts, and the setup of HTTP headers.

To get more insight into this application, please visit the following site: <https://learn.microsoft.com/en-us/power-pages/configure/portal-management-app>.

Adding custom fonts to the portal

The current abilities of Power Pages focus on an easy-to-use platform to build sites connected to Dataverse. Even though it inherits many features from Power Apps portals, some options are still unavailable.

One example is the ability to use any font from the ones listed in the Power Pages editor when making changes to the pages of your site:

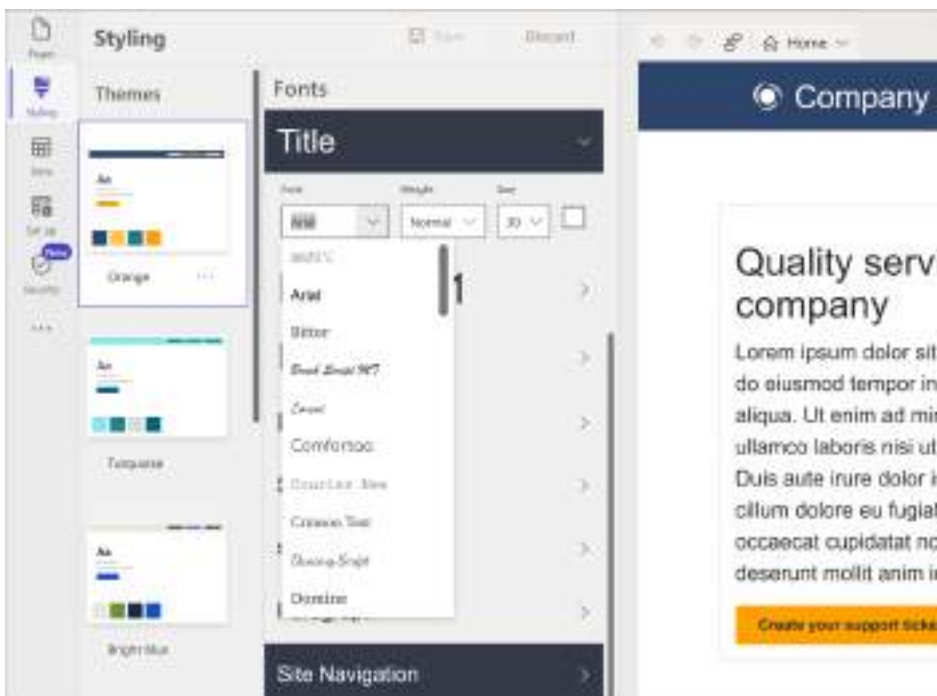


Figure 13.14: Available fonts in Power Pages

This recipe will help you overcome this requirement by leveraging the Power Platform CLI and Visual Studio Code to change the portal contents directly.

Getting ready

We must install the Power Platform CLI and Visual Studio Code to connect directly with our portal contents. Please refer to *Chapter 12, Advanced Techniques with the Power Apps Component Framework*, for step-by-step instructions on installing and configuring these applications.

For up-to-date instructions on how to connect to the portal, please refer to the following link: <https://learn.microsoft.com/en-us/power-pages/configure/power-platform-cli-tutorial>

How to do it...

We will approach this recipe in three parts: *Preparing the local version of the portal*, *Using preinstalled fonts*, and *Using fonts from the Google Fonts repository*.

Preparing the local version of the portal

1. Once you have all the requirements completed, execute the following code from a terminal window inside Visual Studio Code:

```
pac auth create --environment <environmentURL>
Enter
```



To get your environment's URL, go to the Power Pages maker portal, <https://make.powerpages.microsoft.com>, click on the gear icon at the top right of the screen, and then on **Session details**. You can get the URL from the **Instance url**.

Note the following about the executed code:

- Replace <environmentURL> with your environment's URL.
 - This action will open a dialog requesting the credentials to access the environment.
 - If the action gets completed successfully, you should get a message including the text **Authentication profile created**.
2. Enter the following command to get the list of portals in the environment:

```
pac powerpages list
Enter
```

From this list, get the **Website Id** of the portal we will update.

3. Let's create a folder to download the website by issuing the following command:

```
md portals
Enter
cd portals
Enter
```

4. Download the portal to the current folder by entering this command:

```
pac powerpages download --path . --websiteId <WebSiteId>
--modelVersion 2
Enter
```

5. Note the following details about this command:
 - Change <WebsiteId> to the one gathered in *step 2*.
 - A folder will get created with a name starting with the portal's name – for example, `ampi-helpdesk-portal---ampi-helpdesk`
 - `modelVersion 2` refers to the enhanced data model used in Power Pages.
6. Enter the folder by entering:

```
cd <folder-name>
Enter
```

Change <folder-name> to the folder name created in the previous step.

7. Open Visual Studio Code from this folder by entering this command:

```
code . -r
```

8. On the open dialog, click **Allow** to permit the usage of the **Power Platform Tools** extension.

Using preinstalled fonts

9. From the Explorer pane inside Visual Studio Code, expand **web-files** and double-click to open the `portalbasictheme.css` file.
10. From the menu, click on **Edit | Replace**:

- Change:

```
font-family: Arial;
```

to:

```
font-family: "Segoe UI", "Helvetica Neue", Helvetica, Arial,
sans-serif;
```

- Click the **Replace All** icon.
11. Click on **File | Save** from the menu or `Ctrl + S` to save the file.

12. From the menu, click on **Terminal** | **New terminal** and enter the following code to upload the changes:

```
pac powerpages upload --path . --modelVersion 2
Enter
```

13. Once the process finishes, edit your portal and click on **Sync** to ensure the changes get updated.
14. This process works with fonts already installed on users' devices. Follow the next section for another approach.

Using fonts from the Google Fonts repository

15. Browse to the Google Fonts website, <https://fonts.google.com>, and choose the desired font. As an example, I've chosen **Finlandica**: <https://fonts.google.com/specimen/Finlandica>.
16. Click the **Get font** button and then the **<> Get embed code**.
17. From the **Web** tab, click on the **@import** option and copy the **Embed code in the <head> of your html** code.

The following screenshot is an example of setting the Finlandica font:



Figure 13.15: Finlandica Embed code page

18. Repeat the same steps as in the earlier section, *Using preinstalled fonts*, setting the font in *step 9* like so:

- Change:

```
font-family: Arial;
```

to:

```
font-family: 'Finlandica', sans-serif;
```

19. Add the import code gathered from *step 15* of this section (removing the `<style>` tags) at the beginning of the file, just after the comments, as seen in the following screenshot:



Figure 13.16: Finlandica import code

How it works...

Using these workarounds, we can improve the look and feel of our site using preinstalled fonts or fonts from a repository such as the ones found in Google Fonts.

After applying the changes, the home page should look like the following:



Figure 13.17: Finlandica font applied

Keep in mind that applying any other theme from the Power Pages editor will overwrite the `portalbasictheme.css` file, voiding your changes. Just repeat the steps of this recipe to get the fonts back.



packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Learn Microsoft Power Apps, Second Edition

Matthew Weston

Elisa Bárcena Martín

ISBN: 978-1-80107-064-5

- Understand the Power Apps ecosystem and licensing
- Take your first steps building canvas apps
- Develop apps using intermediate techniques such as the barcode scanner and GPS controls
- Explore new connectors to integrate tools across the Power Platform
- Store data in Dataverse using model-driven apps
- Discover the best practices for building apps cleanly and effectively
- Use AI for app development with AI Builder and Copilot



Microsoft 365 and SharePoint Online Cookbook, Second Edition

Gaurav Mahajan

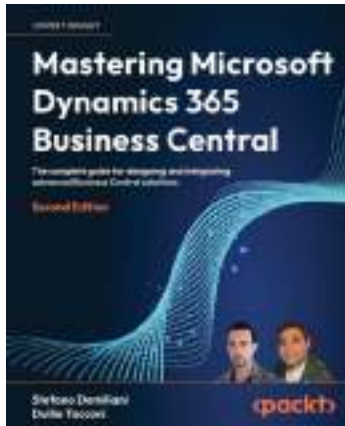
Nate Chamberlain

Sudeep Ghatak

Scott Brewster

ISBN: 978-1-80324-317-7

- Collaborate effectively with SharePoint, Teams, OneDrive, Delve, Search, and Viva
- Boost creativity and productivity with Microsoft Copilot
- Develop and deploy custom applications using Power Apps
- Create custom bots using Power Virtual Agents (Copilot Studio)
- Integrate with other apps, automate workflows and repetitive processes with Power Automate/Desktop (RPA)
- Design reports and engaging dashboards with Power BI
- Utilize Planner, To Do, and gather feedback with polls and surveys in Microsoft Forms
- Experience seamless integration in the mobile platform



Mastering Microsoft Dynamics 365 Business Central, Second Edition

Stefano Demiliani

Duilio Tacconi

ISBN: 978-1-83763-064-6

- Developing a customized solution for Dynamics 365 Business Central
- Writing performant code following extensibility patterns
- Handling reporting, files, and printing on a cloud environment
- Handling Business Central telemetries with Azure
- Writing APIs and integrations for Dynamics 365 Business Central
- Applying DevOps and CI/CD to development projects by using GitHub
- Integrating Business Central with Power Platform
- Publishing your solutions to AppSource marketplace
- Manage Copilot capabilities and create your own generative AI copilot

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Leave a review!

Thank you for purchasing this book from Packt Publishing—we hope you enjoyed it! Your feedback is invaluable and helps us improve and grow. Please take a moment to leave an Amazon review; it will only take a minute, but it makes a big difference for readers like you.

Scan the QR code below to receive a free ebook of your choice.



<https://packt.link/NzOWQ>

Index

A

Accounts table 41

adaptive application
designing 201-209

Adaptive Cards 114
example 114, 115

AI Builder 263

AI prompt
using 372

AIReply
reference link 271

analytics 297

application interfaces design,
enhancing 192, 193, 197
Browse screen, improving 193, 194
Detail screen, refining 195, 196
Edit screen, finishing touches 196, 197

Application Life cycle Management (ALM) 74
reference link 365

application logic
building 361-363

Application Programming
Interface (API) 173

applications
automating, with desktop flows 142-147
building, with Dataverse for Teams 253-259
discovering, through Power
Apps mobile 212-215

Artificial Intelligence (AI) 263

Augmented Reality (AR) 185
automated cloud flow 112

Azure Data Studio
reference link 92

Azure DevOps 76
reference link 76

Azure Key Vault service
reference link 199
setting up 199-201

Azure SQL 12

Azure SQL Database 85, 337
reference link, for issues and limitations 36
setting up 12-15

Azure SQL database views
data source, provisioning 92-95
used, for reducing application
complexity 91-95

Azure SQL Server connections
app, building 338-340
environments, creating 338
exported package, transforming 340-343
updating, in Power Apps 336, 337

B

Browsers based on Chromium section
reference link 216

Build Tools for Visual Studio 354

business process flow (BPF) 57, 113

business process logic
setting up 56-61

C

canvas apps

- component, setting up 155-159
- component, testing 365-368
- creating, from existing data 31-36
- integrating, inside model-driven apps 64-67
- map capabilities, integrating 180-184
- on-premises data, integrating
 - with 96-100
- reference link 27
- text recognition system, building 272-276

canvas apps, best practices

- coding methodology 19
- control usage 18, 19
- Copilot assistance 20
- discovering 18
- reference link 20
- variable naming 18

canvas component properties

- reference link 163

canvas control

- screen real estate, extending with 331-335

canvas solutions

- extending, with Mixed Reality 185-190

Cards for Power Apps 120

Cascading Style Sheets (CSS) styles 317

Center of Excellence (CoE) Starter Kit 301

- reference link 301

Client Access Licenses (CALs) 82

code components, Liquid

- reference link 383

column formatting 220, 225, 347

Command-Line Interface (CLI) 352

component

- testing, on canvas app 365-368

connectors 148

- handling, through data policies 301-304
- reference link, for issues and limitations 37

Contacts table 41

Content Delivery Network (CDN) 357

Content Management System (CMS) 374

Copilot experience

- discovering, when building model-driven apps 77-80

creator kit

- reference link 169

custom connectors

- reference link 174
- used, for expanding communications 173-179

customer success solution

- canvas app, building 267-270
- cloud flow, building 267
- creating, with sentiment analysis 264-270
- Microsoft form, building to receive complaints 266
- SharePoint list, creating to store customer complaints 265

custom fonts

- adding, to portal 387, 388

custom pages

- model-driven apps, improving 67-73
- references 73

custom properties 159

D

dashboards 61

- designing, to visualize data 61-64

data entry forms 31

Dataflows 109

- reference link 109

- synchronization tools 109
 - used, for consuming
 - external data 102-108
 - data policies**
 - connectors, handling through 301-304
 - data source**
 - selection, by performing
 - real-world tests 85-89
 - data structures**
 - defining 47-54
 - Dataverse 27, 40, 85**
 - business-oriented features 40
 - environments 41
 - solutions 41
 - table example 41
 - tables, preparing 4-6
 - Dataverse for Teams 84, 252**
 - apps, building 253-259
 - solution elements 260
 - Dataverse solution**
 - extending 372-374
 - Dataverse table views**
 - example 96
 - Date Picker 209**
 - default environment 23, 30**
 - reference link 292
 - delegation 37, 127**
 - desktop flows 113**
 - applications, automating with 142-147
 - developer environment**
 - reference link 292
 - development environment**
 - configuring 352-356
 - DirectQuery 234**
 - DNN**
 - URL 374
 - Document automation toolkit**
 - reference link 290
 - document processing 271**
 - Dribbble**
 - reference link 198
 - Dynamics 365 Business Central 215**
 - Dynamics 365 Field Service 215**
 - Dynamics 365 Sales 215**
- ## E
- enterprise resource planning (ERP) system 66**
 - Environment groups feature 42**
 - reference link 42
 - environments 41, 292**
 - configuring 41-45
 - creating, with database 295
 - creating, without database 293, 294
 - default 292
 - developer 292
 - Microsoft Teams 292
 - production 292
 - sandbox 292
 - support 292
 - trial 292
 - Extensible Markup Language (XML)**
 - structure 314
- ## F
- Figma**
 - reference link 198
 - file processing automation**
 - building, for SharePoint libraries 120-126
 - Finance and Operations 215**
 - Finlandica**
 - reference link 390

Floating Action Button (FAB) 160

- building 160-162
- reference link 162

Flow Studio

- reference link 301

Font Awesome

- reference link 357
- setting up 356-358

fonts

- using, from Google Fonts repository 390, 391

G**GitHub**

- reference link 76

GitHub Copilot 363**Globally Unique Identifier (GUID) 226****Global Positioning System (GPS) 67, 180****Google Analytic**

- reference link 384

Google Chrome

- applications, running as standalone solutions on 218

Google Fonts repository

- fonts, using from 390, 391

Google Forms

- integrating with Dataverse, cloud flows used 148-152

Google Forms JSON exporter 152**grandfathering period 83****Graph API 244****Group Policy (GPO) 219****Group property 42****H****help desk interface**

- building 374-378

Hypertext Transfer Protocol (HTTP) 174**I****iframes 61, 64****incident tracking solution**

- data source setup 20-23
- user interface, building 23-26

instant cloud flow 112**invoice classification system**

- AI model, configuring 281-283
- approval example notification 288
- cloud flow, building 284-288
- creating, with Power Automate 280, 281

K**Key Performance Indicators (KPIs) 118****L****Liquid**

- reference link 383

Long Term Release (LTS) 353**M****map capabilities**

- integrating, into canvas apps 180-184

map control properties

- reference link 184

Marketing Emails app 228**metadata-enabled storage**

- building 6-9

Microsoft 365

Power Apps/Power Automate 84

Microsoft Azure 31**Microsoft Clarity**

reference link 384

Microsoft Edge

applications, running as standalone solutions on 217

Microsoft Forms 9, 114

information gathering 10-12

sales survey solution, creating with 114-120

Microsoft Lists 6**Microsoft Power Platform CLI**

configuring 355

reference link 355

Microsoft Teams 234

context parameters 242

environment 292

Power Apps, integrating inside 243-251

Microsoft Teams channels

canvas app, creating to distribute through 235-240

custom application upload 241

Microsoft Teams connector

reference link 252

mixed reality

reference link 190

used, for extending canvas solutions 185-190

Mockaroo

reference link 193

model-driven apps 27, 67

building 54-56

canvas apps, integrating inside 64-67

improving, with custom pages 67-73

reference link 27

models

reference link 264

Modern app designer 54**modern browsers**

leveraging, for Power Apps 216-219

modern controls and themes 209**Monitor tool 329****N****Node.js 353**

configuring 353

npm 353

configuring 353

O**Office 365 Groups connector**

reference link 252

offline capabilities 27**OnChange property 276****OneDrive**

app, building 311-313

cloud flow, building to communicate with Power Apps 311

cloud flow, building to obtain Excel data 309, 310

data sharing, between tenants 308

data, uploading 309

Excel data, obtaining from 313, 314

guest access 308

on-premises data

integrating, with canvas apps 96-100

OnSelect property 276

P

Pagination

reference link 88

Pin to Home feature 214-216

pipelines 74

reference link 74

Postman 153

reference link 174

Power Apps

building 232, 233

embedding, in SharePoint
pages 225-228

for Microsoft 365 84

integrating, inside Teams 243-251

modern browsers, leveraging for 216-219

SharePoint document libraries,
improving 220-224

used, for improving SharePoint
list forms 27-30

used, for making Power BI reports
interactive 234

Power Apps component libraries

using 169-172

Power Apps maker

reference link 365

Power Apps mobile

applications, discovering
through 212-215

Power Apps Monitor

troubleshooting with 326

working 329, 330

Power Apps portal

custom fonts, adding 387

local version, preparing 388, 389

preinstalled fonts, using 389, 390

Power Automate

for Microsoft 365 84

modular solution, creating with 135-142

Power Apps delegation,
overcoming with 127-134

using, to create invoice
classification system 280, 281

Power Automate machine runtime app 144

Power BI 229

Power BI Desktop

configuring 230-232

installing 229, 231

Power BI embedded system dashboard

reference link 64

PowerBIIntegration control 234

Power Pages

setting up 370, 371

Power Pages Copilot 379

Power Pages maker portal

reference link 388

Power Pages Management 384

Power Pages security

reference link 380

Power Platform admin center

configuration capabilities 392

Power Platform analytics, gathering

Power Apps maker portal 298, 299

Power Automate 299

Power Platform admin center 297, 298

SharePoint list Power Apps 299

Power Platform environment

configuring 355

Power Platform Licensing Guide

reference link 271

Power Platform solution

building 2, 3, 45, 46

Power Query

reference link 103

prebuilt AI models

reference link 271

prebuilt models

reference link 290

production environment 292

solution, deploying to 74-76

project

creating 358-361

prompts, in AI Builder

reference link 264

R**real-world tests**

performing, for selection of
data source 85-89

record attributes

automating 384-386

Refresh method 234**responsive password manager**

building, in Power Apps 199-209

Robotic Process Automation (RPA) 113**S****sales survey solution**

creating, with Microsoft Forms 114-119

sandbox environment 292**Scalable Vector Graphics (SVG) 314****scheduled cloud flow 113****Scope 124****screen real estate**

extending, with canvas control 331-335

Search function 37**security roles 296**

references 296

sentiment analysis

used, for creating customer
success solution 264-270

SharePoint 85**SharePoint document libraries**

cloud flow, building 345-347

configuring 347

creating 344

file processing automation,
building with 120-126

files, renaming 343, 344

improving, with Power Apps 220-224

SharePoint list apps

destination site, configuring 321, 322

exported package, transforming 323, 324

exporting 320

list, building on source site 319

list, customizing on source site 319, 320

package, importing 324, 325

transferring, from one site to
another 318, 319

SharePoint List Formatting Samples

reference link 225

SharePoint list forms

improving, with Power Apps 27-30

SharePoint Online 6

performance considerations 90

SharePoint pages 225

Power Apps, embedding 225-228

solution 41

deploying, to production
environment 74-76

reference link, for distributing
model-driven app 45

solution package

- building 364
- deploying 364
- environment, deploying 364, 365

static data

- used, for improving application speed and responsiveness 101

support environment

- reference link 292

SVG, in canvas apps

- working with 315-317

T**tables 3**

- in Dataverse 4-6, 41

text recognition 271

- building, with canvas app 272-276

Text Recognizer control 276**Ticket table 47**

- operations 47

trial environment

- reference link 292

troubleshooting, with Power**Apps Monitor 326**

- app, building 327-329
- SharePoint list, provisioning 326

U**user access and security**

- configuring 379, 380
- page access, securing 382, 383
- permissions, setting up 380-382

User Interface (UI) 191

- guidelines 154, 155

V**vertical navigation component**

- creating 163-169

Visual Studio 2017

- configuring 354

Visual Studio Code (VS Code) 353

- configuring 354
- reference link 354

W**WebAppInstallForceList 220****web browsers 216****web forms 114****web parts 225****WordPress**

- URL 374

Wrap feature

- reference link 216

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily.

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below:



<https://packt.link/free-ebook/9781835465158>

2. Submit your proof of purchase.
3. That's it! We'll send your free PDF and other benefits to your email directly.

